Data Science in Engineering

Statistics and Machine Learning with Real Engineering Data

Prof. Dr. Maik Weber

2026-11-11

Table of contents

Preface **8**
What this course is about . . . 8
Prerequisites . . . 8
How to work with this script . . . 9
Software . . . 9
Reading guide . . . 9
Notation . . . 10
Assessment . . . 11

I Part A – Understanding Data **12**

1 The Data Science Workflow in Engineering **13**
1.1 What are data? . . . 14
1.1.1 Data as a table . . . 15
1.1.2 Types of variables . . . 15
1.1.3 Roles of variables: target and features . . . 16
1.1.4 Something that looks like data – but is not . . . 17
1.2 What is data science in engineering? . . . 18
1.3 Statistics, machine learning, deep learning, AI: sorting the terms . . . 19
1.3.1 Extension: And what about AI? . . . 20
1.3.2 Where statistics and machine learning differ – and where they overlap . . . 21
1.4 The CRISP-DM process model . . . 22
1.5 Tools: the Python data stack . . . 24
1.6 Live coding: a first look at the power plant data . . . 26
1.6.1 A first simple model . . . 33
1.7 Back to the decision . . . 33
1.8 Exercises . . . 36
1.8.1 Solutions . . . 36

2 Exploratory Data Analysis and Data Quality **40**
2.1 Reading raw data . . . 41
2.1.1 Live coding: a raw weather file . . . 42
2.1.2 Live coding: the energy data of a house . . . 46

2.2 Describing distributions . . . 48
2.2.1 Live coding: appliance energy use . . . 50
2.3 Patterns over time . . . 51
2.3.1 Live coding: daily and weekly profiles . . . 51
2.4 Data quality and outliers . . . 53
2.4.1 Live coding: outliers in the energy data . . . 54
2.5 Missing data . . . 55
2.5.1 Live coding: how missing values bias a mean . . . 57
2.5.2 Live coding: gaps in semiconductor data . . . 60
2.5.3 Extension: MNAR in more depth . . . 64
2.6 Back to the decision . . . 65
2.7 Exercises . . . 66
2.7.1 Solutions . . . 67

3 Probability and Uncertainty 73
3.1 Random variables and distributions . . . 74
3.2 The Weibull distribution for lifetimes . . . 75
3.2.1 Estimating parameters: maximum likelihood . . . 76
3.3 Live coding: lifetimes of turbofan engines . . . 77
3.4 Confidence intervals . . . 79
3.5 Back to the decision . . . 81
3.6 Bayes' theorem: how much should we trust an alarm? . . . 82
3.7 Extension: Bayesian estimation with prior knowledge . . . 84
3.8 Exercises . . . 86
3.8.1 Solutions . . . 86

II Part B – Statistics for Engineers 90

4 Statistical Testing 91
4.1 The logic of a hypothesis test . . . 92
4.2 Comparing two groups: the t-test . . . 93
4.3 More than two groups, categories and non-normal data . . . 93
4.4 Effect size and practical relevance . . . 94
4.5 The multiple-testing problem . . . 95
4.6 Live coding: NOx emissions 2011–2015 . . . 96
4.7 Live coding: product variant and machine failures . . . 98
4.8 How many measurements do we need? . . . 99
4.9 Back to the decision . . . 100
4.10 Exercises . . . 101
4.10.1 Solutions . . . 101

5 Linear and Nonlinear Regression **105**
5.1 The linear model . . . 106
5.1.1 Estimation by least squares . . . 106
5.1.2 Interpreting the output . . . 106
5.1.3 Model assumptions and residual analysis . . . 107
5.1.4 Collinearity . . . 107
5.2 Live coding: concrete strength . . . 108
5.2.1 Confidence vs. prediction intervals . . . 112
5.3 Nonlinear curve fitting . . . 113
5.4 Back to the decision . . . 116
5.5 Extension: design of experiments – a 2^3 full factorial design . . . 117
5.6 Exercises . . . 118
5.6.1 Solutions . . . 119

6 Model Selection, Regularisation and Gaussian Processes **125**
6.1 Training error, test error and overfitting . . . 126
6.2 Cross-validation . . . 127
6.3 Regularisation: Ridge and Lasso . . . 128
6.4 Live coding: airfoil self-noise . . . 129
6.4.1 How complex should the model be? . . . 130
6.4.2 Regularisation in action . . . 131
6.5 Gaussian process regression: predictions with uncertainty . . . 132
6.6 Back to the decision . . . 135
6.7 Extension: Bayesian optimisation – design of experiments with a GP . . . 136
6.8 Exercises . . . 138
6.8.1 Solutions . . . 138

III Part C – Machine Learning **143**

7 Classification **144**
7.1 Three ways to classify . . . 145
7.1.1 Logistic regression . . . 145
7.1.2 k-nearest neighbours (kNN) . . . 145
7.1.3 Decision trees . . . 145
7.2 Evaluating classifiers . . . 146
7.2.1 Thresholds, ROC and precision–recall curves . . . 147
7.2.2 Strategies for imbalanced classes . . . 147
7.3 Live coding: predictive maintenance . . . 148
7.4 Back to the decision . . . 152
7.5 Exercises . . . 153
7.5.1 Solutions . . . 153

8 Ensembles and Interpretability **158**
8.1 Why ensembles work . . . 159
8.1.1 Bagging and random forests . . . 159
8.1.2 Boosting . . . 160
8.2 Multi-class problems . . . 160
8.3 Interpreting models . . . 161
8.4 Live coding: steel plate faults . . . 162
8.5 Back to the decision . . . 166
8.6 Extension: SHAP values . . . 167
8.7 Exercises . . . 168
8.7.1 Solutions . . . 169

9 Unsupervised Learning and Anomaly Detection **173**
9.1 Principal component analysis . . . 174
9.2 Process monitoring with PCA . . . 174
9.3 Clustering . . . 176
9.4 Anomaly detection . . . 176
9.5 Live coding: PCA monitoring of semiconductor production . . . 177
9.6 Back to the decision . . . 181
9.7 Live coding: operating states of a hydraulic test rig . . . 182
9.8 Exercises . . . 184
9.8.1 Solutions . . . 185

10 Signals and Feature Engineering **188**
10.1 Sampling and aliasing . . . 189
10.2 The frequency domain . . . 189
10.3 Bearing fault frequencies . . . 190
10.4 Statistical features of a signal window . . . 191
10.5 Live coding: healthy vs. inner-race fault . . . 192
10.6 Live coding: amplitude spectrum . . . 193
10.7 Extension: envelope analysis . . . 194
10.8 Live coding: statistical features . . . 196
10.9 Back to the decision . . . 196
10.10 Exercises . . . 197
10.10.1 Solutions . . . 198

IV Part D – Time Series, Deep Learning and Review **203**

11 Time Series Forecasting **204**
11.1 Components and autocorrelation . . . 205
11.2 Defining the forecasting task . . . 205
11.3 Forecasting as regression . . . 206

11.4 Live coding: German electricity load . . . 208
11.4.1 Baselines . . . 210
11.4.2 Feature-based forecasting with gradient boosting . . . 210
11.5 Back to the decision . . . 211
11.6 Extension: seasonal ARIMA . . . 212
11.7 Exercises . . . 213
11.7.1 Solutions . . . 214

12 Deep Learning in Engineering 218
12.1 From linear regression to neural networks . . . 219
12.2 Training: loss, gradients and backpropagation . . . 220
12.3 Remaining useful life from sensor sequences . . . 221
12.4 Live coding: remaining useful life . . . 222
12.5 Back to the decision . . . 226
12.6 Extension: CNNs and transfer learning for visual inspection . . . 227
12.6.1 Live coding: anomaly detection on one MVTec category . . . 227
12.7 Outlook . . . 228
12.8 Exercises . . . 229
12.8.1 Solutions . . . 229

13 Review and Exam Preparation 236
13.1 Which method when? . . . 236
13.2 Core concepts at a glance . . . 237
13.3 Checklist: typical pitfalls . . . 239
13.4 Typical exam question types . . . 239
13.5 Practice questions . . . 240
13.5.1 Solutions . . . 240

Appendices 243

A Python Quick Reference 243
A.1 Getting an overview . . . 243
A.2 Missing values . . . 244
A.3 Selecting rows and columns . . . 245
A.4 Transforming values . . . 246
A.5 Summarising and grouping . . . 248
A.6 Reshaping and combining tables . . . 249
A.7 Reading data . . . 249
A.8 Random numbers and simple models . . . 250
A.9 Plots . . . 250

B Data from Databases **252**
B.1 Tables, keys and relations . . . 252
B.2 Reading data: `SELECT … FROM` . . . 256
B.3 Selecting rows: `WHERE` . . . 257
B.4 Summarising: `GROUP BY` . . . 257
B.5 Combining tables: `JOIN` . . . 259
B.6 Pitfalls . . . 261
B.6.1 A join can multiply rows . . . 261
B.6.2 Inner and left join . . . 261
B.6.3 `NULL` is a missing value . . . 262
B.6.4 Time stamps are often text . . . 263
B.7 Outlook . . . 263
B.8 Exercises . . . 264
B.8.1 Solutions . . . 264

C Datasets **268**

D Further Reading **270**

Preface

Version: 0.1

Welcome to *Data Science in Engineering*, a master's course at Trier University of Applied Sciences for students of mechanical engineering, electrical engineering and industrial engineering.

What this course is about

Every method in this script is introduced together with a real, publicly available engineering dataset – from power plants and gas turbines to rolling bearings, semiconductor production and jet engines. Theory and practice go hand in hand: each chapter starts with a (fictional) engineering scenario that calls for a decision, explains the methods needed to make it, applies them in Python – and returns to the scenario at the end to make the decision with the results.

The course has four parts:

Part	Topics	Chapters
A – Understanding Data	workflow, exploratory analysis, data quality, uncertainty	1–3
B – Statistics for Engineers	hypothesis tests, regression, design of experiments, model selection	4–6
C – Machine Learning	classification, ensembles, unsupervised learning, signal features	7–10
D – Time Series, Deep Learning and Review	forecasting, neural networks, review	11–13

Prerequisites

Basic Python programming and mathematics at bachelor level (linear algebra, calculus, introductory statistics).

How to work with this script

The script is meant to be read in the browser or as a PDF. Every result in it was computed with Python, and the code is part of the script: in the HTML version, click *Code* above a result to show it. You can copy it into your own notebook to reproduce or vary an analysis.

During the semester, the demo code from the lectures is provided as Jupyter notebooks.

Software

You can work with any development environment you like (e.g. JupyterLab, VS Code or PyCharm), as long as it can open, run and edit Jupyter notebooks. We recommend Google Colab; a local installation is an alternative:

- **Google Colab (recommended):** Colab runs notebooks in the browser – no installation required, on a laptop or a tablet (an external keyboard helps a lot). Sign in with a Google account and open a notebook via *File* → *Upload notebook*. Most packages used in this course are pre-installed.
- **Alternative – local installation:** The Anaconda Distribution installs Python, Jupyter and the most important data science packages in one go – a good choice if you prefer to work offline and with files on your own computer.

Packages that are missing can be installed directly from a notebook cell with `%pip install`, e.g. `%pip install ucimlrepo`.

Reading guide

Each chapter opens with a scenario (purple box) and closes it in a section *Back to the decision*. The people and situations are invented; the data are real (or, where stated, simulated).

Coloured boxes highlight what matters most:

i Definitions

Blue boxes define the central concepts.

! Key results

Red boxes contain theorems, key results and the takeaways at the end of each chapter.

> **Pitfalls**
>
> Yellow boxes warn of typical mistakes – in data science, many of them lead to results that look convincing but are wrong.

> **Engineering practice**
>
> Green boxes give practical advice for engineering projects.

Each chapter ends with exercises (highlighted boxes) followed by **solutions**, which are collapsed in the HTML version – try the exercise first, then open the solution. The lab assignments of the course build on the exercises.

Sections and exercises marked **Extension** go beyond the core of the course. They are optional and not required for the exam – use them to dig deeper into topics that matter for your own work.

Notation

The script uses the following conventions throughout:

Symbol	Meaning
$x_1, \dots, x_n$	observations (sample of size n); x_i is the i-th observation
$\bar{x}$, s	sample mean and sample standard deviation
$\tilde{x}$	sample median
μ, σ	mean and standard deviation of the population or distribution
$\hat{\theta}$	estimate of a parameter θ (the hat marks anything computed from data)
y, $\hat{y}$	response (target) and its prediction
X	matrix of inputs: one row per observation, one column per input
$P(A)$, $P(A \mid B)$	probability, conditional probability
$E[X]$, $\mathrm{Var}(X)$	expected value and variance of a random variable X
ln	natural logarithm

The Greek alphabet is small, and the standard symbols of different fields overlap. Where a symbol has different meanings in different chapters, the meaning is stated where it is used. The most important cases: β is the Weibull shape parameter (Chapter 3), the probability of

a type II error (Chapter 4) and a regression coefficient (Chapters 5–7); η is the Weibull scale parameter (Chapter 3) and the learning rate (Chapter 12); σ denotes a standard deviation and, in Chapters 7 and 12, an activation function.

Assessment

- **Lab assignments** (coursework, pass/fail): four assignments during the semester.
- **Written exam** (90 minutes, without computer): method choice, interpretation of results, short calculations and code reading.

Both parts must be passed; they can be completed in different semesters.

Part I

Part A – Understanding Data

1 The Data Science Workflow in Engineering

Engineers have always worked with data: test bench measurements, calibration curves, quality records, field failures. What has changed is the *amount* and the *variety* of data. A modern production line logs hundreds of signals per second, a wind turbine reports its state every few seconds, and simulation studies produce thousands of runs. Data science provides the methods to turn such data into decisions.

This chapter sets the stage. We clarify what data are, what makes data science in engineering special and how it relates to statistics, machine learning and AI. We introduce a process model that structures every project and get to know the Python tools we will use throughout the course. Finally, we load and inspect our first dataset and use it to answer a business question.

Scenario: the day-ahead commitment

You have just joined the trading desk of a combined cycle power plant. Every day at noon, the plant must commit its electrical output for each hour of the next day to the power exchange. Every megawatt-hour delivered more or less than committed is settled at an unfavourable price. Until now, the desk has simply committed the long-term average output for every hour. Your colleague suggests using tomorrow's temperature forecast instead – but the team lead is sceptical: "Is that really worth the effort?"

Your decision: Keep the flat commitment or switch to a temperature-based estimate?

Learning goals. After this chapter you can

- define what we mean by data, determine the type and role of a variable and recognise tables that are not data,
- explain how we use the terms statistics, machine learning, deep learning, artificial intelligence and data science, and where they overlap,
- describe the phases of a data science project (CRISP-DM) and apply them to an engineering problem,
- distinguish the main data science tasks: description, inference, prediction and diagnosis,
- use the Python data stack (NumPy, pandas, Matplotlib, seaborn) in Jupyter or Google Colab,
- load a dataset from a public repository,
- produce a first overview of a dataset (size, types, summary statistics, pair plots) and interpret it,
- turn a model result into a business decision with a simple cost model.

Datasets in this chapter

- Combined Cycle Power Plant (UCI) – original publication: P. Tüfekci, *Prediction of full load electrical power output of a base load operated combined cycle power plant using machine learning methods*, International Journal of Electrical Power & Energy Systems 60 (2014), 126–140, doi:10.1016/j.ijepes.2014.02.027

1.1 What are data?

Before we talk about data *science*, we should be clear about what we mean by *data*. The word is used loosely – for a spreadsheet, a database, a folder of images, "the numbers". For this course we use a sharper definition.

> **Definition: Data**
>
> **Data** are recorded values of **variables** (characteristics such as temperature, torque or fault type), **observed** on **observational units** (parts, machines, test runs, points in time) – together with the **context** needed to interpret them: what was measured, how, where, when and in which unit of measurement.

The four highlighted terms deserve a closer look.

Variables are *what* is recorded – temperature, torque, fault type. Their type determines how they can be analysed (see below).

Observational units are *what the values belong to*: a part, a machine, a test run, a point in time. This is not always obvious – one shaft, or one of several measurements on that shaft? The answer decides what counts as an independent observation, which will matter when data are split into training and test sets (Chapter 10). In the power plant example of this chapter, the unit is one hour of plant operation.

Observed, not specified. Data record how things *actually turned out*: the diameter a coordinate measuring machine found on a particular shaft, the fact that pump no. 17 failed after 3,412 hours. This distinguishes data from specifications, targets and tables computed from a formula, which state how things *should* be (see below). Because data record outcomes, they can contradict our expectations – this is what makes them valuable.

Context (metadata). The value `293` means nothing without its context: a temperature in kelvin? A pressure in mbar? A part number? Unit, sensor, sampling rate, time zone and the definition of each variable belong to the data. They are usually documented separately as **metadata** – and often lost.

> ⚠ Pitfall: numbers without context
>
> In 1999 NASA lost the *Mars Climate Orbiter* because one software component delivered thruster impulse data in pound-force seconds while the receiving component expected newton seconds. Every number was correct – the context was not. Before analysing a dataset, make sure you know the unit and the meaning of every column.

1.1.1 Data as a table

Most methods in this course expect data as a table whose layout follows directly from the definition. This layout is called *tidy data*:

- each **row** is one observation – one unit, or one unit at one point in time,
- each **column** is one variable,
- each **cell** holds one value.

Many real files are not tidy – a separate column for every day, two quantities in one cell – and must be reshaped first (in pandas with `melt` or `pivot`). Signals and images are data, too, even though they do not come as such a table (see below).

1.1.2 Types of variables

The type of a variable determines which summaries, plots and models make sense (Figure 1.1). A variable is either **categorical** – its values are labels such as "supplier B" – or **numeric** – its values are amounts such as 3.2 mm or 5 defects. Categorical variables are *nominal* (unordered) or *ordinal* (ordered); a nominal variable with only two categories, such as failure yes/no, is called **binary**. Numeric variables are *discrete* (countable) or *continuous* (any value in a range).

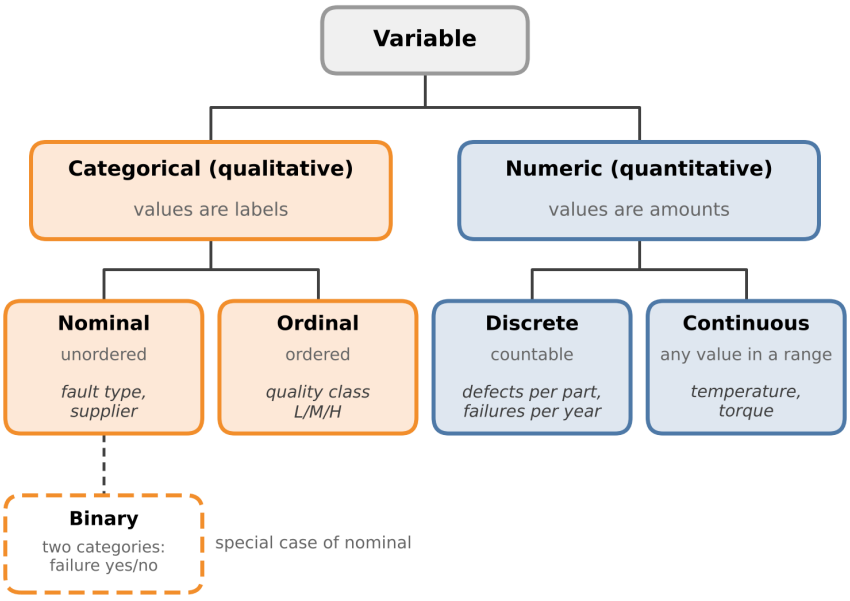


Figure 1.1: Types of variables with engineering examples.

For numeric variables we compute means, standard deviations and histograms; for categorical variables we count frequencies and draw bar charts. The type also decides which calculations are meaningful at all. Coding a category as a number does not make it numeric: the mean of the fault codes 1, 2 and 7 is meaningless. And even for numeric variables, not every calculation makes sense: 20 °C is 10 K warmer than 10 °C, but not "twice as warm", because the zero of the Celsius scale is arbitrary – in kelvin, ratios are meaningful.

In the power plant data, all five variables are continuous, which is typical for process data. Two kinds of data need special treatment:

- **Time stamps** are numeric – the difference between two time stamps is a duration – and mostly serve as the index of a time series.
- **Signals, images and texts** consist of many values per unit, e.g. thousands of samples of a vibration recording. They are condensed into a few informative variables, so-called *features* (next section) – either computed by hand, such as the RMS value of a signal (Chapter 10), or learned by a neural network (Chapter 12).

1.1.3 Roles of variables: target and features

Besides its type, a variable has a role in an analysis. In the power plant example, we want to predict the electrical output from the ambient conditions and the condenser pressure:

- The **target** is the variable we want to explain or predict – here the net electrical output (`PE`). It is also called *output*; in classification (Chapter 7), the target is usually called *label*.
- The **features** are the variables used to explain or predict it – here ambient temperature, ambient pressure, relative humidity and condenser pressure (`AT`, `AP`, `RH`, `V`). They are also called *inputs*.

Features can be measured directly or derived from other variables – the hour of the day from a time stamp, the RMS value from a vibration signal. Constructing such features is called **feature engineering**; it is where engineering knowledge enters a model most directly (Chapters 10 and 11).

The role depends on the question, not on the variable: the ambient temperature is a feature when we predict the power output, but the target when we forecast the weather. And not every analysis has a target – in unsupervised learning (Chapter 9), we look for structure among the features alone.

1.1.4 Something that looks like data – but is not

Consider a table with one row per dimension of a turned shaft – diameter at bearing seat, length, roundness – and columns *nominal value*, *upper tolerance*, *lower tolerance*. It has rows, columns, numbers and units. Is it data?

In the sense of our definition: **no**. The table comes from the technical drawing and describes how the shaft *should* be – a **specification**. It contains no observation of any real shaft. Fitting a model to it would tell us something about the designer's choices, not about the manufacturing process. The **data** are the values the coordinate measuring machine records on the shafts actually produced. Only by comparing them with the specification can we learn something about the process – for example its capability.

Similar cases: a table computed from a formula (a steam table) contains no information beyond the formula; a production plan is not a record of production.

> Simulated data
>
> Data generated by a simulation are observations of a **model**, not of reality. They are extremely useful – for training, for testing methods, when real failures are rare – but every conclusion holds for the simulated system only, and transfers to reality only as far as the simulation is valid. Two datasets in this course are simulated: the AI4I predictive maintenance data and the C-MAPSS turbofan data. We will keep this in mind when interpreting results.

1.2 What is data science in engineering?

Put briefly, data science is the systematic extraction of knowledge and decisions from data. It combines *statistics* (how to reason under uncertainty), *computer science* (how to process data and fit models efficiently) and *domain knowledge* (what the data mean and which questions matter). How this relates to statistics, machine learning and AI is sorted out in the next section.

The third ingredient is where engineers have a decisive advantage. A data scientist without engineering background may find that the vibration level of a pump correlates with the time of day; an engineer knows that this is because the pump runs at higher load during the day shift – and asks the right follow-up question.

Typical tasks fall into four groups:

Task	Question	Engineering example	Chapters
Description	What happened?	Distribution of tensile strength in a batch	2, 3
Inference	Is the effect real?	Did the new supplier change the scrap rate?	4, 5
Prediction	What will happen?	Remaining useful life of a bearing, tomorrow's grid load	5–8, 11, 12
Diagnosis / detection	What is wrong?	Which fault type? Is this run abnormal?	7–10

Engineering data have some characteristic properties that we will meet again and again:

- **Physics behind the data.** Relationships are often known in principle (energy balance, material laws). Good models use this knowledge instead of ignoring it.
- **Rare events.** Failures and defects are – fortunately – rare. This makes them hard to learn from (Chapter 7).
- **Time and sequence.** Sensor data are time series. Neighbouring values are not independent, which has consequences for validation (Chapter 11).
- **Measurement chains.** Every value went through sensor, amplifier, A/D converter, logging system. Each step can introduce errors, offsets, gaps and wrong units (Chapter 2).
- **High cost of errors.** A wrong prediction can mean an unplanned standstill or a safety issue. Uncertainty must be quantified and communicated.

1.3 Statistics, machine learning, deep learning, AI: sorting the terms

You have certainly heard all of these terms – and probably noticed that they are used inconsistently. Product brochures call a linear regression "AI", job advertisements use "data science" and "machine learning" interchangeably, and research communities draw the borders differently. There is no official definition. For this course we therefore fix **working definitions**: not the only possible ones, but clear enough to know what we are talking about.

i Working definitions used in this course

- **Statistics** is the science of learning from data under uncertainty: collecting data (sampling, design of experiments), describing them, and drawing conclusions with a quantified uncertainty (estimates, confidence intervals, tests). Its focus is on *understanding* the process that generated the data.
- **Machine learning (ML)** is the field of algorithms that learn a task from examples instead of being explicitly programmed. Its focus is on *predicting* well on new, unseen data; models are judged by their performance on held-out test data.
- **Deep learning (DL)** is the part of machine learning that uses neural networks with many layers. Such networks learn suitable features by themselves, directly from the raw values – the pixels of an image, the samples of a signal, the words of a text – but typically need large amounts of data and computing power.
- **Artificial intelligence (AI)** is the broadest term: systems that perform tasks which would require intelligence if done by humans – perceiving, reasoning, planning, deciding. Machine learning is today the dominant approach to AI, but not the only one: rule-based expert systems or search and planning algorithms are AI without learning.
- **Data science** is the whole process of turning data into knowledge and decisions – from the question via data acquisition, preparation, analysis and modelling to communication and deployment. It *uses* statistics and machine learning as tools and requires domain knowledge to ask the right questions and interpret the answers.

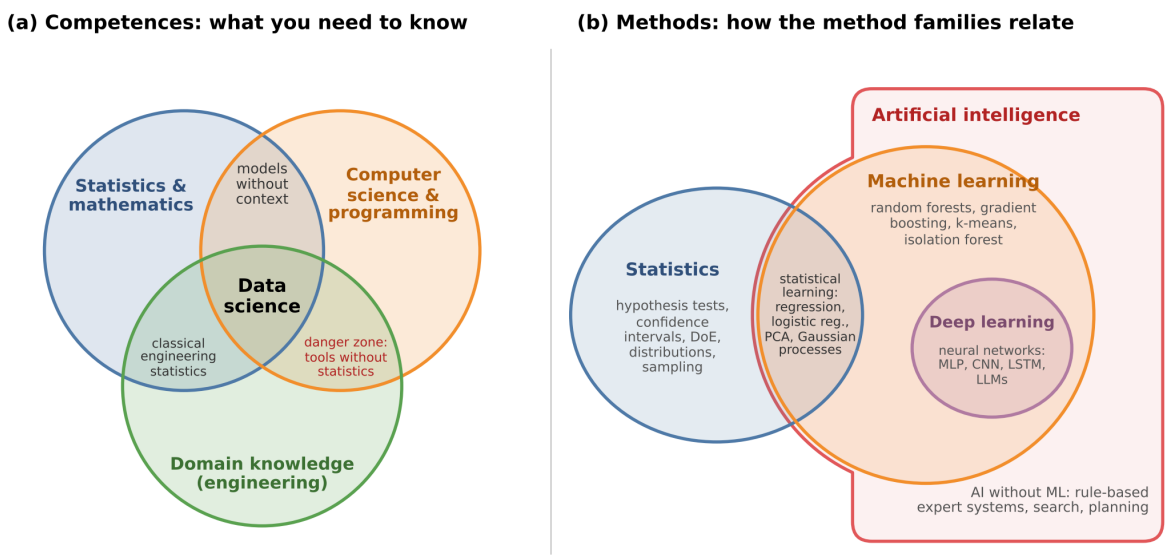


Figure 1.2: Working definitions used in this course. (a) Competences: data science lives where statistics, computer science and domain knowledge meet; any two of them alone fall short. (b) Methods: deep learning is part of machine learning, which is part of AI; statistics overlaps strongly with machine learning ("statistical learning") but also covers methods outside of it, which are not AI.

The two panels answer different questions. Panel (a) asks what you need to *know*: data science combines three **competences**, and leaving out any one of them leads to typical problems – models that answer the wrong engineering question, analyses that do not scale to large data, or powerful tools used without an understanding of uncertainty. Panel (b) asks what kind of *method* you use and arranges the **method families**.

This is why statistics appears in both panels, with different meanings. In (a), it is a competence that every data analysis needs – machine learning included. In (b), it is a family of methods, of which only one part, statistical learning, also belongs to machine learning and thus to AI; a hypothesis test or a confidence interval is statistics, but not AI. Data science itself does not appear in (b) as a separate region, because it is defined by its process and purpose, not by a particular class of methods.

1.3.1 Extension: And what about AI?

"Artificial intelligence" is the oldest and the broadest of the terms – and today the most overloaded one. The field was founded in the 1950s with the goal of making machines perform tasks that require intelligence. For decades, the dominant approach was **symbolic AI**: knowledge was written down by experts as rules ("if bearing temperature $>$ 90 °C and vibration rising,

then …") and combined by logical inference. Such *expert systems* worked in narrow domains but were laborious to build and brittle when reality deviated from the rules.

The alternative is to **learn** the behaviour from data – machine learning. Since around 2012, deep neural networks have achieved breakthroughs in image recognition, speech and text, and today's *generative AI* (large language models such as the chatbots you know, image generators) is deep learning trained on enormous amounts of data. This is why "AI" in everyday language now usually means deep learning – but the term formally covers much more.

Two distinctions help to keep a clear head:

- **Narrow vs. general AI.** All AI systems in practical use, including large language models, are built for specific kinds of tasks. A system that matches humans across all intellectual tasks (*artificial general intelligence*) does not exist and remains a matter of research and debate.
- **Legal vs. technical meaning.** The EU AI Act defines "AI system" broadly – roughly, a machine-based system that infers from its input how to produce outputs such as predictions, recommendations or decisions – and attaches obligations depending on the risk of the application. For engineers this means that even a fairly simple machine learning model in a safety-relevant product can fall under regulation.

In this course we rarely use the word AI. We say precisely which method we use – a test, a regression, a random forest, a neural network – because that determines its assumptions, its limits and how it must be validated.

1.3.2 Where statistics and machine learning differ – and where they overlap

The boundary between statistics and machine learning is the most blurred one, because both fit models to data and many methods belong to both. The difference lies mainly in the **goal** and the **culture** – the statistician Leo Breiman famously spoke of "two cultures":

	Statistics	Machine learning
Main goal	understand and explain: which factors matter, how strong, how certain?	predict accurately on new data
Typical question	"Does the coolant temperature significantly affect the tool wear?"	"Which tool will fail within the next 100 parts?"
Models	few parameters, interpretable, explicit assumptions	flexible, often many parameters, "black box" accepted
Data	often small, carefully designed experiments	often large, observational ("whatever was logged")

	Statistics	Machine learning
Validation	model assumptions, standard errors, tests	performance on held-out test data, cross-validation
Typical output	estimate ± uncertainty, p-value	prediction, score, class

The overlap is large: linear and logistic regression, principal component analysis and Gaussian processes are standard tools in both communities – this common ground is often called **statistical learning**. In this course we use both perspectives: when we fit a regression model in Chapter 5, we look at coefficients and confidence intervals (statistics) *and* at the prediction error on test data (machine learning).

! How this course is positioned

- Chapters 3–5 are mainly **statistics**: distributions, uncertainty, tests, regression, design of experiments.
- Chapters 6–11 are **statistical learning and machine learning**: model selection, classification, ensembles, unsupervised learning, signals and time series.
- Chapter 12 gives a first look at **deep learning**.
- AI methods without learning are not covered. All chapters together follow the **data science** process described next.

⚠ Pitfall: buzzwords instead of methods

When a supplier advertises an "AI-based" monitoring system, ask what is behind it: a threshold rule, a regression model, a random forest or a neural network? How was it validated, on which data, with which error rates? The label says nothing about the quality – the validation does.

1.4 The CRISP-DM process model

Data science projects fail more often for organisational reasons than for mathematical ones: the wrong question was answered, the data did not contain the necessary information, or the model was never used. Process models help to avoid this. The most widely used is CRISP-DM.

i Definition: CRISP-DM

The *Cross-Industry Standard Process for Data Mining* (CRISP-DM) divides a data science project into six phases: business understanding, data understanding, data prepara-

tion, modelling, evaluation and deployment. The phases are passed through iteratively, not strictly one after another.

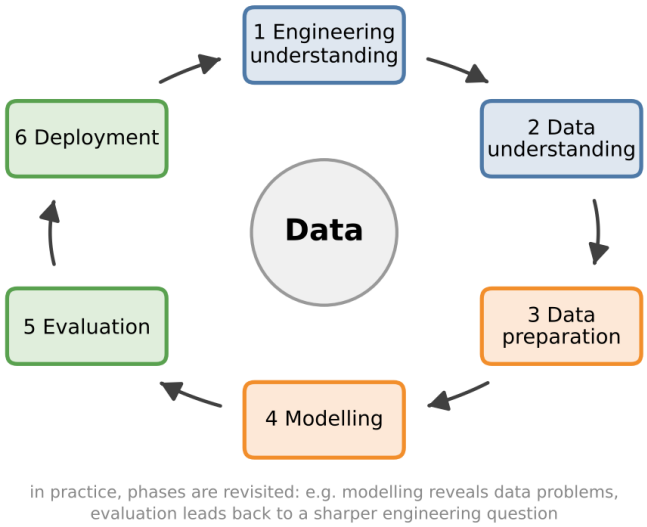


Figure 1.3: The CRISP-DM process model. The phases form a cycle around the data; in practice, the project moves back and forth between them.

What the phases mean in engineering – in general and for the day-ahead commitment of our scenario:

1. **Engineering understanding** (*business understanding* in the original model). Which decision should the analysis support, and what does a wrong answer cost? *Power plant:* which output to commit for each hour of the next day. Every MWh of deviation costs money, so the criterion is economic: switch if the annual savings exceed the annual cost of the temperature-based procedure. At noon, only a temperature *forecast* for tomorrow is available.
2. **Data understanding.** Which data exist, how were they measured, and are the relevant influences measured at all? *Power plant:* 9,568 hourly averages at full load – hours below 420 MW were removed during cleaning, so a model is only valid for this operating state. There are no time stamps, and the temperatures are measured values, not forecasts.
3. **Data preparation.** Cleaning, merging data sources, resampling, handling missing values, constructing features – in practice usually the largest part of a project. *Power plant:* little to do, the table is complete and tidy. In operation, plant records would have to be merged with weather forecasts; Chapter 2 shows the pitfalls of raw weather files.
4. **Modelling.** Choosing and fitting models – always together with a simple baseline. *Power plant:* the baseline is the flat long-term average, the model a straight line for the

output as a function of the ambient temperature.

5. **Evaluation.** Does the model answer the question, under all relevant conditions, and is it better than the baseline? *Power plant:* translate the deviations of both rules into annual costs and compare them (*Back to the decision*).
6. **Deployment.** Integration into a process, a control system or a report – including monitoring whether the model stays valid. *Power plant:* every day before noon, compute the commitment from the forecast and record the actual deviations.

> **Engineering practice: start with a baseline**
>
> Before fitting any sophisticated model, compute a trivial baseline: the mean value, yesterday's value, or a simple physical formula. Every model must beat the baseline – otherwise its complexity is not justified. Surprisingly often, it does not.

> **Pitfall: answering the wrong question**
>
> A model that predicts *when* a machine fails is useless if maintenance can only be scheduled four weeks in advance and the model warns two days ahead. Clarify the decision and its time horizon in phase 1 – it determines which data and models make sense.

1.5 Tools: the Python data stack

Python has become the standard language for data science because of its readable syntax and a large ecosystem of open-source libraries. We use the following packages:

Package	Purpose	Typical objects / functions
NumPy	n-dimensional arrays, linear algebra, random numbers	`np.array`, `np.mean`, `np.linalg`
pandas	tables and time series, reading and writing files	`DataFrame`, `Series`, `read_csv`, `groupby`, `resample`
Matplotlib	plotting (low level, full control)	`plt.plot`, `plt.subplots`
seaborn	statistical plots (high level)	`sns.histplot`, `sns.pairplot`, `sns.boxplot`
SciPy	statistics, signal processing, optimisation	`scipy.stats`, `scipy.signal`, `scipy.optimize`
statsmodels	statistical models with classical output tables	`smf.ols`

Package	Purpose	Typical objects / functions
scikit-learn	machine learning with a uniform interface	`fit`, `predict`, `Pipeline`
PyTorch	neural networks (Chapter 12)	`torch.nn`

The central data structure is the pandas **DataFrame**: a table whose columns have names and data types and whose rows have an index – for time series typically a time stamp. Most operations are *vectorised*: instead of looping over rows, we apply an operation to a whole column at once, which is shorter and much faster.

> **Working with notebooks**
>
> The demo notebooks of the lectures run in Google Colab as well as in a local installation (see the Preface). Packages that are not preinstalled can be installed from a notebook cell with `%pip install`. In Colab, files you download or upload live only as long as the session; save results you want to keep to Google Drive or download them.

```
# Setup - every chapter starts with a cell like this one.

# 1) Install packages that are not pre-installed in Google Colab or Anaconda.
#    The list depends on the chapter (here it is only one package). The loop
#    checks each package and installs it only if it is missing, so running
 ↪  the
#    cell a second time costs nothing.
import importlib.util, subprocess, sys
for pkg in ["ucimlrepo"]:
    if importlib.util.find_spec(pkg) is None:        # package not found?
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg],
 ↪  check=True)

# 2) Import the standard data stack (pre-installed in Colab and Anaconda).
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")                     # common style for all
 ↪  plots
```

1.6 Live coding: a first look at the power plant data

A combined cycle power plant (CCPP) combines a gas turbine with a steam turbine (Figure 1.4). The gas turbine draws in ambient air, compresses it, burns natural gas in it and drives a generator. Its exhaust gas is still hot enough to raise steam in a heat recovery steam generator; the steam drives a second turbine and generator. Behind the steam turbine, the steam is condensed again in a condenser cooled by water or air. The plant that supplied our dataset, located in Turkey, has a nominal capacity of 480 MW: two gas turbines of 160 MW each, two heat recovery steam generators and one steam turbine of 160 MW (Tüfekci 2014).

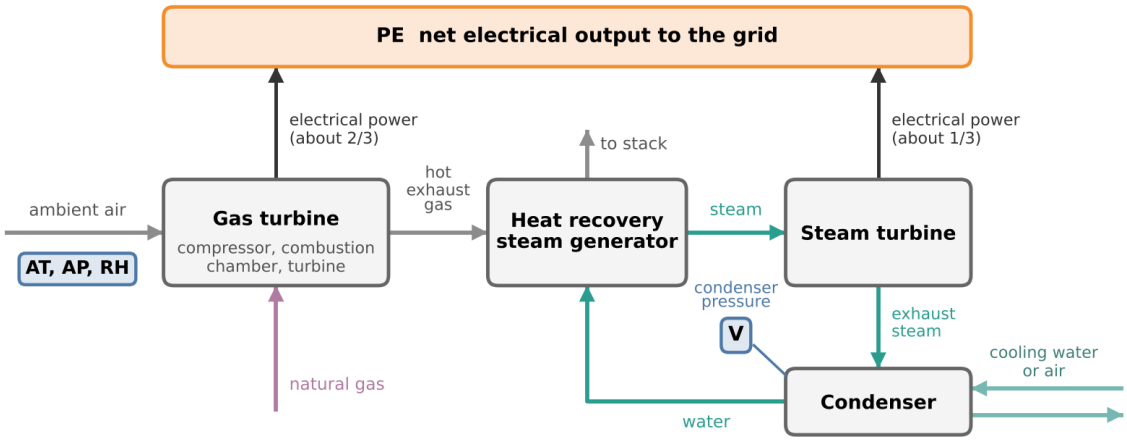


Figure 1.4: Simplified block diagram of a combined cycle power plant with the variables of the dataset. The features (blue) describe the ambient air and the condenser; the target (orange) is the net electrical output of gas turbine and steam turbine together.

Why does the weather matter? Warm weather reduces the output twice:

- **Gas turbine:** warm air is less dense. The compressor draws in a smaller mass of air per second, so less fuel can be burnt and the gas turbine delivers less power.
- **Steam turbine:** a turbine delivers the more power, the larger the pressure difference between its inlet and its outlet. Cold cooling water condenses the steam well and keeps the pressure in the condenser low. On warm days, the cooling water is warmer, the condenser pressure rises and the steam turbine delivers less power.

This is why the condenser pressure is closely linked to the ambient temperature – and why we expect the output to fall when either of them rises.

The dataset contains 9,568 hourly averages from 674 days in six years (2006–2011), all with the plant at full load. The variables are:

Column	Meaning	Unit	Role
AT	ambient temperature	°C	feature
V	exhaust vacuum: pressure in the condenser behind the steam turbine	cm Hg (according to the documentation – see below)	feature
AP	ambient pressure	mbar	feature
RH	relative humidity	%	feature
PE	net electrical output (hourly average)	MW	target

The package `ucimlrepo` downloads datasets from the UCI Machine Learning Repository directly into pandas.

```
from ucimlrepo import fetch_ucirepo

# Download dataset no. 294 (Combined Cycle Power Plant) from the UCI
 ↪  repository.
# The number is part of the address of the dataset page:
# archive.ics.uci.edu/dataset/294/combined+cycle+power+plant
ccpp = fetch_ucirepo(id=294)

# ucimlrepo returns the features (AT, V, AP, RH) and the target (PE) as two
# separate tables (see "Roles of variables"). pd.concat puts them side by
 ↪  side
# into one DataFrame; axis=1 means "join as columns", not as rows.
df = pd.concat([ccpp.data.features, ccpp.data.targets], axis=1)

df.info()   # number of rows, column names, data types, non-missing values
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9568 entries, 0 to 9567
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   AT      9568 non-null   float64
 1   V       9568 non-null   float64
 2   AP      9568 non-null   float64
 3   RH      9568 non-null   float64
 4   PE      9568 non-null   float64
dtypes: float64(5)
```

```
memory usage: 373.9 KB
```

`df.info()` tells us the number of rows, the column names, the data types and the number of non-missing values per column. Check the data types first: pandas stores a column as numbers (`float64`, `int64`) only if *every* entry can be read as a number. A single entry such as `12,5` (decimal comma) or `n/a` (a text marker for a missing value) turns the whole column into text, shown as `object`. Calculations then fail or give nonsense – sorted as text, `"10"` comes before `"9"`. The fix is usually an argument of `read_csv`, e.g. `decimal=","` or `na_values=["n/a"]`. A numeric missing value code such as `-999` does not change the type; it shows up only in the summary statistics (see the weather data in Chapter 2).

Here, all five columns are numeric and complete. The repository also supplies metadata – part of the *context* in the sense of our definition:

```
ccpp.variables   # metadata supplied by the repository: name, role, type,
 ↪  description, unit
```

	name	role	type	demographic	description	units	missing_valu
0	AT	Feature	Continuous	None	in the range 1.81°C and 37.11°C	C	no
1	V	Feature	Continuous	None	in teh range 25.36-81.56 cm Hg	cm Hg	no
2	AP	Feature	Continuous	None	in the range 992.89-1033.30 milibar	milibar	no
3	RH	Feature	Continuous	None	in the range 25.56% to 100.16%	%	no
4	PE	Target	Continuous	None	420.26-495.76 MW	MW	no

Metadata are only as good as the care that went into them – the unit of `V` will show this in a moment.

Next, summary statistics:

```
df.describe().round(2)   # one column per variable; round to 2 decimals for
 ↪  readability
```

	AT	V	AP	RH	PE
count	9568.00	9568.00	9568.00	9568.00	9568.00
mean	19.65	54.31	1013.26	73.31	454.37
std	7.45	12.71	5.94	14.60	17.07
min	1.81	25.36	992.89	25.56	420.26
25%	13.51	41.74	1009.10	63.33	439.75
50%	20.34	52.08	1012.94	74.97	451.55

	AT	V	AP	RH	PE
75%	25.72	66.54	1017.26	84.83	468.43
max	37.11	81.56	1033.30	100.16	495.76

How to read `describe()`

Look at each column with an engineer's eye: Are minimum and maximum physically plausible? Is the mean close to the median (50 %)? A large difference indicates a skewed distribution or outliers. Is the standard deviation large or small compared with the measurement accuracy you would expect?

For the power plant, most ranges are plausible: ambient temperatures from just above freezing to the mid-thirties, ambient pressures around 1013 mbar, and an output of roughly 420 to 500 MW. The relative humidity reaches slightly more than 100 % – physically impossible, but within the accuracy of a humidity sensor near saturation. Worth noticing, not worth correcting.

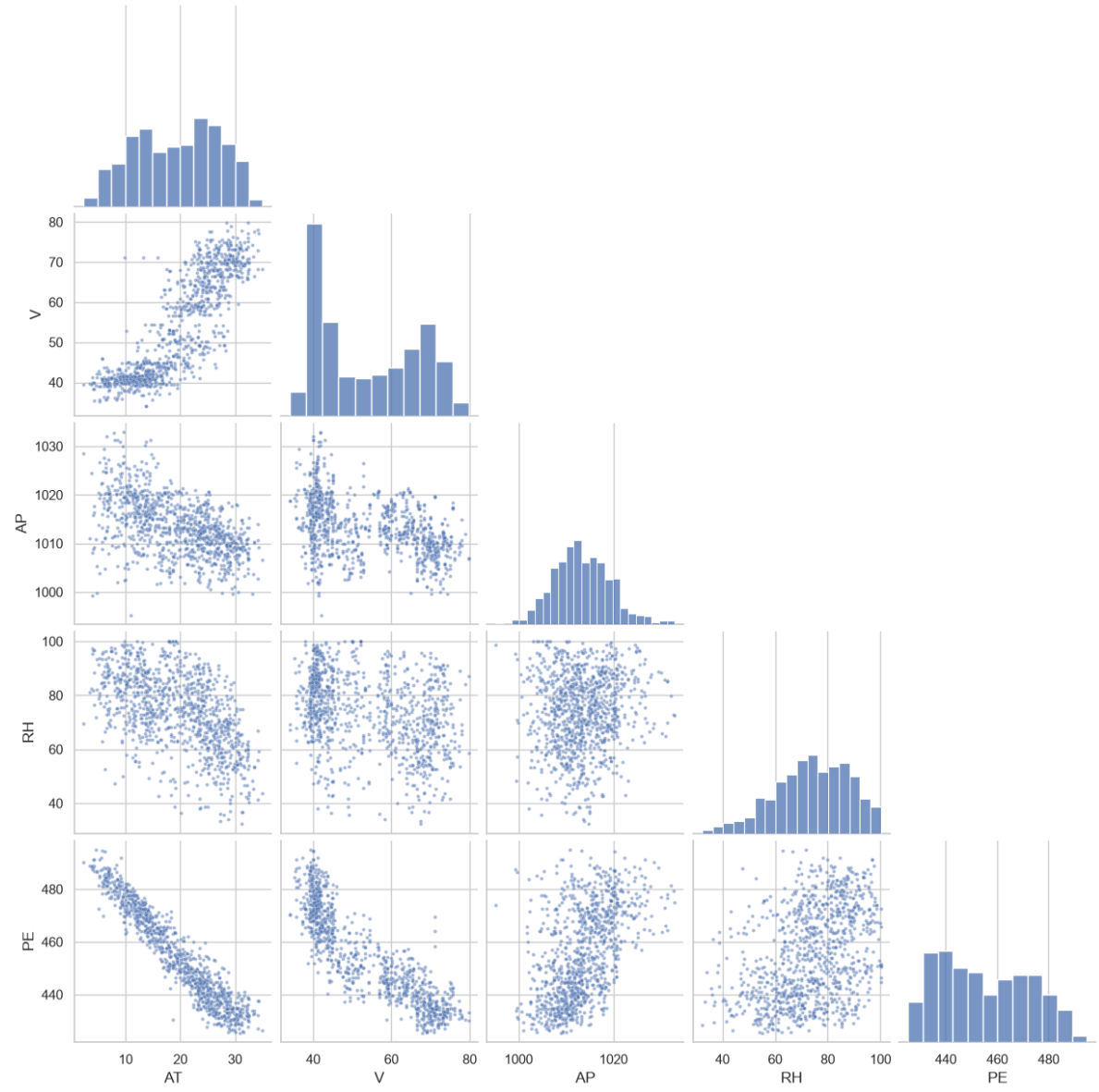
The lower limit of the output is no coincidence: according to the original publication, all hours below 420.26 MW were removed as "out of range" during data cleaning. The dataset therefore contains only hours of high output – a model fitted to it says nothing about weaker operating states.

The exhaust vacuum `V` does not fit, though. According to the documentation – and already the original publication – it is given in cm Hg, but a vacuum of up to 81.6 cm Hg is impossible: the atmosphere itself corresponds to only about 76 cm Hg. Moreover, `V` behaves like a pressure, not like a vacuum: the larger `V`, the lower the output (see the pair plot below). The values would fit the absolute pressure in the condenser if the unit were mm Hg – a single wrong letter – or mbar. The data alone cannot settle this. For predicting `PE`, the unit does not matter; for any physical interpretation, it does. This is exactly what *numbers without context* means.

A pair plot shows all pairwise relationships at once:

```
# Pair plot: a scatter plot for every pair of variables, a histogram of each
# variable on the diagonal.
sns.pairplot(
    df.sample(1000, random_state=0),   # random subset of 1,000 of the 9,568
 ↪  rows
                                       # (avoids overplotting);
                                        ↪  random_state=0 draws
                                       # the same rows every time the cell is
                                        ↪  run
    corner=True,                       # lower triangle only - the upper one
 ↪  would
```

```
                                        # show the same plots mirrored
    plot_kws={"s": 8, "alpha": 0.5},    # small (s = marker size),
↪   semi-transparent
                                        # points (alpha = 0.5), so dense
                                         ↪  regions
                                        # appear darker
)
plt.show()
```

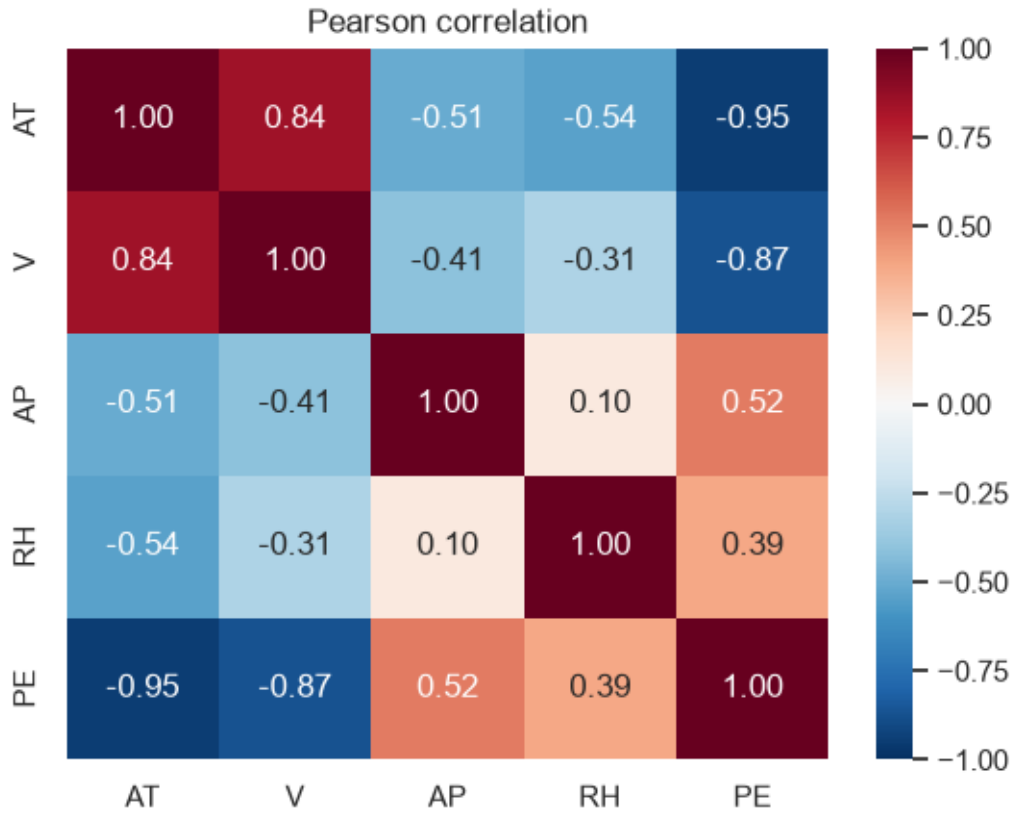


Discussion. The pair plot is the most informative single figure for a small tabular dataset. Several observations stand out:

- `PE` decreases almost linearly with ambient temperature `AT` – this is the strongest relationship in the data and has a clear physical explanation (air density and mass flow).
- `PE` also decreases with the condenser pressure `V`, as expected from the plant (Figure 1.4).
- `AT` and `V` are strongly correlated with each other – warm weather means warmer cooling. Such correlated features (*collinearity*) make it hard to separate their individual effects – we will return to this in Chapter 5.
- Ambient pressure and humidity show weaker, more diffuse relationships with the output.

We plotted a random sample of 1,000 points; with all 9,568 points the scatter plots would be overplotted. Sampling is a simple and legitimate trick for visual exploration – but never for computing the final statistics.

```
corr = df.corr()                          # Pearson correlation of every pair of
 ↪ columns
sns.heatmap(
    corr,
    annot=True, fmt=".2f",                # write each value into its cell, with
 ↪ 2 decimals
    cmap="RdBu_r",                        # diverging colour map: red = positive,
                                          # blue = negative (_r: reversed order
                                           ↪ of "RdBu")
    vmin=-1, vmax=1,                      # colour scale over the full range of
 ↪ r, so that
                                          # white always means r = 0
)
plt.title("Pearson correlation")
plt.show()
```



Definition: Pearson correlation coefficient

For two variables x and y with n observations,

$$r_{xy} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}} \in [-1, 1].$$

It measures the strength of a **linear** relationship: $r = \pm 1$ means all points lie on a straight line, $r = 0$ means no linear relationship.

Pitfall: correlation is not causation – and not every relationship is linear

A correlation near zero does not mean "no relationship": $y = x^2$ on a symmetric interval has $r = 0$. And a strong correlation does not prove a causal effect: both variables may be driven by a third one. In the power plant, `V` correlates with `PE` partly *because* both depend on the ambient temperature.

1.6.1 A first simple model

To make the engineering question concrete, we fit the simplest possible model: a straight line for `PE` as a function of `AT`. We will treat regression properly in Chapter 5; here we only compare the line with the trivial baseline that always predicts the mean.

```
slope, intercept = np.polyfit(df["AT"], df["PE"], deg=1)   # least-squares straight line
                                                            # (polynomial of degree 1)
pred = intercept + slope * df["AT"]                         # predicted output for every hour
rmse = np.sqrt(np.mean((df["PE"] - pred) ** 2))             # root mean squared error
print(f"PE  {intercept:.1f} MW {slope:+.2f} MW/°C · AT,   RMSE = {rmse:.1f} MW")
# The RMSE of the constant prediction "mean" is (almost exactly) the standard deviation.
print(f"Baseline 'always predict the mean': RMSE = {df['PE'].std():.1f} MW")
```

```
PE  497.0 MW -2.17 MW/°C · AT,   RMSE = 5.4 MW
Baseline 'always predict the mean': RMSE = 17.1 MW
```

The line explains most of the variation: its error (root mean squared error, RMSE) is a fraction of the error of the trivial mean-value baseline. Each additional degree Celsius costs about two megawatts of output. Whether an error of a few MW is "good enough" is not a statistical question – it depends on the use case defined in CRISP-DM phase 1.

1.7 Back to the decision

In terms of CRISP-DM, this is phase 5, evaluation: we check the result against the criterion fixed in phase 1. The trading desk compares two rules for the committed output: the flat long-term average and the straight-line estimate from the ambient temperature. To turn the comparison into an economic decision, we use a deliberately simple cost model. All figures are assumptions for illustration:

- **Deviation cost:** every MWh delivered more or less than committed costs 30 € extra. (Real imbalance prices vary strongly from hour to hour and can be much higher.)
- **Operating hours:** the plant runs at full load all 8,760 hours of a year – the data contain full-load hours only.

- **Cost of the new rule:** a weather forecast service, implementation and maintenance cost 25,000 € per year.

The average deviation in MW – the **mean absolute error** (MAE) – times the number of hours gives the deviation in MWh per year, and times the price its cost. We switch if the annual savings exceed the annual cost of the new rule.

```
# Simple cost model - all figures are assumptions for illustration
price_dev = 30          # € per MWh of deviation (extra cost compared with an
 ↪  exact commitment)
hours = 8760            # full-load hours per year (simplification: the plant
 ↪  always runs)
cost_rule = 25_000      # € per year for the temperature-based rule: forecast
 ↪  service,
                        # implementation and maintenance

flat_commitment = df["PE"].mean()                                    # rule
 ↪  1: long-term average
mae_flat = (df["PE"] - flat_commitment).abs().mean()                # mean
 ↪  absolute error (MW)
mae_temp = (df["PE"] - (intercept + slope * df["AT"])).abs().mean()  # rule
 ↪  2: straight line

# MW of deviation x hours = MWh of deviation per year; x price = cost per
 ↪  year
cost_flat = mae_flat * hours * price_dev
cost_temp = mae_temp * hours * price_dev
savings = cost_flat - cost_temp

print(f"Average deviation:  flat {mae_flat:5.1f} MW         temperature-based
 ↪  {mae_temp:5.1f} MW")
print(f"Deviation cost:     flat {cost_flat / 1e6:5.2f} M€ per year
 ↪  temperature-based {cost_temp / 1e6:5.2f} M€ per year")
print(f"Savings: {savings / 1e6:.2f} M€ per year,  cost of the new rule:
 ↪  {cost_rule / 1e6:.3f} M€ per year")
if savings > cost_rule:                                              #
 ↪  criterion from phase 1
    print(f"Decision: switch to the temperature-based estimate "
          f"(net benefit {(savings - cost_rule) / 1e6:.2f} M€ per year).")
else:
    print("Decision: keep the flat commitment - the savings do not cover the
     ↪  costs.")
```

```
# How robust is the decision? Suppose the deviation of rule 2 doubled in
 ↪  operation.
savings_2x = (mae_flat - 2 * mae_temp) * hours * price_dev
print(f"Robustness: with twice the deviation, the savings would still be
 ↪  {savings_2x / 1e6:.2f} M€ per year.")
```

```
Average deviation:  flat  14.8 MW         temperature-based   4.3 MW
Deviation cost:     flat  3.90 M€ per year  temperature-based  1.13 M€ per year
Savings: 2.77 M€ per year,  cost of the new rule: 0.025 M€ per year
Decision: switch to the temperature-based estimate (net benefit 2.75 M€ per year).
Robustness: with twice the deviation, the savings would still be 1.65 M€ per year.
```

Two caveats belong in the recommendation to the team lead. First, we evaluated the rule on the same data we used to fit it; Chapter 6 shows how to evaluate on unseen data. Second, in operation the rule receives a temperature *forecast*, whose error adds to the error of the model. Both points make the real benefit smaller. The last line of the output shows how much room there is: even if the deviation of the temperature-based rule doubled in operation, the savings would still exceed the cost of the rule many times over. And the approach is not a textbook fiction: the plant that supplied the data uses such a model for its next-day commitment, with the temperature forecast of the national weather service as input, and reports a mean relative error below 2 % in operation (Tüfekci 2014).

The cost model is a simplification, too – Exercise 6 asks you to question it. Its value lies elsewhere: it turns "the error is smaller" into "the switch is worth this much per year", which is the language in which the team lead has to decide.

! Key takeaways

- Data are recorded values of variables, observed on observational units, together with their context. Without unit and meaning, numbers cannot be interpreted – as the condenser pressure `V` shows.
- Type and role of a variable determine which summaries and models make sense.
- Every data science project starts with a precise engineering question and a clear criterion for success (CRISP-DM phase 1) – ideally expressed in money, as in the cost model of the power plant.
- Look at the data before modelling: `info()`, `describe()`, pair plots and correlations reveal structure and problems quickly.
- Always compare a model with a simple baseline.

1.8 Exercises

1. Which columns of the power plant data are features, which is the target? Which physical relationships between them do you expect, and which signs of the correlations follow from them?
2. Fit straight lines for `PE` against each of the four features separately and compare their RMSE. Which single feature is the best predictor? Does the ranking agree with the correlation matrix?
3. Sketch the six CRISP-DM phases for a project from your own engineering background. Which phase do you expect to be the most difficult?
4. Place the following in Figure 1.2 (b) and justify: (a) a t-test whether a new supplier changes the scrap rate; (b) a convolutional neural network that detects cracks in X-ray images; (c) route planning for automated guided vehicles with the A* search algorithm; (d) a random forest that predicts tool breakage; (e) a straight line for power output vs. temperature with a confidence interval for the slope.
5. Which of the following are data in the sense of our definition? (a) The torque values logged by a screwdriving station for every screw joint; (b) the maximum torque in the screwdriver's datasheet; (c) a table of thread friction coefficients computed with a formula from a handbook; (d) 10,000 lifetimes generated by a reliability simulation; (e) a column `T` with values between 280 and 320, without any further information.
6. Use the cost model from *Back to the decision*. (a) At which deviation price would the savings just cover the cost of the new rule (break-even)? (b) How large may the average deviation of the temperature-based rule become in operation before switching no longer pays off? (c) Which simplifications of the cost model would you question first, and in which direction would they change the result?

1.8.1 Solutions

Solution to Exercise 1

The features are the ambient conditions `AT`, `AP`, `RH` and the condenser pressure `V`; the target is `PE`. Strictly speaking, `V` is not an ambient condition but a plant variable – it depends on the cooling water temperature and therefore also on the weather (Figure 1.4). Expected signs: higher ambient temperature lowers air density and thus the mass flow and power of the gas turbine → negative correlation of `AT` with `PE`. A higher condenser pressure (worse vacuum, larger `V` in this dataset) reduces the pressure difference across the steam turbine → negative correlation of `V` with `PE`. Higher ambient pressure increases air density → positive correlation. Humidity has a smaller, less obvious effect. The correlation matrix confirms these signs.

Solution to Exercise 2

```
rows = []
for col in ["AT", "V", "AP", "RH"]:                          # one straight
 ↪  line per feature
    b1, b0 = np.polyfit(df[col], df["PE"], deg=1)            # slope b1,
 ↪  intercept b0
    rmse_col = np.sqrt(np.mean((df["PE"] - (b0 + b1 * df[col])) ** 2))
    rows.append({"feature": col, "slope": b1, "RMSE (MW)": rmse_col, "r":
 ↪  df[col].corr(df["PE"])})
pd.DataFrame(rows).sort_values("RMSE (MW)").round(3)         # best feature
 ↪  first
```

	feature	slope	RMSE (MW)	r
0	AT	-2.171	5.425	-0.948
1	V	-1.168	8.421	-0.870
2	AP	1.490	14.594	0.518
3	RH	0.456	15.716	0.390

`AT` gives by far the smallest RMSE, followed by `V`. For a straight-line fit, the ranking by RMSE is exactly the ranking by $|r|$, because the fraction of explained variance of a simple linear regression is r^2. The original publication reports practically the same values (Tüfekci 2014).

Solution to Exercise 3

There is no single correct answer. A good sketch states a concrete decision in phase 1 (e.g. "decide weekly which tools to replace"), names the data sources and their owners in phase 2, lists expected preparation steps in phase 3 (merging, time alignment), names a baseline in phase 4, an evaluation criterion tied to the decision in phase 5 (e.g. cost of unplanned stops), and a realistic deployment path in phase 6. Many practitioners name phase 2/3 (data access and quality) or phase 6 (deployment and acceptance) as the hardest.

Solution to Exercise 4

(a) **Statistics** – inference about a difference with quantified uncertainty, no prediction task.
(b) **Deep learning** (and thus ML and AI) – a neural network learns features from raw

images.
(c) **AI without machine learning** – A* is a search algorithm; nothing is learned from data.
(d) **Machine learning** – a flexible model optimised for predictive accuracy; it is not deep learning.
(e) The **overlap of statistics and machine learning**: the same linear model, used here in the statistical way (interpretation of the slope and its uncertainty). Used purely to predict output for the day-ahead market and validated on test data, it would be viewed as machine learning.
The examples show that the method alone does not determine the category – the purpose matters too.

Solution to Exercise 5

(a) **Data** – recorded observations of real screw joints (provided we know the unit and which joint each value belongs to).
(b) **Not data** – a specification of what the tool is designed for, not an observation.
(c) **Not data** about any real joint – the table only re-expresses the formula; a model fitted to it would rediscover the formula.
(d) **Data about a model** – valid observations of the simulation; conclusions hold for real products only as far as the simulation is valid.
(e) **Not usable as data** – the numbers might be temperatures in kelvin, but without unit, sensor and time stamp they cannot be interpreted. They become data once the context is recovered.

Solution to Exercise 6

```
# (a) price at which the savings equal the cost of the rule
price_break = cost_rule / ((mae_flat - mae_temp) * hours)
# (b) deviation of rule 2 at which the savings equal the cost of the rule
mae_break = mae_flat - cost_rule / (hours * price_dev)
print(f"(a) break-even price:     {price_break:.2f} € per MWh (assumed:
 ↪  {price_dev} € per MWh)")
print(f"(b) break-even deviation: {mae_break:.1f} MW (today:
 ↪  {mae_temp:.1f} MW, flat rule: {mae_flat:.1f} MW)")
```

```
(a) break-even price:     0.27 € per MWh (assumed: 30 € per MWh)
(b) break-even deviation: 14.7 MW (today: 4.3 MW, flat rule: 14.8 MW)
```

(a) Even a deviation price far below 1 € per MWh would cover the cost of the rule – real prices are much higher.

(b) The rule pays off as long as it is even slightly better than the flat commitment: the break-even deviation lies only about 0.1 MW below that of the flat rule. With well over 100,000 MWh of deviation per year, even small improvements are worth a lot of money.
(c) For example: deviations in the two directions are usually priced differently, and the price depends on the state of the grid; the plant does not run at full load in every hour (maintenance, partial load), so fewer hours mean smaller savings; the forecast error of the temperature increases the deviation of the new rule; and the cost of the new rule may be underestimated (staff time, IT). None of these is likely to reverse the decision here – but in a project with a smaller margin, each of them could.

2 Exploratory Data Analysis and Data Quality

Exploratory data analysis (EDA) is the systematic first look at a dataset before any model is built. Its purpose is twofold: to understand the structure of the data – distributions, relationships, patterns over time – and to find problems – gaps, outliers, implausible values, inconsistencies. In engineering, a large share of failed data science projects can be traced back to data problems that a careful EDA would have revealed.

The statistician John Tukey, who coined the term, put it this way: EDA is detective work. We look for clues without knowing in advance what we will find. The tools are simple – summary statistics and plots – but using them well requires domain knowledge.

In terms of the CRISP-DM process from Chapter 1, EDA is the core of phase 2, *data understanding*. The original CRISP-DM model names four tasks for this phase, and this chapter follows them: **collect** the data (Section 2.1), **describe** them (Section 2.2), **explore** them (Section 2.3) and **verify their quality** (Section 2.4 and Section 2.5). The findings determine what happens in phase 3, *data preparation* – which outliers to keep, how to fill gaps, which sensors to drop. And they often lead back to phase 1: can the question be answered with these data at all? The scenario of this chapter is exactly such a decision.

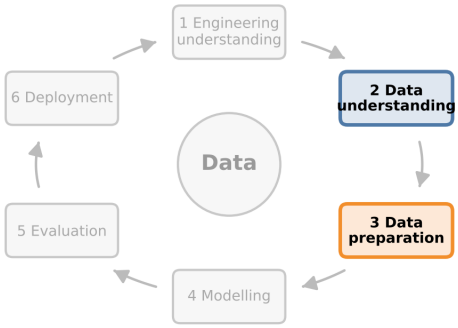


Figure 2.1: Where this chapter belongs in the CRISP-DM process: data understanding and data preparation.

Scenario: go or no-go for a failure prediction project

A semiconductor fab wants to predict failed production runs from its process data. A project budget for one year has been requested, and the fab manager asks you for a quick assessment

before signing: “We have almost 600 sensors and more than 1,500 runs – that must be enough, right?” You have one week to look at the data.

Your decision: Recommend starting the project now, or first investing in better data?

Learning goals. After this chapter you can

- read a raw sensor file correctly (separator, missing value code, time stamps, time zone),
- describe distributions numerically (location, spread, shape) and visually (histogram, box plot),
- explore a sensor dataset over time with rolling means, resampling and heatmaps,
- detect outliers with the IQR rule and robust z-scores and decide how to treat them,
- characterise missing data (MCAR, MAR, MNAR) and choose an appropriate treatment,
- judge whether a dataset is fit for the intended analysis.

Datasets in this chapter

- DWD Climate Data Center – hourly station data
- Appliances Energy Prediction (UCI)
- SECOM (UCI)

2.1 Reading raw data

Before data can be described, they have to be read – and read correctly. Real measurement data rarely come as a ready-made table: they arrive as text files with their own separators, codes for missing values and time formats. This section shows two typical cases – a raw file from a weather service and a sensor dataset with an unusual time format. First, the usual setup:

```
# Setup (as in every chapter): install packages that are not pre-installed in
# Google Colab or Anaconda, then import the standard data stack.
import importlib.util, subprocess, sys
for pkg in ["ucimlrepo"]:
    if importlib.util.find_spec(pkg) is None:        # package not found?
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg],
 ↪  check=True)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")                     # common style for all
 ↪  plots
```

2.1.1 Live coding: a raw weather file

In Chapter 1, the trading desk of the power plant needed temperature data to commit its output – and weather services deliver such data as raw files. As an example, we load hourly air temperature and humidity from the German Weather Service (DWD), station Trier-Petrisberg (station ID 05100). The power plant of Chapter 1 is not located in Trier, and the file contains measured values, not forecasts – but the steps are the same for any weather service and any raw sensor file.

The DWD provides the data as a zip archive containing a semicolon-separated text file plus metadata files. Missing values are coded as `-999`. This is typical for raw data: we have to know the file structure, the separator, the missing value code and the time format before we can use the data. All of this is documented: the DWD describes the dataset in a dataset description (columns, formats, the products `recent` and `historical`), and every archive contains metadata files with further details, e.g. on missing values. Reading this documentation is part of the task *collect* in CRISP-DM phase 2. When reading the file, we replace the code `-999` by `NaN` ("not a number"), the marker that pandas uses for a missing value. Otherwise, `-999` would count as a real temperature of −999 °C and distort every mean.

Because the file is only needed once, we use a small helper function `download()`: it fetches a file from the internet on the first run and keeps a copy in the folder `data`; every later run reads this copy.

```
from pathlib import Path
import requests

DATA = Path("data")
DATA.mkdir(exist_ok=True)                          # create the folder ./data
 ↪  if it does not exist

def download(url, fname=None):
    """Download a file once and cache it in ./data."""
    if fname is None:                              # no file name given:
        fname = url.split("/")[-1]                 # use the last part of the
 ↪  URL
    path = DATA / fname                            # "/" joins folder and file
 ↪  name
    if not path.exists():                          # download only if not
     ↪  cached yet
        r = requests.get(url, timeout=120)         # give up after 120 s
 ↪  without response
```

```
        r.raise_for_status()                        # stop with an error if the
 ↪  download failed
        path.write_bytes(r.content)                 # open, write and close the
 ↪  file in one step
    return path
```

```
import zipfile

# The URL is long; we build it from three parts with "+":
server  =
 ↪  "https://opendata.dwd.de/climate_environment/CDC/observations_germany/"
product = "climate/hourly/air_temperature/recent/"   # hourly air
 ↪  temperature, recent data
fname   = "stundenwerte_TU_05100_akt.zip"           # 05100 = station ID of
 ↪  Trier-Petrisberg
url = server + product + fname

with zipfile.ZipFile(download(url)) as z:       # open the archive (closed
 ↪  after the block)
    print(z.namelist())                         # what is inside?
    # The data file is the one whose name starts with "produkt". Its full
     ↪  name contains
    # the dates of the measurement period and changes daily, so we search for
     ↪  it:
    for n in z.namelist():
        if n.startswith("produkt"):
            name = n
            break                               # found: stop searching
    dwd = pd.read_csv(z.open(name),             # read the file directly from
 ↪  the archive
                      sep=";",                  # columns are separated by
 ↪  semicolons
                      na_values=[-999],         # replace the code -999 by
 ↪  NaN ("missing")
                      skipinitialspace=True)    # ignore blanks after each
 ↪  separator

dwd.columns = dwd.columns.str.strip()           # remove blanks around the
 ↪  column names
dwd.head()
```

```
['Metadaten_Stationsname_Betreibername_05100.html', 'Metadaten_Stationsname_Betreibername_05
```

	STATIONS_ID	MESS_DATUM	QN_9	TT_TU	RF_TU	eor
0	5100	2025041100	3	4.9	83.0	eor
1	5100	2025041101	3	4.5	85.0	eor
2	5100	2025041102	3	3.6	88.0	eor
3	5100	2025041103	3	2.8	91.0	eor
4	5100	2025041104	3	1.8	92.0	eor

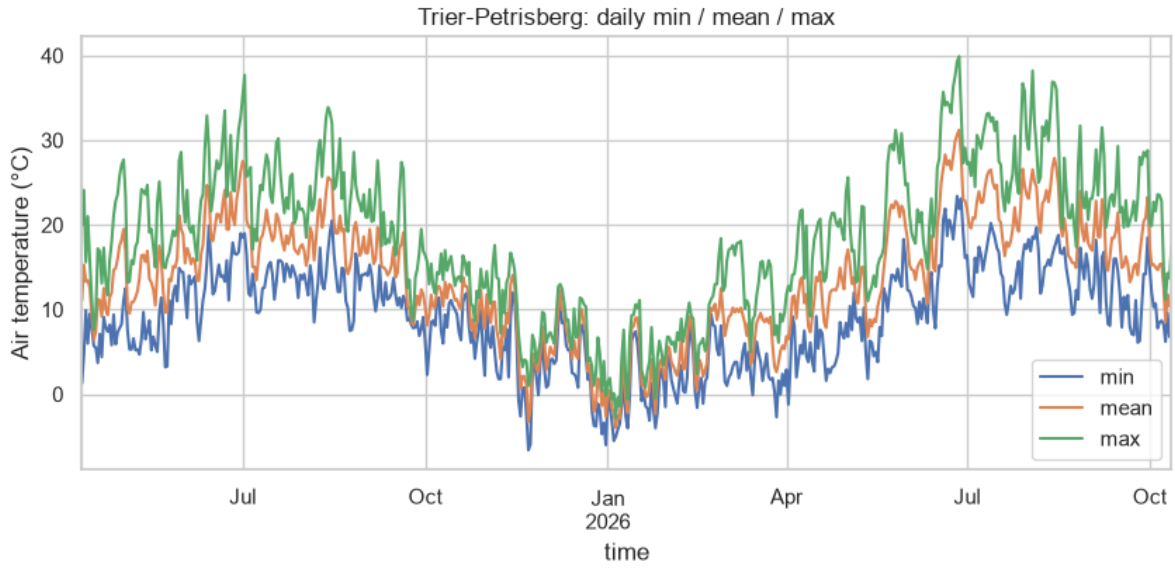
The raw table has cryptic column names (`TT_TU` = air temperature in °C, `RF_TU` = relative humidity in %, `QN_9` = quality level). `STATIONS_ID` and `QN_9` are stored as numbers but are codes, i.e. categorical variables (Chapter 1). The time stamp `MESS_DATUM` is an integer such as `2025041113` (year, month, day, hour). We convert the time stamp into a proper `datetime`, use it as the index and give the columns meaningful names. The format codes such as `%Y` for the year are listed in Appendix A.

```
# MESS_DATUM is an integer such as 2025041113 (year, month, day, hour):
# convert it to text and parse it with the matching format into a real time
 ↪  stamp.
# Format codes: %Y = year (4 digits); %m = month, %d = day, %H = hour (2
 ↪  digits each).
dwd["time"] = pd.to_datetime(dwd["MESS_DATUM"].astype(str),
 ↪  format="%Y%m%d%H")
weather = (dwd.set_index("time")[["TT_TU", "RF_TU"]]           # time as
 ↪  index, two columns
              .rename(columns={"TT_TU": "temp_C", "RF_TU":
 ↪  "rel_humidity_pct"}))
print(weather.isna().sum())     # number of missing values per column
weather.tail()                  # last five rows
```

```
temp_C              19
rel_humidity_pct    19
dtype: int64
```

time	temp_C	rel_humidity_pct
2026-10-12 19:00:00	12.3	71.0
2026-10-12 20:00:00	11.4	74.0
2026-10-12 21:00:00	10.9	76.0
2026-10-12 22:00:00	9.9	82.0
2026-10-12 23:00:00	9.8	78.0

```
# resample("D") groups the hourly values by day; agg computes three
 ↪  statistics per day.
daily = weather["temp_C"].resample("D").agg(["min", "mean", "max"])
ax = daily.plot(figsize=(10, 4))                  # one line per column: min,
 ↪  mean, max
ax.set(ylabel="Air temperature (°C)", title="Trier-Petrisberg: daily min /
 ↪  mean / max")
plt.show()
```



Discussion. With a time index, pandas offers powerful operations: `resample("D")` aggregates hourly values to days, and we can compute several statistics at once. The plot shows the seasonal cycle and the day-to-day weather variability. Note the time zone: the DWD files of this product use UTC. When you combine such data with local production data, time zones and daylight saving time must be handled explicitly – a common source of one-hour shifts.

The output above also reports a few missing hours – the former `-999` codes, now `NaN`. Why they are missing is documented in the metadata file `Metadaten_Fehldaten_…` in the archive. Reading it is the first step before choosing a treatment (Section 2.5).

Engineering practice: data from databases

In companies, engineering data often come from databases rather than files – process data from a historian, quality and order data from a manufacturing execution system. They are read with SQL: a few commands such as `SELECT`, `WHERE`, `GROUP BY` and `JOIN` cover most needs, and `pd.read_sql` loads the result directly into pandas. The questions

of this section remain the same: units, missing values (in SQL: `NULL`) and time zones. Appendix B gives a short introduction.

2.1.2 Live coding: the energy data of a house

Our second dataset accompanies us through the next three sections. It was recorded in a low-energy house in Belgium. A wireless sensor network measured temperature and humidity in nine rooms every 10 minutes for about 4.5 months; the electricity consumption of the appliances was logged by an energy meter, and outdoor weather was taken from a nearby airport. Key columns:

Column	Meaning	Unit
`Appliances`	energy use of appliances in the 10-minute interval	Wh
`lights`	energy use of light fixtures	Wh
`T1` … `T9`, `RH_1` … `RH_9`	temperature and relative humidity in the rooms	°C, %
`T_out`, `RH_out`, `Windspeed`, …	outdoor weather	various
`rv1`, `rv2`	random variables, added deliberately by the authors	–

The two random variables are a nice feature of this dataset: a sound analysis should find that they carry no information.

```
# Helper: parse time stamps with an explicit format. Several common formats
 ↪  are tried one
# after the other; a format is accepted only if it parses *all* time stamps.
def parse_timestamps(s, formats=("%Y-%m-%d %H:%M:%S", "%Y-%m-%d%H:%M:%S",
 ↪  "%Y-%m-%d %H:%M", "%d-%m-%Y %H:%M:%S",
                                  "%d-%m-%Y %H:%M", "%d/%m/%Y %H:%M",
                                   ↪  "%m/%d/%Y %H:%M",
                                  "%d.%m.%Y %H:%M")):
    for fmt in formats:
        t = pd.to_datetime(s, format=fmt, errors="coerce")   #
 ↪  errors="coerce": NaT instead of
                                                              # an error if a
                                                               ↪  string does
                                                               ↪  not fit
        if t.notna().all():                                  # this format
         ↪  fits every time stamp
```

```
            print("Time stamp format:", fmt)
            return t
    # no format fits: show some of the problematic strings in the error
     ↪ message
    bad = s[pd.to_datetime(s, format="mixed", errors="coerce").isna()]
    raise ValueError(f"Unknown time stamp format, e.g. {bad.head(3).tolist()
     ↪ or s.head(3).tolist()}")
```

```
from ucimlrepo import fetch_ucirepo

appl = fetch_ucirepo(id=374)                          # Appliances Energy
 ↪ Prediction (UCI)
energy = appl.data.original.copy()                    # whole table incl. time,
 ↪ as a copy

raw = energy["date"].astype(str).str.strip()          # look at the raw time
 ↪ stamps first ...
print("Raw time stamps:", raw.head(3).tolist())       # first three, as a plain
 ↪ list
energy["date"] = parse_timestamps(raw)                # ... then parse them with
 ↪ an explicit format
energy = energy.set_index("date").sort_index()        # time as index, in
 ↪ chronological order

# rows and columns, first and last time stamp:
print(energy.shape, energy.index.min(), "to", energy.index.max())
# isna() gives True where a value is missing; True counts as 1, so sum()
 ↪ counts
# the missing values per column, and the second sum() adds up all columns:
print("Missing values:", energy.isna().sum().sum())
# Regular grid: is every time step exactly 10 minutes?
steps = energy.index.to_series().diff().dropna()      # time between consecutive
 ↪ rows
print("Regular 10-min grid:", (steps == pd.Timedelta("10min")).all())
```

```
Raw time stamps: ['2016-01-1117:00:00', '2016-01-1117:10:00', '2016-01-1117:20:00']
Time stamp format: %Y-%m-%d%H:%M:%S
(19735, 28) 2016-01-11 17:00:00 to 2016-05-27 18:00:00
Missing values: 0
Regular 10-min grid: True
```

Pitfall: letting the software guess date formats

Is `03/04/2016` the 3rd of April or the 4th of March? If you let pandas guess, it may parse some time stamps one way and others the other way – or fail on a single unusual entry. Always look at the raw strings and give the format explicitly. Because the measurement period contains days after the 12th, only the correct order of day and month can parse *all* time stamps – the helper `parse_timestamps` (folded above) uses exactly this check.

A real-world surprise: in the version of this dataset served by the UCI repository, date and time are glued together without a space (`2016-01-1117:00:00` for 11 January 2016, 17:00). pandas cannot guess this format – but the explicit format `%Y-%m-%d%H:%M:%S` reads it without problems.

Checking that the time index is regular is a quick but important test. Gaps in the time grid are "hidden" missing values: `df.isna()` does not see them because the rows do not exist at all.

Good practice: downloading and reading raw files

1. **Keep the raw file untouched.** Never correct values by hand in the file. Every step from the raw file to the clean table is code – so anyone can repeat it, including you next semester.
2. **Download once, keep a copy** – like the helper `download()`. Check that the download succeeded (`raise_for_status()`); otherwise an error page may be stored as "data".
3. **Note source and version:** URL, download date, version. Many sources are updated: the DWD product `recent` covers roughly the last 500 days and changes daily, so the same code can give slightly different results next month.
4. **Read the documentation first:** separator, decimal mark, missing value code, units, time format and time zone.
5. **State everything explicitly** instead of letting pandas guess: `sep=";"`, `decimal=","` (numbers with a decimal comma, common in German files), `na_values=[-999]`, `format=` for time stamps.
6. **Check right after reading:** `df.head()`, `df.info()`, `df.describe()`. A numeric column of type `object` contains at least one text entry; a table with only one column or unexpectedly many missing values points to a wrong separator or code.

2.2 Describing distributions

Which description fits a single variable depends on its type (Chapter 1):

- A **categorical** variable is described by the **frequencies** of its categories – counts or shares – and shown in a bar chart. The most frequent category is called the **mode**. For an *ordinal* variable, the median is meaningful as well (quality class M lies in the middle), the mean is not.
- A **numeric** variable is described by its **location** (where are the values?), its **spread** (how much do they vary?) and its **shape** (symmetric, skewed, multimodal?).

Engineering data are mostly numeric, and so is the rest of this section. A categorical example – the pass/fail label of this chapter's scenario – follows in Section 2.5.

Definition: Location and spread

For numeric observations $x_1, \dots, x_n$:

- **Mean** $\bar{x} = \frac{1}{n}\sum_i x_i$ and **median** (the middle value of the sorted data).
- **Sample standard deviation** $s = \sqrt{\frac{1}{n-1}\sum_i (x_i - \bar{x})^2}$.
- **Quartiles** Q_1, Q_3 (25 % and 75 % quantiles) and the **interquartile range** $\mathrm{IQR} = Q_3 - Q_1$.
- **Median absolute deviation:** with the median $\tilde{x} = \mathrm{median}(x_1, \dots, x_n)$,

$$\mathrm{MAD} = \mathrm{median}\left(|x_1 - \tilde{x}|, \dots, |x_n - \tilde{x}|\right),$$

  i.e. the median of the absolute deviations from the median.

Mean and standard deviation are sensitive to extreme values: a single wrong reading of 10^6 can dominate both. Median, IQR and MAD are **robust**: they barely change if a few values are wrong. For engineering data, where sensor glitches are common, it pays to look at both and compare them.

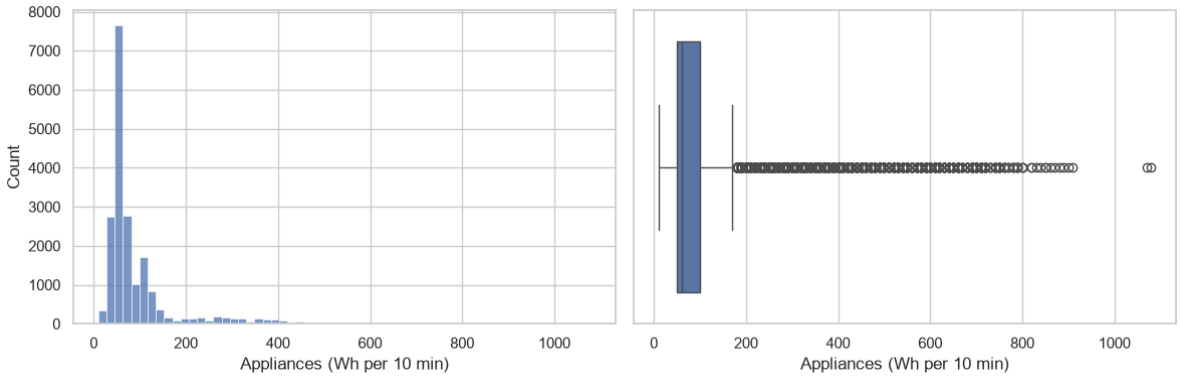
Which plot for which question?

- *Frequencies of a categorical variable:* bar chart – sorted by frequency, or in the natural order of an ordinal variable.
- *Distribution of a numeric variable:* histogram (shape) or box plot (compact, good for comparing groups).
- *Relationship between two variables:* scatter plot; with many points use transparency or hexbin plots.
- *Development over time:* line plot, possibly with a rolling mean.
- *Patterns over two time scales* (hour of day × weekday): heatmap.
- *Many variables at once:* correlation heatmap, pair plot of a subset.

2.2.1 Live coding: appliance energy use

The target variable of the energy dataset is the energy use of the appliances. Histogram and box plot show its distribution:

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))          # two plots side by
 ↪  side
sns.histplot(energy["Appliances"], bins=60, ax=axes[0])  # histogram with 60
 ↪  bins (left)
axes[0].set(xlabel="Appliances (Wh per 10 min)")
sns.boxplot(x=energy["Appliances"], ax=axes[1])          # box plot (right);
 ↪  x= draws it horizontally
axes[1].set(xlabel="Appliances (Wh per 10 min)")
plt.tight_layout(); plt.show()                           # tight_layout:
 ↪  avoid overlapping labels
energy["Appliances"].describe().round(1)                 # the same
 ↪  distribution in numbers
```




```
count    19735.0
mean        97.7
std        102.5
min         10.0
25%         50.0
50%         60.0
75%        100.0
max       1080.0
Name: Appliances, dtype: float64
```

Discussion. The distribution is strongly **right-skewed**: most of the time the consumption is low (base load from fridge, standby), but occasionally it rises sharply (cooking, washing

machine). Consequently, the mean is clearly larger than the median, and the box plot flags many points as "outliers" (Section 2.4). Here these are not measurement errors but real usage events – deleting them would remove the most interesting part of the data.

For skewed, positive quantities a **log transformation** often helps: it makes the distribution more symmetric and turns multiplicative effects into additive ones.

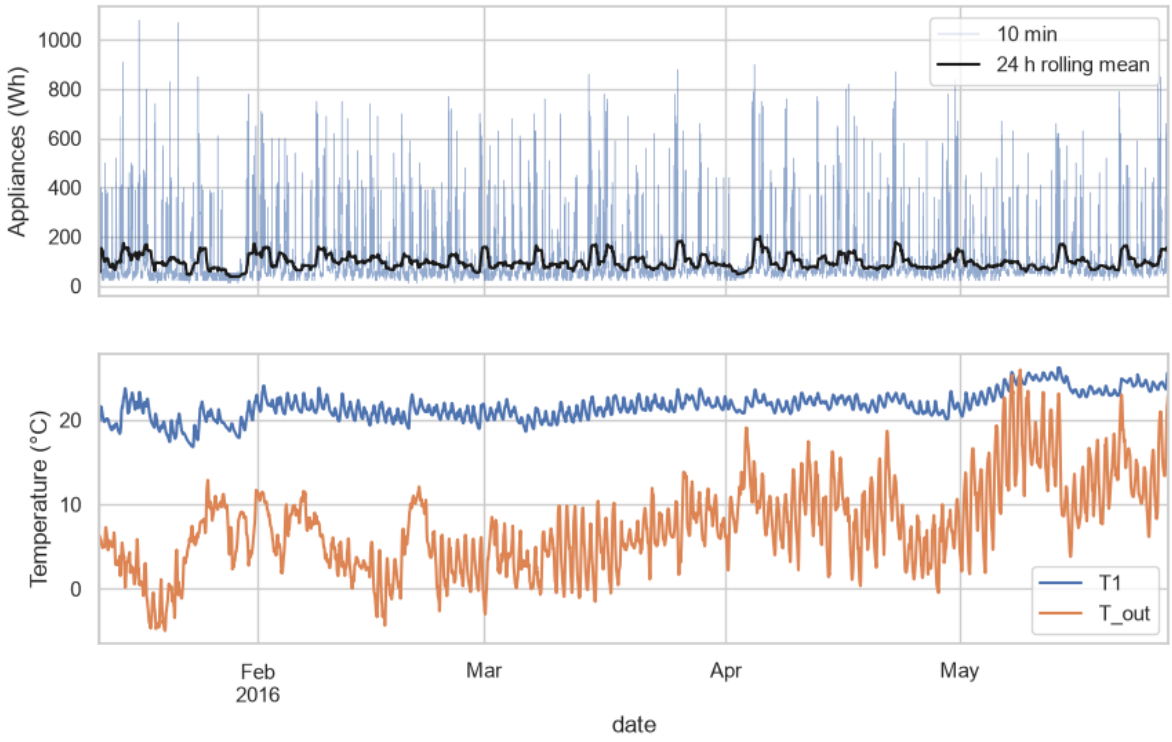
2.3 Patterns over time

Engineering data are mostly time series. Besides the distribution of each variable, its development over time matters: trends, cycles, sudden steps. Two tools help to see them in noisy data: a **rolling mean** smooths short-term fluctuations, and **resampling** aggregates the values to a coarser time grid, e.g. from 10 minutes to hours.

2.3.1 Live coding: daily and weekly profiles

First the whole measurement period: the 10-minute values of the appliance energy use with a 24-hour rolling mean, and the indoor and outdoor temperature as hourly means:

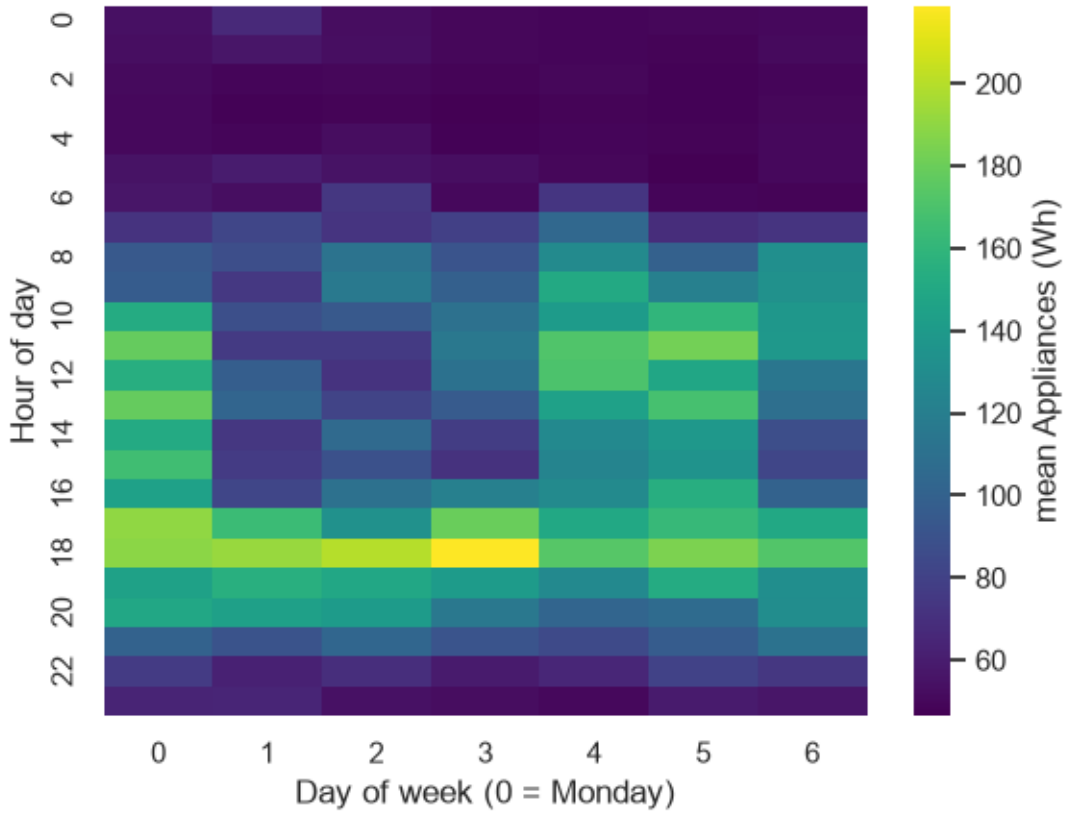
```
fig, axes = plt.subplots(2, 1, figsize=(10, 6), sharex=True)   # two plots,
 ↪  one time axis
# top: 10-minute values (thin, transparent line) and their 24-hour rolling
 ↪  mean
energy["Appliances"].plot(ax=axes[0], lw=0.3, alpha=0.6, label="10 min")
energy["Appliances"].rolling("1D").mean().plot(ax=axes[0], color="k",
 ↪  label="24 h rolling mean")
axes[0].set(ylabel="Appliances (Wh)"); axes[0].legend()
# bottom: indoor and outdoor temperature, resampled from 10 minutes to hourly
 ↪  means
energy[["T1", "T_out"]].resample("h").mean().plot(ax=axes[1])
axes[1].set(ylabel="Temperature (°C)")
plt.show()
```



The rolling mean reveals the level of consumption over time. Indoor temperature `T1` is much more stable than the outdoor temperature – as expected for a well-insulated building.

Consumption depends on human behaviour, so we expect patterns over the hour of the day and the day of the week. A heatmap shows both at once:

```
# Mean consumption for every combination of weekday (0 = Monday) and hour of
 ↪ day.
# unstack(0) turns the weekday into columns: a table with 24 rows (hours) and
 ↪ 7 columns.
profile = energy.groupby([energy.index.dayofweek,
 ↪ energy.index.hour])["Appliances"].mean().unstack(0)
sns.heatmap(profile, cmap="viridis",                    # colour = mean
 ↪ consumption
            cbar_kws={"label": "mean Appliances (Wh)"})   # label of the
 ↪ colour bar
plt.xlabel("Day of week (0 = Monday)"); plt.ylabel("Hour of day")
plt.show()
```



Discussion. Low consumption at night, a rise in the morning and a peak in the evening hours – a typical residential profile. Such patterns are important for later modelling: hour of day and weekday are strong features for forecasting (Chapter 11).

2.4 Data quality and outliers

Measured data pass through a long chain – sensor, signal conditioning, A/D conversion, data logger, database, export – and every link can introduce errors. Typical problems:

Problem	Typical cause	How to detect
Missing values	sensor dropout, logging gap, value not measured for every product	`df.isna()`, gaps in the time index (Section 2.5)
Outliers	real rare event *or* measurement error	box plot, IQR rule, robust z-score
Implausible values	sensor fault, wrong scaling	physical limits (e.g. humidity > 100 %)

Problem	Typical cause	How to detect
Frozen values	sensor stuck, communication lost	long runs of identical values
Coarse resolution	A/D converter or logger rounds values (e.g. to 0.1 K)	many identical values, gaps in the histogram
Wrong units / time stamps	°C vs. K, local time vs. UTC	domain knowledge, comparison of sources (Section 2.1)
Drift and offsets	sensor ageing, recalibration	trends, steps in the time series

Two of these problems need more than a quick check: outliers, treated in this section, and missing values (Section 2.5).

Definition: Outlier rules

- **IQR rule (Tukey):** a value is flagged if it lies below $Q_1 - 1.5\,\mathrm{IQR}$ or above $Q_3 + 1.5\,\mathrm{IQR}$. These are the whiskers of the box plot.
- **Robust z-score:** $z_i = \dfrac{x_i - \tilde{x}}{1.4826 \cdot \mathrm{MAD}}$, flag if $|z_i| > 3.5$, where $\tilde{x}$ is the median. The factor 1.4826 makes MAD comparable to the standard deviation for normally distributed data.

The ordinary z-score $(x_i - \bar{x})/s$ is less suitable for outlier detection: the outliers themselves inflate $\bar{x}$ and s and can hide each other (*masking*).

Pitfall: deleting "outliers" automatically

An outlier is a value that is unusual *with respect to a model* of the data – it is not automatically an error. The rare high current peaks of a motor may be exactly the events that announce a failure. Never delete data before you understand why they are extreme, and always document what you removed.

2.4.1 Live coding: outliers in the energy data

We apply both rules to the appliance energy use and to three variables with roughly symmetric distributions:

```
def iqr_outliers(x, k=1.5):
    """IQR rule: True for values more than k * IQR below Q1 or above Q3."""
    q1, q3 = x.quantile([0.25, 0.75])
```

```
    return (x < q1 - k * (q3 - q1)) | (x > q3 + k * (q3 - q1))

def robust_z(x):
    """Robust z-score: distance from the median in units of 1.4826 * MAD."""
    mad = (x - x.median()).abs().median()            # median absolute
 ↪  deviation
    return (x - x.median()) / (1.4826 * mad)

for col in ["Appliances", "T1", "T_out", "RH_1"]:
    x = energy[col]
    share_iqr = iqr_outliers(x).mean()               # mean of True/False =
 ↪  share of flagged values
    share_z = (robust_z(x).abs() > 3.5).mean()
    print(f"{col:11s} IQR rule: {share_iqr:6.1%}   |robust z| > 3.5:
     ↪  {share_z:6.1%}")
```

```
Appliances  IQR rule:  10.8%   |robust z| > 3.5:  11.2%
T1          IQR rule:   2.6%   |robust z| > 3.5:   0.2%
T_out       IQR rule:   2.2%   |robust z| > 3.5:   0.3%
RH_1        IQR rule:   0.7%   |robust z| > 3.5:   0.2%
```

For the room temperature and humidity, very few values are flagged; for `Appliances`, many. This confirms that a fixed rule cannot decide alone: the flags tell us where to *look*, the engineer decides what the values mean.

2.5 Missing data

Almost every real dataset has gaps. How to treat them depends less on *how many* values are missing than on *why* they are missing. Statisticians distinguish three mechanisms. Because the names are notoriously confusing, we explain them with one running example.

Example: a hardening furnace. A thermocouple measures the furnace temperature once per minute; the target is about 850 °C. The actual temperature depends on the load: a fully loaded furnace runs hotter than a nearly empty one. The load is logged reliably by the control system. Now suppose that some temperature values are missing. Three different reasons are possible:

1. **The radio link drops messages at random.** About 10 % of the readings are lost, regardless of load, temperature or time of day. Whether a value is missing has nothing to do with the process.

2. **The logger runs in an energy-saving mode at low load.** When the furnace is nearly empty, only every fifth reading is stored. Missing values are therefore much more frequent at low load – and since low load means lower temperatures, the missing values tend to be the lower ones. But the reason for the gap (low load) is itself recorded in the data.
3. **The amplifier saturates above 900 °C.** Whenever the furnace gets too hot, the measurement fails and nothing is stored. The values are missing *because* they are high. The load gives a hint – very hot readings occur at high load – but it cannot tell us which readings actually exceeded 900 °C.

Definition: Missing data mechanisms

Let R indicate whether a value is missing.

- **MCAR** (missing completely at random): R does not depend on any variable, observed or unobserved. *Case 1 – random radio losses.*
- **MAR** (missing at random): R depends only on **observed** variables. Once we know those (here: the load), the missingness tells us nothing more about the missing value: at a given load, the missing temperatures do not differ from the recorded ones. *Case 2 – energy-saving mode at low load.*
- **MNAR** (missing not at random): R depends on the **missing value itself** (or on other unobserved quantities), even after accounting for everything observed. *Case 3 – saturation at high temperatures.*

Pitfall: "missing at random" is not random

The name MAR is misleading: in case 2 the values are *not* missing randomly – low temperatures are missing much more often. "At random" only means "random once we take the observed variables into account". Do not read MAR as "harmless".

What the mechanism means in practice:

Mechanism	Mean of the remaining values	What helps
MCAR	unbiased – we just have fewer values	deleting incomplete rows is acceptable – only the sample gets smaller; filling gaps with the mean or median keeps the mean unbiased, but makes the spread look too small

Mechanism	Mean of the remaining values	What helps
MAR	**biased** (case 2: too high, because low values are missing)	use the observed variable that explains the gaps – e.g. impute temperature from load with a regression, or average per load level
MNAR	**biased** (case 3: too low, because high values are missing)	the data alone cannot fix it; we need knowledge of the mechanism (e.g. "missing means > 900 °C", see Section 2.5.3) or a better sensor

The first live coding below simulates the furnace and shows these effects in numbers. Common strategies in practice:

- drop variables with a very high share of missing values,
- drop rows if few are affected and the mechanism is plausibly MCAR,
- impute with the median (simple, but it shrinks the spread and ignores *why* values are missing) or with a model that uses the variables explaining the gaps (MAR),
- for time series: interpolate short gaps, but not long ones,
- add an indicator variable "value was missing" – missingness itself can be informative,
- above all: find out *why* the values are missing – ask the people who run the process.

2.5.1 Live coding: how missing values bias a mean

We simulate the furnace from the example above: 10,000 readings, load between 0 and 1, temperature rising with load. Then we delete values according to the three mechanisms and compare four ways of dealing with the gaps: dropping them, filling them with the mean or the median of the remaining values, and filling them from the load with a straight-line regression fitted on the complete rows. For each, we check two numbers against the truth: the mean and the standard deviation.

```
# --- 1. Simulate the "true" furnace data
 ↪  ----------------------------------------
# A random number generator with a fixed start value (seed = 0): every run
 ↪  produces
# the same random numbers, so the results in the script are reproducible.
# default_rng is NumPy's recommended way to create such a generator.
rng = np.random.default_rng(0)
```

```
n = 10_000                                        # number of readings (_ only for
 ↪  readability)
load = rng.uniform(0, 1, n)                       # load between 0 (empty) and 1
 ↪  (full);
                                                  # always recorded - no gaps
noise = rng.normal(0, 15, n)                      # random scatter, standard
 ↪  deviation 15 K
temp = 820 + 80 * load + noise                    # true temperature: 820 °C
 ↪  empty, 900 °C full

# --- 2. Delete values according to the three mechanisms
 ↪  ---------------------------
# Each mask is an array of n truth values: True means "this reading is
 ↪  missing".
# rng.random(n) draws n numbers between 0 and 1; "< p" is True with
 ↪  probability p.
missing = {
    "MCAR": rng.random(n) < 0.10,                 # random radio losses: always
     ↪  10 %
    "MAR":  rng.random(n) < 0.8 * (1 - load),     # energy-saving mode: 80 % if
     ↪  empty, 0 % if full
    "MNAR": temp > 900,                           # saturation: missing because
     ↪  hot
}

# --- 3. Treat the gaps in four ways and compare with the truth
 ↪  ----------------------
rows = []
for name, m in missing.items():                   # name = mechanism, m = mask of
 ↪  missing values
    obs = ~m                                      # ~ negates every element of the
 ↪  mask:
                                                  # True now means "value
                                                   ↪  observed"
    # straight line temperature vs. load, fitted on the observed readings
     ↪  only
    slope, intercept = np.polyfit(load[obs], temp[obs], deg=1)
    # np.where(m, a, b): where a value is missing (m is True) take a,
     ↪  otherwise b
    treatments = {
        "drop gaps":  temp[obs],                                          # use
         ↪  remaining values only
```

```
        "mean":       np.where(m, temp[obs].mean(), temp),           # fill
         ↪ with their mean
        "median":     np.where(m, np.median(temp[obs]), temp),       # fill
         ↪ with their median
        "regression": np.where(m, intercept + slope * load, temp),   # fill
         ↪ from the load
    }
    for method, values in treatments.items():
        rows.append({"mechanism": name, "treatment": method,
                     "mean (°C)": values.mean(), "std (K)":
                      ↪ values.std(ddof=1)})

print(f"Truth: mean {temp.mean():.1f} °C, standard deviation
 ↪ {temp.std(ddof=1):.1f} K")
print("Share missing:", ", ".join(f"{name} {m.mean():.0%}" for name, m in
 ↪ missing.items()))
result = pd.DataFrame(rows).pivot(index="mechanism", columns="treatment")   #
 ↪ one row per mechanism
result.loc[list(missing), (slice(None), list(treatments))].round(1)        #
 ↪ keep the order above
```

```
Truth: mean 860.0 °C, standard deviation 27.5 K
Share missing: MCAR 10%, MAR 40%, MNAR 8%
```

	mean (°C)				std (K)			
treatment	drop gaps	mean	median	regression	drop gaps	mean	median	regression
mechanism								
MCAR	860.0	860.0	860.0	860.1	27.5	26.1	26.1	27.1
MAR	868.9	868.9	869.7	860.0	26.0	20.2	20.2	26.0
MNAR	856.0	856.0	856.1	858.3	24.5	23.6	23.6	25.0

Discussion. Read the table column by column:

- **Dropping the gaps** keeps the mean unbiased only under MCAR. Under MAR it is too high, because the cooler low-load readings are under-represented; under MNAR it is too low, because the hottest readings are missing.
- **Filling with the mean or the median** of the remaining values looks like a repair, but it is none. Filling with the mean leaves the mean exactly where it was, and the median hardly changes it – the bias under MAR and MNAR remains completely. On top of that, the spread shrinks, because all gaps receive the same value: under MAR, the standard

deviation drops from 27.5 K to 20.2 K. Every later analysis would consider the process more stable than it is.

- **Regression imputation** uses the load, which explains *why* values are missing. Under MAR, it repairs the mean completely and almost restores the spread. Under MNAR, it removes only part of the bias: the line was fitted on readings below 900 °C and cannot know what happens above the saturation limit.

Even regression imputation slightly underestimates the spread, because the filled-in values lie exactly on the line, without the scatter of real readings. A few degrees may not sound like much – but for a heat treatment with a tolerance of ±10 °C, they can decide whether a batch is released.

2.5.2 Live coding: gaps in semiconductor data

The dataset of the scenario, SECOM, contains data from a semiconductor manufacturing process: 1,567 production runs, each described by 591 sensor and process measurements, and a label pass (−1) or fail (1). The sensor names are anonymised. Many signals have gaps. A small obstacle on the way: in the repository's metadata the label is not marked as the target variable – another example of incomplete metadata. The helper function below therefore identifies the label column from its values.

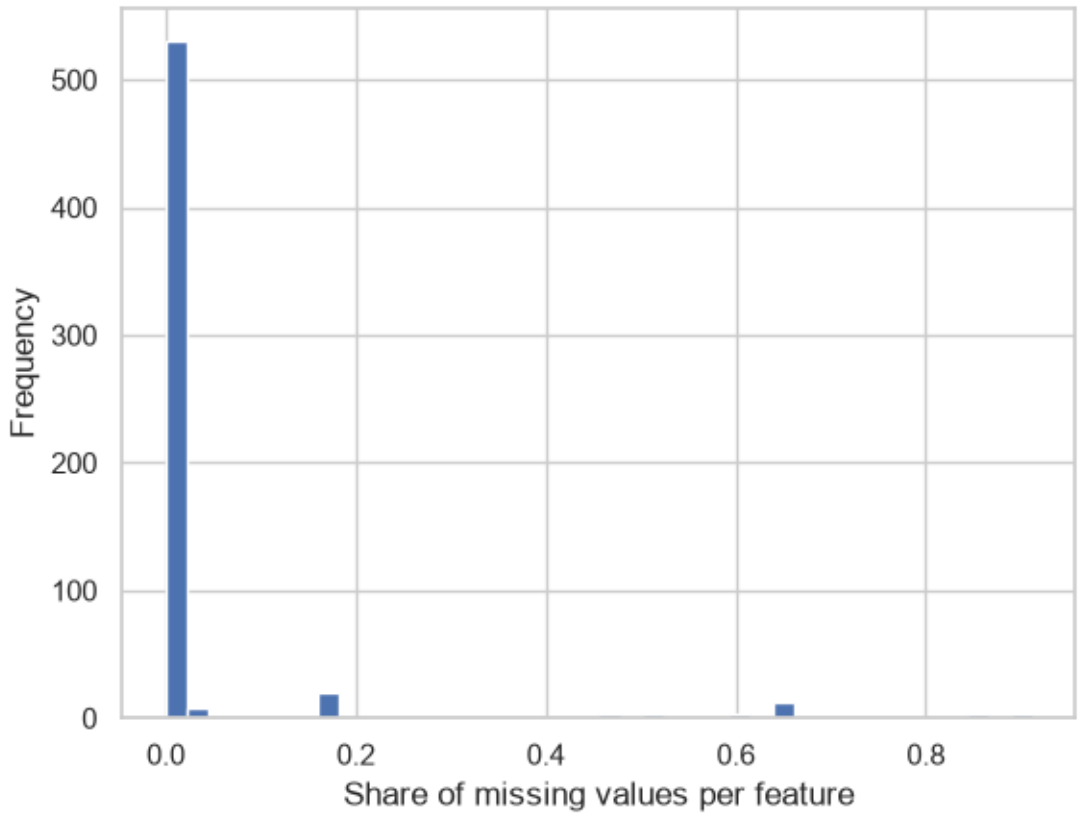
```
def load_secom():
    """SECOM: return the process measurements X and the label y (-1 = pass, 1
     ↪  = fail).

    In the UCI repository the label is not marked as target variable, so we
 ↪  find it
    ourselves: the only numeric column whose values are exactly -1 and 1.
    """
    df = fetch_ucirepo(id=179).data.original           # all columns of the
 ↪  dataset
    num = df.select_dtypes("number")                   # numeric columns only
    labels = [c for c in num.columns if set(num[c].dropna().unique()) == {-1,
 ↪  1}]
    if len(labels) != 1:                               # fallback: look at
     ↪  the column names
        labels = [c for c in df.columns if any(k in str(c).lower() for k in
 ↪  ("pass", "fail", "label"))]
    assert len(labels) == 1, f"Label column not found unambiguously:
     ↪  {labels}"
    y = df[labels[0]].astype(int)                      # the label
    X = num.drop(columns=labels, errors="ignore")      # all other numeric
 ↪  columns: measurements
```

```
    print(f"Label column: {labels[0]!r}, measurement columns: {X.shape[1]}, "
          f"ignored non-numeric columns: {[c for c in df.columns if c not in
           ↪ num.columns]}")
    return X, y

X, y = load_secom()
missing = X.isna().mean().sort_values(ascending=False)   # share of missing
 ↪ values per feature
print(f"Runs: {len(X)}, failed runs: {(y == 1).sum()}")
print(f"Features: {X.shape[1]}, with any missing value: {(missing >
 ↪ 0).sum()}, "
      f"more than 50 % missing: {(missing > 0.5).sum()}")
missing.plot.hist(bins=40, xlabel="Share of missing values per feature")   #
 ↪ how are the shares distributed?
plt.show()
```

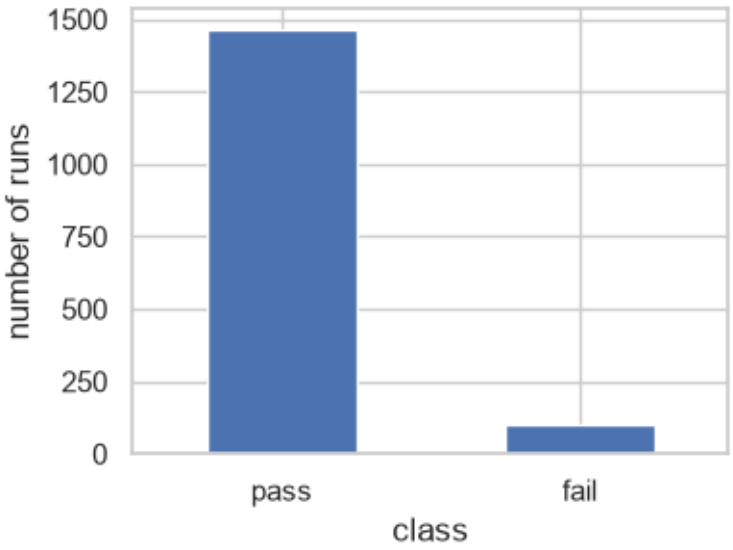
```
Label column: 'class', measurement columns: 590, ignored non-numeric columns: ['timestamp']
Runs: 1567, failed runs: 104
Features: 590, with any missing value: 538, more than 50 % missing: 28
```



The label is a categorical – more precisely, a binary – variable. As explained in Section 2.2, it is described by the frequencies of its two categories:

```
label = y.map({-1: "pass", 1: "fail"})        # readable category names
 ↪ instead of codes
counts = label.value_counts()                 # frequency of each category
print(counts.to_string())
print(f"Share of failed runs: {counts['fail'] / counts.sum():.1%}")
fig, ax = plt.subplots(figsize=(4, 3))        # a small figure of its own
counts.plot.bar(ax=ax, rot=0, ylabel="number of runs")   # bar chart; rot=0:
 ↪ horizontal labels
plt.show()
```

```
class
pass    1463
fail     104
Share of failed runs: 6.6%
```



Only about one run in fifteen failed. For the decision, this share matters more than the number of sensors: a model can only learn what a failure looks like from the failed runs, and rare events are hard to learn from (Chapter 7).

```
# Is the missingness itself related to the outcome?
miss_rate = X.isna().mean(axis=1)                  # axis=1: share of missing
 ↪  values per run (row)
print(miss_rate.groupby(y).describe().round(3))    # compare passed (-1) and
 ↪  failed (1) runs

# Constant features carry no information
n_constant = (X.nunique(dropna=True) <= 1).sum()   # at most one different
 ↪  value (gaps ignored)
print("Constant features:", n_constant)
```

```
        count   mean    std    min    25%    50%    75%    max
class
-1     1463.0  0.046  0.023  0.007  0.034  0.041  0.054  0.258
 1      104.0  0.043  0.017  0.014  0.034  0.041  0.047  0.108
Constant features: 116
```

Discussion. Most features are complete or nearly complete, but a group of features has a large share of missing values – often entire sensors that were only recorded for part of the runs. In addition, a considerable number of features are constant and can be removed without loss. Comparing the share of missing values between passed and failed runs is a quick check of

whether missingness could be informative: if the shares differ, the gaps are not MCAR. Here they are similar – no sign that the gaps carry information about the outcome, although this does not prove MCAR.

A pragmatic preparation for SECOM, which we will use in Chapter 9:

1. remove constant features,
2. remove features with more than 50 % missing values,
3. impute the remaining gaps with the median,
4. standardise all features.

> ⚠ Pitfall: preprocessing before splitting
>
> As soon as a model is trained, the median for imputation must be computed on the **training data only** and then applied unchanged to all new data – otherwise information from the test data leaks into the model. Chapter 6 explains why, and how a scikit-learn `Pipeline` takes care of it.

2.5.3 Extension: MNAR in more depth

Censored values. In case 3 we do not know how hot the furnace was – but we do know that it was *above 900 °C*. Such a value is called **censored**: it is not completely unknown, only bounded on one side. Censored values are everyday business in reliability engineering. A life test ends after 1,000 hours, and the parts that are still running at that point have a lifetime of "more than 1,000 hours" – missing *because* it is long. Methods for such data, e.g. a Weibull analysis with censoring, use exactly this partial information instead of discarding it. MNAR is therefore hardest when the mechanism is *unknown*.

MAR or MNAR depends on what is measured. The three mechanisms are defined relative to the variables in the dataset – not to everything one could have measured. In case 2, the logged load explains the gaps. If the load had not been logged, exactly the same logger behaviour would be MNAR: the gaps would depend on a quantity that is not in the data. Two practical consequences follow:

- A variable that explains the gaps only helps if it is *used*. A plain mean of the remaining temperatures is biased under MAR just as under MNAR.
- It pays to log quantities that might explain *why* values are missing – operating mode, load, sensor status. They turn an MNAR problem, which the data cannot solve, into an MAR problem, which they can.

Whether gaps are MAR or MNAR cannot be tested with the data themselves, because the missing values are unknown. It remains an assumption that has to be justified with knowledge of the process.

2.6 Back to the decision

In CRISP-DM terms, this is the step back from phase 2 to phase 1: what we learned about the data decides whether and how the project continues. For a go/no-go recommendation, three questions matter: Are enough features usable? Are there enough examples of the event we want to predict? And are the gaps understood? We use simple, transparent criteria: at least half of the features should be non-constant with at most 50 % missing values, and there should be at least 50 failed runs to learn from.

```
# A feature is usable if it is not constant and has at most 50 % missing
 ↪  values.
usable = (X.nunique(dropna=True) > 1) & (X.isna().mean() <= 0.5)
n_fail = int((y == 1).sum())                        # number of failed runs
print(f"Usable features: {usable.sum()} of {X.shape[1]}
 ↪  ({usable.mean():.0%})")
print(f"Failed runs: {n_fail} of {len(y)} ({n_fail / len(y):.1%})")
if usable.mean() >= 0.5 and n_fail >= 50:           # the two criteria from
 ↪  the text
    print("Decision: GO - with a data-quality work package in the first
     ↪  project phase.")
else:
    print("Decision: NO-GO for now - improve data acquisition first.")
```

```
Usable features: 446 of 590 (76%)
Failed runs: 104 of 1567 (6.6%)
Decision: GO - with a data-quality work package in the first project phase.
```

Whatever the code prints, the recommendation should name the conditions: the sensor names are anonymised, so no process engineer can yet judge which signals are physically relevant; the missing values must be explained (are they MAR or MNAR?); and roughly one hundred failures is a small number for a problem with hundreds of candidate features. Chapter 9 will show how hard the prediction actually is. These conditions are at the same time the tasks for the next round of the CRISP-DM cycle: with better-understood data, phases 2 and 3 are passed through again.

> **EDA toolbox**
>
> The functions used most in this chapter – all of them, with examples, in the Python quick reference (Appendix A):
>
> - **overview:** `df.info()`, `df.describe()`, `df.head()`, `df.nunique()`
> - **missing values:** `df.isna().sum()`, `df.fillna(...)`, `pd.read_csv(...,`

`na_values=...)`

- **time stamps:** `pd.to_datetime(..., format=...)`, `df.set_index(...)`, `s.resample(...)`, `s.rolling(...)`, `s.tz_localize(...)`, `s.tz_convert(...)`
- **groups and summaries:** `df.groupby(...).mean()`, `s.median()`, `s.quantile(...)`
- **masks and selection:** `df[m]`, `&`, `|`, `~`, `np.where(m, a, b)`
- **plots:** `sns.histplot`, `sns.boxplot`, `sns.heatmap`

! Key takeaways

- Read raw files deliberately: separator, missing value code, time format, units and time zone are documented in the metadata.
- Describe distributions by location, spread and shape; compare classical (mean, s) and robust (median, IQR, MAD) statistics.
- Check data quality systematically: missing values, gaps in time, implausible values, frozen sensors, outliers.
- Outlier rules flag candidates – domain knowledge decides whether they are errors or important events.
- The treatment of missing data depends on the mechanism (MCAR, MAR, MNAR) – and whether gaps are MAR or MNAR depends on what else is measured.

2.7 Exercises

1. Change the DWD code to load a weather station near your home town. The station list `TU_Stundenwerte_Beschreibung_Stationen.txt` is in the same DWD folder as the data files.
2. Determine type and role (see Chapter 1) of each column of the raw DWD file (`STATIONS_ID`, `MESS_DATUM`, `QN_9`, `TT_TU`, `RF_TU`, `eor`) for the question "How does the air temperature in Trier vary over the day?".
3. Compute the mean temperature per hour of the day for the DWD data and plot the daily cycle. At which hour is it warmest – and does the result change if you convert the time stamps from UTC to German local time?
4. Resample the energy data to hourly values. Which aggregation (mean, sum, max, last) is correct for `Appliances`, which for the temperatures? Explain.
5. Show that the random variables `rv1` and `rv2` carry no information about `Appliances`. Compare their correlation with that of the outdoor temperature.
6. Compare the IQR rule and the robust z-score for `Appliances` after a log transformation, `np.log(energy["Appliances"])`. How many values are flagged now? What does this tell you about the "outliers" in the original scale?
7. Check all temperature columns (`T1`–`T9`, `T_out`) and humidity columns (`RH_1`–`RH_9`,

`RH_out`) of the energy dataset for physically implausible values and for "frozen" periods, i.e. more than 6 identical consecutive values.

8. For SECOM: propose a strategy for features with 0–5 %, 5–50 % and more than 50 % missing values, and implement steps 1–3 of the preparation described above. How many features remain?

2.7.1 Solutions

Solution to Exercise 1

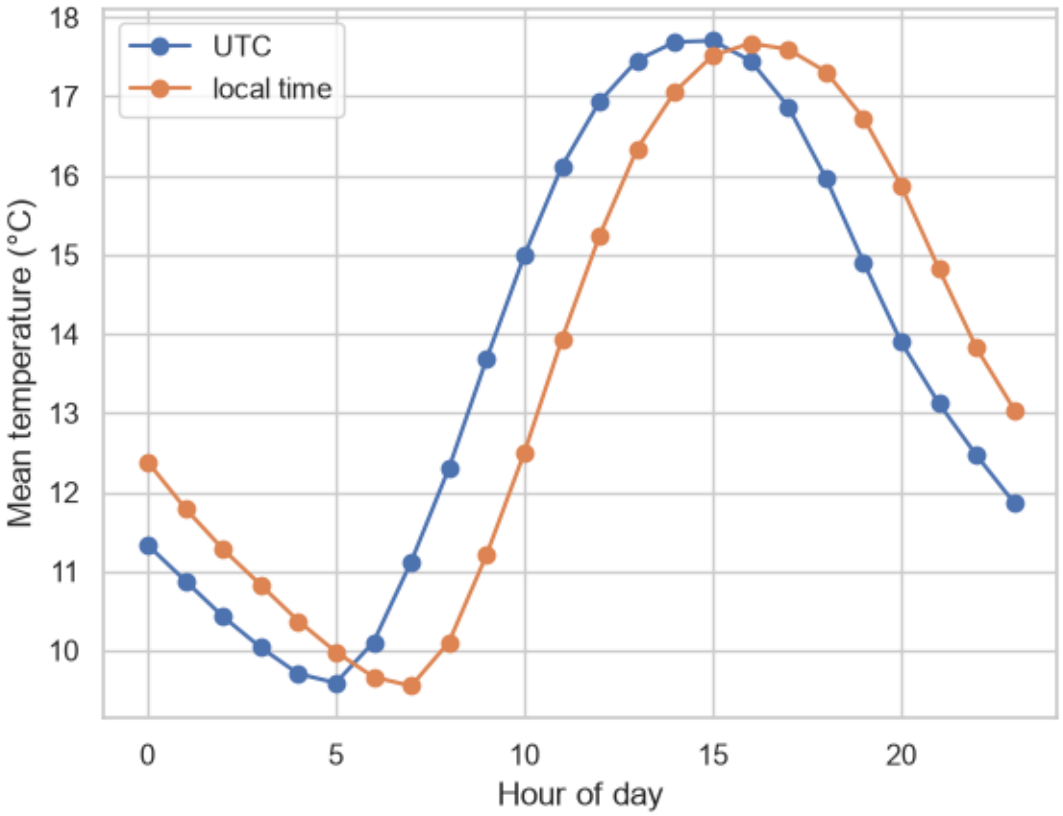
Open the station list in a browser or read it with pandas, find your station and replace `05100` in the file name. The file name of the `recent` product always has the form `stundenwerte_TU_<ID>_akt.zip`, where `<ID>` is the five-digit station ID with leading zeros. Not all stations measure all variables – if the file does not exist, choose a neighbouring station.

Solution to Exercise 2

- `STATIONS_ID`: categorical (nominal) – a code, although stored as a number; constant within the file.
- `MESS_DATUM`: time stamp – numeric (differences are durations); after conversion, the index of the time series.
- `QN_9`: categorical – a code for the quality level of the record. Whether the levels are ordered must be looked up in the DWD documentation; computing a mean is meaningless either way.
- `TT_TU`: continuous (°C) – the variable of interest for this question; there is no target, because nothing is predicted.
- `RF_TU`: continuous (%) – not needed for this question.
- `eor`: not a variable at all – a marker for "end of record" in each line of the file.

Solution to Exercise 3

```
# The DWD time stamps are in UTC: declare this (tz_localize), then
 ↪  convert to German time.
local = weather.tz_localize("UTC").tz_convert("Europe/Berlin")
cycle = pd.DataFrame({                                          # mean
 ↪  temperature per hour of day
    "UTC": weather.groupby(weather.index.hour)["temp_C"].mean(),
    "local time": local.groupby(local.index.hour)["temp_C"].mean(),
})
ax = cycle.plot(marker="o")
ax.set(xlabel="Hour of day", ylabel="Mean temperature (°C)")
plt.show()
cycle.idxmax()                                                  # hour of the
 ↪  maximum
```




```
UTC           15
local time    16
```

```
dtype: int32
```

The maximum typically occurs in the early or mid afternoon. In local time the curve is shifted by one hour in winter and two hours in summer, so the hour of the maximum changes accordingly. This is exactly the kind of shift that causes errors when weather data are merged with production data recorded in local time.

Solution to Exercise 4

```
# One aggregation per column: energies are summed, temperatures are
 ↪  averaged.
hourly = energy.resample("h").agg({"Appliances": "sum", "lights": "sum",
                                   "T1": "mean", "T_out": "mean"})
hourly.head()
```

date	Appliances	lights	T1	T_out
2016-01-11 17:00:00	330	210	19.890000	6.308333
2016-01-11 18:00:00	1060	310	19.897778	5.941667
2016-01-11 19:00:00	1040	150	20.495556	6.000000
2016-01-11 20:00:00	750	210	20.961111	6.000000
2016-01-11 21:00:00	620	140	21.311667	5.833333

`Appliances` is an energy per interval (Wh in 10 minutes). The energy of one hour is the **sum** of the six 10-minute values (unit: Wh per hour, i.e. the mean power in W). Temperatures are state variables: the hourly value is the **mean** (or, for an instantaneous value, the last one). Summing temperatures would be physically meaningless. `max` is useful for questions about peaks, e.g. dimensioning a fuse.

Solution to Exercise 5

```
# Correlation of each listed column with the appliance energy use
energy[["rv1", "rv2", "T_out", "T2",
 ↪  "RH_out"]].corrwith(energy["Appliances"]).round(3)
```

```
rv1      -0.011
rv2      -0.011
T_out     0.099
T2        0.120
```

```
RH_out   -0.152
dtype: float64
```

The correlations of `rv1` and `rv2` with `Appliances` are close to zero. (Check also with `energy["rv1"].equals(energy["rv2"])` whether the two random columns are identical copies – a nice example of a redundant feature.) The weather variables show small but non-zero correlations. Note that with almost 20,000 observations even tiny correlations can be "statistically significant" (Chapter 4) – the size of the correlation matters more than its p-value.

Solution to Exercise 6

```
log_app = np.log(energy["Appliances"])            # log transformation
 ↪  (all values are positive)
print(f"IQR rule:          {iqr_outliers(log_app).mean():.1%}")
print(f"|robust z| > 3.5:  {(robust_z(log_app).abs() > 3.5).mean():.1%}")
```

```
IQR rule:          6.5%
|robust z| > 3.5:  7.4%
```

After the log transformation far fewer values are flagged. Many "outliers" on the original scale were simply the long right tail of a skewed distribution. Outlier rules implicitly assume a roughly symmetric distribution – applying them to skewed data without transformation flags normal behaviour.

Solution to Exercise 7

```
temp_cols = [f"T{i}" for i in range(1, 10)] + ["T_out"]      # T1 ... T9
 ↪  and T_out
rh_cols = [f"RH_{i}" for i in range(1, 10)] + ["RH_out"]     # RH_1 ...
 ↪  RH_9 and RH_out
# physical plausibility: overall temperature range, humidity outside
 ↪  0-100 %
print("Temperature range (°C):", energy[temp_cols].min().min(), "to",
 ↪  energy[temp_cols].max().max())
print("Humidity outside 0-100 %:", ((energy[rh_cols] < 0) |
 ↪  (energy[rh_cols] > 100)).sum().sum())

def longest_constant_run(x):
    """Length of the longest run of identical consecutive values."""
    runs = (x != x.shift()).cumsum()     # new run number whenever the
 ↪  value changes
    return x.groupby(runs).size().max()  # longest run

frozen = {c: longest_constant_run(energy[c]) for c in temp_cols +
 ↪  rh_cols}
pd.Series(frozen).sort_values(ascending=False).head()          # the five
 ↪  longest runs
```

```
Temperature range (°C): -6.065 to 29.85666667
Humidity outside 0-100 %: 0

RH_6    353
T9       79
T7       78
T5       62
RH_8     42
dtype: int64
```

The temperature and humidity ranges should be physically plausible. Runs of identical values occur in practice because sensors report with limited resolution and indoor conditions change slowly; a run is suspicious only if it is long compared with the normal dynamics (e.g. several hours for an outdoor temperature). Here, the outdoor weather comes from an airport station with lower update rate and was interpolated to 10 minutes – this explains smooth or repeated values in the `_out` columns.

Solution to Exercise 8

```
X1 = X.loc[:, X.nunique(dropna=True) > 1]                # 1. drop constant
 ↪  features
X2 = X1.loc[:, X1.isna().mean() <= 0.5]                  # 2. drop > 50 %
 ↪  missing
X3 = X2.fillna(X2.median())                              # 3. median
 ↪  imputation
print(f"{X.shape[1]} -> {X1.shape[1]} (constant removed) -> {X2.shape[1]}
 ↪  (sparse removed)")
print("Remaining missing values:", X3.isna().sum().sum())
```

```
590 -> 474 (constant removed) -> 446 (sparse removed)
Remaining missing values: 0
```

Strategy: 0–5 % missing → median imputation (little influence on results); 5–50 % → median or model-based imputation plus an indicator variable "was missing"; more than 50 % → remove the feature, unless domain experts know that it is important (then investigate why it is missing). In a real project the imputation would be fitted on the training data only (see the warning above).

3 Probability and Uncertainty

No two measurements are identical, no two components have the same lifetime, and every estimate computed from a sample would come out slightly differently with another sample. Engineers deal with this variability every day – in tolerances, safety factors, reliability targets and maintenance plans. Probability theory provides the language to describe it, and statistics provides the tools to quantify how certain our estimates are.

In this chapter we model the lifetimes of jet engines with a probability distribution, derive reliability figures such as the B10 life, and quantify the uncertainty of our estimates with confidence intervals – computed both with a classical formula and with the bootstrap. Finally we meet Bayes' theorem, which tells us how to update probabilities when new information arrives.

Scenario: the replacement interval

You are the reliability engineer of an airline that operates a fleet of turbofan engines of one type. The airworthiness authority requires a maintenance programme in which, with 95 % confidence, at most 10 % of the engines reach a critical degradation state before they are taken off wing for overhaul. Your data: the complete run-to-failure histories of 100 engines of the same type. The fleet manager wants the interval as long as possible – every additional cycle on wing saves money.

Your decision: After how many flight cycles should engines be overhauled preventively?

Learning goals. After this chapter you can

- describe random quantities with probability density, distribution function, expected value and variance,
- choose suitable distributions for engineering data (normal, lognormal, Weibull, exponential),
- fit a distribution to lifetime data by maximum likelihood and check the fit with a probability plot,
- interpret the Weibull shape parameter in terms of failure behaviour and derive the B10 life,
- compute and correctly interpret confidence intervals, analytically and with the bootstrap,
- apply Bayes' theorem to diagnostic alarms and explain the base rate effect,
- (extension) update prior knowledge with test data (beta–binomial model) and interpret a credible interval.

Datasets in this chapter

- NASA C-MAPSS turbofan degradation data

3.1 Random variables and distributions

> **Definition: Random variable, PDF and CDF**
>
> A **random variable** X assigns a number to the outcome of a random experiment, e.g. the lifetime of a component. For a continuous random variable,
>
> - the **cumulative distribution function (CDF)** $F(x) = P(X \le x)$ gives the probability of a value up to x,
> - the **probability density function (PDF)** $f(x) = F'(x)$ describes how probability is distributed; $P(a < X \le b) = \int_a^b f(x)\,dx$.
>
> The **expected value** $E[X] = \int x\, f(x)\,dx$ is the long-run average, the **variance** $\mathrm{Var}(X) = E[(X - E[X])^2]$ measures the spread.

In reliability engineering, $F(t)$ is the probability that a component has failed by time t, and the **reliability** (survival function) is $R(t) = 1 - F(t)$.

Choosing a distribution is a modelling decision, guided by physics and by the data:

Distribution	Parameters	Typical use	Physical reasoning
Normal	μ, σ	measurement errors, machined dimensions	sum of many small independent effects
Lognormal	μ, σ of $\ln X$	fatigue life, particle sizes, repair times	product of many small independent effects
Weibull	shape β, scale η	lifetimes, strength of brittle materials	"weakest link": the first of many flaws fails
Exponential	rate λ	random failures, times between events	constant failure rate, no memory
Poisson	rate λ	number of defects per unit or per time	rare independent events

The prominence of the normal distribution is explained by one of the most important results of probability theory:

! Theorem: Central limit theorem

Let $X_1, \ldots, X_n$ be independent random variables with the same distribution, mean μ and finite variance σ^2. Then the distribution of the sample mean $\bar{X}_n$ approaches a normal distribution with mean μ and standard deviation $\sigma/\sqrt{n}$ as n grows – **whatever the distribution of the** X_i.

Two consequences matter for engineers: quantities that arise as sums of many small effects are approximately normal, and the uncertainty of a mean decreases with $1/\sqrt{n}$ – four times as many measurements halve the uncertainty.

3.2 The Weibull distribution for lifetimes

The two-parameter Weibull distribution has the CDF and the **hazard rate** (instantaneous failure rate of the survivors)

$$F(t) = 1 - \exp\left[-\left(\frac{t}{\eta}\right)^{\beta}\right], \qquad h(t) = \frac{f(t)}{R(t)} = \frac{\beta}{\eta}\left(\frac{t}{\eta}\right)^{\beta-1}.$$

The scale parameter η is the **characteristic life**: by $t = \eta$, 63.2 % of all units have failed, regardless of β. The shape parameter β determines how the failure rate changes over time:

! Interpretation of the Weibull shape parameter

- $\beta < 1$: decreasing failure rate – **early failures** (manufacturing defects, infant mortality).
- $\beta = 1$: constant failure rate – **random failures**; the Weibull becomes the exponential distribution.
- $\beta > 1$: increasing failure rate – **wear-out** (fatigue, corrosion, abrasion). $\beta \approx 3.5$ gives a nearly symmetric distribution.

Together, the three regimes form the well-known *bathtub curve* of a product's life.

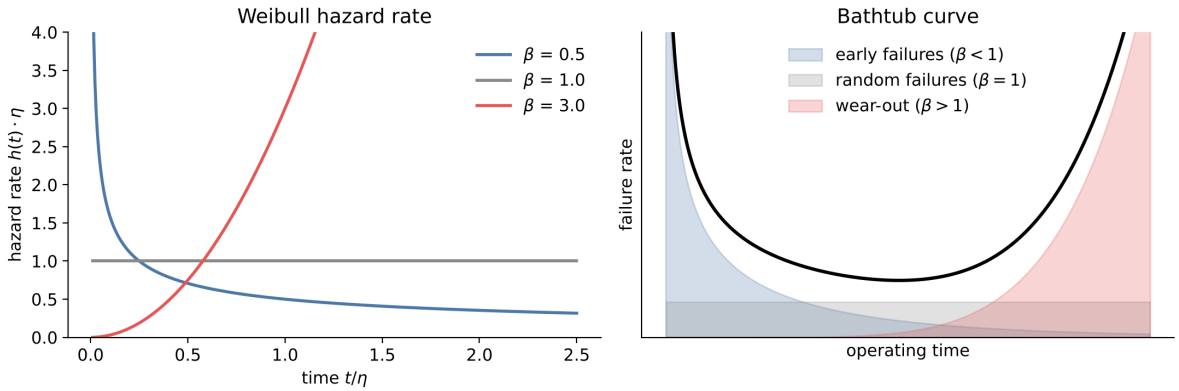


Figure 3.1: Left: Weibull hazard rate for different shape parameters. Right: the bathtub curve of a product's life as the superposition of early failures, random failures and wear-out.

The shape parameter has direct consequences for maintenance: preventive replacement only makes sense if the failure rate increases ($\beta > 1$). With $\beta \le 1$, a new part is not more reliable than an old one, and replacing it early wastes money.

A frequently used reliability figure is the **B10 life**: the time by which 10 % of the units have failed, i.e. $F(B_{10}) = 0.1$. Solving the Weibull CDF gives

$$B_{10} = \eta \, (-\ln 0.9)^{1/\beta} .$$

3.2.1 Estimating parameters: maximum likelihood

> **Definition: Maximum likelihood estimation (MLE)**
>
> Given independent observations $x_1, \dots, x_n$ and a model with density $f(x;\theta)$, the **likelihood** $L(\theta) = \prod_i f(x_i;\theta)$ measures how plausible the data are for a parameter value θ. The maximum likelihood estimate $\hat{\theta}$ maximises L, in practice its logarithm $\ell(\theta) = \sum_i \ln f(x_i;\theta)$.

For the normal distribution, MLE leads to the familiar mean and (biased) standard deviation. For the Weibull distribution, there is no closed-form solution; `scipy.stats.weibull_min.fit` maximises the likelihood numerically.

To check whether the fitted distribution describes the data, we use a **probability plot** (Q–Q plot): the sorted data are plotted against the corresponding quantiles of the fitted distribution.

If the model fits, the points lie on a straight line. Systematic curvature at the ends reveals a wrong tail behaviour – exactly where reliability questions are decided.

```
# Setup: works locally and in Google Colab
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

sns.set_theme(style="whitegrid")
```

3.3 Live coding: lifetimes of turbofan engines

C-MAPSS (*Commercial Modular Aero-Propulsion System Simulation*) is a NASA simulation of a large turbofan engine. The dataset contains run-to-failure trajectories of a fleet of engines with different initial wear and manufacturing variation. In the training file `train_FD001.txt`, every one of the 100 engines runs until failure, so the last recorded cycle of each engine is its lifetime (in flight cycles). We will use the sensor data of the same file in Chapter 12.

Data access. The cell below downloads `CMAPSSData.zip` (about 12 MB) from the NASA Open Data Portal once and extracts it into the folder `data/CMAPSSData/`.

```
from pathlib import Path
import zipfile, io, requests

def cmapss_dir(root=Path("data")):
    """Download and extract the NASA C-MAPSS data once; return the folder
     ↪  with train_FD001.txt."""
    root.mkdir(exist_ok=True)
    target = root / "CMAPSSData"
    if not any(target.rglob("train_FD001.txt")):
        zip_path = root / "CMAPSSData.zip"
        if not zip_path.exists():
            r =
 ↪  requests.get("https://data.nasa.gov/docs/legacy/CMAPSSData.zip",
 ↪  timeout=300)
            r.raise_for_status()
            zip_path.write_bytes(r.content)
        with zipfile.ZipFile(zip_path) as z:
            z.extractall(target)
```

```
        for inner in list(target.rglob("*.zip")):          # some versions
         ↪ contain a nested zip
            zipfile.ZipFile(inner).extractall(inner.parent)
    return next(target.rglob("train_FD001.txt")).parent

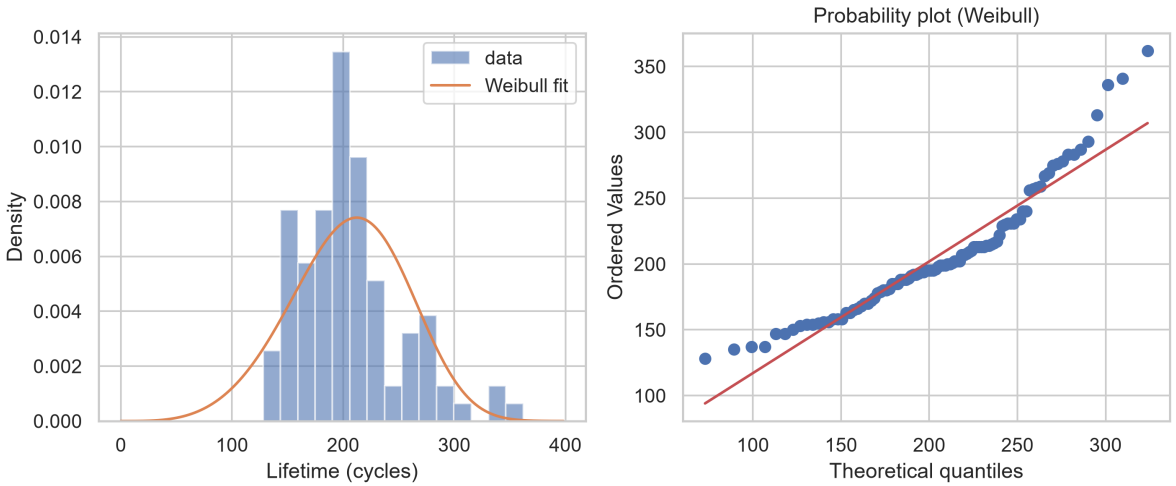
CMAPSS = cmapss_dir()
cols = ["unit", "cycle", "op1", "op2", "op3"] + [f"s{i}" for i in range(1,
 ↪ 22)]
train = pd.read_csv(CMAPSS / "train_FD001.txt", sep=r"\s+", header=None,
 ↪ names=cols)
lifetimes = train.groupby("unit")["cycle"].max()
lifetimes.describe().round(1)
```

```
count    100.0
mean     206.3
std       46.3
min      128.0
25%      177.0
50%      199.0
75%      229.2
max      362.0
Name: cycle, dtype: float64
```

```
beta, loc, eta = stats.weibull_min.fit(lifetimes, floc=0)
print(f"Weibull fit: shape beta = {beta:.2f}, scale eta = {eta:.1f} cycles")

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
t = np.linspace(0, lifetimes.max() * 1.1, 300)
axes[0].hist(lifetimes, bins=15, density=True, alpha=0.6, label="data")
axes[0].plot(t, stats.weibull_min.pdf(t, beta, scale=eta), label="Weibull
 ↪ fit")
axes[0].set(xlabel="Lifetime (cycles)", ylabel="Density"); axes[0].legend()
stats.probplot(lifetimes, dist=stats.weibull_min, sparams=(beta, 0, eta),
 ↪ plot=axes[1])
axes[1].set_title("Probability plot (Weibull)")
plt.show()
```

```
Weibull fit: shape beta = 4.41, scale eta = 225.0 cycles
```



Discussion. We fixed the location parameter at zero (`floc=0`), i.e. we use the two-parameter Weibull distribution. The estimated shape parameter is clearly larger than 1 – the engines show **wear-out behaviour**, as expected for a degradation simulation. The probability plot tells us whether the Weibull model is adequate over the whole range; pay particular attention to the lower tail, which determines the B10 life.

Note that no engine fails very early: the shortest lifetime is well above zero. A three-parameter Weibull distribution with a *failure-free time* (location parameter $t_0 > 0$) could describe this better – a question for Exercise 1.

```
b10 = stats.weibull_min.ppf(0.10, beta, scale=eta)
print(f"B10 life from the fit:    {b10:.0f} cycles")
print(f"B10 from the formula:     {eta * (-np.log(0.9)) ** (1 / beta):.0f}
 ↪  cycles")
print(f"Empirical 10 % quantile:  {lifetimes.quantile(0.10):.0f} cycles")
```

```
B10 life from the fit:    135 cycles
B10 from the formula:     135 cycles
Empirical 10 % quantile:  155 cycles
```

3.4 Confidence intervals

The mean lifetime we computed is an **estimate** from 100 engines. Another fleet of 100 engines would give a somewhat different value. A confidence interval quantifies this sampling uncertainty.

Definition: Confidence interval

A 95 % **confidence interval** is computed from the data by a procedure that, if the experiment were repeated many times, would contain the true parameter in 95 % of the repetitions.
For the mean of approximately normal data (or large n by the central limit theorem):

$$\bar{x} \pm t_{n-1,\,0.975}\,\frac{s}{\sqrt{n}},$$

where $t_{n-1,\,0.975}$ is the 97.5 % quantile of the t-distribution with $n-1$ degrees of freedom (≈ 1.98 for $n = 100$).

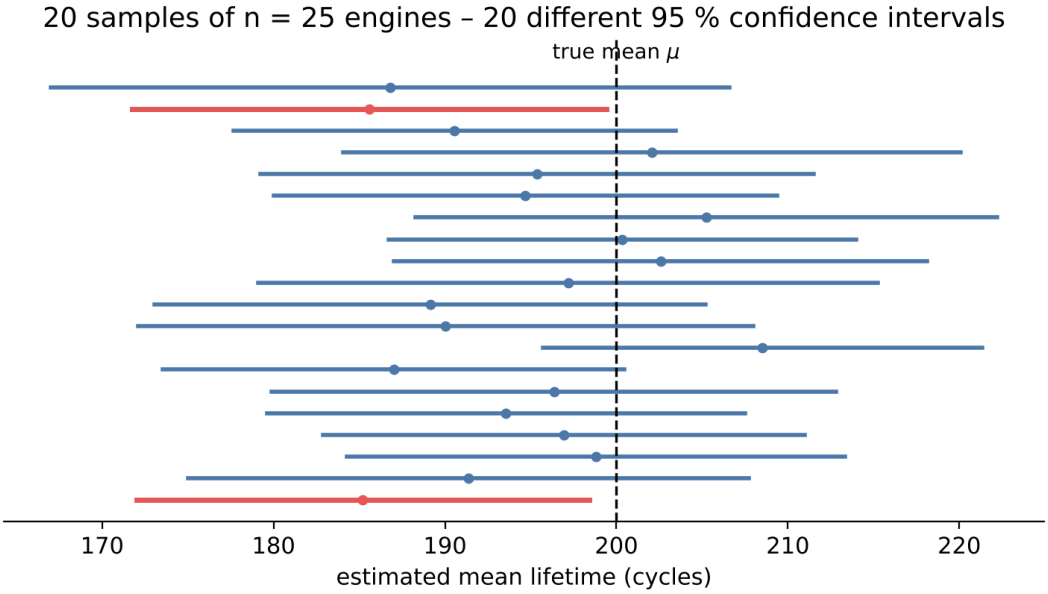


Figure 3.2: Twenty repetitions of the same experiment, each with its own 95 % confidence interval for the mean. On average, 19 of 20 intervals contain the true mean; here two miss it (red). For a single computed interval we never know whether it is one of the lucky ones.

Pitfall: what a confidence interval does *not* say

A computed interval, say [195, 215] cycles, does **not** contain the true mean "with 95 % probability" – in the classical view, the true mean is fixed and either inside or not. The 95 % refers to the reliability of the method. Also, a confidence interval for the *mean* says nothing about the spread of *individual* lifetimes: most single engines lie far outside it.

For many quantities – the B10 life, a median, a ratio – there is no simple formula for the interval. The bootstrap solves this by simulation:

> **Definition: Bootstrap**
>
> Draw B (e.g. 10,000) samples of size n **with replacement** from the observed data and compute the statistic of interest for each. The distribution of these bootstrap replicates approximates the sampling distribution of the statistic; its 2.5 % and 97.5 % quantiles give an approximate 95 % confidence interval (percentile method; SciPy uses the refined BCa method by default).

```
ci_t = stats.t.interval(0.95, df=len(lifetimes) - 1,
                        loc=lifetimes.mean(), scale=stats.sem(lifetimes))
boot = stats.bootstrap((lifetimes.values,), np.mean, confidence_level=0.95,
                       random_state=0)
print(f"Mean lifetime: {lifetimes.mean():.1f} cycles")
print("t-interval:        ", np.round(ci_t, 1))
print("bootstrap interval:", np.round(boot.confidence_interval, 1))
```

```
Mean lifetime: 206.3 cycles
t-interval:         [197.1 215.5]
bootstrap interval: [197.9 215.8]
```

Discussion. For the mean of 100 observations both methods give almost the same interval – the central limit theorem makes the t-interval reliable here. The bootstrap becomes valuable for statistics without a known formula (Exercise 2) and for small or strongly skewed samples.

3.5 Back to the decision

The requirement "at most 10 % failures with 95 % confidence" has two parts. The 10 % defines the B10 life. The 95 % confidence means that we must not use the point estimate of B10 but a value that is, with 95 % confidence, *below* the true B10 – the lower bound of a one-sided 95 % confidence interval, which we obtain with the bootstrap. And preventive overhaul only makes sense if the failure rate increases with age, i.e. $\beta > 1$.

```
def b10_of(x):
    c, _, scale = stats.weibull_min.fit(x, floc=0)
    return stats.weibull_min.ppf(0.10, c, scale=scale)

boot_b10 = stats.bootstrap((lifetimes.values,), b10_of, n_resamples=1000,
 ↪  confidence_level=0.90,
```

```
                            method="percentile", random_state=1)
b10_lower = boot_b10.confidence_interval.low      # lower end of a 90 %
 ↪  two-sided = 95 % one-sided bound
print(f"Weibull shape beta = {beta:.2f}  ->  {'wear-out: preventive overhaul
 ↪  is sensible' if beta > 1 else 'no wear-out: preventive overhaul does not
 ↪  help'}")
print(f"B10 point estimate: {b10:.0f} cycles, 95 % lower confidence bound:
 ↪  {b10_lower:.0f} cycles")
print(f"Decision: overhaul after {int(np.floor(b10_lower))} flight cycles at
 ↪  the latest.")
```

```
Weibull shape beta = 4.41  ->  wear-out: preventive overhaul is sensible
B10 point estimate: 135 cycles, 95 % lower confidence bound: 128 cycles
Decision: overhaul after 128 flight cycles at the latest.
```

The difference between the point estimate and the lower bound is the price of having data from only 100 engines – more data would allow a longer interval. In the real world, the decision would also consider that C-MAPSS is a *simulation*: before an interval is approved, the model would have to be validated against field data of real engines.

3.6 Bayes' theorem: how much should we trust an alarm?

So far we have treated probabilities as long-run frequencies. **Conditional probabilities** describe how the probability of an event changes when we know that another event has occurred.

! Theorem: Bayes' theorem

For events F and A with $P(A) > 0$:

$$P(F \mid A) = \frac{P(A \mid F)\,P(F)}{P(A)} = \frac{P(A \mid F)\,P(F)}{P(A \mid F)\,P(F) + P(A \mid \bar{F})\,P(\bar{F})}.$$

$P(F)$ is the **prior** probability, $P(F \mid A)$ the **posterior** probability after observing A.

Example: a condition monitoring system detects a bearing fault F with probability $P(A \mid F) = 0.95$ (*sensitivity*) and raises a false alarm on a healthy bearing with probability $P(A \mid \bar{F}) = 0.05$. Only 1 % of the monitored bearings are actually damaged. If the system raises an alarm A, how likely is a real fault?

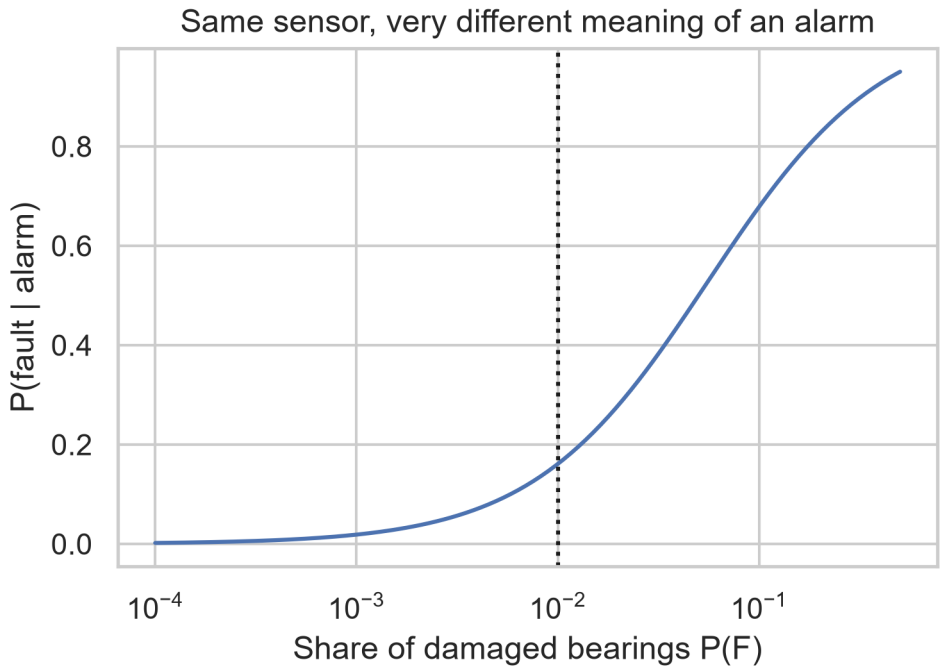
$$P(F \mid A) = \frac{0.95 \cdot 0.01}{0.95 \cdot 0.01 + 0.05 \cdot 0.99} \approx 0.16$$

Five out of six alarms are false – not because the system is bad, but because faults are rare.

> **Pitfall: base rate neglect**
>
> Intuition tends to confuse $P(A \mid F)$ ("the system detects 95 % of all faults") with $P(F \mid A)$ ("95 % of all alarms are real"). When the base rate $P(F)$ is small, the two differ dramatically. This effect returns in Chapter 7 as the *precision* of a classifier.

```
sens, false_alarm = 0.95, 0.05
prevalence = np.logspace(-4, -0.3, 200)
posterior = sens * prevalence / (sens * prevalence + false_alarm * (1 -
 ↪  prevalence))

plt.semilogx(prevalence, posterior)
plt.axvline(0.01, ls=":", c="k")
plt.xlabel("Share of damaged bearings P(F)"); plt.ylabel("P(fault | alarm)")
plt.title("Same sensor, very different meaning of an alarm")
plt.show()
```



3.7 Extension: Bayesian estimation with prior knowledge

Extension – optional material, not required for the exam.

Classical (frequentist) statistics uses only the current data. Bayesian statistics treats an unknown parameter θ itself as uncertain, describes the knowledge before the experiment by a **prior distribution** and updates it with the data to a **posterior distribution**:

$$p(\theta \mid \text{data}) \propto p(\text{data} \mid \theta)\, p(\theta).$$

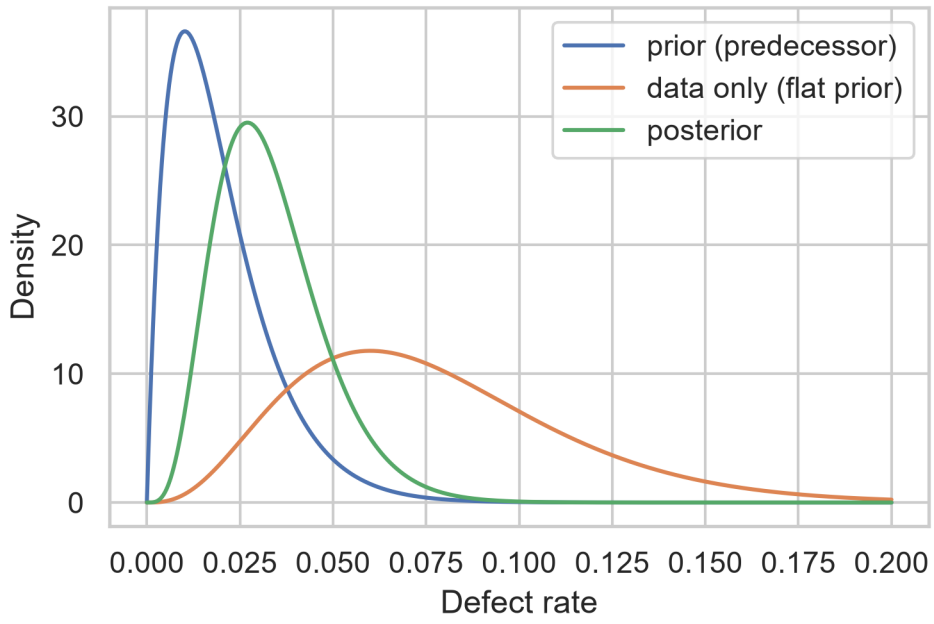
This is attractive in engineering, where tests are expensive but experience from predecessor products exists. Example: a new product variant is tested, 3 of 50 parts are defective. From the predecessor product we know a defect rate of about 2 %.

For a proportion, the **beta distribution** is a convenient prior: with prior $\text{Beta}(a, b)$ and k defects in n parts, the posterior is again a beta distribution, $\text{Beta}(a+k,\ b+n-k)$. (We write a, b for the beta parameters to avoid confusion with the Weibull shape β.) The prior mean is $a/(a+b)$, so $\text{Beta}(2, 98)$ has mean $2/(2+98) = 2\,\%$ and carries roughly the weight of 100 previously inspected parts.

```
k, n = 3, 50                          # new tests: defects, parts
a0, b0 = 2, 98                        # prior from the predecessor product
a1, b1 = a0 + k, b0 + n - k           # posterior

p = np.linspace(0, 0.20, 500)
plt.plot(p, stats.beta.pdf(p, a0, b0), label="prior (predecessor)")
plt.plot(p, stats.beta.pdf(p, 1 + k, 1 + n - k), label="data only (flat
 ↪  prior)")
plt.plot(p, stats.beta.pdf(p, a1, b1), label="posterior")
plt.xlabel("Defect rate"); plt.ylabel("Density"); plt.legend()
plt.show()

ci_freq = stats.binomtest(k, n).proportion_ci(0.95)
print(f"Frequentist: {k/n:.1%}, 95 % CI [{ci_freq.low:.1%},
 ↪  {ci_freq.high:.1%}]")
lo, hi = stats.beta.interval(0.95, a1, b1)
print(f"Bayesian:    {a1/(a1+b1):.1%}, 95 % credible interval [{lo:.1%},
 ↪  {hi:.1%}]")
```




```
Frequentist: 6.0%, 95 % CI [1.3%, 16.5%]
Bayesian:    3.3%, 95 % credible interval [1.1%, 6.7%]
```

Discussion. The posterior lies between prior and data: the new tests suggest a higher defect rate than the predecessor, but 50 parts are not enough to overrule the experience from 100 parts completely. The Bayesian **credible interval** has the intuitive interpretation that the classical confidence interval lacks: given prior and data, the defect rate lies in this interval with 95 % probability. The price is that the result depends on the prior – which must therefore be justified and documented. Whether the predecessor is really comparable is an engineering judgement, not a statistical one.

! Key takeaways

- Distributions are modelling choices; physics (sum, product, weakest link) guides the choice, probability plots check it.
- The Weibull shape parameter distinguishes early failures, random failures and wear-out – and decides whether preventive maintenance makes sense.
- Confidence intervals quantify the uncertainty of an estimate; the bootstrap provides them for almost any statistic.
- Bayes' theorem: with rare events, even good detectors produce mostly false alarms.

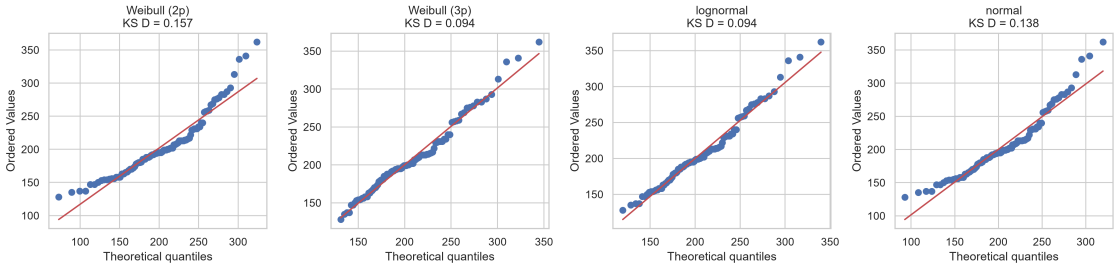
3.8 Exercises

1. Fit a lognormal, a normal and a three-parameter Weibull distribution (free location parameter) to the lifetimes. Compare the fits with probability plots and the Kolmogorov–Smirnov statistic.
2. Use the bootstrap to compute a 95 % confidence interval for the B10 life. Why is it much wider (relative to the estimate) than the interval for the mean?
3. How does the width of the confidence interval of the mean change if only 20 engines are available? Simulate it by drawing many random subsamples of 20 engines.
4. Interpret the shape parameter for the maintenance strategy: does preventive replacement make sense? At which age would you replace an engine if at most 5 % of the engines may fail before replacement?
5. The fault share in a well-maintained fleet is only 0.1 %. How reliable is an alarm of the monitoring system above? Which false alarm rate would be needed to reach $P(F \mid A) = 50\,\%$?
6. (Extension) Repeat the defect-rate analysis with a weak prior Beta(0.2, 9.8) and a strong prior Beta(20, 980). When does the prior dominate, when the data?

3.8.1 Solutions

Solution to Exercise 1

```
fits = {
    "Weibull (2p)": (stats.weibull_min, stats.weibull_min.fit(lifetimes,
     ↪  floc=0)),
    "Weibull (3p)": (stats.weibull_min,
     ↪  stats.weibull_min.fit(lifetimes)),
    "lognormal":    (stats.lognorm, stats.lognorm.fit(lifetimes,
     ↪  floc=0)),
    "normal":       (stats.norm, stats.norm.fit(lifetimes)),
}
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, (name, (dist, params)) in zip(axes, fits.items()):
    stats.probplot(lifetimes, dist=dist, sparams=params, plot=ax)
    ks = stats.kstest(lifetimes, dist.cdf, args=params)
    ax.set_title(f"{name}\nKS D = {ks.statistic:.3f}")
plt.tight_layout(); plt.show()
```



The model with the smallest KS statistic and the straightest probability plot fits best. The three-parameter Weibull has one more parameter and therefore always fits at least as well as the two-parameter version; its location parameter estimates a failure-free time. Caution: the p-values of the KS test are not valid when the parameters were estimated from the same data, so use the statistic for comparison only. With 100 observations, several distributions usually fit acceptably – the choice should also be guided by physics (Weibull for wear-out).

Solution to Exercise 2

```
def b10_stat(x):
    c, _, scale = stats.weibull_min.fit(x, floc=0)
    return stats.weibull_min.ppf(0.10, c, scale=scale)

boot_b10 = stats.bootstrap((lifetimes.values,), b10_stat,
 ↪  n_resamples=1000,
                           method="percentile", random_state=0)
lo, hi = boot_b10.confidence_interval
print(f"B10 = {b10:.0f} cycles, 95 % CI [{lo:.0f}, {hi:.0f}]  (relative
 ↪  width {(hi - lo) / b10:.0%})")
print(f"Mean: relative width {(ci_t[1] - ci_t[0]) /
 ↪  lifetimes.mean():.0%}")
```

```
B10 = 135 cycles, 95 % CI [127, 146]  (relative width 14%)
Mean: relative width 9%
```

The B10 life is a statement about the **lower tail** of the distribution, where only about 10 of the 100 engines contribute information. In addition, it depends on two estimated parameters, and the shape parameter enters as an exponent. The mean uses all observations equally. Reliability figures in the tails are therefore always much more uncertain than averages – a reason for large safety factors.

Solution to Exercise 3

```
rng = np.random.default_rng(0)
widths = []
for _ in range(2000):
    sub = rng.choice(lifetimes.values, size=20, replace=False)
    lo_, hi_ = stats.t.interval(0.95, df=19, loc=sub.mean(),
 ↪  scale=stats.sem(sub))
    widths.append(hi_ - lo_)
print(f"n = 100: width {ci_t[1] - ci_t[0]:.1f} cycles")
print(f"n = 20:  mean width {np.mean(widths):.1f} cycles (ratio
 ↪  {np.mean(widths) / (ci_t[1] - ci_t[0]):.2f})")
```

```
n = 100: width 18.4 cycles
n = 20:  mean width 42.6 cycles (ratio 2.32)
```

The width scales approximately with $1/\sqrt{n}$, so going from 100 to 20 engines should make the interval about $\sqrt{5} \approx 2.2$ times wider; the t-quantile for 19 degrees of freedom (2.09 instead of 1.98) adds a little more. (Subsampling without replacement from only 100 engines slightly reduces the spread, so the simulated ratio can be a bit smaller.)

Solution to Exercise 4

```
t05 = stats.weibull_min.ppf(0.05, beta, scale=eta)
print(f"beta = {beta:.2f} > 1 -> wear-out; replacement age for 5 %
 ↪  failures: {t05:.0f} cycles")
```

```
beta = 4.41 > 1 -> wear-out; replacement age for 5 % failures: 115 cycles
```

Since $\beta > 1$, the failure rate increases with age, so preventive replacement reduces failures. The replacement age for at most 5 % failures is the 5 % quantile $B_5 = \eta(-\ln 0.95)^{1/\beta}$. Given the large uncertainty of tail estimates (Exercise 2), one would in practice use the lower confidence bound of B_5 rather than the point estimate. Whether replacement is economical depends additionally on the cost ratio of planned replacement and unplanned failure.

Solution to Exercise 5

With $P(F) = 0.001$:

$$P(F \mid A) = \frac{0.95 \cdot 0.001}{0.95 \cdot 0.001 + 0.05 \cdot 0.999} \approx 0.019,$$

i.e. only about 2 % of the alarms are real. For $P(F \mid A) = 0.5$ we need $0.95 \cdot 0.001 = q \cdot 0.999$, where $q = P(A \mid \bar{F})$ is the false alarm rate, so $q \approx 0.00095$ – roughly one false alarm per thousand healthy bearings. The better the fleet, the more specific the monitoring must be.

Solution to Exercise 6

```
for name, (a, b) in {"weak": (0.2, 9.8), "medium": (2, 98), "strong":
 ↪  (20, 980)}.items():
    post = stats.beta(a + k, b + n - k)
    lo_, hi_ = post.interval(0.95)
    print(f"{name:6s} prior: posterior mean {post.mean():.1%}, 95 %
     ↪  credible interval [{lo_:.1%}, {hi_:.1%}]")
```

```
weak   prior: posterior mean 5.3%, 95 % credible interval [1.2%, 12.2%]
medium prior: posterior mean 3.3%, 95 % credible interval [1.1%, 6.7%]
strong prior: posterior mean 2.2%, 95 % credible interval [1.4%, 3.2%]
```

All three priors have a mean of 2 % but different weights ($a + b = 10$, 100, 1000 “virtual parts”). The weak prior is dominated by the 50 real parts: the posterior mean is close to the observed 6 %. The strong prior hardly moves – 50 parts are not enough to overrule 1,000 virtual ones. Rule of thumb: the data dominate once n is large compared with $a + b$.

Part II

Part B – Statistics for Engineers

4 Statistical Testing

A process engineer changes the supplier of a raw material, and the mean scrap rate in the following week is slightly lower. Is the new material better – or was it a good week by chance? Questions like this are everyday engineering questions, and statistical hypothesis tests are the standard tool to answer them. They provide a disciplined way to decide whether an observed difference is larger than what random variation alone would plausibly produce.

Tests are also among the most misunderstood tools of statistics. A small p-value does not mean that an effect is large or important, and a large p-value does not prove that there is no effect. With the big datasets of modern production, almost every difference becomes "significant". This chapter therefore puts as much weight on **effect sizes** and **practical relevance** as on p-values.

Scenario: the consultant's bonus

A gas turbine power plant hired a consultant in 2011 to reduce its NOx emissions through better operating practices. The contract promises a bonus if the emissions in 2015 are "demonstrably and relevantly lower" than in 2011; the contract defines *relevant* as a reduction of at least 5 % of the 2011 mean. The consultant presents a test with a tiny p-value and asks for payment. You are asked to review the claim for the plant management.

Your decision: Confirm the claim and approve the bonus – or not?

Learning goals. After this chapter you can

- formulate engineering questions as statistical hypotheses,
- explain test statistic, p-value, significance level, type I and type II errors and power,
- choose and apply t-tests, ANOVA, chi-squared and non-parametric tests,
- distinguish statistical significance from practical relevance (effect size),
- recognise and correct for the multiple-testing problem,
- estimate the sample size needed to detect a relevant effect.

Datasets in this chapter

- Gas Turbine CO and NOx Emission (UCI)
- AI4I 2020 Predictive Maintenance (UCI)

4.1 The logic of a hypothesis test

A hypothesis test is a proof by contradiction with uncertainty. We assume that there is *no* effect and check whether the data are compatible with this assumption.

Definition: Elements of a hypothesis test

- **Null hypothesis** H_0: the "nothing special" statement, e.g. $\mu_1 = \mu_2$ (both suppliers give the same mean).
- **Alternative hypothesis** H_1: what we want to detect, e.g. $\mu_1 \neq \mu_2$.
- **Test statistic** T: a number computed from the data that measures the deviation from H_0.
- **p-value**: the probability of obtaining a test statistic at least as extreme as the observed one, **assuming H_0 is true**.
- **Significance level** α (often 0.05): H_0 is rejected if $p < \alpha$.

Two kinds of wrong decisions are possible:

	H_0 true (no effect)	H_0 false (effect exists)
H_0 **rejected**	type I error (false alarm), probability α	correct decision, probability $1 - \beta =$ **power**
H_0 **not rejected**	correct decision	type II error (missed effect), probability β

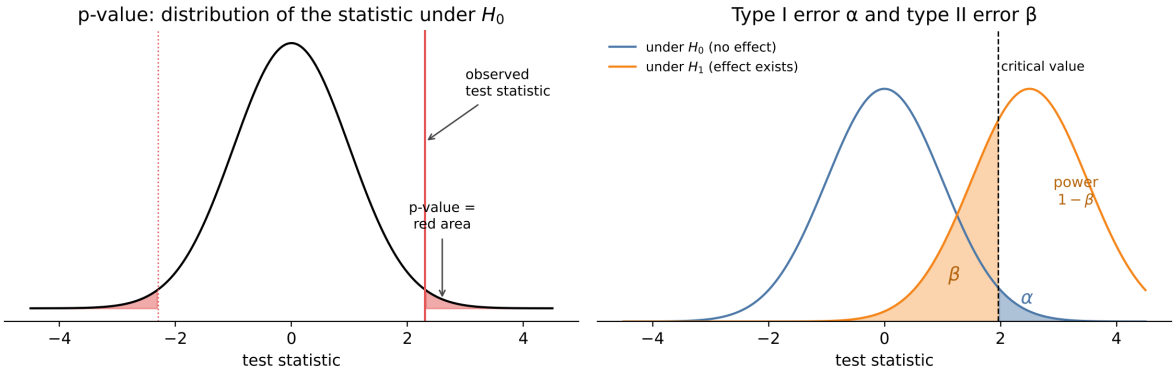


Figure 4.1: Left: the p-value is the probability, computed under the null hypothesis, of a test statistic at least as extreme as the observed one (two-sided test). Right: the two error types. The critical value is set such that the false alarm probability is 5 %; the probability of missing a real effect depends on the effect size and the sample size.

The significance level controls the false alarm rate. The power depends on the size of the effect, the noise level and the sample size – small samples often have too little power to detect relevant effects.

> ⚠ Pitfall: what the p-value is not
>
> - It is **not** the probability that H_0 is true.
> - $p \geq 0.05$ does **not** prove that there is no effect – maybe the sample was just too small ("absence of evidence is not evidence of absence").
> - $p < 0.05$ says nothing about the **size** or the **importance** of the effect.

4.2 Comparing two groups: the t-test

For two independent samples with means $\bar{x}_1, \bar{x}_2$, standard deviations s_1, s_2 and sizes n_1, n_2, the **Welch t-test** uses

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{s_1^2/n_1 + s_2^2/n_2}}.$$

Under H_0, t follows approximately a t-distribution. Welch's version does not assume equal variances and should be the default (the classical Student t-test assumes them). The test assumes independent observations and approximately normal data – for large samples, the central limit theorem makes the normality assumption less critical.

> ❗ Tests and confidence intervals are two sides of one coin
>
> A two-sided test at level $\alpha = 0.05$ rejects $H_0 : \mu_1 - \mu_2 = 0$ exactly when the 95 % confidence interval for $\mu_1 - \mu_2$ does not contain 0. The interval is more informative: it shows not only *whether* there is a difference but also *how large* it plausibly is. Report intervals whenever possible.

4.3 More than two groups, categories and non-normal data

Question	Data	Test	SciPy
Do two groups differ in their mean?	continuous, approx. normal	Welch t-test	`ttest_ind(equal_var=False)`
… without normality assumption	continuous / ordinal	Mann–Whitney U	`mannwhitneyu`

Question	Data	Test	SciPy
Paired measurements (before/after on the same parts)	continuous	paired t-test / Wilcoxon	`ttest_rel` / `wilcoxon`
Do k groups differ?	continuous	one-way ANOVA / Kruskal–Wallis	`f_oneway` / `kruskal`
Are two categorical variables related?	counts in a contingency table	chi-squared test	`chi2_contingency`

The **one-way ANOVA** (analysis of variance) compares the variation *between* group means with the variation *within* the groups:

$$F = \frac{\text{variance between groups}}{\text{variance within groups}} = \frac{\sum_j n_j(\bar{x}_j - \bar{x})^2/(k-1)}{\sum_j \sum_i (x_{ij} - \bar{x}_j)^2/(N-k)}.$$

Here x_{ij} is observation i in group j, $\bar{x}_j$ the mean of group j with n_j observations, $\bar{x}$ the overall mean, k the number of groups and N the total number of observations. A large F indicates that at least one group mean differs. ANOVA does not say *which* one – this requires pairwise comparisons, which lead to the multiple-testing problem below.

The **chi-squared test** compares the observed counts O in a contingency table with the counts E expected if the variables were independent: $\chi^2 = \sum (O-E)^2/E$.

4.4 Effect size and practical relevance

i Definition: Cohen's d

The standardised mean difference

$$d = \frac{\bar{x}_1 - \bar{x}_2}{s_p}, \qquad s_p = \sqrt{\frac{s_1^2 + s_2^2}{2}},$$

expresses a difference in units of the standard deviation. Rough conventions: $|d| \approx 0.2$ small, 0.5 medium, 0.8 large.

In engineering, the most meaningful effect size is often the raw difference in physical units, compared with a tolerance or a requirement: "the mean diameter differs by 0.2 µm, the tolerance is ±20 µm".

> ⚠ Pitfall: significance with large samples
>
> The standard error shrinks with $1/\sqrt{n}$. With thousands of observations, differences far too small to matter become highly significant. With big data, always ask: *Is the difference large enough to matter for the process?*

4.5 The multiple-testing problem

If we perform 20 independent tests at $\alpha = 0.05$ and no effect exists anywhere, we still expect one "significant" result by chance. The probability of at least one false alarm is $1 - 0.95^{20} \approx 64\,\%$.

> 💡 Correcting for multiple tests
>
> The simplest correction is **Bonferroni**: with m tests, use α/m for each. It is conservative. The **Holm** method is uniformly better and equally simple. Both are available in `statsmodels.stats.multitest.multipletests`. Most importantly: decide *before* looking at the data which comparisons you will test.

```
# Setup: works locally and in Google Colab
import importlib.util, subprocess, sys
for pkg in ["ucimlrepo"]:
    if importlib.util.find_spec(pkg) is None:
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg],
 ↪  check=True)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

sns.set_theme(style="whitegrid")
```

```
# Download helper, as in Chapter 2
from pathlib import Path
import requests

DATA = Path("data")
DATA.mkdir(exist_ok=True)                          # create the folder ./data
 ↪  if it does not exist
```

```
def download(url, fname=None):
    """Download a file once and cache it in ./data."""
    if fname is None:                               # no file name given:
        fname = url.split("/")[-1]                  # use the last part of the
 ↪  URL
    path = DATA / fname                             # "/" joins folder and file
 ↪  name
    if not path.exists():                           # download only if not
     ↪  cached yet
        r = requests.get(url, timeout=120)          # give up after 120 s
 ↪  without response
        r.raise_for_status()                        # stop with an error if the
 ↪  download failed
        path.write_bytes(r.content)                 # open, write and close the
 ↪  file in one step
    return path
```

4.6 Live coding: NOx emissions 2011–2015

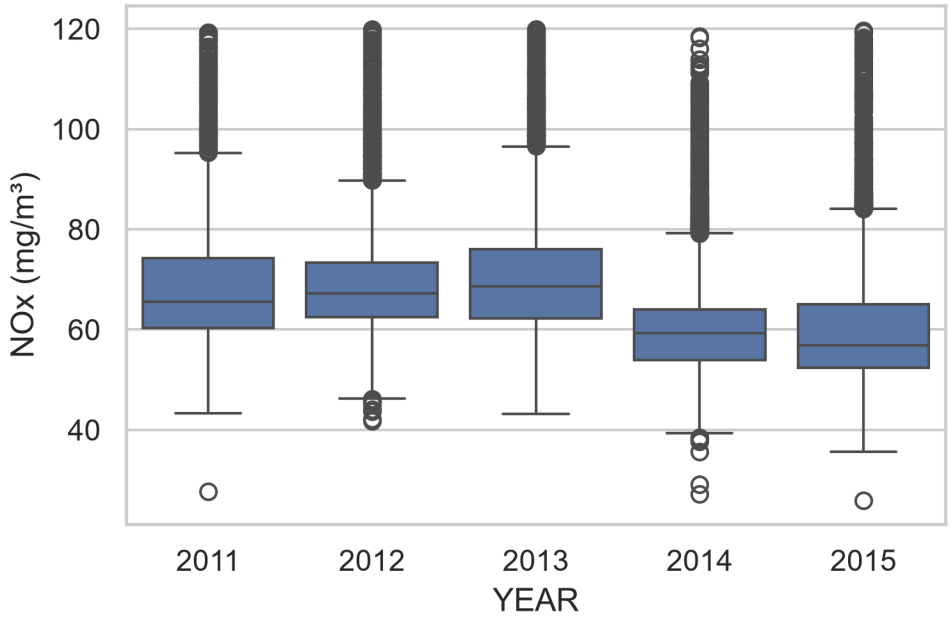
The dataset contains 36,733 hourly sensor values of a gas turbine in north-western Turkey, recorded over five years, with one CSV file per year. Besides ambient conditions and turbine process variables, it contains the emissions of carbon monoxide (`CO`) and nitrogen oxides (`NOX`), both in mg/m³. Emission limits make NOx a relevant quantity for plant operators and authorities.

```
import zipfile

url =
 ↪  "https://archive.ics.uci.edu/static/public/551/gas+turbine+co+and+nox+emission+data+set.z
frames = []
with zipfile.ZipFile(download(url, "gas_turbine.zip")) as z:
    for name in sorted(n for n in z.namelist() if n.endswith(".csv")):
        year = int(name.split("_")[-1][:4])
        frames.append(pd.read_csv(z.open(name)).assign(year=year))
gt = pd.concat(frames, ignore_index=True)
gt.columns = gt.columns.str.upper()
gt.groupby("YEAR")["NOX"].describe().round(1)
```

YEAR	count	mean	std	min	25%	50%	75%	max
2011	7411.0	67.6	10.7	27.8	60.4	65.5	74.3	119.3
2012	7628.0	68.8	10.2	41.8	62.5	67.2	73.4	119.9
2013	7152.0	70.0	12.0	43.2	62.3	68.7	76.0	119.9
2014	7158.0	60.1	10.0	27.2	53.9	59.3	64.0	118.5
2015	7384.0	59.9	11.1	25.9	52.4	56.8	65.1	119.7

```
sns.boxplot(data=gt, x="YEAR", y="NOX")
plt.ylabel("NOx (mg/m³)")
plt.show()
```



The box plots show the distribution of NOx per year. Before testing, look at the size of the differences between the medians compared with the spread within each year – this is what the tests will formalise.

```
groups = [g["NOX"].values for _, g in gt.groupby("YEAR")]
print("ANOVA:         ", stats.f_oneway(*groups))
print("Kruskal-Wallis:", stats.kruskal(*groups))

a, b = gt.loc[gt.YEAR == 2011, "NOX"], gt.loc[gt.YEAR == 2015, "NOX"]
res = stats.ttest_ind(a, b, equal_var=False)
```

```
ci = res.confidence_interval(0.95)
cohens_d = (a.mean() - b.mean()) / np.sqrt((a.var() + b.var()) / 2)
print(f"Welch t-test 2011 vs 2015: p = {res.pvalue:.2e}")
print(f"difference = {a.mean() - b.mean():.1f} mg/m³, 95 % CI [{ci.low:.1f},
 ↪  {ci.high:.1f}], Cohen's d = {cohens_d:.2f}")
```

```
ANOVA:          F_onewayResult(statistic=np.float64(1495.9585853305857), pvalue=np.float64(0
Kruskal-Wallis: KruskalResult(statistic=np.float64(6795.540640131331), pvalue=np.float64(0.0)
Welch t-test 2011 vs 2015: p = 0.00e+00
difference = 7.7 mg/m³, 95 % CI [7.3, 8.0], Cohen's d = 0.70
```

Discussion. With several thousand hourly values per year, the p-values are astronomically small – any systematic difference between years is detected. The informative numbers are the difference in mg/m³ with its confidence interval and Cohen's d. Whether the change matters depends on the distance to the emission limit and on the cause.

And here an engineering caution is needed: the hourly values are **not independent** – consecutive hours are strongly correlated, so the effective sample size is much smaller than the number of rows, and the p-values are too optimistic. Moreover, differences between years can be caused by different ambient conditions (a hot year), different load profiles or maintenance work. A test can establish *that* the years differ, not *why*. To attribute the change to a cause we would need a model that accounts for these influences (Chapter 5).

4.7 Live coding: product variant and machine failures

The AI4I dataset is a synthetic dataset modelled on a real milling process. Each of the 10,000 records belongs to a product of quality variant L (low, 50 % of products), M (medium, 30 %) or H (high, 20 %). Is the failure rate independent of the variant? Both variables are categorical, so we use the chi-squared test.

```
from ucimlrepo import fetch_ucirepo

ai4i = fetch_ucirepo(id=601).data.original
table = pd.crosstab(ai4i["Type"], ai4i["Machine failure"])
chi2, p, dof, expected = stats.chi2_contingency(table)
print(table, f"\nchi2 = {chi2:.2f}, dof = {dof}, p = {p:.3f}")
(table[1] / table.sum(axis=1)).rename("failure rate").round(4)
```

```
Machine failure     0    1
Type
H                 982   21
L                5765  235
M                2914   83
chi2 = 13.75, dof = 2, p = 0.001

Type
H    0.0209
L    0.0392
M    0.0277
Name: failure rate, dtype: float64
```

Discussion. Compare the failure rates per variant and the p-value. The chi-squared test needs sufficiently large expected counts (rule of thumb: at least 5 in every cell) – check `expected`. Even if the test is significant, ask whether the difference in failure rates is relevant, and whether the variants differ in other respects (e.g. tool wear or load) that could explain it.

4.8 How many measurements do we need?

Planning comes before testing. If a difference of δ is relevant, how many samples are needed to detect it with a power of, say, 80 %? For the two-sample t-test, the required number per group is approximately

$$n \approx \frac{2\,(z_{1-\alpha/2} + z_{1-\beta})^2}{d^2}, \qquad d = \delta/\sigma,$$

where δ is the smallest relevant difference, σ the standard deviation and z_q the q-quantile of the standard normal distribution. This gives about $16/d^2$ for $\alpha = 0.05$ and power 0.8. Halving the detectable effect requires four times as many samples.

```python
from statsmodels.stats.power import TTestIndPower

power_analysis = TTestIndPower()
for d in [0.2, 0.5, 0.8]:
    n = power_analysis.solve_power(effect_size=d, alpha=0.05, power=0.8)
    print(f"d = {d}: n = {np.ceil(n):.0f} per group")
```

```
d = 0.2: n = 394 per group
d = 0.5: n = 64 per group
d = 0.8: n = 26 per group
```

4.9 Back to the decision

The contract requires two things: the reduction must be *demonstrable* (larger than random variation) and *relevant* (at least 5 % of the 2011 mean). A small p-value only covers the first part. We therefore look at the confidence interval of the difference: the bonus is justified only if the whole interval lies above the relevance threshold.

```
nox_2011 = gt.loc[gt.YEAR == 2011, "NOX"]
nox_2015 = gt.loc[gt.YEAR == 2015, "NOX"]
res_bonus = stats.ttest_ind(nox_2011, nox_2015, equal_var=False)
ci_red = res_bonus.confidence_interval(0.95)              # CI of mean(2011)
 ↪  - mean(2015)
threshold = 0.05 * nox_2011.mean()
print(f"Reduction 2011 -> 2015: {nox_2011.mean() - nox_2015.mean():.1f}
 ↪  mg/m³, "
      f"95 % CI [{ci_red.low:.1f}, {ci_red.high:.1f}], relevance threshold
       ↪  {threshold:.1f} mg/m³")
if ci_red.low > threshold:
    print("Decision: claim confirmed - the reduction is demonstrable and
     ↪  relevant.")
elif ci_red.low > 0:
    print("Decision: reduction demonstrable, but not demonstrably relevant -
     ↪  no bonus under the contract.")
else:
    print("Decision: no demonstrable reduction - claim rejected.")
```

```
Reduction 2011 -> 2015: 7.7 mg/m³, 95 % CI [7.3, 8.0], relevance threshold 3.4 mg/m³
Decision: claim confirmed - the reduction is demonstrable and relevant.
```

Two further points belong in the review. The hourly values are autocorrelated, so the true uncertainty is larger than the interval suggests – a borderline case should be decided against the claim. And even a confirmed reduction does not prove that the consultant *caused* it: different weather, load profiles or maintenance could have the same effect. A fair contract would have specified how such influences are accounted for – a lesson for the next one.

! Key takeaways

- A test checks whether data are compatible with "no effect"; the p-value is a probability *under* H_0, not *of* H_0.
- Report effect sizes and confidence intervals, not just p-values; judge relevance in physical units.

- Large samples make tiny effects significant; small samples miss relevant effects – plan the sample size.
- Many tests produce false alarms; correct (Bonferroni, Holm) and plan comparisons in advance.
- Tests assume independent observations – time series data violate this.

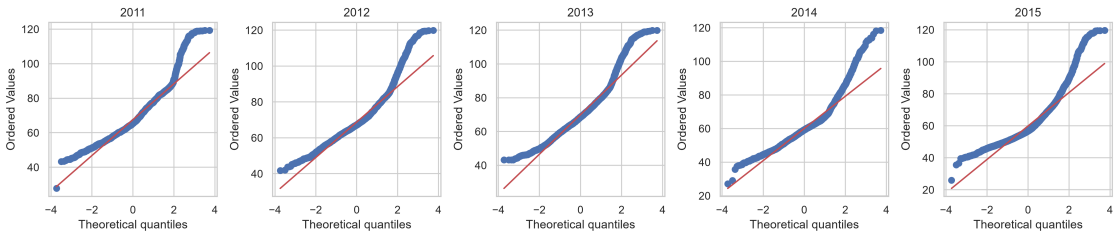
4.10 Exercises

1. Check the normality assumption for NOx per year with QQ plots. Does a violation matter for the ANOVA with these sample sizes?
2. Draw a random sample of 30 hourly values per year and repeat the ANOVA and the 2011 vs. 2015 t-test. What changes and why? Repeat the sampling a few times.
3. Run pairwise Welch t-tests between all years. How many comparisons are there? Apply a Bonferroni and a Holm correction.
4. Formulate an engineering conclusion from the NOx analysis in two or three sentences – for a plant manager, not for a statistician.
5. A new coating is supposed to reduce the wear of a tool by 10 µm. From previous tests the standard deviation of the wear is 25 µm. How many tools must be tested per variant to detect the improvement with 80 % power ($\alpha = 0.05$)?

4.10.1 Solutions

Solution to Exercise 1

```
years = sorted(gt["YEAR"].unique())
fig, axes = plt.subplots(1, len(years), figsize=(16, 3.5))
for ax, yr in zip(axes, years):
    stats.probplot(gt.loc[gt.YEAR == yr, "NOX"], dist="norm", plot=ax)
    ax.set_title(str(yr))
plt.tight_layout(); plt.show()
```



Deviations from the straight line at the ends indicate heavier or lighter tails than the

normal distribution, curvature indicates skewness. With thousands of observations per group, the central limit theorem makes the group *means* approximately normal, so the ANOVA is robust against moderate non-normality. More critical are the unequal variances between years and above all the dependence of consecutive hourly values. The Kruskal–Wallis test, which does not assume normality, serves as a cross-check.

Solution to Exercise 2

```
for seed in range(3):
    small = gt.groupby("YEAR", group_keys=False).sample(30,
 ↪  random_state=seed)
    groups_s = [g["NOX"].values for _, g in small.groupby("YEAR")]
    a_s = small.loc[small.YEAR == 2011, "NOX"]; b_s =
 ↪  small.loc[small.YEAR == 2015, "NOX"]
    print(f"seed {seed}: ANOVA p =
     ↪  {stats.f_oneway(*groups_s).pvalue:.3g}, "
          f"t-test p = {stats.ttest_ind(a_s, b_s,
           ↪  equal_var=False).pvalue:.3g}, "
          f"difference = {a_s.mean() - b_s.mean():.1f} mg/m³")
```

```
seed 0: ANOVA p = 0.000462, t-test p = 0.203, difference = 4.5 mg/m³
seed 1: ANOVA p = 5.7e-07, t-test p = 0.000116, difference = 8.8 mg/m³
seed 2: ANOVA p = 7.32e-06, t-test p = 0.0237, difference = 7.0 mg/m³
```

The estimated differences stay of similar size, but the p-values are much larger and vary from sample to sample: with 30 values per year, the standard error is roughly $\sqrt{7000/30} \approx 15$ times larger. Small differences that were "highly significant" with all data may no longer be detectable. The effect did not change – only the power of the test.

Solution to Exercise 3

```
import itertools
from statsmodels.stats.multitest import multipletests

pairs, pvals = [], []
for y1, y2 in itertools.combinations(years, 2):
    pairs.append(f"{y1}-{y2}")
    pvals.append(stats.ttest_ind(gt.loc[gt.YEAR == y1, "NOX"],
 ↪  gt.loc[gt.YEAR == y2, "NOX"],
                                 equal_var=False).pvalue)
result = pd.DataFrame({"p": pvals,
                       "Bonferroni": multipletests(pvals,
                        ↪  method="bonferroni")[1],
                       "Holm": multipletests(pvals, method="holm")[1]},
                        ↪  index=pairs)
result.style.format("{:.2e}")
```

Table 4.4

	p	Bonferroni	Holm
2011-2012	1.18e-12	1.18e-11	3.55e-12
2011-2013	1.01e-37	1.01e-36	4.06e-37
2011-2014	0.00e+00	0.00e+00	0.00e+00
2011-2015	0.00e+00	0.00e+00	0.00e+00
2012-2013	3.97e-11	3.97e-10	7.94e-11
2012-2014	0.00e+00	0.00e+00	0.00e+00
2012-2015	0.00e+00	0.00e+00	0.00e+00
2013-2014	0.00e+00	0.00e+00	0.00e+00
2013-2015	0.00e+00	0.00e+00	0.00e+00
2014-2015	3.14e-01	1.00e+00	3.14e-01

With 5 years there are $\binom{5}{2} = 10$ comparisons. Bonferroni multiplies each p-value by 10 (capped at 1); Holm multiplies the smallest by 10, the next by 9 and so on, and is therefore less conservative. With these large samples most comparisons remain significant even after correction – the correction matters most when p-values are close to α.

Solution to Exercise 4

A good answer states the size of the change in physical units, relates it to a meaningful reference and names the limits of the analysis, for example: "The mean NOx emission in 2015 was X mg/m³ lower (higher) than in 2011, a change of about Y % of the typical level. This difference is clearly larger than random variation. The data alone do not show whether the change was caused by maintenance, different operation or different weather; to find out, we should model the emissions as a function of load and ambient conditions." (Insert X and Y from your results.) Avoid statistical jargon such as "p < 0.001" without explanation.

Solution to Exercise 5

```
d = 10 / 25
n = TTestIndPower().solve_power(effect_size=d, alpha=0.05, power=0.8)
print(f"d = {d}, n = {np.ceil(n):.0f} tools per variant (rule of thumb
 ↪  16/d² = {16 / d**2:.0f})")
```

```
d = 0.4, n = 100 tools per variant (rule of thumb 16/d² = 100)
```

The effect size is $d = 10/25 = 0.4$, which requires about 100 tools per variant. If that is too expensive, options are: reduce the noise (more homogeneous test conditions, paired design with both coatings on the same machine), accept a lower power, or test for a larger minimum effect.

5 Linear and Nonlinear Regression

Regression is the workhorse of engineering data analysis. Calibration curves, empirical material laws, characteristic maps of engines and process models are all regression models: they describe how a quantity of interest – the *response* – depends on one or more influencing quantities – the *inputs*, *factors* or *features*. A regression model can be used to **understand** a process (which factors matter, how strongly?) and to **predict** the response for new input values.

This chapter starts with the multiple linear regression model, its estimation by least squares and the statistical tools to interpret and check it. We then fit nonlinear models with physical meaning, which are common in engineering. An extension section shows how designed experiments make sure that the data actually contain the information we need.

Scenario: releasing a precast element

A precast concrete plant wants to use a new, cheaper mix: 350 kg cement and 180 kg water per m^3, 5 kg superplasticizer, no slag or fly ash. The elements require a compressive strength of 35 MPa before they may be loaded. No test specimens of the new mix exist yet, and the production manager asks: "Can we promise the customer 35 MPa after 28 days – and if not, when?"

Your decision: Approve the mix for the 28-day delivery promise – or name a later date?

Learning goals. After this chapter you can

- fit and interpret multiple linear regression models with statsmodels,
- explain least squares, coefficient standard errors, R^2 and the difference between confidence and prediction intervals,
- check model assumptions with residual analysis and recognise collinearity,
- include transformations and interaction terms,
- fit nonlinear physical or empirical models with `scipy.optimize.curve_fit` and quantify parameter uncertainty,
- (extension) plan a full factorial experiment and estimate main effects and interactions.

Datasets in this chapter

- Concrete Compressive Strength (UCI)

5.1 The linear model

Definition: Multiple linear regression

The response y is modelled as a linear function of p inputs plus a random error:

$$y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \varepsilon_i, \qquad \varepsilon_i \sim N(0, \sigma^2) \text{ independent.}$$

In matrix form, $y = X\beta + \varepsilon$, where the design matrix X has a first column of ones for the intercept.

"Linear" refers to the **parameters**, not to the inputs. Models like $y = \beta_0 + \beta_1 \ln x + \beta_2 x^2$ or $y = \beta_0 + \beta_1 x_1 x_2$ are linear models, because they are linear in β. This makes the linear model much more flexible than its name suggests.

5.1.1 Estimation by least squares

The coefficients are chosen to minimise the sum of squared residuals $\sum_i (y_i - \hat{y}_i)^2$. Setting the derivatives to zero gives the **normal equations** and their solution:

Least squares estimate

$$\hat{\beta} = (X^\top X)^{-1} X^\top y, \qquad \widehat{\mathrm{Cov}}(\hat{\beta}) = \hat{\sigma}^2 (X^\top X)^{-1}, \qquad \hat{\sigma}^2 = \frac{1}{n-p-1} \sum_i (y_i - \hat{y}_i)^2.$$

The square roots of the diagonal of $\widehat{\mathrm{Cov}}(\hat{\beta})$ are the **standard errors** of the coefficients.

(Numerically, software never inverts $X^\top X$ explicitly but uses a QR or SVD decomposition – more stable, same result.)

5.1.2 Interpreting the output

- **Coefficient $\hat{\beta}_j$:** expected change of y when x_j increases by one unit *and all other inputs stay constant*. The last part is crucial – and often unrealistic when inputs are correlated.
- **Standard error, t-value, p-value:** test of $H_0 : \beta_j = 0$ given the other inputs in the model; the 95 % confidence interval is $\hat{\beta}_j \pm t_{0.975}\,\mathrm{SE}(\hat{\beta}_j)$.
- **Coefficient of determination** $R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$: share of the variance of y explained by the model. It never decreases when inputs are added; the **adjusted** R^2 penalises additional inputs.

Pitfall: R^2 measures fit, not validity

A high R^2 does not mean that the model is correct, that it predicts well on new data or that the relationships are causal. Conversely, a low R^2 can be perfectly acceptable if the process is inherently noisy. Always look at the residuals – and, for prediction, at the error on test data (Chapter 6).

5.1.3 Model assumptions and residual analysis

The statistical statements (standard errors, p-values, intervals) rely on assumptions about the errors. The **residuals** $e_i = y_i - \hat{y}_i$ are our window to check them:

Assumption	Check	Typical violation in engineering data
Linearity (correct functional form)	residuals vs. fitted values and vs. each input: no pattern	saturation, threshold effects
Constant variance	spread of residuals constant	relative errors: spread grows with the response
Independence	residuals vs. time / order	time series, batches, repeated measurements on the same part
Normality	QQ plot of residuals	outliers, skewed responses

Engineering practice: transformations

Many engineering relationships are multiplicative or saturating. A log transformation of the response or of an input often linearises the relationship and stabilises the variance at the same time. Physical knowledge tells you which transformation makes sense: for concrete, strength grows quickly at first and then levels off, which suggests ln(age).

5.1.4 Collinearity

When inputs are strongly correlated, their individual effects cannot be separated well: the coefficients get large standard errors, can have implausible signs and change strongly when another input is added. The **variance inflation factor** $\mathrm{VIF}_j = 1/(1 - R_j^2)$, where R_j^2 is the R^2 of regressing x_j on all other inputs, quantifies this; values above about 5–10 indicate problematic collinearity. Collinearity does not harm predictions within the range of the data, but it ruins the interpretation of individual coefficients.

```
# Setup: works locally and in Google Colab
import importlib.util, subprocess, sys
for pkg in ["ucimlrepo"]:
    if importlib.util.find_spec(pkg) is None:
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg],
 ↪  check=True)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
import statsmodels.api as sm

sns.set_theme(style="whitegrid")
```

5.2 Live coding: concrete strength

The dataset contains 1,030 laboratory tests of concrete. Each test is described by the amounts of seven mixture components in kg per m^3 of concrete and the age of the specimen at testing (1 to 365 days); the response is the compressive strength in MPa. Concrete strength is a classic example of a highly nonlinear material property: it depends on the water/cement ratio, on supplementary materials such as blast furnace slag and fly ash, on the superplasticizer and on the curing time.

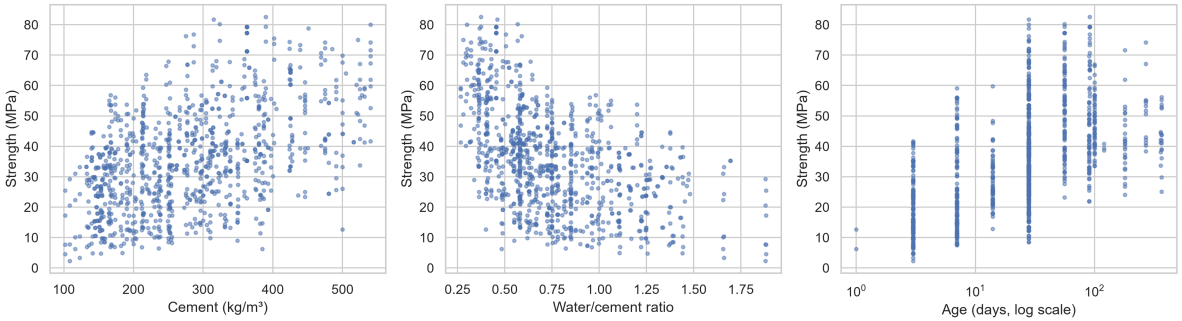
```
from ucimlrepo import fetch_ucirepo

concrete = fetch_ucirepo(id=165)
conc = concrete.data.features.copy()
conc.columns = ["cement", "slag", "fly_ash", "water", "superplasticizer",
                "coarse_agg", "fine_agg", "age"]          # kg/m³, age in
                 ↪  days
conc["strength"] = concrete.data.targets.iloc[:, 0]        # MPa
conc.describe().T.round(1)
```

	count	mean	std	min	25%	50%	75%	max
cement	1030.0	281.2	104.5	102.0	192.4	272.9	350.0	540.0
slag	1030.0	73.9	86.3	0.0	0.0	22.0	143.0	359.4
fly_ash	1030.0	54.2	64.0	0.0	0.0	0.0	118.3	200.1

	count	mean	std	min	25%	50%	75%	max
water	1030.0	181.6	21.4	121.8	164.9	185.0	192.0	247.0
superplasticizer	1030.0	6.2	6.0	0.0	0.0	6.4	10.2	32.2
coarse_agg	1030.0	972.9	77.8	801.0	932.0	968.0	1029.4	1145.0
fine_agg	1030.0	773.6	80.2	594.0	731.0	779.5	824.0	992.6
age	1030.0	45.7	63.2	1.0	7.0	28.0	56.0	365.0
strength	1030.0	35.8	16.7	2.3	23.7	34.4	46.1	82.6

```
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
axes[0].scatter(conc["cement"], conc["strength"], s=8, alpha=0.5)
axes[0].set(xlabel="Cement (kg/m³)", ylabel="Strength (MPa)")
axes[1].scatter(conc["water"] / conc["cement"], conc["strength"], s=8,
 ↪  alpha=0.5)
axes[1].set(xlabel="Water/cement ratio", ylabel="Strength (MPa)")
axes[2].scatter(conc["age"], conc["strength"], s=8, alpha=0.5)
axes[2].set(xscale="log", xlabel="Age (days, log scale)", ylabel="Strength
 ↪  (MPa)")
plt.tight_layout(); plt.show()
```



Strength increases with the cement content, decreases with the water/cement ratio (a classical rule in concrete technology, known as Abrams' law) and grows with age – roughly linearly in the *logarithm* of the age. We therefore enter age as ln(age):

```
model = smf.ols("strength ~ cement + slag + fly_ash + water +
 ↪  superplasticizer"
                " + coarse_agg + fine_agg + np.log(age)", data=conc).fit()
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
```

```
Dep. Variable:                strength   R-squared:                       0.819
Model:                             OLS   Adj. R-squared:                  0.817
Method:                  Least Squares   F-statistic:                     575.6
Date:                 Tue, 10 Nov 2026   Prob (F-statistic):              0.00
Time:                         19:58:09   Log-Likelihood:                -3482.4
No. Observations:                1030   AIC:                             6983.
Df Residuals:                    1021   BIC:                             7027.
Df Model:                           8
Covariance Type:            nonrobust
================================================================================
                   coef    std err          t      P>|t|      [0.025      0.975]
--------------------------------------------------------------------------------
Intercept       -77.5334     18.331     -4.230      0.000    -113.504     -41.563
cement            0.1349      0.006     23.055      0.000       0.123       0.146
slag              0.1159      0.007     16.647      0.000       0.102       0.130
fly_ash           0.0944      0.009     10.944      0.000       0.077       0.111
water            -0.1288      0.028     -4.673      0.000      -0.183      -0.075
superplasticizer  0.1188      0.064      1.846      0.065      -0.007       0.245
coarse_agg        0.0297      0.006      4.593      0.000       0.017       0.042
fine_agg          0.0360      0.007      4.886      0.000       0.022       0.050
np.log(age)       8.7466      0.192     45.610      0.000       8.370       9.123
==============================================================================
Omnibus:                       19.634   Durbin-Watson:                   1.305
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               22.742
Skew:                           0.266   Prob(JB):                     1.15e-05
Kurtosis:                       3.496   Cond. No.                     1.06e+05
==============================================================================

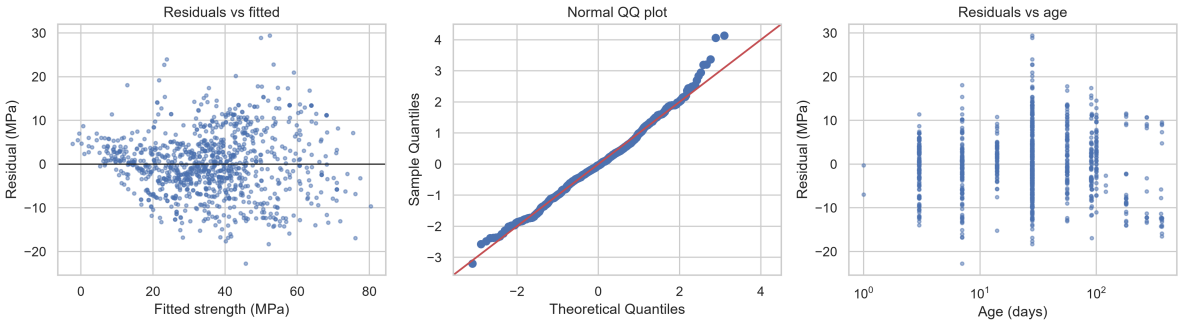
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.06e+05. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Discussion. Read the summary from top to bottom:

- **R^2 and adjusted R^2** tell you how much of the variation in strength the model explains.
- **Coefficients:** each value is the change in MPa per additional kg/m³ of the component (or per unit of ln age), holding all other components constant. Cement, slag and fly ash act as binders and should have positive coefficients; water should have a negative one.
- **Holding constant is tricky here:** the components add up to roughly the volume of one cubic metre. If we add more cement, something else must be reduced. This built-in dependence causes collinearity, which explains why some aggregate coefficients can be small or have surprising signs.

- **Notes at the bottom:** statsmodels warns, for example, about a large condition number – a hint at collinearity or very differently scaled inputs.

```
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
axes[0].scatter(model.fittedvalues, model.resid, s=8, alpha=0.5)
axes[0].axhline(0, color="k", lw=1)
axes[0].set(xlabel="Fitted strength (MPa)", ylabel="Residual (MPa)",
 ↪  title="Residuals vs fitted")
sm.qqplot(model.resid, line="45", fit=True, ax=axes[1]);
 ↪  axes[1].set_title("Normal QQ plot")
axes[2].scatter(conc["age"], model.resid, s=8, alpha=0.5)
axes[2].set(xscale="log", xlabel="Age (days)", ylabel="Residual (MPa)",
 ↪  title="Residuals vs age")
plt.tight_layout(); plt.show()
```



Discussion. Look for patterns: a curved band in the residuals vs. fitted values indicates a missing nonlinearity; a funnel shape indicates non-constant variance; structure in the residuals vs. age shows that the ln transformation captures the age effect only approximately. Systematic patterns tell us that the linear model is an approximation – which is fine for interpretation of the main effects but limits its predictive accuracy. Flexible models in Chapters 6 and 8 will do better.

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

Xv = sm.add_constant(conc[["cement", "slag", "fly_ash", "water",
 ↪  "superplasticizer",
                           "coarse_agg",
                            ↪  "fine_agg"]].assign(log_age=np.log(conc["age"])))
vif = pd.Series([variance_inflation_factor(Xv.values, i) for i in range(1,
 ↪  Xv.shape[1])],
                index=Xv.columns[1:], name="VIF")
vif.round(1)
```

```
cement              7.5
slag                7.3
fly_ash             6.1
water               7.0
superplasticizer    3.0
coarse_agg          5.1
fine_agg            7.0
log_age             1.1
Name: VIF, dtype: float64
```

5.2.1 Confidence vs. prediction intervals

A regression model gives two different kinds of intervals for a new input x_0:

> **Definition: Confidence interval vs. prediction interval**
>
> - The **confidence interval** for the *mean response* $E[y \mid x_0]$ reflects only the uncertainty of the estimated coefficients. It shrinks towards zero as n grows.
> - The **prediction interval** for a *single new observation* y_0 additionally contains the scatter σ of individual observations around the mean. It never becomes narrower than about $\pm 2\sigma$.

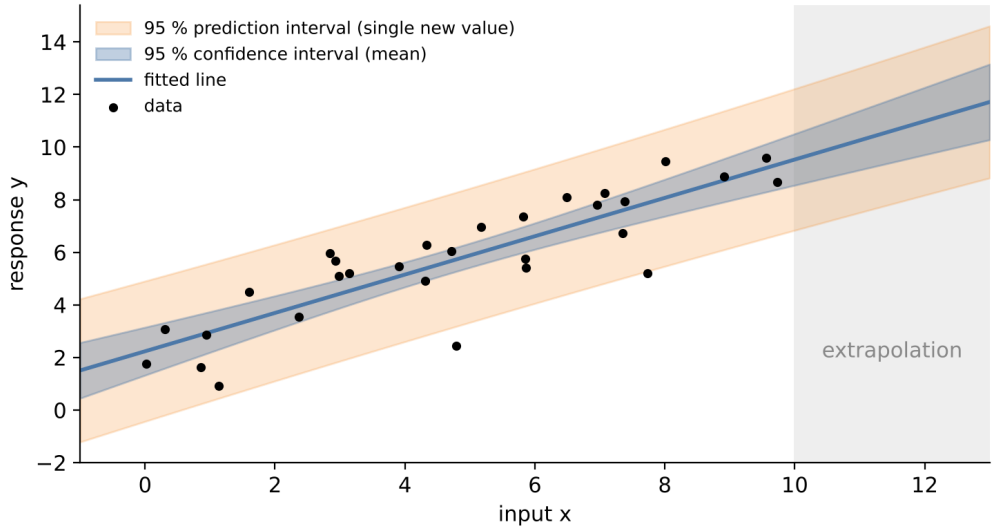


Figure 5.1: Confidence band of the regression line (mean response) and prediction band for single new observations. Both widen with distance from the centre of the data – and in the extrapolation region, neither is supported by data.

For an engineer who needs to know the strength of *the next* concrete batch, the prediction interval is the relevant one.

```
new_mix = pd.DataFrame({"cement": [350], "slag": [0], "fly_ash": [0],
 ↪  "water": [180],
                         "superplasticizer": [5], "coarse_agg": [1000],
                          ↪  "fine_agg": [780],
                         "age": [28]})
model.get_prediction(new_mix).summary_frame(alpha=0.05).round(1)
```

	mean	mean_se	mean_ci_lower	mean_ci_upper	obs_ci_lower	obs_ci_upper
0	34.0	0.5	32.9	35.0	19.9	48.0

`mean_ci_lower/upper` is the confidence interval of the mean strength of such mixes at 28 days, `obs_ci_lower/upper` the prediction interval for a single specimen. The difference shows how much individual tests scatter.

> **Pitfall: extrapolation**
>
> Regression models are only supported by data within the range of the inputs – and, with correlated inputs, only within the region where input *combinations* were observed. A mix with an unusual combination of components may lie within every single input range and still be far from the data.

5.3 Nonlinear curve fitting

Many engineering models are nonlinear in their parameters: hardening and creep curves, exponential decay, saturation curves, characteristic curves of components. They cannot be written as a linear model, but their parameters can still be estimated by least squares – numerically, by iteration:

$$\hat{\theta} = \arg\min_{\theta} \sum_{i} \left(y_i - f(x_i; \theta) \right)^2 .$$

`scipy.optimize.curve_fit` solves this problem with the Levenberg–Marquardt algorithm or a trust-region method.

Engineering practice: fitting physical models

- **Starting values** matter: the iteration can converge to a local minimum or not at all. Derive them from physics or read them off a plot.
- **Parameter uncertainty:** the covariance matrix `pcov` returned by `curve_fit` gives standard errors (based on a linear approximation around the optimum; valid for approximately normal, independent errors).
- **Identifiability:** if two parameters are strongly correlated, the data cannot separate them – the model is too complex for the data or the experimental design is poor.
- **Linearisation** (e.g. fitting $\ln y$) is tempting but changes the error structure; a direct nonlinear fit is usually preferable.
- **Physically meaningful parameters** are an advantage over a polynomial: they can be compared between materials and checked for plausibility.

As an example we model the strength development of one concrete mix over time with the hyperbolic model

$$f_c(t) = f_\infty \, \frac{t}{a + t},$$

with the final strength f_∞ (MPa) and the time a (days) at which half of it is reached. Physically it describes a hydration process that slows down over time.

```
from scipy.optimize import curve_fit

mix_cols = ["cement", "slag", "fly_ash", "water", "superplasticizer",
 ↪  "coarse_agg", "fine_agg"]
n_ages = conc.groupby(mix_cols)["age"].nunique()
best = n_ages.idxmax()                                    # the mix tested at
 ↪  most ages
mix = conc[(conc[mix_cols] == pd.Series(best,
 ↪  index=mix_cols)).all(axis=1)].sort_values("age")
print(f"Mix tested at {n_ages.max()} different ages:", dict(zip(mix_cols,
 ↪  best)))

def strength_model(t, f_inf, a):
    return f_inf * t / (a + t)

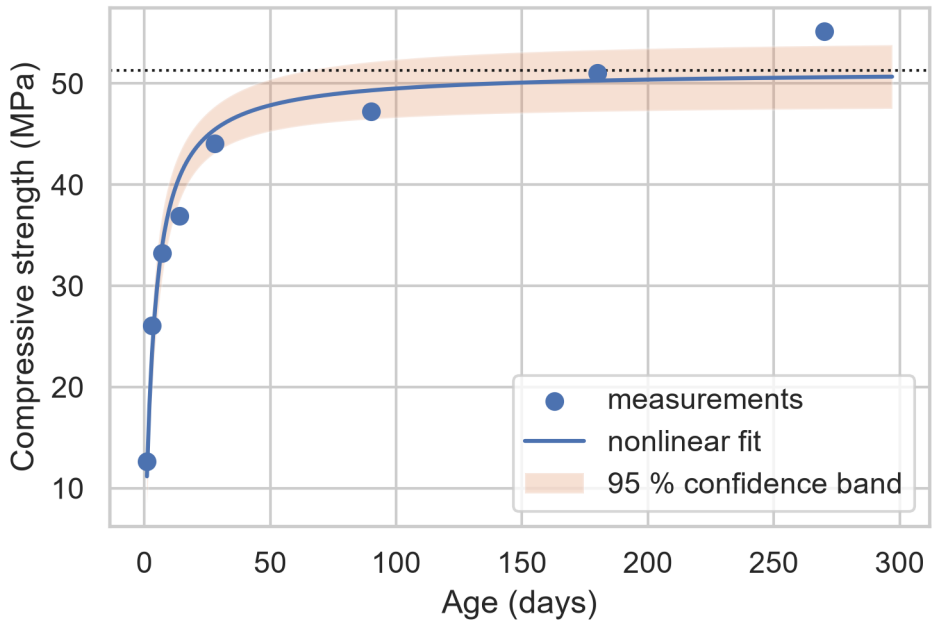
p0 = [mix["strength"].max(), 7]                           # starting values
 ↪  from the data
popt, pcov = curve_fit(strength_model, mix["age"], mix["strength"], p0=p0)
perr = np.sqrt(np.diag(pcov))
corr = pcov[0, 1] / (perr[0] * perr[1])
```

```
print(f"f_inf = {popt[0]:.1f} ± {perr[0]:.1f} MPa,  a = {popt[1]:.1f} ±
 ↪  {perr[1]:.1f} days,  "
      f"correlation = {corr:.2f}")
```

```
Mix tested at 8 different ages: {'cement': np.float64(500.0), 'slag': np.float64(0.0), 'fly_a
f_inf = 51.3 ± 1.7 MPa,  a = 3.6 ± 0.6 days,  correlation = 0.62
```

```
# 95 % confidence band of the fitted curve (delta method)
t = np.linspace(1, mix["age"].max() * 1.1, 300)
f_inf, a = popt
J = np.column_stack([t / (a + t), -f_inf * t / (a + t) ** 2])   # partial
 ↪  derivatives
se = np.sqrt(np.einsum("ij,jk,ik->i", J, pcov, J))
fit = strength_model(t, *popt)

plt.scatter(mix["age"], mix["strength"], label="measurements", zorder=3)
plt.plot(t, fit, label="nonlinear fit")
plt.fill_between(t, fit - 1.96 * se, fit + 1.96 * se, alpha=0.25, label="95 %
 ↪  confidence band")
plt.axhline(f_inf, ls=":", c="k", lw=1)
plt.xlabel("Age (days)"); plt.ylabel("Compressive strength (MPa)");
 ↪  plt.legend()
plt.show()
```



Discussion. The two parameters have a direct engineering meaning and can be compared between mixes. The confidence band is computed with the **delta method**: the variance of the fitted curve is approximated by $J\,\mathrm{Cov}(\hat{\theta})\,J^{\top}$, where J contains the partial derivatives of the model with respect to the parameters. The band is narrow where many measurements exist and widens towards long ages. A strong correlation between $\hat{f}_{\infty}$ and $\hat{a}$ is typical for saturation models: if the data do not reach the plateau, a higher final strength reached later explains them almost equally well.

5.4 Back to the decision

A strength promise concerns *individual* elements, not the average of many – so the relevant interval is the **prediction interval**. To be on the safe side, we require that the lower end of the 95 % prediction interval reaches 35 MPa. We check 28 days and, if necessary, search for the earliest age at which the requirement is met.

```
required = 35.0
ages = np.arange(1, 366)
candidates = pd.concat([new_mix.drop(columns="age")] * len(ages),
 ↪  ignore_index=True).assign(age=ages)
pi_low =
 ↪  model.get_prediction(candidates).summary_frame(alpha=0.05)["obs_ci_lower"].values
low_28 = pi_low[ages == 28][0]
print(f"Lower 95 % prediction bound at 28 days: {low_28:.1f} MPa")
if low_28 >= required:
    print("Decision: approve the mix for the 28-day promise.")
elif (pi_low >= required).any():
    print(f"Decision: not at 28 days - promise {ages[np.argmax(pi_low >=
     ↪  required)]} days, or confirm with test specimens.")
else:
    print("Decision: the model cannot guarantee 35 MPa within a year - adjust
     ↪  the mix (e.g. lower w/c ratio).")
```

```
Lower 95 % prediction bound at 28 days: 19.9 MPa
Decision: not at 28 days - promise 158 days, or confirm with test specimens.
```

The recommendation should also state the limits of the model: it is a linear approximation fitted to laboratory specimens from one study, and the residual analysis showed systematic deviations. In practice, no plant would release a new mix on the basis of a regression model alone – but the model tells us whether the trial batches are worth casting and which test age to plan.

5.5 Extension: design of experiments – a 2^3 full factorial design

Extension – optional material, not required for the exam.

The concrete data are *observational*: the mixes were not chosen to estimate effects efficiently, and the inputs are correlated. In a **designed experiment** we choose the factor settings ourselves, so that the effects can be separated cleanly with as few runs as possible.

> **Definition: Full factorial design**
>
> In a 2^k full factorial design, each of k factors is set to a low (-1) and a high ($+1$) level, and all 2^k combinations are run. The columns of the coded design matrix are **orthogonal**, so all main effects and interactions are estimated independently of each other and with the smallest possible variance.

With three factors, 8 runs estimate 3 main effects, 3 two-factor interactions and 1 three-factor interaction. **Replicates** (repeating all runs) provide an estimate of the experimental error. **Randomising** the run order protects against drifts in the test setup.

The example simulates such an experiment with a known "true" model, so we can check whether the analysis recovers it.

```
import itertools

design = pd.DataFrame(list(itertools.product([-1, 1], repeat=3)),
                      columns=["cement", "water", "age"])
rng = np.random.default_rng(42)
design = pd.concat([design, design], ignore_index=True)        # 2 replicates
design["strength"] = (40 + 8 * design.cement - 6 * design.water + 5 *
 ↪  design.age
                      + 2 * design.cement * design.age + rng.normal(0, 1.5,
                       ↪  len(design)))

doe = smf.ols("strength ~ cement * water * age", data=design).fit()
print(doe.params.round(2))
print("Correlation between design columns:\n",
      np.round(np.corrcoef(design[["cement", "water", "age"]].T.values), 2))
```

```
Intercept          39.92
cement              7.59
water              -5.40
cement:water       -0.33
age                 4.80
```

```
cement:age           2.19
water:age           -0.11
cement:water:age    -0.54
dtype: float64
Correlation between design columns:
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

The estimated coefficients reproduce the true values (8, −6, 5 and 2 for the cement × age interaction) up to the noise, and the non-existing interactions are estimated near zero. With coded factors, a coefficient is half the **effect** (change of the response from the low to the high level). The design columns are uncorrelated – the exact opposite of the observational concrete data.

> **!** Key takeaways
>
> - Linear regression is linear in the parameters; transformations and interactions make it flexible.
> - Coefficients mean "change per unit, all else constant" – collinearity makes this hard to interpret.
> - Check residuals for patterns; R^2 alone does not validate a model.
> - Use prediction intervals for single new observations; beware of extrapolation.
> - Nonlinear models with physically meaningful parameters are fitted with `curve_fit`; good starting values and parameter correlations deserve attention.

5.6 Exercises

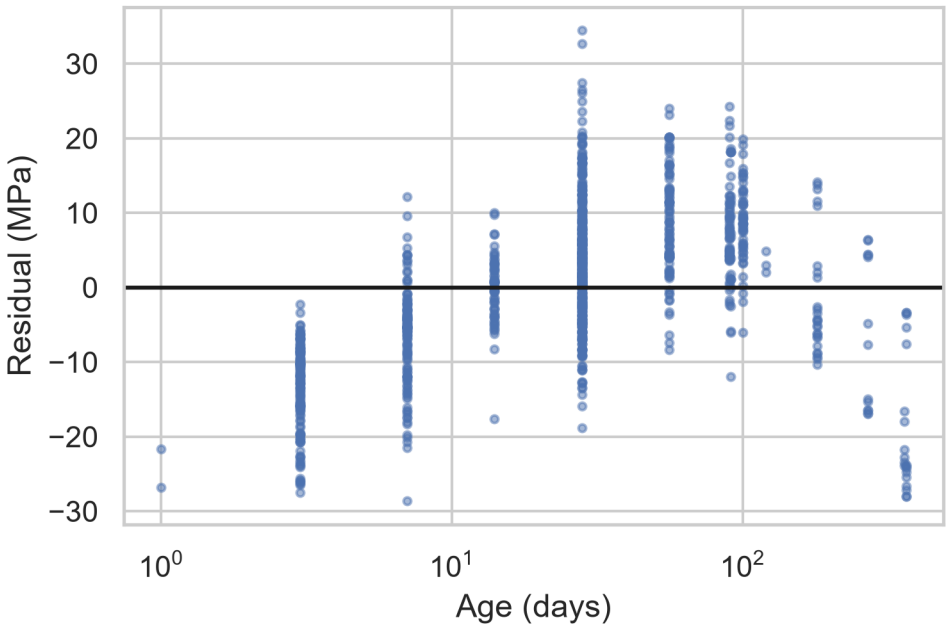
1. Replace `np.log(age)` by `age` in the linear model. How do R^2 and the residual plot change? Explain physically why the log transform helps.
2. Compute the water/cement ratio and use it as an input instead of `water` and `cement`. Compare R^2 and interpretability with the original model.
3. Which coefficients of the concrete model are statistically significant? Are the signs plausible from a materials point of view? Relate your answer to the VIF values.
4. Fit the strength model $f_\infty\, t/(a+t)$ to all mixes with at least four test ages. How does the final strength f_∞ depend on the water/cement ratio?
5. Compare the hyperbolic model with a logarithmic model $f(t) = c_0 + c_1 \ln t$ for the selected mix. Which one extrapolates more plausibly to 1000 days, and why is extrapolation risky anyway?
6. (Extension) Extend the DoE example to a 2^4 design with a fourth factor. How many runs are needed? What is a fractional factorial design, and what is its price?

5.6.1 Solutions

Solution to Exercise 1

```
model_lin_age = smf.ols("strength ~ cement + slag + fly_ash + water +
 ↪  superplasticizer"
                        " + coarse_agg + fine_agg + age",
                         ↪  data=conc).fit()
print(f"R² with log(age): {model.rsquared:.3f},  with age:
 ↪  {model_lin_age.rsquared:.3f}")
plt.scatter(conc["age"], model_lin_age.resid, s=8, alpha=0.5)
plt.axhline(0, c="k"); plt.xscale("log"); plt.xlabel("Age (days)");
 ↪  plt.ylabel("Residual (MPa)")
plt.show()
```

```
R² with log(age): 0.819,  with age: 0.616
```



With a linear age term, R^2 drops noticeably and the residuals show a clear curved pattern over age: the model overestimates young and very old specimens and underestimates intermediate ages. Physically, hydration of cement is fast in the first days and slows down later – the strength gain per day decreases with age. ln(age) captures this diminishing return.

Solution to Exercise 2

```
conc["wc"] = conc["water"] / conc["cement"]
model_wc = smf.ols("strength ~ wc + slag + fly_ash + superplasticizer"
                   " + coarse_agg + fine_agg + np.log(age)",
                    ↪  data=conc).fit()
print(f"R²: original {model.rsquared:.3f}, with w/c ratio
 ↪  {model_wc.rsquared:.3f}")
model_wc.params.round(3)
```

```
R²: original 0.819, with w/c ratio 0.759

Intercept           21.087
wc                 -35.379
slag                 0.083
fly_ash              0.036
superplasticizer     0.475
coarse_agg           0.001
fine_agg             0.003
np.log(age)          8.240
dtype: float64
```

The fit is of similar quality – the w/c ratio carries most of the information of water and cement together. The advantage is interpretability: the coefficient of `wc` directly expresses the well-known strength loss with increasing water/cement ratio, a quantity concrete engineers think in. Strictly, Abrams' law is nonlinear (strength $B^{-w/c}$ with a material constant B), so ln(strength) vs. `wc` might fit even better.

Solution to Exercise 3

```
pd.DataFrame({"coef": model.params, "p": model.pvalues, "VIF":
 ↪  vif}).round(4)
```

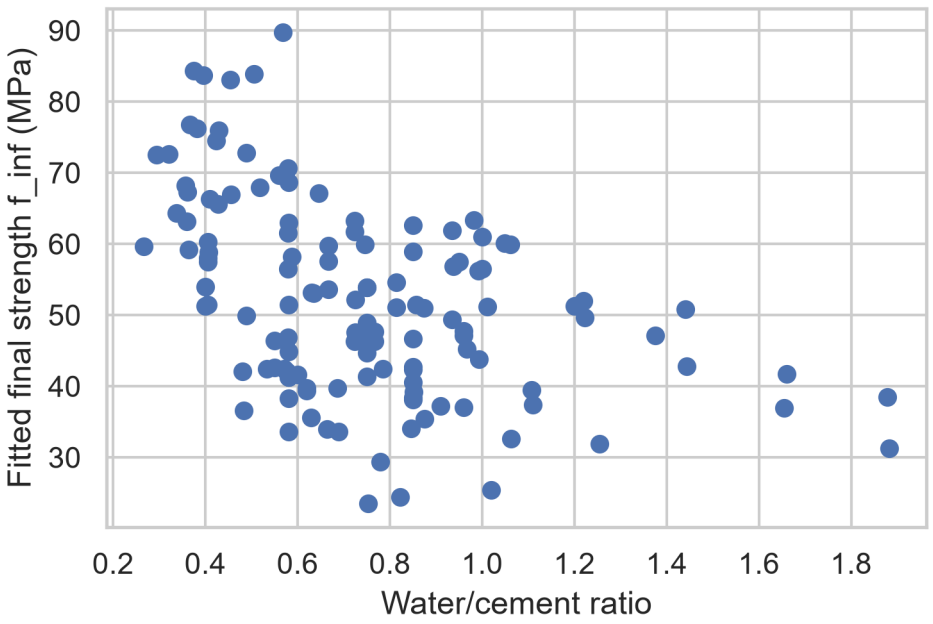
	coef	p	VIF
Intercept	-77.5334	0.0000	NaN
cement	0.1349	0.0000	7.5324
coarse_agg	0.0297	0.0000	5.0890
fine_agg	0.0360	0.0000	7.0304
fly_ash	0.0944	0.0000	6.1411
log_age	NaN	NaN	1.0523

np.log(age)	8.7466	0.0000	NaN
slag	0.1159	0.0000	7.2792
superplasticizer	0.1188	0.0652	2.9803
water	-0.1288	0.0000	6.9839

Expected signs: cement, slag and fly ash (binders) and superplasticizer positive; water negative; age positive. Coefficients that are not significant or have counterintuitive signs typically belong to inputs with high VIF – the aggregates, which are strongly linked to the other components through the fixed total volume. Their individual coefficients cannot be interpreted reliably, even though the model as a whole predicts reasonably.

Solution to Exercise 4

```
rows = []
for key, g in conc.groupby(mix_cols):
    if g["age"].nunique() < 4:
        continue
    try:
        p, _ = curve_fit(strength_model, g["age"], g["strength"],
                         p0=[g["strength"].max(), 7], maxfev=5000)
    except RuntimeError:
        continue                                          # fit did not
         ↪ converge
    rows.append({"wc": g["water"].iloc[0] / g["cement"].iloc[0], "f_inf":
 ↪ p[0], "a": p[1]})
fits = pd.DataFrame(rows)
fits = fits[(fits["a"] > 0) & (fits["f_inf"] < 200)]        # drop
 ↪ physically meaningless fits
plt.scatter(fits["wc"], fits["f_inf"])
plt.xlabel("Water/cement ratio"); plt.ylabel("Fitted final strength f_inf
 ↪ (MPa)")
plt.show()
print(f"{len(fits)} mixes fitted")
```

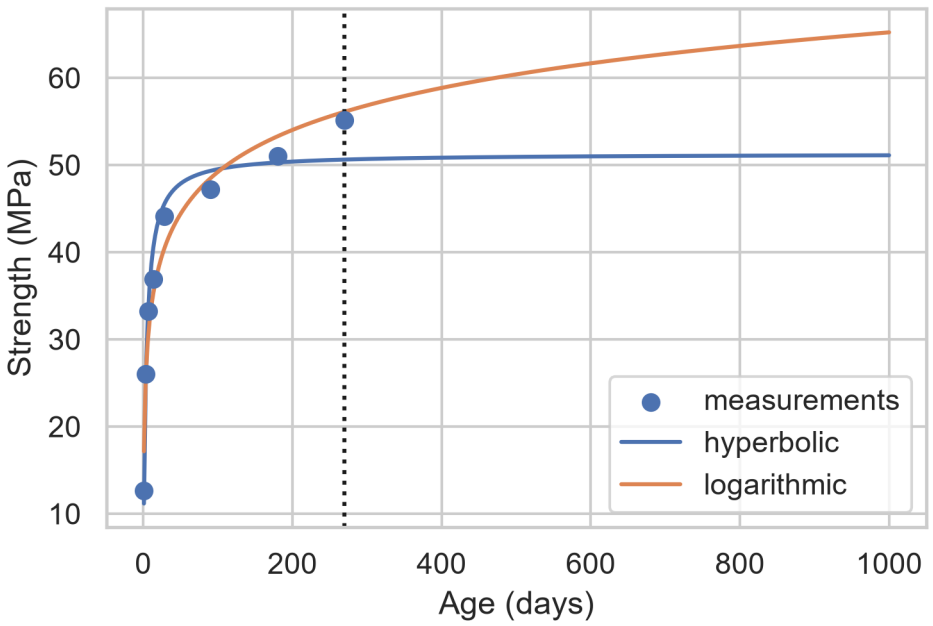



```
131 mixes fitted
```

The final strength decreases clearly with increasing water/cement ratio – the same relationship as in the linear model, but now expressed through a physically meaningful parameter. Some fits must be discarded because the mix was tested at too few or too early ages to determine the plateau (the parameters are then not identifiable). Checking plausibility of fitted parameters is an essential step when fitting many curves automatically.

Solution to Exercise 5

```
c1, c0 = np.polyfit(np.log(mix["age"]), mix["strength"], deg=1)
t_long = np.linspace(1, 1000, 400)
plt.scatter(mix["age"], mix["strength"], zorder=3, label="measurements")
plt.plot(t_long, strength_model(t_long, *popt), label="hyperbolic")
plt.plot(t_long, c0 + c1 * np.log(t_long), label="logarithmic")
plt.axvline(mix["age"].max(), ls=":", c="k")
plt.xlabel("Age (days)"); plt.ylabel("Strength (MPa)"); plt.legend()
plt.show()
```



Within the measured range both models can fit similarly well. Beyond it, the logarithmic model grows without bound, while the hyperbolic model approaches a final strength – physically more plausible, since hydration eventually stops. But neither model is validated beyond the last measurement: long-term effects (e.g. carbonation, drying) are not in the data. Extrapolation relies on the model *form*, not on data – and different plausible forms can give very different answers.

Solution to Exercise 6

A 2^4 full factorial design needs $2^4 = 16$ runs (32 with two replicates). It estimates 4 main effects, 6 two-factor, 4 three-factor and 1 four-factor interaction. A **fractional factorial design** 2^{4-1} uses only 8 runs, chosen such that the main effects remain orthogonal. The price is **aliasing**: some effects can no longer be distinguished – in the 2^{4-1} design with generator $D = ABC$, each main effect is aliased with a three-factor interaction and the two-factor interactions are aliased in pairs. This is acceptable when higher-order interactions are negligible, which is often the case in screening experiments.

```
design4 = pd.DataFrame(list(itertools.product([-1, 1], repeat=3)),
 ↪  columns=list("ABC"))
design4["D"] = design4["A"] * design4["B"] * design4["C"]       #
 ↪  generator D = ABC
design4
```

	A	B	C	D
0	-1	-1	-1	-1
1	-1	-1	1	1
2	-1	1	-1	1
3	-1	1	1	-1
4	1	-1	-1	1
5	1	-1	1	-1
6	1	1	-1	-1
7	1	1	1	1

6 Model Selection, Regularisation and Gaussian Processes

A model that reproduces its training data perfectly is usually useless. Given enough parameters, any model can pass through every data point – including the noise. What we really want is a model that **generalises**: that predicts well for new operating points, new parts, next week's data. This chapter introduces the central idea of machine learning: models must be judged on data they have not seen during fitting.

We study how model complexity affects the prediction error, learn to estimate this error honestly with cross-validation, and meet two ways to control complexity: regularisation, which penalises large coefficients, and Gaussian processes, a Bayesian approach that also tells us how uncertain each prediction is.

Scenario: a surrogate model instead of wind tunnel time

An aerodynamics team is screening thousands of airfoil configurations for a quiet fan blade. Each wind tunnel measurement costs half a day. The team lead asks whether a data-driven model, trained on the existing NASA measurements, could replace the tunnel for the screening stage. The acoustics expert sets the requirement: the prediction error on *new* conditions must be below 2 dB (RMSE), otherwise the ranking of configurations is unreliable.

Your decision: Which model, if any, do you release for the screening?

Learning goals. After this chapter you can

- explain overfitting and the bias–variance trade-off,
- evaluate models with train/test splits and k-fold cross-validation,
- build preprocessing and model as a scikit-learn pipeline and avoid data leakage,
- use Ridge and Lasso regression to control model complexity,
- fit Gaussian process regression models and interpret their prediction uncertainty,
- (extension) explain the idea of Bayesian optimisation for expensive experiments.

Datasets in this chapter

- Airfoil Self-Noise (UCI)
- Combined Cycle Power Plant (UCI)

6.1 Training error, test error and overfitting

Suppose the data are generated by $y = f(x) + \varepsilon$ with an unknown function f and noise ε with variance σ^2. We fit a model $\hat{f}$ on a training set. Its error on the **training data** is always optimistic: the model was tuned to exactly these points. The relevant quantity is the expected error on **new data** from the same process – the *generalisation error* or *test error*.

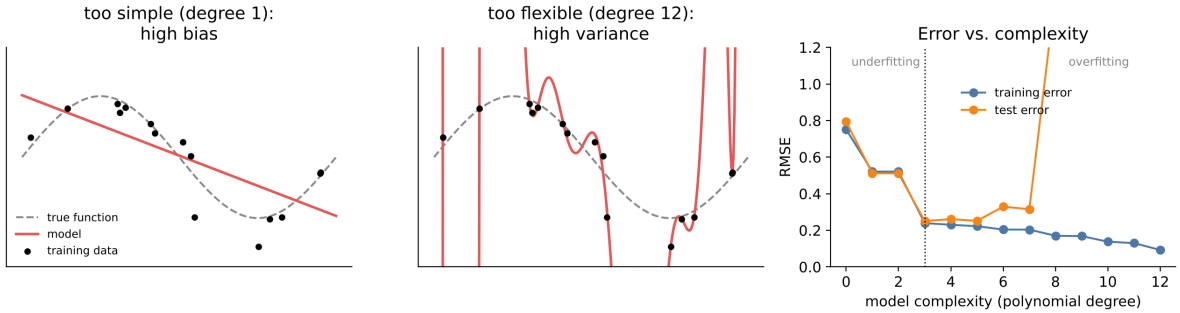


Figure 6.1: Under- and overfitting. A model that is too simple (left) misses the structure; a model that is too flexible (centre) follows the noise of the training data. The test error is U-shaped in model complexity (right), while the training error keeps decreasing.

! Bias–variance decomposition

For squared error loss, the expected test error at a point x decomposes into

$$E\big[(y - \hat{f}(x))^2\big] = \underbrace{\big(E[\hat{f}(x)] - f(x)\big)^2}_{\text{bias}^2} + \underbrace{\operatorname{Var}\big(\hat{f}(x)\big)}_{\text{variance}} + \underbrace{\sigma^2}_{\text{irreducible noise}}.$$

Simple models have high **bias** (systematic error, *underfitting*); flexible models have high **variance** (they change strongly with the training sample, *overfitting*). The best model balances both. The noise σ^2 cannot be reduced by any model.

The practical consequence: **never judge a model by its training error.** Set data aside for testing, or use cross-validation.

6.2 Cross-validation

Definition: k-fold cross-validation

Split the data randomly into k parts (*folds*) of equal size. For each fold, train the model on the other $k-1$ folds and compute the error on the held-out fold. The **cross-validation error** is the average over the k folds. Typical values are $k = 5$ or $k = 10$.

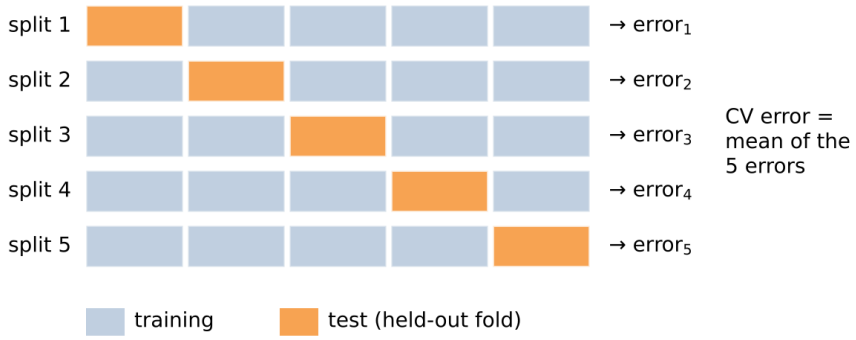


Figure 6.2: 5-fold cross-validation: each observation is used for testing exactly once and for training four times.

Cross-validation uses the data efficiently and gives, besides the mean, an idea of the variability of the error across folds. It is the standard tool to **select** a model or a hyperparameter (polynomial degree, regularisation strength). If the selected model's performance must be reported, use a separate **test set** that was never touched during selection – otherwise the reported error is again optimistic.

Pitfall: data leakage

Any information from the test data that influences training makes the test error too optimistic. Typical leaks: scaling or imputing with statistics of the full dataset, selecting features by their correlation with the target on all data, random splits of time series (Chapter 11), or several rows of the same part/machine in both training and test set. In scikit-learn, putting all preprocessing steps into a `Pipeline` ensures that they are fitted inside each training fold only.

Engineering practice: one median for training and test data

In Chapter 2, the gaps in SECOM were filled with medians. Which median fills the gaps in the *test* data? The **training** median – not one computed from the test data. The median is a parameter of the model, just like a regression coefficient: it is estimated once from the training data and then applied unchanged to all new data. Three reasons:

- In operation there is no "test median": new data often arrive one observation at a time – a single production run, a single sensor reading – and a median cannot be computed from one value.
- The model must always receive the same preprocessing. If a gap was filled with 52.3 during training but with 48.9 during testing, the model sees inputs it was not trained on.
- A test median would itself be a small leak: the prediction for one test point would depend on the other test points.

The same holds for scaling, PCA and every other preprocessing step: `fit` on the training data, `transform` on training *and* test data – exactly what a `Pipeline` does. If the distribution of new data drifts away from the training data, the remedy is to monitor the drift and retrain – not to compute statistics on the test data. (One legitimate exception: if a whole batch – say all wafers of a lot – is always available at prediction time, statistics per batch may be used, but then in exactly the same way during training.)

6.3 Regularisation: Ridge and Lasso

Instead of choosing between few and many features, we can keep a flexible model and penalise large coefficients:

Definition: Ridge and Lasso regression

$$\text{Ridge:} \quad \min_{\beta} \sum_i (y_i - x_i^\top \beta)^2 + \lambda \sum_j \beta_j^2, \qquad \text{Lasso:} \quad \min_{\beta} \sum_i (y_i - x_i^\top \beta)^2 + \lambda \sum_j |\beta_j|.$$

The hyperparameter $\lambda \geq 0$ (in scikit-learn: `alpha`) controls the strength of the penalty: $\lambda = 0$ gives ordinary least squares, large λ shrinks all coefficients towards zero.

Ridge shrinks coefficients smoothly and copes well with collinear inputs. Lasso can set coefficients **exactly to zero** and thus performs feature selection. Because the penalty depends on the size of the coefficients, the inputs must be **standardised** first – otherwise the penalty depends on the units. λ is chosen by cross-validation (`RidgeCV`, `LassoCV`).

```
# Setup: works locally and in Google Colab
import importlib.util, subprocess, sys
for pkg in ["ucimlrepo"]:
    if importlib.util.find_spec(pkg) is None:
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg],
 ↪  check=True)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
```

6.4 Live coding: airfoil self-noise

NASA measured the self-noise of airfoil sections (NACA 0012 profiles of different chord lengths) in an anechoic wind tunnel. The noise is generated by the interaction of the turbulent boundary layer with the trailing edge – a physically complex mechanism that is hard to model from first principles. The dataset has 1,503 measurements:

Input	Meaning	Unit
`frequency`	frequency	Hz
`angle`	angle of attack	deg
`chord`	chord length	m
`velocity`	free-stream velocity	m/s
`thickness`	suction side displacement thickness	m
`spl` (target)	scaled sound pressure level	dB

```
from ucimlrepo import fetch_ucirepo

air = fetch_ucirepo(id=291)
X = air.data.features.copy()
X.columns = ["frequency", "angle", "chord", "velocity", "thickness"]   # Hz,
 ↪  deg, m, m/s, m
y = air.data.targets.iloc[:, 0].rename("spl")                           # dB
X.describe().T.round(4)
```

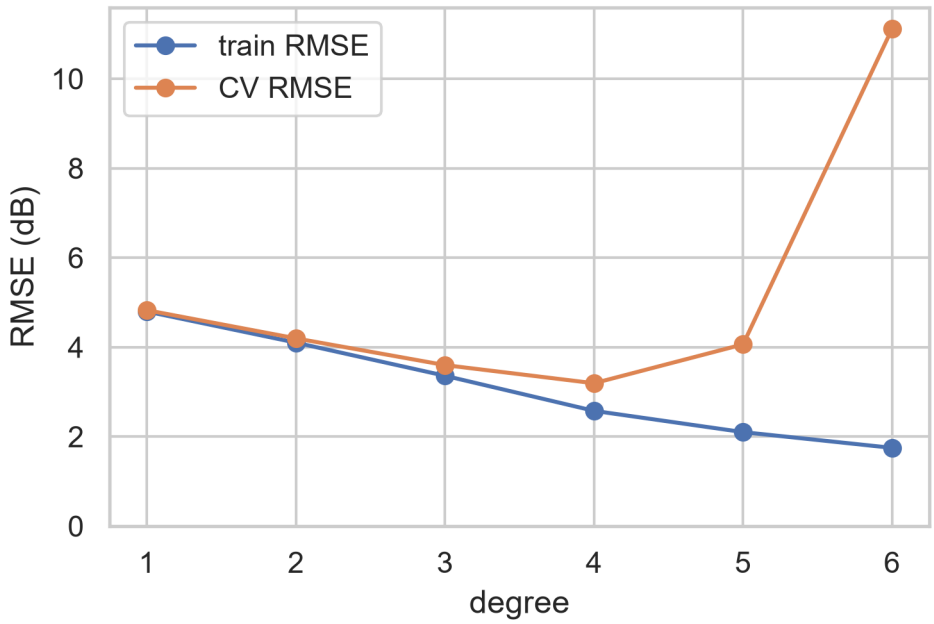
	count	mean	std	min	25%	50%	75%	max
frequency	1503.0	2886.3806	3152.5731	200.0000	800.0000	1600.0000	4000.0000	20000.0000
angle	1503.0	6.7823	5.9181	0.0000	2.0000	5.4000	9.9000	22.2000
chord	1503.0	0.1365	0.0935	0.0254	0.0508	0.1016	0.2286	0.3048
velocity	1503.0	50.8607	15.5728	31.7000	39.6000	39.6000	71.3000	71.3000
thickness	1503.0	0.0111	0.0132	0.0004	0.0025	0.0050	0.0156	0.0584

6.4.1 How complex should the model be?

We fit polynomial models of increasing degree – with five inputs, degree 6 already produces several hundred terms – and compare training and cross-validation errors.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression, RidgeCV
from sklearn.model_selection import KFold, cross_validate

cv = KFold(n_splits=5, shuffle=True, random_state=0)
rows = []
for degree in range(1, 7):
    pipe = make_pipeline(PolynomialFeatures(degree), StandardScaler(),
 ↪  LinearRegression())
    res = cross_validate(pipe, X, y, cv=cv,
 ↪  scoring="neg_root_mean_squared_error",
                         return_train_score=True)
    rows.append({"degree": degree, "terms":
 ↪  PolynomialFeatures(degree).fit(X).n_output_features_,
                 "train RMSE": -res["train_score"].mean(), "CV RMSE":
                  ↪  -res["test_score"].mean()})
scores = pd.DataFrame(rows).set_index("degree")
ax = scores[["train RMSE", "CV RMSE"]].plot(marker="o")
ax.set(ylabel="RMSE (dB)", ylim=(0, None))
plt.show()
scores.round(2)
```



degree	terms	train RMSE	CV RMSE
1	6	4.80	4.83
2	21	4.10	4.20
3	56	3.36	3.60
4	126	2.58	3.19
5	252	2.10	4.06
6	462	1.75	11.12

Discussion. The training error decreases steadily with the degree – more terms can always fit the training data better. The cross-validation error first decreases (less bias) and eventually increases again (more variance); its minimum indicates a good degree. The growing gap between training and CV error is the signature of overfitting. Note also the scale: a straight-line model leaves errors of several dB, which matters for acoustic requirements.

6.4.2 Regularisation in action

Instead of picking the degree exactly, we can use a rich model and let Ridge regression control its complexity. `RidgeCV` selects λ by internal cross-validation; the outer cross-validation evaluates the whole procedure.

```
ridge = make_pipeline(PolynomialFeatures(5), StandardScaler(),
                      RidgeCV(alphas=np.logspace(-4, 3, 30)))
res_ridge = cross_validate(ridge, X, y, cv=cv,
 ↪  scoring="neg_root_mean_squared_error")
print(f"Ridge, degree 5: CV RMSE = {-res_ridge['test_score'].mean():.2f} dB")
```

```
Ridge, degree 5: CV RMSE = 3.14 dB
```

Engineering practice: nested model selection

Here, cross-validation happens at two levels: `RidgeCV` chooses λ on the training part of each outer fold, and the outer loop measures how well the *whole procedure* (including the choice of λ) generalises. This **nested cross-validation** gives an honest estimate. Choosing λ on the same folds used to report the error would again be optimistic.

6.5 Gaussian process regression: predictions with uncertainty

Polynomial and linear models assume a fixed functional form. A **Gaussian process (GP)** places a probability distribution directly over functions – it is the Bayesian approach to regression.

Definition: Gaussian process

A Gaussian process is a collection of random variables $f(x)$, any finite number of which have a joint normal distribution. It is specified by a mean function (often 0) and a **kernel** (covariance function) $\kappa(x, x')$ that states how strongly the function values at x and x' are correlated. A common choice is the squared exponential (RBF) kernel

$$\kappa(x, x') = \sigma_f^2 \exp\left(-\frac{\|x - x'\|^2}{2\ell^2}\right),$$

with signal variance σ_f^2 and **length scale** ℓ (we write κ to avoid confusion with the k of k-fold cross-validation): points closer than about ℓ have similar function values.

Given training data, the GP posterior at a new point is again normal: its **mean** is the prediction and its **standard deviation** quantifies the uncertainty. Near the training points the uncertainty is small, far away it grows back to the prior level. The kernel hyperparameters and the noise level are fitted by maximising the marginal likelihood – no cross-validation needed.

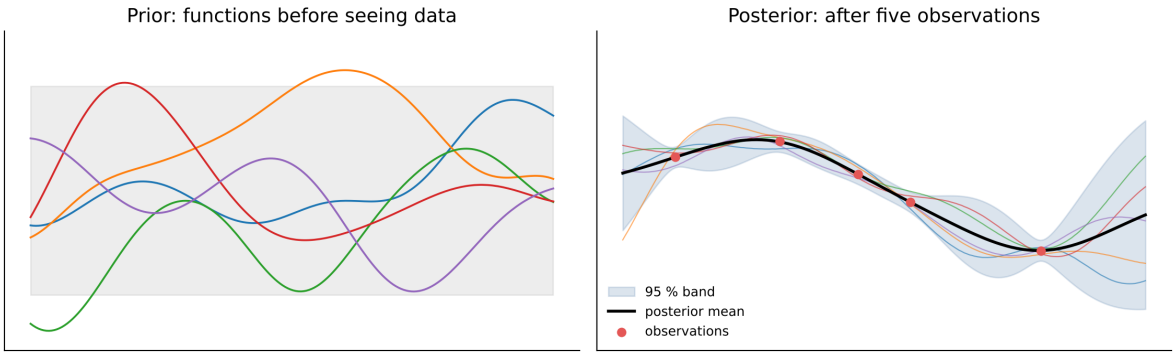


Figure 6.3: A Gaussian process before and after observing data. Left: functions drawn from the prior. Right: after five noisy observations, the posterior mean interpolates the data and the uncertainty band is narrow near the observations and wide in between and outside.

GPs are popular as **surrogate models** for expensive simulations and experiments, where data are scarce and knowing the uncertainty matters. Their limitation: the computational cost grows with n^3, so they suit datasets up to a few thousand points.

We first look at a one-dimensional example: the power output of the combined cycle power plant as a function of ambient temperature, learned from only 15 measurements.

```
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, WhiteKernel, ConstantKernel
 ↪  as C

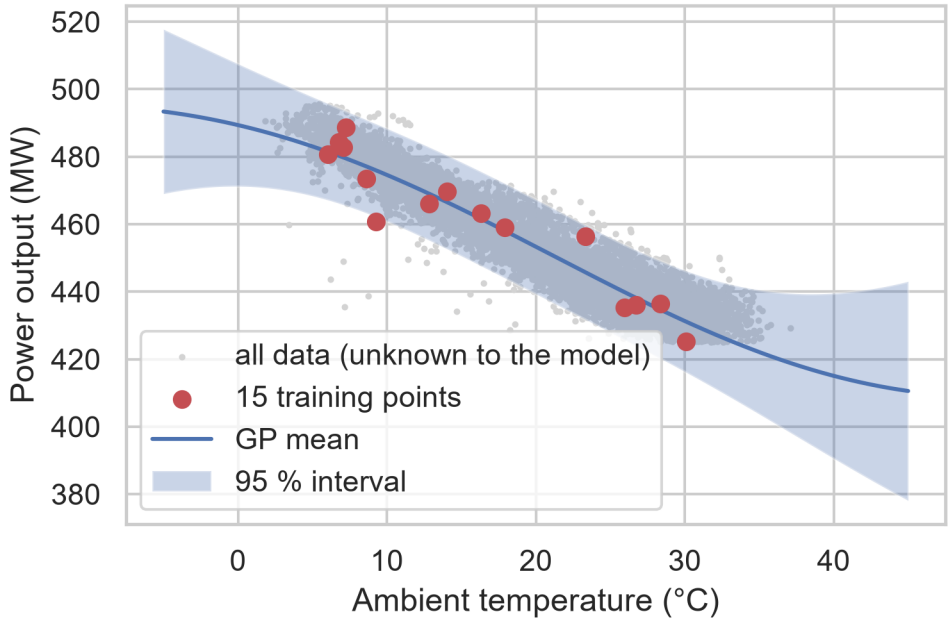
ccpp = fetch_ucirepo(id=294)
pp = pd.concat([ccpp.data.features, ccpp.data.targets], axis=1)
few = pp.sample(15, random_state=1)

kernel = C(100.0) * RBF(length_scale=10.0) + WhiteKernel(noise_level=10.0)
gp = GaussianProcessRegressor(kernel=kernel, normalize_y=True,
 ↪  random_state=0)
gp.fit(few[["AT"]], few["PE"])
print("Fitted kernel:", gp.kernel_)

T = np.linspace(-5, 45, 300).reshape(-1, 1)
mean, std = gp.predict(pd.DataFrame(T, columns=["AT"]), return_std=True)
plt.scatter(pp["AT"], pp["PE"], s=2, c="lightgray", label="all data (unknown
 ↪  to the model)")
plt.scatter(few["AT"], few["PE"], c="C3", zorder=3, label="15 training
 ↪  points")
```

```
plt.plot(T, mean, label="GP mean")
plt.fill_between(T.ravel(), mean - 1.96 * std, mean + 1.96 * std, alpha=0.3,
 ↪  label="95 % interval")
plt.xlabel("Ambient temperature (°C)"); plt.ylabel("Power output (MW)");
 ↪  plt.legend()
plt.show()
```

```
Fitted kernel: 2.27**2 * RBF(length_scale=29.5) + WhiteKernel(noise_level=0.118)
```



Discussion. With only 15 points, the GP recovers the almost linear trend. The fitted `WhiteKernel` noise level estimates the scatter of the individual measurements, the RBF length scale how quickly the function can change. Outside the measured temperature range (below the coldest and above the warmest training point), the band widens: the model honestly reports that it does not know. A polynomial would extrapolate with the same confidence everywhere.

```
# GP on the full airfoil problem (5 inputs), compared with the polynomial
 ↪  ridge model
gp_air = make_pipeline(StandardScaler(), GaussianProcessRegressor(
    kernel=C(1.0) * RBF(length_scale=np.ones(5)) + WhiteKernel(0.1),
 ↪  normalize_y=True))
res_gp = cross_validate(gp_air, X, y, cv=cv,
 ↪  scoring="neg_root_mean_squared_error")
```

```
print(f"Gaussian process: CV RMSE = {-res_gp['test_score'].mean():.2f} dB")
```

```
Gaussian process: CV RMSE = 1.89 dB
```

Here we use one length scale per input (an *anisotropic* or ARD kernel). After fitting, a long length scale indicates that the output changes slowly with that input – a built-in measure of relevance.

6.6 Back to the decision

The requirement refers to new conditions, so we compare the cross-validation errors – never the training errors. Among the models that meet the 2 dB requirement, we release the simplest one; if none meets it, the tunnel cannot yet be replaced.

```
cv_rmse = {
    "linear model": scores.loc[1, "CV RMSE"],
    f"polynomial (degree {scores['CV RMSE'].idxmin()})": scores["CV
     ↪  RMSE"].min(),
    "polynomial degree 5 + Ridge": -res_ridge["test_score"].mean(),
    "Gaussian process": -res_gp["test_score"].mean(),
}
for name, e in cv_rmse.items():
    print(f"{name:32s} CV RMSE = {e:.2f} dB {' ' if e < 2.0 else ' '}")
ok = [name for name, e in cv_rmse.items() if e < 2.0]
print("Decision:", f"release '{ok[0]}' for the screening." if ok else "no
 ↪  model is accurate enough - keep using the tunnel.")
```

```
linear model                     CV RMSE = 4.83 dB
polynomial (degree 4)            CV RMSE = 3.19 dB
polynomial degree 5 + Ridge      CV RMSE = 3.14 dB
Gaussian process                 CV RMSE = 1.89 dB
Decision: release 'Gaussian process' for the screening.
```

The models are listed from simple to complex, so the first model that meets the requirement is released. Two caveats belong in the recommendation: cross-validation estimates the error for conditions *similar* to the measured ones – for configurations outside the measured ranges (e.g. larger chords or higher speeds) the error can be much larger, and the Gaussian process at least signals this through its uncertainty. And the best candidates of the screening should still be confirmed in the tunnel.

6.7 Extension: Bayesian optimisation – design of experiments with a GP

Extension – optional material, not required for the exam.

When every experiment is expensive – a test bench run, a CFD simulation, a production trial – we want to find the optimum with as few runs as possible. **Bayesian optimisation** fits a GP to the runs so far and chooses the next run where an *acquisition function* is largest. The popular **expected improvement** (EI) is

$$\mathrm{EI}(x) = (\mu(x) - y^*)\,\Phi(z) + \sigma(x)\,\varphi(z), \qquad z = \frac{\mu(x) - y^*}{\sigma(x)},$$

where $\mu(x)$ and $\sigma(x)$ are the GP's predicted mean and standard deviation at x, y^* is the best result so far and Φ, φ are the standard normal CDF and PDF. EI is large where the predicted mean is high (*exploitation*) or the uncertainty is large (*exploration*).

The example uses a simulated "experiment": the yield of a process as a function of temperature, which the algorithm does not know.

```
from scipy.stats import norm

def experiment(temp):                          # unknown to the optimiser
    return 40 + 45 * np.exp(-((temp - 172) / 25) ** 2) + 5 * np.sin(temp / 9)

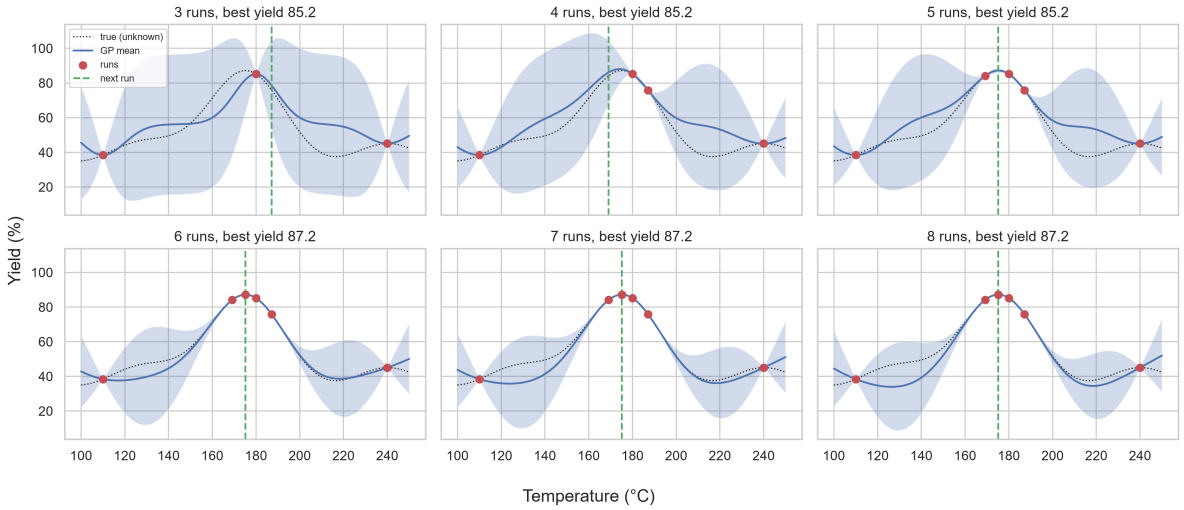
grid = np.linspace(100, 250, 500).reshape(-1, 1)
X_runs = np.array([[110.0], [180.0], [240.0]])            # three initial
 ↪ runs
y_runs = experiment(X_runs.ravel())

fig, axes = plt.subplots(2, 3, figsize=(14, 6), sharex=True, sharey=True)
for ax in axes.ravel():
    gp_bo = GaussianProcessRegressor(C(1.0) * RBF(20.0), normalize_y=True,
 ↪ alpha=1e-6)
    gp_bo.fit(X_runs, y_runs)
    mu, sd = gp_bo.predict(grid, return_std=True)
    best = y_runs.max()
    z = (mu - best) / np.maximum(sd, 1e-9)
    ei = (mu - best) * norm.cdf(z) + sd * norm.pdf(z)       # expected
 ↪ improvement
    x_next = grid[np.argmax(ei)]

    ax.plot(grid, experiment(grid.ravel()), "k:", lw=1, label="true
 ↪ (unknown)")
```

```
    ax.plot(grid, mu, label="GP mean")
    ax.fill_between(grid.ravel(), mu - 1.96 * sd, mu + 1.96 * sd, alpha=0.25)
    ax.scatter(X_runs, y_runs, c="C3", zorder=3, label="runs")
    ax.axvline(x_next[0], c="C2", ls="--", label="next run")
    ax.set_title(f"{len(y_runs)} runs, best yield {best:.1f}")

    X_runs = np.vstack([X_runs, x_next])
    y_runs = np.append(y_runs, experiment(x_next[0]))
axes[0, 0].legend(fontsize=8)
fig.supxlabel("Temperature (°C)"); fig.supylabel("Yield (%)")
plt.tight_layout(); plt.show()
```



Discussion. After a few iterations the algorithm concentrates its runs near the optimum, while regions that cannot contain a better result are not explored further. Compared with a classical grid (Chapter 5), Bayesian optimisation adapts the design sequentially to the results – very efficient when runs are expensive and can be done one after another. Once the optimum is found, EI becomes small everywhere and the proposals cluster; in practice one stops when the expected improvement falls below a threshold.

! Key takeaways

- The training error is optimistic; judge models by their error on unseen data.
- The test error is U-shaped in model complexity: too simple → bias, too flexible → variance.
- k-fold cross-validation selects models and hyperparameters; preprocessing belongs inside the pipeline.

- Ridge and Lasso control complexity by penalising coefficients (standardise first!).
- Gaussian processes give predictions with uncertainty – valuable for small data and expensive experiments.

6.8 Exercises

1. Repeat the degree study for the power plant data (`pp`, target `PE`). Why does the curve look so different from the airfoil case?
2. Why must the `StandardScaler` be part of the pipeline instead of being fitted on all data beforehand? Construct an example in which the difference becomes visible.
3. Plot predicted vs. measured sound pressure level for the Ridge model, using `cross_val_predict`. In which range does the model fail?
4. GP: increase the number of training points from 15 to 50 and 200. How does the uncertainty band change? What happens outside the measured temperature range?
5. (Extension) Replace Ridge by `LassoCV` (degree 3). How many of the polynomial features survive?
6. (Extension) Bayesian optimisation: start with a different set of three runs. Does the algorithm still find the optimum? Compare with a regular grid of the same number of runs.

6.8.1 Solutions

Solution to Exercise 1

```
X_pp, y_pp = pp[["AT", "V", "AP", "RH"]], pp["PE"]
rows = []
for degree in range(1, 6):
    pipe = make_pipeline(PolynomialFeatures(degree), StandardScaler(),
 ↪  LinearRegression())
    res = cross_validate(pipe, X_pp, y_pp, cv=cv,
 ↪  scoring="neg_root_mean_squared_error",
                         return_train_score=True)
    rows.append({"degree": degree, "train RMSE":
 ↪  -res["train_score"].mean(),
                 "CV RMSE": -res["test_score"].mean()})
pd.DataFrame(rows).set_index("degree").round(3)
```

degree	train RMSE	CV RMSE
1	4.557	4.559
2	4.254	4.260
3	4.149	4.162
4	4.088	4.118
5	4.030	4.073

The power plant relationship is nearly linear, there are only four inputs and almost 10,000 observations. Higher degrees improve the fit only slightly, and training and CV errors stay close together – with this much data relative to the number of parameters, overfitting hardly occurs. The remaining error is dominated by the irreducible noise. The airfoil problem is more nonlinear and has six times less data, so flexibility helps first and then overfits.

Solution to Exercise 2

Fitting the scaler on all data uses the means and standard deviations of the test folds – a (here mild) form of leakage. The effect is small for scaling with many observations but becomes large for steps that use the target, e.g. selecting the features most correlated with y before cross-validation:

```
from sklearn.feature_selection import SelectKBest, f_regression

rng = np.random.default_rng(0)
noise = pd.DataFrame(rng.normal(size=(100, 2000)))          # pure noise
 ↪ features
target = pd.Series(rng.normal(size=100))                    # unrelated
 ↪ target

leaky_X = SelectKBest(f_regression, k=20).fit_transform(noise, target)
 ↪ # selection on ALL data
leaky = cross_validate(LinearRegression(), leaky_X, target, cv=cv,
 ↪ scoring="r2")["test_score"].mean()
proper = cross_validate(make_pipeline(SelectKBest(f_regression, k=20),
 ↪ LinearRegression()),
                        noise, target, cv=cv,
 ↪ scoring="r2")["test_score"].mean()
print(f"CV R² with leaky selection: {leaky:.2f},  inside the pipeline:
 ↪ {proper:.2f}")
```

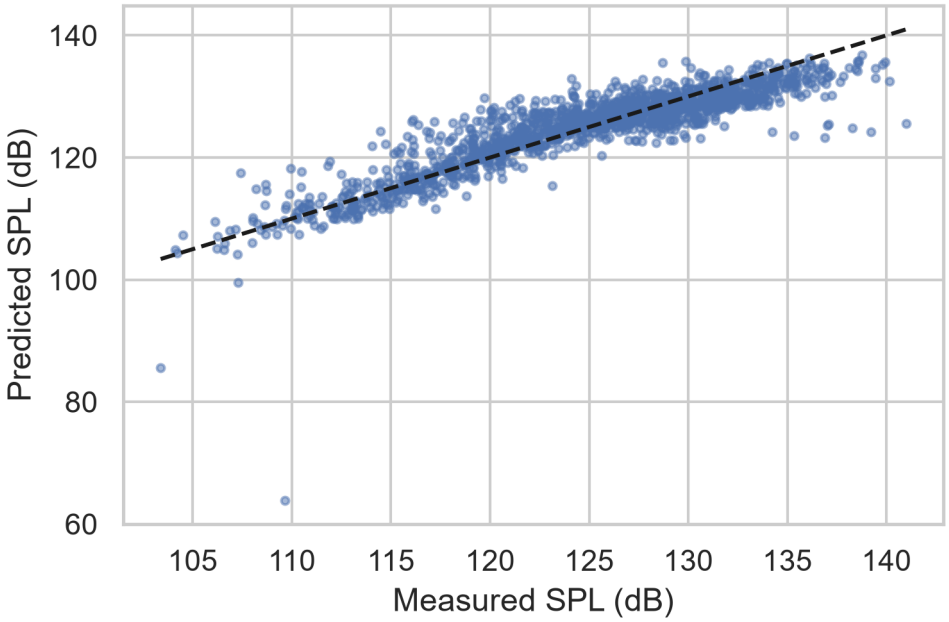
```
CV R² with leaky selection: 0.47,  inside the pipeline: -0.32
```

With selection outside the pipeline, the "model" appears to predict pure noise; inside the pipeline the CV R^2 is correctly around zero or negative.

Solution to Exercise 3

```python
from sklearn.model_selection import cross_val_predict

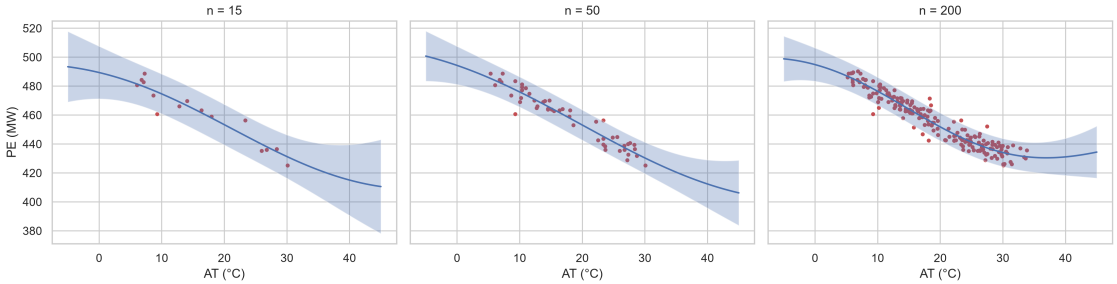
y_hat = cross_val_predict(ridge, X, y, cv=cv)
plt.scatter(y, y_hat, s=8, alpha=0.5)
lims = [y.min(), y.max()]
plt.plot(lims, lims, "k--")
plt.xlabel("Measured SPL (dB)"); plt.ylabel("Predicted SPL (dB)")
plt.show()
```



`cross_val_predict` returns for each observation the prediction of the model that did *not* see it during training. Deviations are typically largest at the extremes – the quietest and loudest conditions are underrepresented, and the model shrinks predictions towards the mean (a typical effect of regularisation). Plotting the residuals against frequency shows that some frequency ranges are captured less well than others.

Solution to Exercise 4

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharey=True)
for ax, n_train in zip(axes, [15, 50, 200]):
    sub = pp.sample(n_train, random_state=1)
    g = GaussianProcessRegressor(kernel=kernel, normalize_y=True,
    ↪ random_state=0).fit(sub[["AT"]], sub["PE"])
    m, s = g.predict(pd.DataFrame(T, columns=["AT"]), return_std=True)
    ax.scatter(sub["AT"], sub["PE"], s=8, c="C3")
    ax.plot(T, m); ax.fill_between(T.ravel(), m - 1.96 * s, m + 1.96 * s,
    ↪ alpha=0.3)
    ax.set(title=f"n = {n_train}", xlabel="AT (°C)")
axes[0].set_ylabel("PE (MW)")
plt.tight_layout(); plt.show()
```



With more data the band becomes narrower, but it does not shrink to zero: because the kernel contains a `WhiteKernel`, the predicted standard deviation includes the measurement noise, so the band approaches the scatter of individual observations (similar to a prediction interval). The uncertainty about the *trend* itself, however, decreases. Outside the data range the band still widens; with more points the trend is better determined, so the widening is less dramatic. Where there are no data at all, data elsewhere can only reduce the uncertainty as far as the kernel assumes smoothness.

Solution to Exercise 5

```
from sklearn.linear_model import LassoCV

lasso = make_pipeline(PolynomialFeatures(3), StandardScaler(),
                      LassoCV(cv=5, random_state=0,
 ↪  max_iter=50000)).fit(X, y)
coefs = lasso[-1].coef_
names = lasso[0].get_feature_names_out(X.columns)
print(f"{(coefs != 0).sum()} of {len(coefs)} features survive")
pd.Series(coefs, index=names)[coefs != 0].sort_values(key=abs,
 ↪  ascending=False).head(10).round(3)
```

```
22 of 56 features survive

frequency chord          -5.804
frequency thickness      -4.006
frequency^2 chord         1.969
frequency angle chord    -1.947
angle                    -1.946
angle chord velocity      1.935
frequency chord^2         1.800
chord                    -1.715
frequency thickness^2     1.547
frequency^2              -1.390
dtype: float64
```

Lasso keeps only part of the polynomial terms. The surviving terms show which inputs and interactions matter most; frequency usually dominates, since the noise spectrum depends strongly on it. Note that with correlated features Lasso picks one of a group somewhat arbitrarily – the selection is not a proof of physical relevance.

Solution to Exercise 6

Change `X_runs` to, e.g., `[[100.0], [130.0], [250.0]]` and rerun the loop. Usually the algorithm still finds the region of the optimum, because the large uncertainty in unexplored regions makes EI large there. With an unlucky start and a misleading local maximum (the sine term), it may need more iterations. A regular grid of 8 runs over 100–250 °C has a spacing of about 21 °C; the best grid point can easily be several percent below the true optimum, and the grid cannot refine itself. Bayesian optimisation pays off when each run is expensive and runs can be performed one after another.

Part III

Part C – Machine Learning

7 Classification

Many engineering decisions are categorical: a part is OK or not OK, a machine will fail within the next shift or not, a fault belongs to type A, B or C. Predicting such a category from measured data is called **classification**. It is the core of automated quality inspection, predictive maintenance and fault diagnosis.

Classification raises questions that regression does not. How do we measure the quality of a classifier when failures are rare? What is worse – a false alarm or a missed failure? And how do we choose the decision threshold that turns a predicted probability into an action? This chapter introduces three basic classifiers and, just as importantly, the metrics to evaluate them honestly.

Scenario: should we install the failure alarm?

A milling line runs 10,000 operations per period. Every machine failure that is not anticipated costs about 10,000 € (scrap, repair, downtime). A vendor offers to install a failure alarm: when it fires, the machine is stopped and inspected, which costs about 500 € – whether or not a failure was imminent. Today there is no alarm at all, so every failure is unplanned. The plant manager wants a clear answer.

Your decision: Install an alarm – and if so, based on which model?

Learning goals. After this chapter you can

- explain how logistic regression, k-nearest neighbours and decision trees classify,
- evaluate classifiers with confusion matrix, precision, recall, F1, ROC and precision–recall curves,
- handle strongly imbalanced classes with class weights, suitable metrics and threshold choice,
- choose a decision threshold based on the costs of errors,
- recognise and avoid data leakage.

Datasets in this chapter

- AI4I 2020 Predictive Maintenance (UCI)

7.1 Three ways to classify

In classification, the response y takes one of K classes; in the **binary** case $y \in \{0, 1\}$ (e.g. 1 = failure). Most classifiers estimate the probability $P(y = 1 \mid x)$ and predict class 1 if this probability exceeds a **threshold**, by default 0.5.

7.1.1 Logistic regression

i Definition: Logistic regression

The probability of class 1 is modelled as

$$P(y = 1 \mid x) = \sigma(\beta_0 + \beta^\top x), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Equivalently, the **log-odds** are linear in the inputs: $\ln \frac{P(y=1|x)}{1-P(y=1|x)} = \beta_0 + \beta^\top x$. The parameters are estimated by maximum likelihood.

Logistic regression is the classification counterpart of linear regression: simple, fast, interpretable. A coefficient β_j means that one unit more of x_j multiplies the odds of failure by e^{β_j} (the **odds ratio**). Its decision boundary – the set of x where the probability equals the threshold – is a straight line (a hyperplane). Nonlinear boundaries require engineered features such as products or squares.

7.1.2 k-nearest neighbours (kNN)

kNN stores the training data and classifies a new point by majority vote of its k nearest training points. It makes no assumption about the shape of the boundary, but it depends entirely on the **distance measure** – inputs must be standardised, otherwise the input with the largest numerical range dominates. Small k gives ragged boundaries (high variance), large k smooth ones (high bias).

7.1.3 Decision trees

A decision tree splits the input space step by step with simple rules such as "torque $>$ 55 Nm". At each node, the split is chosen that makes the resulting groups as **pure** as possible, measured e.g. by the Gini impurity $G = 1 - \sum_k p_k^2$, where p_k is the share of class k in the node. Trees are easy to explain to colleagues and can capture interactions and thresholds, but a single deep tree overfits strongly; its depth must be limited. Chapter 8 shows how combining many trees fixes this.

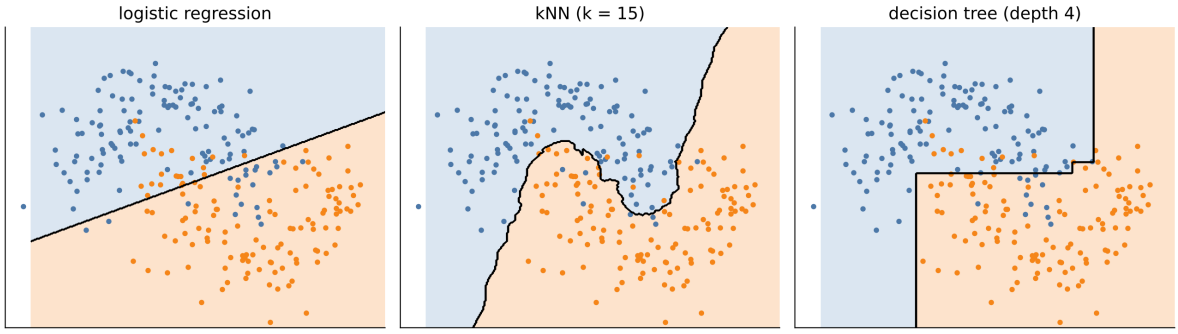


Figure 7.1: Decision boundaries of the three classifiers on a two-dimensional toy problem. Logistic regression draws a straight line, kNN follows the local structure of the data, and a decision tree uses axis-parallel steps.

7.2 Evaluating classifiers

Definition: Confusion matrix and derived metrics

	predicted OK (0)	predicted failure (1)
actual OK (0)	true negative (TN)	false positive (FP) – false alarm
actual failure (1)	false negative (FN) – missed failure	true positive (TP)

- **Accuracy** = (TP + TN) / all
- **Precision** = TP / (TP + FP): how many alarms are real?
- **Recall** (sensitivity, detection rate) = TP / (TP + FN): how many failures are detected?
- **F1 score** = harmonic mean of precision and recall
- **Balanced accuracy** = mean of the recall of both classes

Pitfall: accuracy on imbalanced data

If 3 % of the records are failures, a "model" that always predicts "OK" reaches 97 % accuracy – and detects nothing. With rare events, accuracy is almost meaningless. Use precision, recall, F1 or balanced accuracy, and always look at the confusion matrix.

Precision is nothing but $P(\text{failure} \mid \text{alarm})$ – Bayes' theorem from Chapter 3. When failures

are rare, even a detector with high recall and a low false alarm rate can have a low precision.

7.2.1 Thresholds, ROC and precision–recall curves

Lowering the threshold produces more alarms: recall increases, precision usually decreases. Instead of judging a single threshold, we can look at all thresholds at once:

- The **ROC curve** plots the true positive rate (recall) against the false positive rate FP/(FP + TN). The area under the curve (**ROC AUC**) is the probability that a randomly chosen failure gets a higher score than a randomly chosen OK record; 0.5 = random guessing, 1 = perfect ranking.
- The **precision–recall curve** plots precision against recall. For rare events it is more informative than ROC, because the false positive *rate* stays small even when the false alarms far outnumber the true alarms.

> Engineering practice: choose the threshold by costs
>
> The threshold 0.5 is arbitrary. If a missed failure costs c_{FN} and a false alarm c_{FP}, choose the threshold that minimises the total cost $c_{FN} \cdot \mathrm{FN} + c_{FP} \cdot \mathrm{FP}$ – on validation data, not on the test data. For well-calibrated probabilities, the optimal threshold is $c_{FP}/(c_{FP}+c_{FN})$: if a miss is 20 times as expensive as a false alarm, alarm already at a failure probability of about 5 %.

7.2.2 Strategies for imbalanced classes

- **Stratified splitting:** keep the class proportions equal in training and test data (`stratify=y`).
- **Class weights:** weight errors on the rare class more strongly during training (`class_weight="balanced"`).
- **Resampling:** undersample the majority class or oversample the minority class (only in the training data!).
- **Threshold adjustment** as described above – often the simplest and most effective measure.

```
# Setup: works locally and in Google Colab
import importlib.util, subprocess, sys
for pkg in ["ucimlrepo"]:
    if importlib.util.find_spec(pkg) is None:
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg],
 ↪  check=True)
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
```

7.3 Live coding: predictive maintenance

The AI4I 2020 dataset is a synthetic dataset that reflects a real milling process. Each of the 10,000 records contains the product quality variant (L/M/H), air and process temperature, rotational speed, torque and tool wear, and the label `Machine failure`. The failures are generated by five documented mechanisms: tool wear failure (TWF), heat dissipation failure (HDF), power failure (PWF), overstrain failure (OSF) and random failures (RNF). Because the mechanisms are known, the dataset is ideal to check whether a model has learned something physically sensible.

```
from ucimlrepo import fetch_ucirepo

ai4i = fetch_ucirepo(id=601).data.original

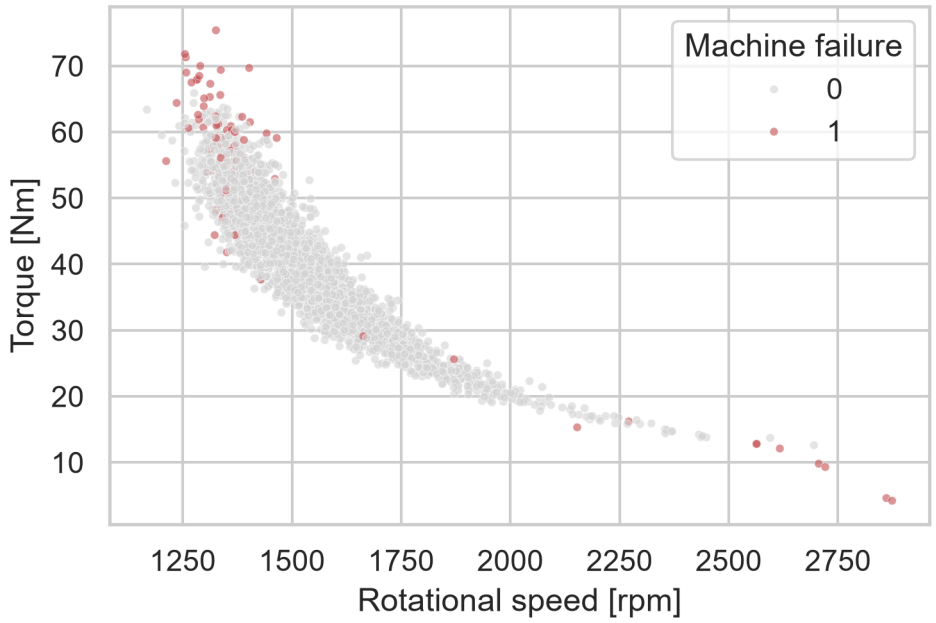
# Column names differ between versions of the dataset (with or without units
 ↪  in brackets,
# blanks or underscores). We map them to one canonical spelling with units.
import re
canonical = {"air temperature": "Air temperature [K]", "process temperature":
 ↪  "Process temperature [K]",
             "rotational speed": "Rotational speed [rpm]", "torque": "Torque
              ↪  [Nm]",
             "tool wear": "Tool wear [min]", "machine failure": "Machine
              ↪  failure", "type": "Type"}
def standardise(col):
    base = re.sub(r"[_\s]+", " ", str(col).split("[")[0]).strip().lower()
    return canonical.get(base, col)
ai4i = ai4i.rename(columns=standardise)
print("Columns:", ai4i.columns.tolist())

features = ["Type", "Air temperature [K]", "Process temperature [K]",
            "Rotational speed [rpm]", "Torque [Nm]", "Tool wear [min]"]
X = pd.get_dummies(ai4i[features], columns=["Type"], drop_first=True)
```

```
y = ai4i["Machine failure"]
print(f"Failure rate: {y.mean():.1%}")
# NOTE: the columns TWF, HDF, PWF, OSF, RNF describe the failure mode.
# Using them as features would be data leakage!
```

```
Columns: ['UID', 'Product ID', 'Type', 'Air temperature [K]', 'Process temperature [K]', 'Rot
Failure rate: 3.4%
```

```
sns.scatterplot(data=ai4i.sample(3000, random_state=0), x="Rotational speed
↪ [rpm]", y="Torque [Nm]",
                hue="Machine failure", s=10, alpha=0.6, palette=["lightgray",
↪ "C3"])
plt.show()
```



The failures are not spread randomly: they concentrate at very high torque with low speed and at very low torque with high speed – regions of high and low mechanical power. A linear boundary in this plane cannot separate them; this matters for the choice of model.

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=0)

models = {
    "logistic regression": make_pipeline(StandardScaler(),
     ↪  LogisticRegression(class_weight="balanced", max_iter=1000)),
    "kNN (k=5)": make_pipeline(StandardScaler(), KNeighborsClassifier(5)),
    "decision tree": DecisionTreeClassifier(max_depth=5,
     ↪  class_weight="balanced", random_state=0),
}
for name, m in models.items():
    m.fit(X_train, y_train)
    print(f"--- {name}")
    print(classification_report(y_test, m.predict(X_test), digits=3))
```

```
--- logistic regression
              precision    recall  f1-score   support

           0      0.990     0.827     0.901      2415
           1      0.135     0.765     0.229        85

    accuracy                          0.825      2500
   macro avg      0.562     0.796     0.565      2500
weighted avg      0.961     0.825     0.878      2500

--- kNN (k=5)
              precision    recall  f1-score   support

           0      0.977     0.996     0.986      2415
           1      0.737     0.329     0.455        85

    accuracy                          0.973      2500
   macro avg      0.857     0.663     0.721      2500
weighted avg      0.969     0.973     0.968      2500

--- decision tree
              precision    recall  f1-score   support
```

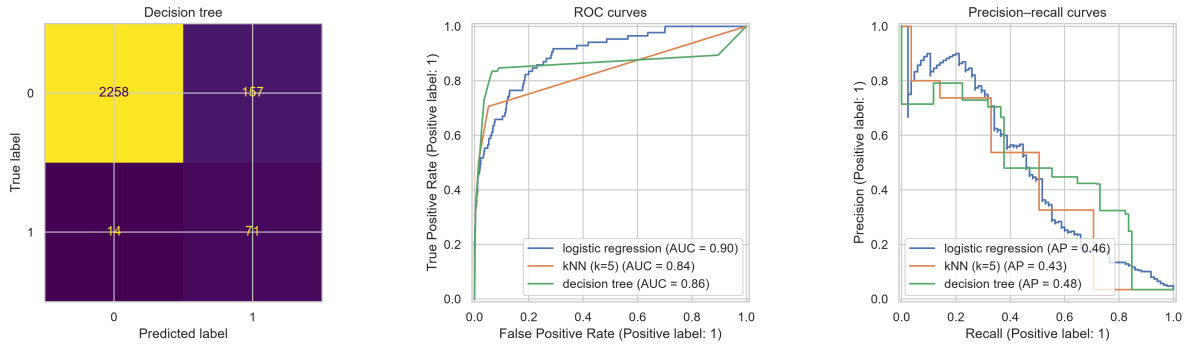
```
              0       0.994     0.935     0.964       2415
              1       0.311     0.835     0.454         85

    accuracy                            0.932       2500
   macro avg        0.653     0.885     0.709       2500
weighted avg        0.971     0.932     0.946       2500
```

Discussion. Read the reports row by row: the line for class 1 (failure) is the interesting one. Typically, logistic regression with class weights detects many failures (high recall) but raises many false alarms (low precision), because its linear boundary cannot isolate the failure regions. kNN without class weights is conservative: few false alarms, but many missed failures. The decision tree can represent the thresholds in torque, speed and tool wear that generate the failures and usually achieves the best balance. The accuracy row is high for all models – and says little.

```
from sklearn.metrics import ConfusionMatrixDisplay, PrecisionRecallDisplay,
 ↪  RocCurveDisplay

fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
ConfusionMatrixDisplay.from_estimator(models["decision tree"], X_test,
 ↪  y_test, ax=axes[0], colorbar=False)
axes[0].set_title("Decision tree")
for name, m in models.items():
    RocCurveDisplay.from_estimator(m, X_test, y_test, ax=axes[1], name=name)
    PrecisionRecallDisplay.from_estimator(m, X_test, y_test, ax=axes[2],
 ↪  name=name)
axes[1].set_title("ROC curves"); axes[2].set_title("Precision-recall curves")
plt.tight_layout(); plt.show()
```



Discussion. The ROC curves may look reassuringly good for all models, while the precision–recall curves reveal clear differences – the typical picture for rare events. The precision–recall

curve directly shows the trade-off an engineer faces: “If I want to catch 80 % of the failures, how many of my alarms will be false?”

7.4 Back to the decision

We compare the total cost per test period for the status quo (every failure is a missed failure) and for each model at its default threshold. A missed failure costs 10,000 €, a false alarm 500 €. Detected failures are assumed to be prevented and to cost only the inspection.

```
from sklearn.metrics import confusion_matrix

c_fn, c_fp = 10_000, 500
status_quo = c_fn * int(y_test.sum())
costs = {}
for name, m in models.items():
    tn, fp, fn, tp = confusion_matrix(y_test, m.predict(X_test)).ravel()
    costs[name] = c_fn * fn + c_fp * (fp + tp)          # every alarm
 ↪  triggers an inspection
print(f"{'status quo (no alarm)':22s} {status_quo / 1000:8.1f} k€")
for name, c in costs.items():
    print(f"{name:22s} {c / 1000:8.1f} k€")
best = min(costs, key=costs.get)
if costs[best] < status_quo:
    print(f"Decision: install the alarm based on the {best} - saving
     ↪  {(status_quo - costs[best]) / 1000:.1f} k€ "
          f"({1 - costs[best] / status_quo:.0%}) per test period.")
else:
    print("Decision: no model beats the status quo - do not install.")
```

```
status quo (no alarm)     850.0 k€
logistic regression       441.5 k€
kNN (k=5)                 589.0 k€
decision tree             254.0 k€
Decision: install the alarm based on the decision tree - saving 596.0 k€ (70%) per test perio
```

The cost comparison translates the confusion matrix into the language of the plant manager. Exercise 2 shows that choosing the threshold by cost improves the result further. Before the final decision, two things should be checked: the AI4I data are *simulated*, so the savings must be confirmed in a pilot on the real line; and the assumption that every detected failure can be prevented is optimistic.

! Key takeaways

- Classifiers estimate class probabilities; a threshold turns them into decisions – choose it by the costs of errors.
- Logistic regression: linear boundary, interpretable; kNN: local, needs scaling; trees: thresholds and interactions, must be kept shallow.
- With rare events, accuracy and ROC can look good while the model is useless – use precision, recall and precision–recall curves.
- Features that encode the outcome (here: the failure mode columns) cause data leakage.

7.5 Exercises

1. Add the failure mode columns (TWF, HDF, PWF, OSF, RNF) as features and retrain the decision tree. What happens to the scores – and why is this result worthless?
2. Assume a missed failure costs 10,000 € and a false alarm 500 €. Choose the decision threshold of the logistic regression that minimises the total cost. Use a validation split of the training data, not the test data.
3. Visualise the decision tree (`sklearn.tree.plot_tree`, depth 3). Do the rules make physical sense?
4. Construct the feature `power [W]` = torque × angular velocity and the temperature difference between process and air. Do they improve the logistic regression? Why?
5. A test set has TN = 950, FP = 30, FN = 5, TP = 15. Compute accuracy, precision, recall and F1 by hand. Is the model useful?

7.5.1 Solutions

Solution to Exercise 1

```
X_leak = X.join(ai4i[["TWF", "HDF", "PWF", "OSF", "RNF"]])
Xl_tr, Xl_te, yl_tr, yl_te = train_test_split(X_leak, y, test_size=0.25,
 ↪  stratify=y, random_state=0)
tree_leak = DecisionTreeClassifier(max_depth=5,
 ↪  random_state=0).fit(Xl_tr, yl_tr)
print(classification_report(yl_te, tree_leak.predict(Xl_te), digits=3))
```

```
              precision    recall  f1-score   support

           0      0.998     1.000     0.999      2415
```

```
           1      1.000     0.941     0.970        85

    accuracy                          0.998      2500
   macro avg      0.999     0.971     0.984      2500
weighted avg      0.998     0.998     0.998      2500
```

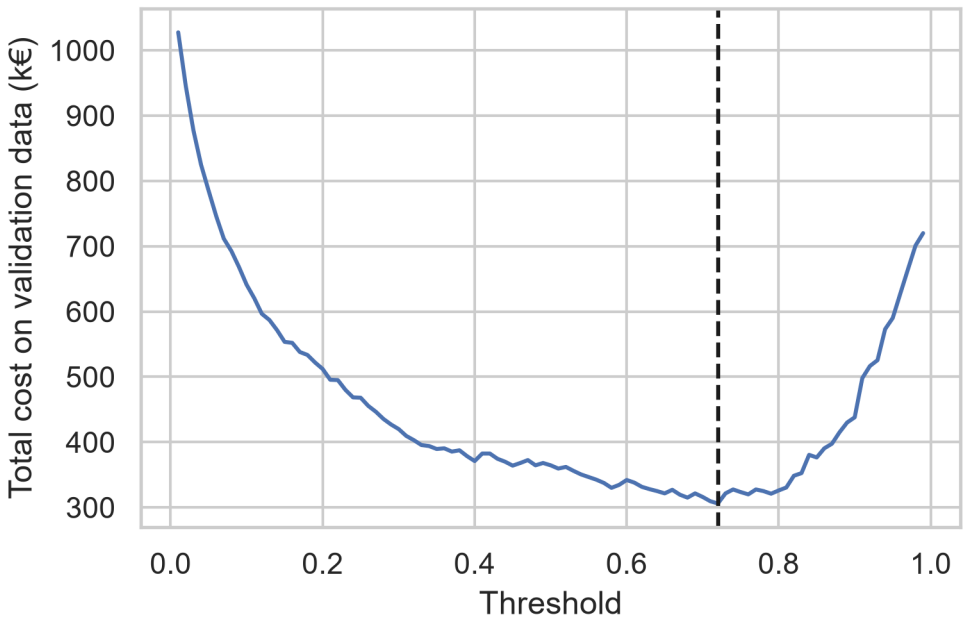
Precision and recall jump to nearly 100 %. The failure mode columns are only known *after* a failure has been diagnosed – they essentially encode the label. In operation, at prediction time, they are not available. Such leakage produces impressive but completely useless models. Rule: for each feature, ask "Would I know this value at the moment I need the prediction?"

Solution to Exercise 2

```
X_fit, X_val, y_fit, y_val = train_test_split(X_train, y_train,
 ↪  test_size=0.3,
                                               stratify=y_train,
 ↪  random_state=1)
logreg = make_pipeline(StandardScaler(),
 ↪  LogisticRegression(class_weight="balanced", max_iter=1000))
logreg.fit(X_fit, y_fit)
p_val = logreg.predict_proba(X_val)[:, 1]

c_fn, c_fp = 10_000, 500
thresholds = np.linspace(0.01, 0.99, 99)
cost = [c_fn * ((p_val < t) & (y_val == 1)).sum() + c_fp * ((p_val >= t)
 ↪  & (y_val == 0)).sum()
        for t in thresholds]
t_best = thresholds[np.argmin(cost)]
plt.plot(thresholds, np.array(cost) / 1000)
plt.axvline(t_best, ls="--", c="k")
plt.xlabel("Threshold"); plt.ylabel("Total cost on validation data (k€)")
plt.show()

p_test = logreg.predict_proba(X_test)[:, 1]
cost_test = c_fn * ((p_test < t_best) & (y_test == 1)).sum() + c_fp *
 ↪  ((p_test >= t_best) & (y_test == 0)).sum()
print(f"best threshold {t_best:.2f}, cost on test data {cost_test /
 ↪  1000:.0f} k€")
```



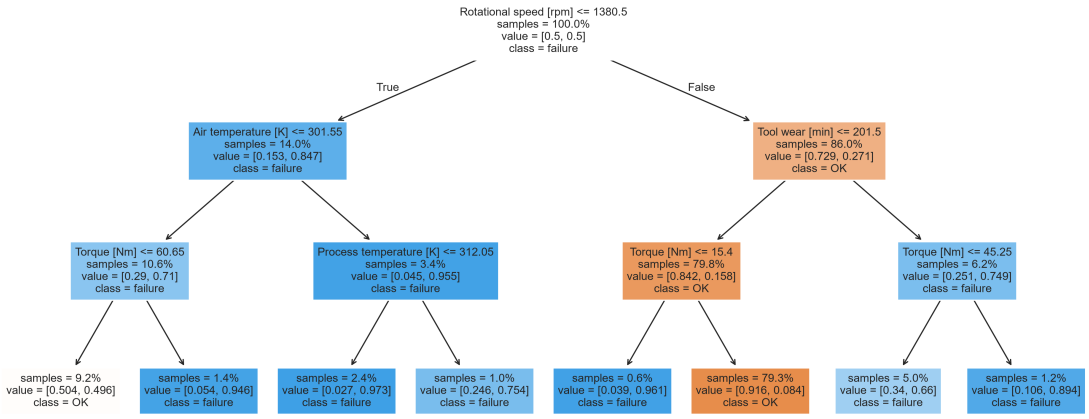

```
best threshold 0.72, cost on test data 404 k€
```

The cost curve typically has a broad minimum. Note that with `class_weight="balanced"` the predicted probabilities are no longer calibrated (the model behaves as if failures were as frequent as OK records), so the optimal threshold differs from the theoretical $c_{FP}/(c_{FP}+c_{FN}) \approx 0.048$. Choosing the threshold on validation data and reporting the cost on the untouched test data keeps the evaluation honest.

Solution to Exercise 3

```
from sklearn.tree import plot_tree

tree3 = DecisionTreeClassifier(max_depth=3, class_weight="balanced",
 ↪  random_state=0).fit(X_train, y_train)
plt.figure(figsize=(16, 7))
plot_tree(tree3, feature_names=X.columns, class_names=["OK", "failure"],
 ↪  filled=True,
          impurity=False, proportion=True, fontsize=9)
plt.show()
```



The top splits typically use torque, rotational speed and tool wear – exactly the quantities behind the documented failure mechanisms: overstrain (high torque × tool wear), power failure (power too low or too high, i.e. extreme torque–speed combinations) and tool wear failure (large tool wear). A tree that splits mainly on product ID or random variables would be suspicious. Checking learned rules against physics is one of the best plausibility tests for a model.

Solution to Exercise 4

```
X_eng = X.copy()
X_eng["power [W]"] = ai4i["Torque [Nm]"] * ai4i["Rotational speed [rpm]"]
 ↪  * 2 * np.pi / 60
X_eng["delta T [K]"] = ai4i["Process temperature [K]"] - ai4i["Air
 ↪  temperature [K]"]
X_eng["strain [min Nm]"] = ai4i["Tool wear [min]"] * ai4i["Torque [Nm]"]

from sklearn.model_selection import cross_val_score, StratifiedKFold
skf = StratifiedKFold(5, shuffle=True, random_state=0)
for name, data in [("original features", X), ("with engineered features",
 ↪  X_eng)]:
    s = cross_val_score(make_pipeline(StandardScaler(),
 ↪  LogisticRegression(class_weight="balanced", max_iter=2000)),
                        data, y, cv=skf, scoring="average_precision")
    print(f"{name:26s} average precision = {s.mean():.3f}")
```

```
original features          average precision = 0.429
with engineered features   average precision = 0.453
```

Engineered features based on physics can help logistic regression a lot, because they turn curved boundaries into simpler ones. But power failures occur for *both* too low and too high power – a linear model in `power` can only represent one side. Squared or binned terms, or a tree-based model, are needed to capture both. This is a typical example of how domain knowledge and model choice interact.

Solution to Exercise 5

Total = 1000. Accuracy = (950 + 15)/1000 = 96.5 %. Precision = 15/(15 + 30) = 33 %. Recall = 15/(15 + 5) = 75 %. F1 = 2 · 0.33 · 0.75/(0.33 + 0.75) 0.46.
The accuracy is lower than that of the trivial model "always OK" (98 %)! Still, the model may be useful: it detects three of four failures, at the price of two false alarms per real one. Whether that is acceptable depends on the costs – if a missed failure is expensive and checking an alarm is cheap, this model is valuable despite its "bad" accuracy.

8 Ensembles and Interpretability

A single decision tree is easy to understand but a poor predictor: it overfits, and a small change in the data can produce a completely different tree. The remedy is surprisingly simple – combine many trees. **Ensemble methods** such as random forests and gradient boosting are, for tabular data, among the most accurate general-purpose methods available. In industrial practice they are often the first choice for quality prediction, fault classification and virtual sensors.

Their accuracy comes at a price: an ensemble of hundreds of trees is no longer transparent. Yet engineers need to know *why* a model predicts a defect – to trust it, to check it against physics and to derive process improvements. The second half of this chapter therefore introduces methods to interpret such models.

Scenario: which defects can be classified automatically?

The automatic inspection line of a steel mill detects surface defects reliably, but a human inspector still has to assign every defect to a defect class – the class decides whether a plate is reworked, downgraded or scrapped. The inspectors are overloaded. The quality manager proposes to let a model classify those defect classes it recognises reliably and to send only the rest to the inspectors. "Reliable" means: precision and recall of at least 90 % for that class.

Your decision: Which defect classes may be classified automatically, and how much inspector workload does that save?

Learning goals. After this chapter you can

- explain bagging and boosting and why they improve on a single tree,
- train random forests and gradient boosting models and tune their hyperparameters with cross-validated search,
- solve and evaluate multi-class problems,
- interpret models with permutation importance and partial dependence plots, knowing their limitations,
- (extension) explain individual predictions with SHAP values.

Datasets in this chapter

- Steel Plates Faults (UCI)

8.1 Why ensembles work

Recall the bias–variance decomposition from Chapter 6. A deep decision tree has low bias but high variance. If we could average many trees trained on *different* datasets from the same process, the variance would shrink while the bias stays low. Ensembles approximate this idea in two different ways.

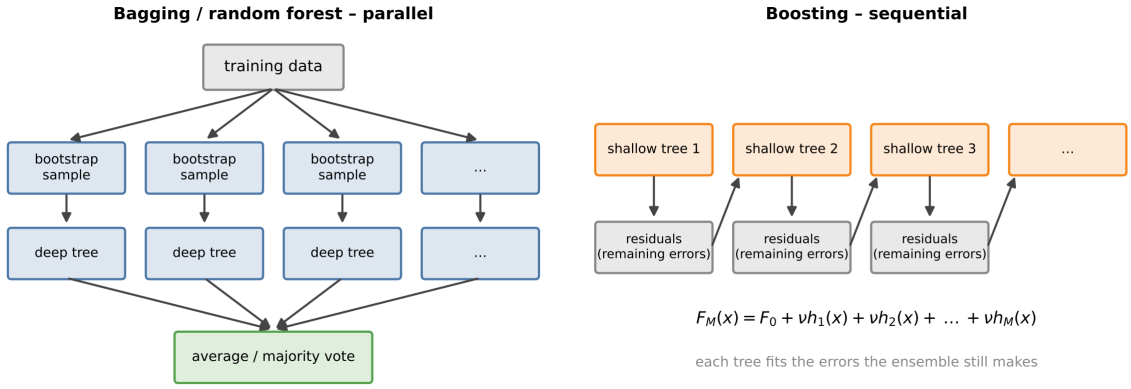


Figure 8.1: Bagging and boosting. Bagging trains many deep trees independently on bootstrap samples and averages them (variance reduction). Boosting trains shallow trees one after another, each one correcting the remaining errors of the ensemble so far (bias reduction).

8.1.1 Bagging and random forests

> **Definition: Bagging and random forest**
>
> **Bagging** (bootstrap aggregating) trains B models on B bootstrap samples of the training data and averages their predictions (regression) or takes a majority vote (classification). A **random forest** is bagging with decision trees plus one additional trick: at each split, only a random subset of the features (e.g. $\sqrt{p}$ of p features) is considered. This decorrelates the trees and makes the averaging more effective.

Why the extra randomness helps: averaging B predictions with variance σ^2 and pairwise correlation ρ gives a variance of $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$. More trees reduce only the second term; reducing the correlation ρ between the trees reduces the first. Random forests have few critical hyperparameters, rarely overfit when more trees are added, and provide a free validation estimate: each tree did not see about a third of the data (*out-of-bag* samples), on which it can be evaluated.

8.1.2 Boosting

Definition: Gradient boosting

Gradient boosting builds the model step by step:

$$F_m(x) = F_{m-1}(x) + \nu\, h_m(x),$$

where each new tree h_m is fitted to the negative gradient of the loss with respect to the current predictions – for squared error simply the **residuals** of F_{m-1}. The **learning rate** $0 < \nu \leq 1$ shrinks each step.

Boosting uses shallow trees (depth 3–8) that individually are weak, and reduces mainly the bias. Unlike random forests, boosting *can* overfit if too many trees are added; the number of trees, the learning rate and the tree depth must be tuned together, typically with early stopping on validation data. Modern implementations (scikit-learn's `HistGradientBoostingClassifier`, XGBoost, LightGBM) are very fast and handle missing values natively.

Engineering practice: which method first?

For a new tabular problem, a random forest with default settings is a robust first model that needs almost no tuning. Gradient boosting is often a few percent more accurate but requires tuning. A linear or logistic model remains the essential baseline – if the ensemble is only marginally better, the simpler model may be preferable for transparency.

8.2 Multi-class problems

With $K > 2$ classes, the confusion matrix becomes $K \times K$, and precision and recall are computed per class. They can be averaged in two ways: the **macro** average treats every class equally (important if rare classes matter), the **weighted** average weights them by their frequency. Balanced accuracy is the macro average of the recalls.

8.3 Interpreting models

> **Definition: Permutation importance**
>
> The importance of feature j is the decrease in a performance score (on validation or test data) when the values of feature j are randomly shuffled, which destroys its relationship with the target while keeping its distribution.

Permutation importance works for any model. The impurity-based `feature_importances_` of tree models are cheaper but biased towards features with many distinct values and computed on training data.

> **Pitfall: importance is not causality – and correlated features share importance**
>
> A high importance means that the *model* relies on a feature, not that the feature *causes* the defect. If two features are strongly correlated (e.g. two dimensions of a defect), shuffling one of them hurts little because the other still carries the information – both may appear unimportant although the information is essential.

A **partial dependence plot** (PDP) shows how the predicted probability changes on average when one feature is varied over its range while the others keep their observed values. It answers the engineering question "in which direction and how strongly does this measurement influence the prediction?" and reveals thresholds and nonlinearities.

```
# Setup: works locally and in Google Colab
import importlib.util, subprocess, sys
for pkg in ["ucimlrepo", "shap"]:
    if importlib.util.find_spec(pkg) is None:
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg],
 ↪  check=True)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
```

8.4 Live coding: steel plate faults

The dataset comes from the automatic inspection of steel plates. For 1,941 detected surface defects, 27 features were extracted from the inspection images and the production data: position and size of the defect (`X_Minimum`, `Pixels_Areas`, `Length_of_Conveyer`, …), luminosity statistics, shape indices, the steel type and the plate thickness. Each defect belongs to one of seven classes: *Pastry*, *Z_Scratch*, *K_Scatch*, *Stains*, *Dirtiness*, *Bumps* and *Other_Faults*.

```
from ucimlrepo import fetch_ucirepo

steel = fetch_ucirepo(id=198)
X = steel.data.features
y = steel.data.targets.idxmax(axis=1)      # 7 one-hot columns -> one label
y.value_counts()
```

```
Other_Faults    673
Bumps           402
K_Scratch       391
Z_Scratch       190
Pastry          158
Stains           72
Dirtiness        55
Name: count, dtype: int64
```

The classes are imbalanced: *Other_Faults* is by far the largest class, while *Dirtiness* and *Stains* are rare. "Other faults" is a heterogeneous rest category – we must expect it to be hard to classify.

```
from sklearn.model_selection import train_test_split, cross_val_score,
 ↪  StratifiedKFold
from sklearn.ensemble import RandomForestClassifier,
 ↪  HistGradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
 ↪  stratify=y, random_state=0)
cv = StratifiedKFold(5, shuffle=True, random_state=0)

candidates = {
```

```
    "logistic regression (baseline)": make_pipeline(StandardScaler(),
     ↪  LogisticRegression(max_iter=3000)),
    "random forest": RandomForestClassifier(n_estimators=300, random_state=0,
     ↪  n_jobs=-1),
    "gradient boosting": HistGradientBoostingClassifier(random_state=0),
}
for name, m in candidates.items():
    s = cross_val_score(m, X_train, y_train, cv=cv,
 ↪  scoring="balanced_accuracy")
    print(f"{name:32s} balanced accuracy = {s.mean():.3f} ± {s.std():.3f}")
```

```
logistic regression (baseline)   balanced accuracy = 0.717 ± 0.019
random forest                    balanced accuracy = 0.750 ± 0.019
gradient boosting                balanced accuracy = 0.785 ± 0.022
```

Discussion. Both ensembles should clearly outperform the logistic regression baseline. The standard deviation across folds indicates how much of the difference between the ensembles is real – differences smaller than this spread should not be over-interpreted. With seven classes, guessing would reach a balanced accuracy of about 1/7 0.14.

```
from sklearn.metrics import classification_report, ConfusionMatrixDisplay

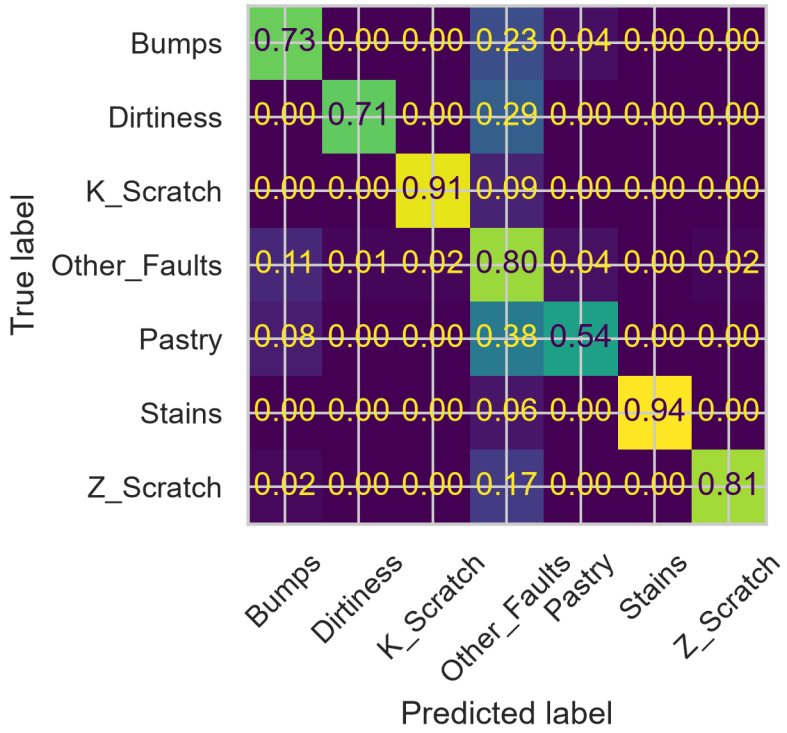
rf = candidates["random forest"].fit(X_train, y_train)
print(classification_report(y_test, rf.predict(X_test), digits=3))
ConfusionMatrixDisplay.from_estimator(rf, X_test, y_test, xticks_rotation=45,
 ↪  colorbar=False,
                                      normalize="true", values_format=".2f")
plt.show()
```

```
              precision    recall  f1-score   support

       Bumps      0.771     0.733     0.751       101
   Dirtiness      0.833     0.714     0.769        14
   K_Scratch      0.967     0.908     0.937        98
Other_Faults      0.692     0.804     0.744       168
      Pastry      0.656     0.538     0.592        39
      Stains      1.000     0.944     0.971        18
   Z_Scratch      0.929     0.812     0.867        48

    accuracy                          0.792       486
```

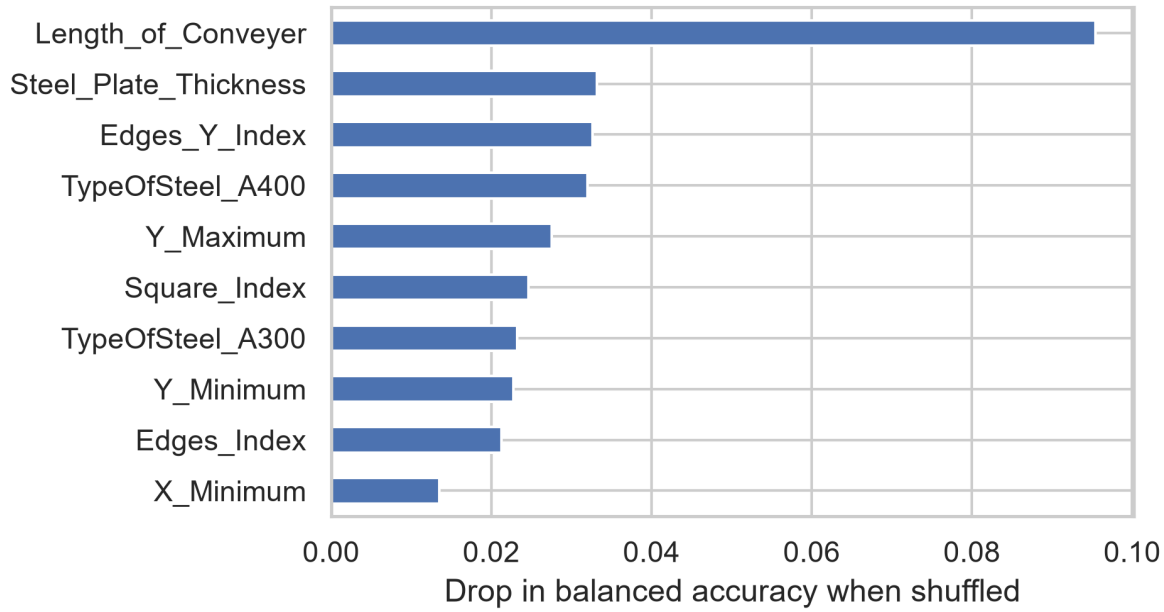
```
   macro avg      0.836     0.779     0.804       486
weighted avg      0.800     0.792     0.793       486
```



The row-normalised confusion matrix shows for each true class where its defects end up. Typically, geometrically distinctive defects (scratches, pastry) are recognised well, while *Other_Faults* and *Bumps* are confused with each other.

```
from sklearn.inspection import permutation_importance

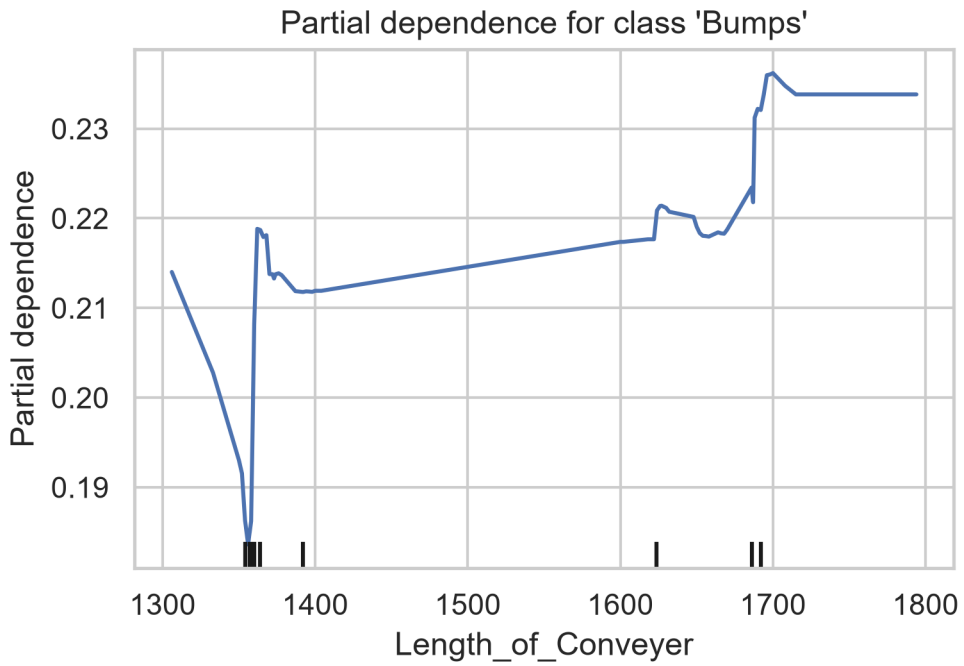
pi = permutation_importance(rf, X_test, y_test, n_repeats=10, random_state=0,
                            scoring="balanced_accuracy")
imp = pd.Series(pi.importances_mean, index=X.columns).sort_values()
imp.tail(10).plot.barh(xlabel="Drop in balanced accuracy when shuffled")
plt.show()
```




```
from sklearn.inspection import PartialDependenceDisplay

# The most important feature with a continuous range of values (binary
 ↪  features such as
# the steel type have no meaningful curve). Partial dependence needs
 ↪  floating-point data.
top = next(c for c in imp.index[::-1] if X_test[c].nunique() > 10)
print("Feature:", top)
PartialDependenceDisplay.from_estimator(rf, X_test.astype(float), [top],
 ↪  target=rf.classes_[0], kind="average")
plt.title(f"Partial dependence for class '{rf.classes_[0]}'")
plt.show()
```

```
Feature: Length_of_Conveyer
```



Discussion. The importance ranking tells us which measurements the model relies on – for steel defects typically luminosity and geometry features as well as the steel type and plate thickness. The partial dependence plot shows *how* the predicted probability of one class changes along the most important continuous feature. Look for thresholds: a steep step suggests a physical limit or a property of the inspection system worth discussing with the process engineers.

8.5 Back to the decision

For each class we need both high precision (when the model says "scratch", it really is one) and high recall (it finds the scratches). We read both from the test set and apply the 90 % criterion.

```
from sklearn.metrics import precision_recall_fscore_support

prec, rec, _, support = precision_recall_fscore_support(y_test,
 ↪  rf.predict(X_test), labels=rf.classes_, zero_division=0)
per_class = pd.DataFrame({"precision": prec, "recall": rec, "test cases":
 ↪  support}, index=rf.classes_)
per_class["automate"] = (per_class["precision"] >= 0.9) &
 ↪  (per_class["recall"] >= 0.9)
```

```
print(per_class.round(2))
share = per_class.loc[per_class["automate"], "test cases"].sum() /
 ↪  per_class["test cases"].sum()
auto = list(per_class.index[per_class["automate"]])
print(f"Decision: automate {auto if auto else 'no class yet'} - about
 ↪  {share:.0%} of the defects would no longer need an inspector.")
```

```
              precision  recall  test cases  automate
Bumps              0.77    0.73         101     False
Dirtiness          0.83    0.71          14     False
K_Scratch          0.97    0.91          98      True
Other_Faults       0.69    0.80         168     False
Pastry             0.66    0.54          39     False
Stains             1.00    0.94          18      True
Z_Scratch          0.93    0.81          48     False
Decision: automate ['K_Scratch', 'Stains'] - about 24% of the defects would no longer need a
```

With only a few dozen test cases per class, precision and recall are themselves uncertain – a class that just passes the 90 % mark should be monitored closely after go-live, for example by having inspectors check a random sample of automatically classified plates. Classes that fail the criterion are good candidates for better labels or additional features.

8.6 Extension: SHAP values

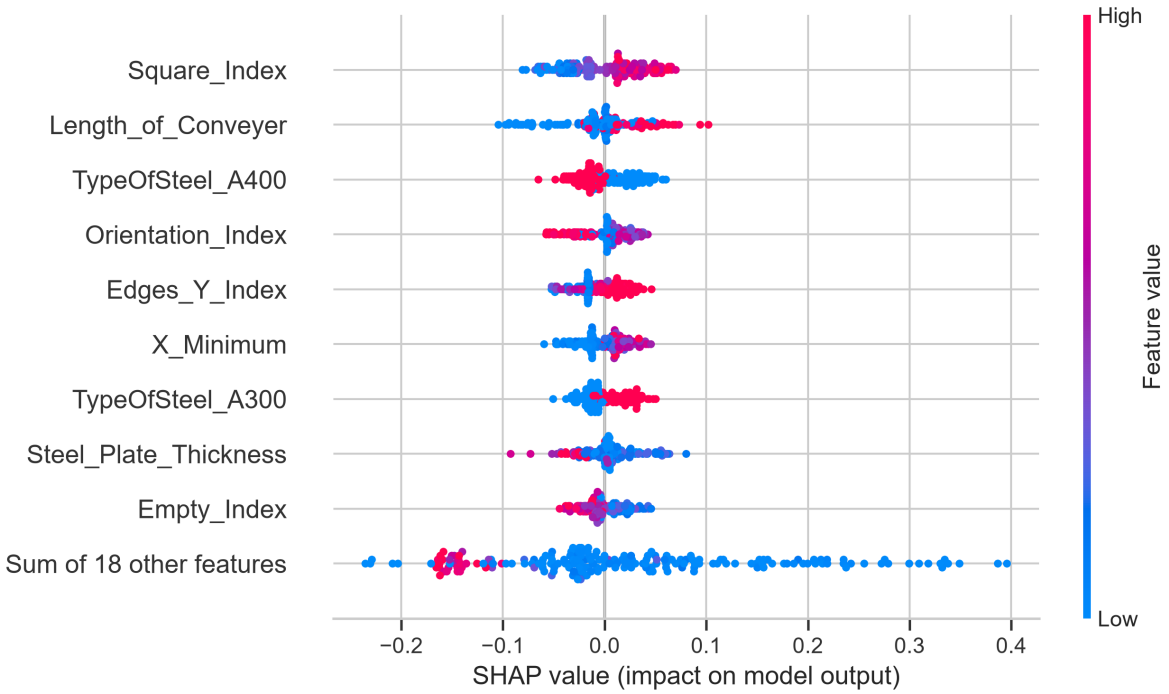
Extension – optional material, not required for the exam.

SHAP (*SHapley Additive exPlanations*) distributes a single prediction among the features, based on the Shapley value from cooperative game theory: each feature receives its average marginal contribution over all possible orders in which features could be added. The contributions add up exactly to the difference between the prediction and the average prediction. SHAP answers the question: *why* did the model classify **this** plate as K_Scatch?

```
import shap

explainer = shap.TreeExplainer(rf)
sv = explainer(X_test.iloc[:200])
# SHAP values for one class, e.g. "K_Scatch"
cls = list(rf.classes_).index("K_Scatch") if "K_Scatch" in rf.classes_ else 0
print("SHAP values for class:", rf.classes_[cls])
shap.plots.beeswarm(sv[:, :, cls], max_display=10)
```

```
SHAP values for class: Bumps
```



Each dot is one plate; its horizontal position is the contribution of the feature to the K_Scatch score, its colour the feature value. Features at the top have the largest overall influence. Unlike permutation importance, the beeswarm plot also shows the *direction* of the effect.

! Key takeaways

- Bagging/random forests reduce variance by averaging decorrelated deep trees; boosting reduces bias by adding shallow trees sequentially.
- Random forests are robust defaults; gradient boosting needs tuning but is often slightly more accurate.
- In multi-class problems, use per-class metrics and the confusion matrix; macro averages protect rare classes.
- Permutation importance and partial dependence make black-box models inspectable – but importance is not causality.

8.7 Exercises

1. Tune `max_depth`, `learning_rate` and `max_iter` of the gradient boosting model with `RandomizedSearchCV`. How much does the tuning gain compared with the default?

2. Which fault classes are confused most often? Look at the row-normalised confusion matrix and discuss whether the confusions are plausible.
3. Compare impurity-based importance (`rf.feature_importances_`) with permutation importance. Why can they differ?
4. Train the random forest with `class_weight="balanced"`. How do the recalls of the rare classes *Dirtiness* and *Stains* change?
5. (Extension) Explain one single prediction with a SHAP waterfall plot, as you would to a quality engineer.

8.7.1 Solutions

Solution to Exercise 1

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform, randint

search = RandomizedSearchCV(
    HistGradientBoostingClassifier(random_state=0),
    {"learning_rate": loguniform(0.01, 0.3), "max_depth": randint(2, 10),
     "max_iter": randint(100, 600), "l2_regularization": loguniform(1e-4,
      ↪ 1)},
    n_iter=25, cv=cv, scoring="balanced_accuracy", random_state=0,
 ↪ n_jobs=-1)
search.fit(X_train, y_train)
print("best parameters:", search.best_params_)
print(f"CV balanced accuracy: tuned {search.best_score_:.3f}")
```

```
best parameters: {'l2_regularization': np.float64(0.028079081078857277), 'learning_rate': n
CV balanced accuracy: tuned 0.783
```

Random search samples hyperparameter combinations instead of trying a full grid – usually more efficient when only a few parameters matter. The gain over the default is often modest (a few percentage points); compare it with the fold-to-fold standard deviation before concluding that tuning was worthwhile. The CV score of the best configuration is slightly optimistic, because it was selected among 25 candidates – an untouched test set gives the honest estimate.

Solution to Exercise 2

The largest off-diagonal entries of the normalised confusion matrix show the most frequent confusions. Typically, *Other_Faults* is confused with *Bumps* and vice versa: both are less distinct in shape, and "other" is a collection of different defect types without a common signature. Geometrically clear defects such as *K_Scatch* or *Pastry* are rarely confused. From a process point of view, refining the *Other_Faults* category (better labels) may improve the classification more than any change of the algorithm.

Solution to Exercise 3

```
cmp = pd.DataFrame({"impurity": pd.Series(rf.feature_importances_,
 ↪  index=X.columns),
                    "permutation": imp})
cmp.rank(ascending=False).sort_values("permutation").head(10)
```

	impurity	permutation
Length_of_Conveyer	2.0	1.0
Steel_Plate_Thickness	6.0	2.0
Edges_Y_Index	18.0	3.0
TypeOfSteel_A400	25.0	4.0
Y_Maximum	17.0	5.0
Square_Index	14.0	6.0
TypeOfSteel_A300	26.0	7.0
Y_Minimum	19.0	8.0
Edges_Index	13.0	9.0
X_Minimum	8.0	10.0

Impurity importance is computed on the training data during tree building and favours continuous features with many possible split points; it also distributes importance over correlated features. Permutation importance is computed on test data and measures what the model actually needs for its predictions. Strongly correlated features (e.g. different measures of the defect size) can have high impurity importance but low permutation importance, because shuffling one leaves the others intact.

Solution to Exercise 4

```
from sklearn.metrics import recall_score

rf_bal = RandomForestClassifier(n_estimators=300,
 ↪  class_weight="balanced", random_state=0, n_jobs=-1)
rf_bal.fit(X_train, y_train)
pd.DataFrame({
    "recall (unweighted)": recall_score(y_test, rf.predict(X_test),
     ↪  average=None, labels=rf.classes_),
    "recall (balanced)": recall_score(y_test, rf_bal.predict(X_test),
     ↪  average=None, labels=rf.classes_),
}, index=rf.classes_).round(2)
```

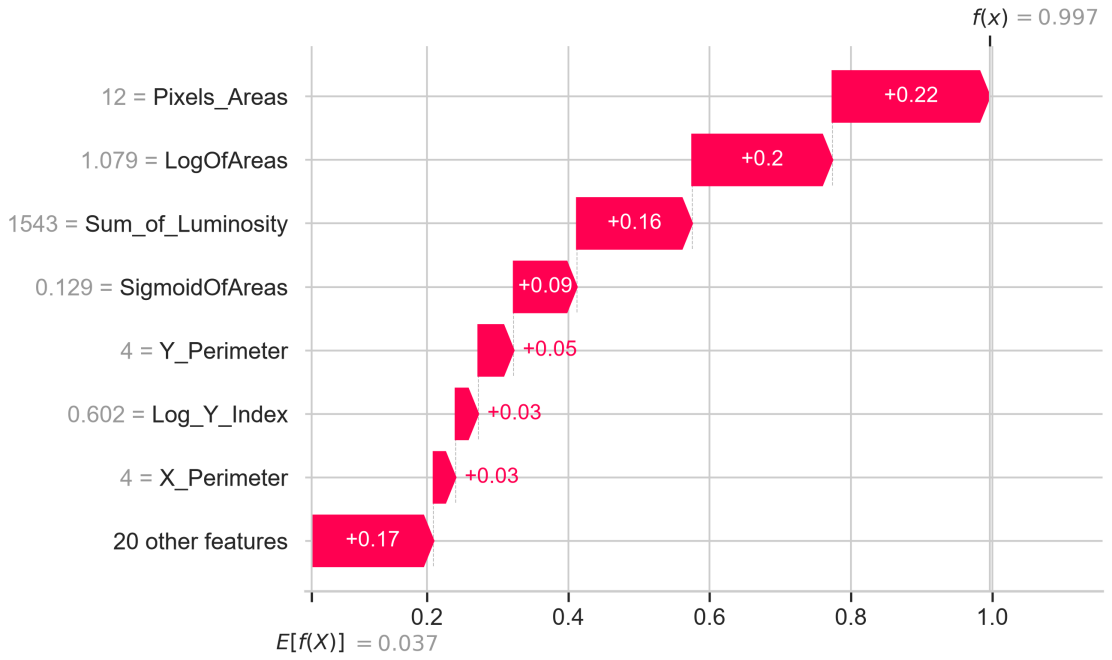
	recall (unweighted)	recall (balanced)
Bumps	0.73	0.81
Dirtiness	0.71	0.86
K_Scratch	0.91	0.91
Other_Faults	0.80	0.62
Pastry	0.54	0.77
Stains	0.94	1.00
Z_Scratch	0.81	0.92

Class weights often increase the recall of rare classes slightly at the expense of frequent ones. For random forests the effect is usually small, because deep trees fit even small classes in the training data. With few test examples per rare class (only a dozen or so), recall estimates are very uncertain – one more or one fewer correct plate changes them by several percentage points.

Solution to Exercise 5

```
i = 0
pred_class = rf.predict(X_test.iloc[[i]])[0]
k = list(rf.classes_).index(pred_class)
print("predicted class:", pred_class, " true class:", y_test.iloc[i])
shap.plots.waterfall(sv[i, :, k], max_display=8)
```

```
predicted class: Stains  true class: Stains
```



The waterfall plot starts at the average model output for this class and adds the contribution of each feature until the prediction for this plate is reached. A good explanation for a quality engineer names the two or three largest contributions in physical terms, for example: "The model considers this a scratch mainly because the defect is long and narrow and its luminosity is low – typical for K_Scatch." It should also state what the explanation is *not*: a causal statement about the production process.

9 Unsupervised Learning and Anomaly Detection

In the previous chapters every training example came with a label: the strength of the concrete, the failure of the machine, the type of defect. In engineering, labels are often the scarcest resource. Failures are rare, new fault types appear that nobody has seen before, and expert labelling is slow and expensive. Yet we usually have plenty of data from **normal operation**.

Unsupervised learning finds structure in data without labels. In this chapter we use it for three purposes: to compress hundreds of correlated sensor signals into a few meaningful components (PCA), to monitor a process by detecting deviations from its normal behaviour, and to group operating states or products into clusters.

Scenario: an online alarm for the fab

After the analysis in Chapter 2, the semiconductor fab decided to try an unsupervised approach: a monitoring system that learns normal runs and raises an alarm for unusual ones, without needing labelled failures. Process engineering accepts an alarm only if it catches at least half of the failed runs while raising false alarms on at most 5 % of the good runs – otherwise operators will ignore it.

Your decision: Install the PCA-based alarm in the production line?

Learning goals. After this chapter you can

- distinguish supervised and unsupervised learning and name engineering uses of the latter,
- reduce high-dimensional process data with principal component analysis (PCA) and interpret scores, loadings and explained variance,
- monitor a process with PCA-based statistics (Hotelling's T^2, SPE) and control limits,
- group data with k-means and DBSCAN and choose the number of clusters,
- detect anomalies with Isolation Forest and evaluate detectors when a few labels are available.

Datasets in this chapter

- SECOM (UCI)
- Condition Monitoring of Hydraulic Systems (UCI)

9.1 Principal component analysis

Process data are highly redundant: dozens of temperatures in the same furnace, pressures along the same line, currents of coupled drives all move together. PCA exploits this redundancy.

Definition: Principal component analysis (PCA)

Given standardised data $Z \in \mathbb{R}^{n \times p}$ (each column with mean 0 and standard deviation 1), PCA finds orthogonal directions $v_1, v_2, \ldots$ (the **loadings**) such that the projection of the data onto v_1 has maximal variance, the projection onto v_2 has maximal variance among all directions orthogonal to v_1, and so on. The projections Zv_j of the data onto the loadings are the **scores** of component j. Computationally, PCA is the singular value decomposition

$$Z = U \Sigma V^{\top},$$

where the columns of V are the loadings and the variance explained by component j is proportional to the squared singular value σ_j^2.

Keeping the first k components gives the best possible approximation of the data by a k-dimensional plane (in the least-squares sense). The **scree plot** (explained variance per component) and the cumulative explained variance help to choose k.

Pitfall: standardise before PCA

PCA maximises variance, and variance depends on units. Without standardisation, a pressure in Pa (values around 10^5) dominates a temperature in °C. Standardise unless all variables share a unit and their natural variances are meaningful.

9.2 Process monitoring with PCA

The idea of multivariate statistical process control: build a PCA model from data of **normal operation** (phase I) and then check every new observation against it (phase II). Two complementary statistics measure the deviation:

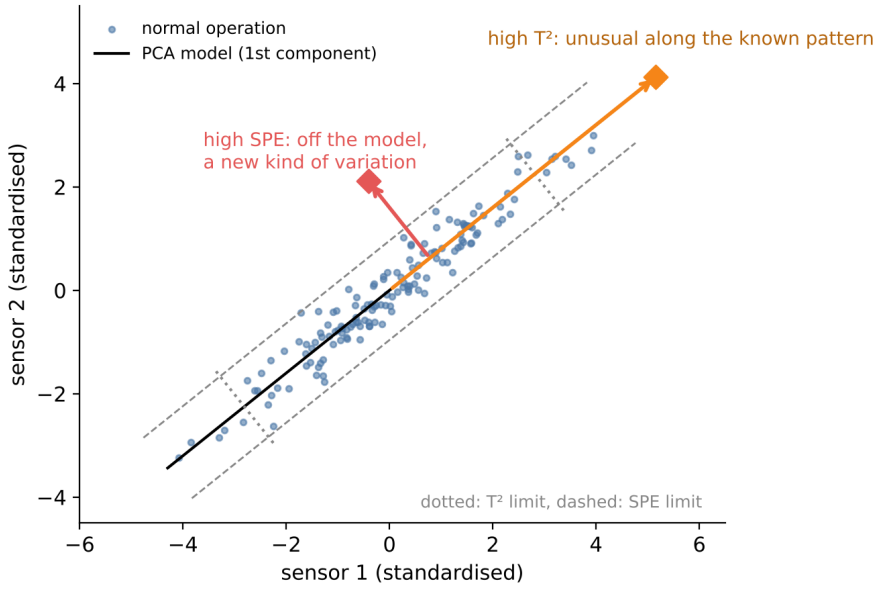


Figure 9.1: Geometry of PCA monitoring, shown for two correlated sensors and a one-component model (in practice: many sensors and a model plane of several components). Hotelling's T^2 measures how unusual an observation is *within* the model (an extreme value along the known pattern); the squared prediction error SPE measures its distance *from* the model (a new kind of variation the model has never seen).

Definition: Hotelling's T^2 and SPE

Let $V_k = [v_1, \dots, v_k]$ contain the first k loadings and λ_j be the variance of the scores of component j in the normal-operation data. For a new standardised observation z with scores $t = V_k^\top z = (t_1, \dots, t_k)^\top$:

$$T^2 = \sum_{j=1}^{k} \frac{t_j^2}{\lambda_j}, \qquad \mathrm{SPE} = \|z - V_k t\|^2.$$

Control limits are derived from the normal-operation data, e.g. as the 99 % quantile of each statistic.

When an alarm occurs, **contribution plots** show which original variables contribute most to the large T^2 or SPE value – a first hint for the operator where to look.

9.3 Clustering

Clustering groups observations so that members of a group are similar and different groups are dissimilar. Engineering uses: identifying operating modes of a machine (idle, part load, full load), grouping products or customers, finding families of similar faults.

Definition: k-means

k-means partitions the data into k clusters by minimising the within-cluster sum of squared distances to the cluster centres. The algorithm alternates two steps until convergence: (1) assign each point to the nearest centre, (2) move each centre to the mean of its points. The result depends on the starting centres, so the algorithm is run several times.

k-means assumes roughly spherical clusters of similar size, and k must be chosen: the **elbow** of the within-cluster sum of squares or the **silhouette score** (how much closer a point is to its own cluster than to the next one, between -1 and 1) help. **DBSCAN** instead defines clusters as dense regions: it finds clusters of arbitrary shape, determines their number itself and labels points in sparse regions as noise – a natural link to anomaly detection.

9.4 Anomaly detection

Definition: Isolation Forest

An Isolation Forest builds random trees that split the data at random features and random thresholds. Anomalies are "few and different": they are isolated after few splits. The **anomaly score** is derived from the average path length needed to isolate a point – short paths mean anomalous.

Engineering practice: evaluating unsupervised detectors

Even if the detector is trained without labels, a few labelled faults are invaluable for **evaluating** it. Use them only for evaluation and threshold selection, never for training – and fit the model on normal data that is separate from the normal data used for evaluation.

```
# Setup: works locally and in Google Colab
import importlib.util, subprocess, sys
for pkg in ["ucimlrepo"]:
    if importlib.util.find_spec(pkg) is None:
```

```
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg],
↪   check=True)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
```

9.5 Live coding: PCA monitoring of semiconductor production

We use the SECOM data introduced in Chapter 2: 1,567 production runs, 590 process measurements, and a pass/fail label. Following the preparation from Chapter 2, we remove sparse and constant features, impute the remaining gaps with the median and standardise. All preprocessing steps and the PCA model are fitted on **passed runs only** – this is our model of normal operation.

```
from ucimlrepo import fetch_ucirepo
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.feature_selection import VarianceThreshold
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

def load_secom():
    """SECOM: return the process measurements X and the label y (-1 = pass, 1
↪    = fail).

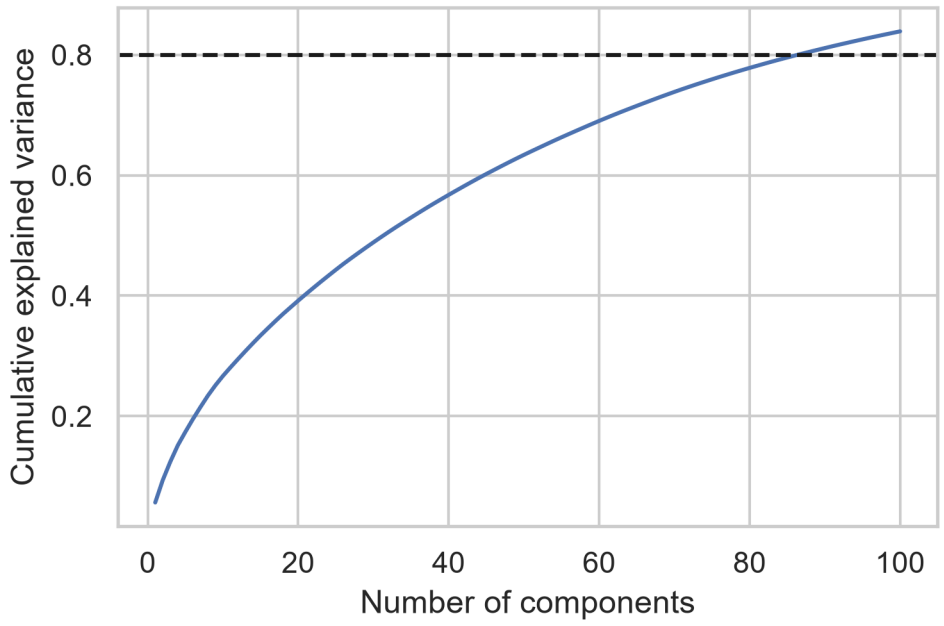
    In the UCI repository the label is not marked as target variable, so we
↪   find it
    ourselves: the only numeric column whose values are exactly -1 and 1.
    """
    df = fetch_ucirepo(id=179).data.original
    num = df.select_dtypes("number")
    labels = [c for c in num.columns if set(num[c].dropna().unique()) == {-1,
↪   1}]
    if len(labels) != 1:
        labels = [c for c in df.columns if any(k in str(c).lower() for k in
↪   ("pass", "fail", "label"))]
```

```
    assert len(labels) == 1, f"Label column not found unambiguously:
     ↪ {labels}"
    y = df[labels[0]].astype(int)
    X = num.drop(columns=labels, errors="ignore")
    print(f"Label column: {labels[0]!r}, measurement columns: {X.shape[1]}, "
          f"ignored non-numeric columns: {[c for c in df.columns if c not in
           ↪ num.columns]}")
    return X, y

X, y = load_secom()                          # helper from Chapter 2
X = X.loc[:, X.isna().mean() < 0.5]          # drop features with > 50 %
 ↪ missing

prep = make_pipeline(SimpleImputer(strategy="median"),
 ↪ VarianceThreshold(1e-8), StandardScaler())
Z = prep.fit_transform(X[y == -1])           # model of normal operation
pca = PCA().fit(Z)
cum = np.cumsum(pca.explained_variance_ratio_)
k = int(np.searchsorted(cum, 0.80) + 1)
plt.plot(np.arange(1, 101), cum[:100]); plt.axhline(0.8, ls="--", c="k")
plt.xlabel("Number of components"); plt.ylabel("Cumulative explained
 ↪ variance")
plt.show()
print("Components for 80 % variance:", k)
```

```
Label column: 'class', measurement columns: 590, ignored non-numeric columns: ['timestamp']
```




```
Components for 80 % variance: 87
```

Discussion. The cumulative explained variance rises steeply for the first components and then slowly. For SECOM, many components are needed to reach 80 % – the process has many partly independent sources of variation, and much of the variance is noise. The choice of 80 % is a convention; in monitoring, the number of components is often chosen by cross-validation of the reconstruction error or by the scree plot's elbow.

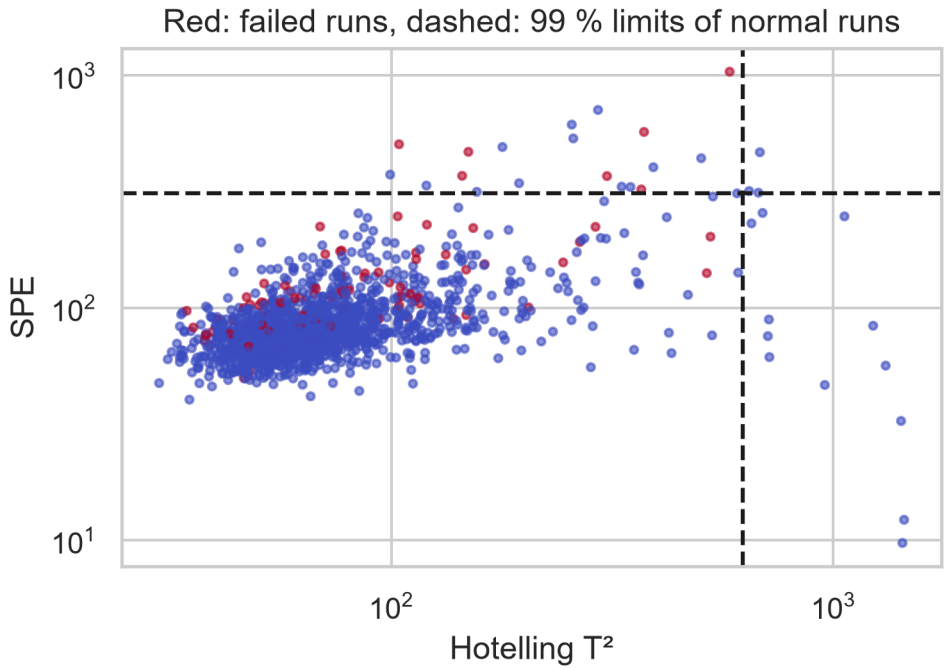
```
pca_k = PCA(k).fit(Z)

def t2_spe(Zx):
    scores = pca_k.transform(Zx)
    t2 = np.sum(scores**2 / pca_k.explained_variance_, axis=1)
    spe = np.sum((Zx - pca_k.inverse_transform(scores))**2, axis=1)
    return t2, spe

t2, spe = t2_spe(prep.transform(X))
t2_lim, spe_lim = np.percentile(t2[y == -1], 99), np.percentile(spe[y == -1],
 ↪  99)

plt.scatter(t2, spe, c=(y == 1), cmap="coolwarm", s=8, alpha=0.6)
plt.axvline(t2_lim, c="k", ls="--"); plt.axhline(spe_lim, c="k", ls="--")
plt.xscale("log"); plt.yscale("log"); plt.xlabel("Hotelling T²");
 ↪  plt.ylabel("SPE")
```

```
plt.title("Red: failed runs, dashed: 99 % limits of normal runs")
plt.show()
```



Discussion. The dashed lines are the 99 % limits of the normal runs, so about 1 % of normal runs exceed each limit by construction (false alarms). Failed runs that exceed a limit are detected; those inside the limits look normal to the model. If most failures lie inside the limits, the measured variables may not contain the cause of the failure, or the failure manifests itself in variables that were removed, or in a way that PCA does not capture.

```
from sklearn.ensemble import IsolationForest
from sklearn.metrics import roc_auc_score

iso = IsolationForest(n_estimators=300, random_state=0).fit(Z)
score = -iso.score_samples(prep.transform(X))
print(f"ROC AUC  Isolation Forest: {roc_auc_score(y == 1, score):.3f}")
print(f"ROC AUC  T²:               {roc_auc_score(y == 1, t2):.3f}")
print(f"ROC AUC  SPE:              {roc_auc_score(y == 1, spe):.3f}")
```

```
ROC AUC  Isolation Forest: 0.571
ROC AUC  T²:               0.533
ROC AUC  SPE:              0.731
```

> ⚠ Pitfall: evaluating on the training data
>
> In this quick comparison, the normal runs used to fit the models are also part of the evaluation. They naturally look "very normal" to the models, which makes the result optimistic. Exercise 1 repeats the evaluation properly with separate normal runs.

For SECOM, expect modest AUC values: fault detection in semiconductor manufacturing from summary process data is known to be hard. This is a realistic lesson – many sensors do not guarantee that the relevant information is in the data.

9.6 Back to the decision

The requirement fixes the false alarm rate at 5 %, so we set the alarm limit at the 95 % quantile of the statistic on *good* runs and then measure which share of the failed runs exceeds it. To avoid the optimism discussed above, we fit preprocessing and PCA on 70 % of the good runs and evaluate on the remaining good runs plus all failed runs.

```
from sklearn.model_selection import train_test_split

X_good_fit, X_good_eval = train_test_split(X[y == -1], test_size=0.3,
 ↪  random_state=0)
prep_d = make_pipeline(SimpleImputer(strategy="median"),
 ↪  VarianceThreshold(1e-8), StandardScaler())
Z_fit = prep_d.fit_transform(X_good_fit)
pca_d = PCA(k).fit(Z_fit)

def spe_of(Zx):
    return np.sum((Zx - pca_d.inverse_transform(pca_d.transform(Zx)))**2,
     ↪  axis=1)

limit = np.percentile(spe_of(prep_d.transform(X_good_eval)), 95)
detection = (spe_of(prep_d.transform(X[y == 1])) > limit).mean()
print(f"False alarm rate on good runs: 5 % (by construction), detection rate
 ↪  of failed runs: {detection:.0%}")
print("Decision:", "install the alarm." if detection >= 0.5 else "do not
 ↪  install - the alarm would miss too many failures.")
```

```
False alarm rate on good runs: 5 % (by construction), detection rate of failed runs: 11%
Decision: do not install - the alarm would miss too many failures.
```

(Strictly, the limit should be set on one part of the held-out good runs and the false alarm rate checked on another; with a few hundred good runs we accept this simplification.) If the detection rate falls short, the result is still valuable: it tells the fab that the summary data do not contain enough information about the failures – and that investing in different measurements is more promising than tuning the algorithm.

9.7 Live coding: operating states of a hydraulic test rig

The hydraulic dataset was recorded on a test rig in which the condition of four components – cooler, valve, pump and accumulator – was varied deliberately. Each of the 2,205 load cycles lasts 60 seconds; 17 sensors were recorded at up to 100 Hz. The file `profile.txt` contains the component conditions, e.g. the cooler efficiency (3 %, 20 % or 100 %). We compute simple features per cycle from the temperature sensors and the cooling efficiency and see whether PCA and k-means recover the cooler condition **without** using the labels.

```
import zipfile, io, requests
from pathlib import Path

DATA = Path("data"); DATA.mkdir(exist_ok=True)
hz = DATA / "hydraulic.zip"
if not hz.exists():
    hz.write_bytes(requests.get("https://archive.ics.uci.edu/static/public/447/"
                                "condition+monitoring+of+hydraulic+systems.zip",
                                timeout=300).content)

def read_sensor(z, name):
    member = next(n for n in z.namelist() if n.endswith(name))
    return pd.read_csv(z.open(member), sep="\t", header=None).values

with zipfile.ZipFile(hz) as z:
    sensors = {s: read_sensor(z, f"{s}.txt") for s in ["TS1", "TS2", "TS3",
    "TS4", "CE", "CP"]}
    profile = pd.DataFrame(read_sensor(z, "profile.txt"),
                           columns=["cooler", "valve", "pump_leak",
    "accumulator", "stable"])

feat = pd.DataFrame({f"{s}_{stat}": getattr(np, stat)(v, axis=1)
                     for s, v in sensors.items() for stat in ["mean",
                      "std"]})
feat.shape
```

```
(2205, 12)
```

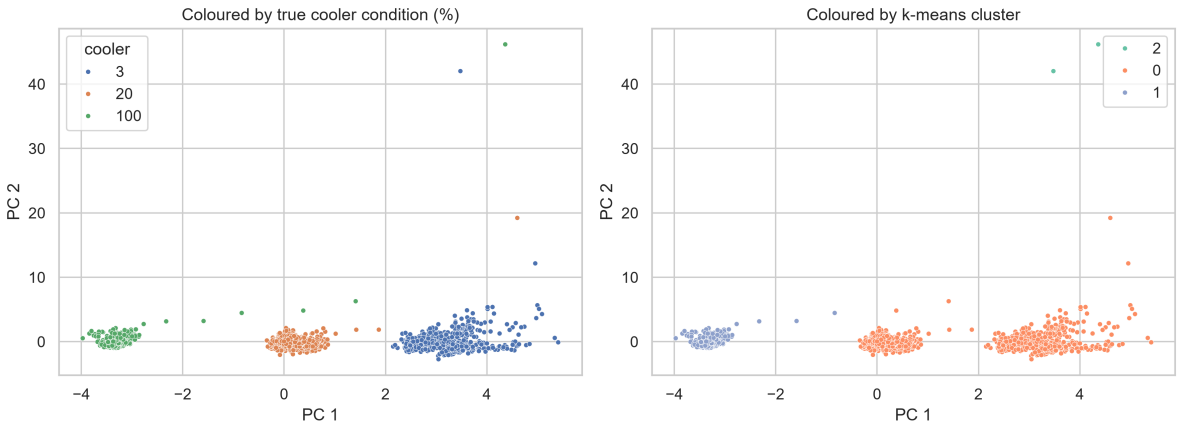
```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

Zh = StandardScaler().fit_transform(feat)
scores_h = PCA(2).fit_transform(Zh)

sil = {k_: silhouette_score(Zh, KMeans(k_, n_init=10,
 ↪  random_state=0).fit_predict(Zh)) for k_ in range(2, 7)}
print("Silhouette scores:", {k_: round(v, 3) for k_, v in sil.items()})
clusters = KMeans(3, n_init=10, random_state=0).fit_predict(Zh)

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
sns.scatterplot(x=scores_h[:, 0], y=scores_h[:, 1],
 ↪  hue=profile["cooler"].astype(int).astype(str),
                ax=axes[0], s=12).set(title="Coloured by true cooler
 ↪  condition (%)", xlabel="PC 1", ylabel="PC 2")
sns.scatterplot(x=scores_h[:, 0], y=scores_h[:, 1], hue=clusters.astype(str),
 ↪  ax=axes[1], s=12,
                palette="Set2").set(title="Coloured by k-means cluster",
 ↪  xlabel="PC 1", ylabel="PC 2")
plt.tight_layout(); plt.show()
pd.crosstab(profile["cooler"], clusters, rownames=["cooler %"],
 ↪  colnames=["cluster"])
```

```
Silhouette scores: {2: 0.543, 3: 0.551, 4: 0.542, 5: 0.558, 6: 0.563}
```



cluster cooler %	0	1	2
3	731	0	1
20	732	0	0
100	2	738	1

Discussion. Temperatures and cooling efficiency depend directly on the cooler condition, so the operating states form well-separated groups in the score plot, and k-means finds them without ever seeing the labels. The cross-table compares clusters and true conditions. This is the ideal case for unsupervised learning: the dominant source of variation in the chosen features *is* the condition of interest. With features dominated by other influences, clusters would reflect those instead – feature choice is decisive.

! Key takeaways

- Unsupervised learning finds structure without labels: compression (PCA), grouping (clustering), deviations (anomaly detection).
- PCA on standardised data summarises correlated sensors; T^2 and SPE monitor a process against its normal behaviour.
- k-means needs a chosen k and spherical clusters; DBSCAN finds arbitrary shapes and noise.
- Fit detectors on normal data, evaluate them with held-out normal data and a few labelled faults.

9.8 Exercises

1. Split the passed SECOM runs into a training part (70 %) and a test part (30 %). Fit preprocessing, PCA and Isolation Forest on the training part only and evaluate on the test part plus all failed runs. How much does the AUC change compared with the optimistic evaluation?
2. Plot the first two principal component scores of SECOM coloured by pass/fail. Then run k-means with 2–6 clusters. Do clusters correspond to failures?
3. Hydraulic system: repeat the analysis with the pressure sensors `PS1`–`PS6` instead of the temperatures. Which component condition (cooler, valve, pump, accumulator) do the clusters now reflect best?
4. Discuss: why is failure detection in SECOM so hard, even though there are almost 600 sensors?

9.8.1 Solutions

Solution to Exercise 1

```
from sklearn.model_selection import train_test_split

X_pass, X_fail = X[y == -1], X[y == 1]
X_pass_tr, X_pass_te = train_test_split(X_pass, test_size=0.3,
 ↪  random_state=0)

prep2 = make_pipeline(SimpleImputer(strategy="median"),
 ↪  VarianceThreshold(1e-8), StandardScaler())
Z_tr = prep2.fit_transform(X_pass_tr)
X_eval = pd.concat([X_pass_te, X_fail])
y_eval = np.r_[np.zeros(len(X_pass_te)), np.ones(len(X_fail))]
Z_eval = prep2.transform(X_eval)

iso2 = IsolationForest(n_estimators=300, random_state=0).fit(Z_tr)
pca2 = PCA(k).fit(Z_tr)
spe2 = np.sum((Z_eval -
 ↪  pca2.inverse_transform(pca2.transform(Z_eval)))**2, axis=1)
print(f"ROC AUC Isolation Forest (proper): {roc_auc_score(y_eval,
 ↪  -iso2.score_samples(Z_eval)):.3f}")
print(f"ROC AUC SPE (proper):              {roc_auc_score(y_eval,
 ↪  spe2):.3f}")
```

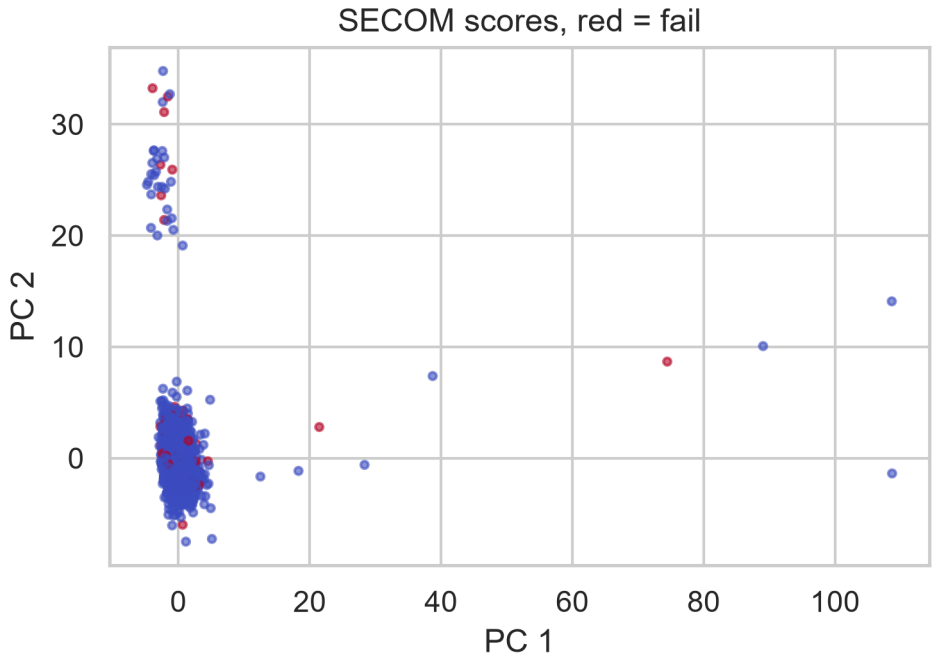
```
ROC AUC Isolation Forest (proper): 0.577
ROC AUC SPE (proper):              0.596
```

The SPE in particular drops when evaluated properly: the PCA model reconstructs its own training runs better than new runs, so in the optimistic evaluation the normal runs looked "too normal" compared with the failed ones. Proper separation of training and evaluation data is just as important in unsupervised as in supervised learning.

Solution to Exercise 2

```
scores_s = pca_k.transform(prep.transform(X))[:, :2]
plt.scatter(scores_s[:, 0], scores_s[:, 1], c=(y == 1), cmap="coolwarm",
 ↪  s=8, alpha=0.6)
plt.xlabel("PC 1"); plt.ylabel("PC 2"); plt.title("SECOM scores, red =
 ↪  fail")
plt.show()

Zall = prep.transform(X)
for k_ in range(2, 7):
    lab = KMeans(k_, n_init=10, random_state=0).fit_predict(Zall)
    rates = pd.Series(y.values == 1).groupby(lab).mean()
    print(f"k = {k_}: failure rate per cluster", rates.round(3).tolist())
```




```
k = 2: failure rate per cluster [0.2, 0.066]
k = 3: failure rate per cluster [0.049, 0.084, 0.25]

k = 4: failure rate per cluster [0.08, 0.25, 0.047, 0.212]
k = 5: failure rate per cluster [0.06, 0.072, 0.05, 0.25, 0.219]
k = 6: failure rate per cluster [0.137, 0.041, 0.055, 0.219, 0.051, 0.25]
```

The failed runs are scattered among the passed ones in the first two components, and the failure rate per cluster stays close to the overall rate of about 6–7 %. The dominant sources of variation in SECOM (e.g. tool or recipe changes) are not related to the pass/fail outcome. Clustering finds the largest structure in the data – which need not be the structure we are interested in.

Solution to Exercise 3

```
with zipfile.ZipFile(hz) as z:
    ps = {s: read_sensor(z, f"{s}.txt") for s in ["PS1", "PS2", "PS3",
 ↪  "PS4", "PS5", "PS6"]}
feat_ps = pd.DataFrame({f"{s}_{stat}": getattr(np, stat)(v, axis=1)
                        for s, v in ps.items() for stat in ["mean",
                         ↪  "std", "max"]})
Zp = StandardScaler().fit_transform(feat_ps)
lab_ps = KMeans(4, n_init=10, random_state=0).fit_predict(Zp)

from sklearn.metrics import adjusted_rand_score
{c: round(adjusted_rand_score(profile[c], lab_ps), 3) for c in ["cooler",
 ↪  "valve", "pump_leak", "accumulator"]}
```

```
{'cooler': 0.585, 'valve': 0.034, 'pump_leak': 0.094, 'accumulator': 0.04}
```

The adjusted Rand index (0 = random agreement, 1 = identical grouping) shows which component condition the clusters reflect. Pressure features respond strongly to the valve condition (switching behaviour) and the accumulator pre-charge pressure, while the temperatures reflected the cooler. Different sensors "see" different faults – a central insight for designing condition monitoring systems.

Solution to Exercise 4

Possible reasons: (1) the measurements are summary values per run and may not capture the physical cause of the failure; (2) the failures may have several different causes, so there is no single "failure direction"; (3) many features are noise, and in high dimensions the distance to normal data grows for *every* point, which dilutes anomaly scores; (4) only about 100 failures are available, and labels may be noisy; (5) part of the failures may originate in later process steps that are not measured at all. More sensors do not help if they do not measure the right thing.

10 Signals and Feature Engineering

Condition monitoring systems record vibration, current, pressure or acoustic signals at sampling rates of several kilohertz. A single minute of vibration data from one accelerometer at 12 kHz contains 720,000 numbers. Machine learning models cannot – and should not – work on such raw data directly. They need a small set of **features** that capture the information relevant for the question at hand.

This chapter shows how to get from a raw signal to meaningful features. We look at signals in the time and in the frequency domain, use the physics of rolling bearings to know *where* in the spectrum a fault should appear, and compute statistical features that summarise a signal window. Feature engineering is where engineering knowledge pays off most in data science.

Scenario: a cheap sensor for the pump station

A water utility operates dozens of pumps with rolling bearings. A full condition monitoring system is too expensive, but the maintenance team could fit each pump with a low-cost vibration sensor that computes a single number per second and raises an alarm above a threshold. The team asks: which number, which threshold – and will it work?

Your decision: Is a simple threshold on one statistical feature good enough to detect an inner-race fault, and which threshold do you set?

Learning goals. After this chapter you can

- explain sampling, the Nyquist frequency and aliasing,
- compute and interpret amplitude spectra with the FFT, including frequency resolution and leakage,
- calculate characteristic bearing fault frequencies and relate spectral peaks to them,
- extract statistical features (RMS, peak, crest factor, kurtosis) from signal windows,
- split windowed data correctly for training and testing,
- (extension) explain and apply envelope analysis.

Datasets in this chapter

- CWRU Bearing Data Center
- Condition Monitoring of Hydraulic Systems (UCI)

10.1 Sampling and aliasing

A sensor signal is continuous; the data acquisition system samples it at discrete times $t_n = n/f_s$ with the **sampling rate** f_s.

> **!** Sampling theorem (Nyquist–Shannon)
>
> A signal can be reconstructed from its samples if it contains no frequencies at or above half the sampling rate, the **Nyquist frequency** $f_N = f_s/2$. Frequencies above f_N are not lost silently – they are folded back and appear as false lower frequencies (**aliasing**).

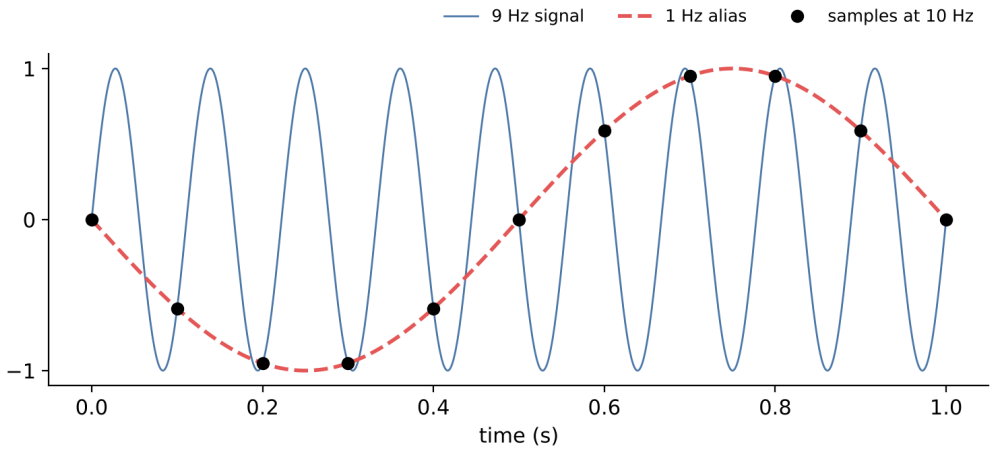


Figure 10.1: Aliasing. A 9 Hz sine sampled at 10 Hz (dots) produces exactly the same samples as a 1 Hz sine. After sampling, the two cannot be distinguished.

A component at frequency f with $f_N < f < f_s$ appears at $|f - f_s|$. This is why data acquisition systems use an analogue **anti-aliasing filter** before the A/D converter. Once aliasing has happened, no digital processing can undo it.

10.2 The frequency domain

Rotating machines produce periodic excitations: imbalance at the shaft frequency, gear meshing at the tooth frequency, bearing defects at characteristic frequencies. In the time domain these components overlap; in the frequency domain they separate into peaks.

Definition: Discrete Fourier transform and amplitude spectrum

For N samples $x_0, \dots, x_{N-1}$, the discrete Fourier transform is

$$X_k = \sum_{n=0}^{N-1} x_n \, e^{-2\pi i \, kn/N}, \qquad k = 0, \dots, N-1,$$

computed efficiently by the **fast Fourier transform (FFT)**. Bin k corresponds to the frequency $f_k = k\, f_s/N$. The single-sided **amplitude spectrum** $A_k = 2|X_k|/N$ (for $0 < k < N/2$) gives the amplitude of a sine component at f_k.

Two practical facts:

- **Frequency resolution** $\Delta f = f_s/N = 1/T$: it depends only on the measurement duration T. To separate two peaks 1 Hz apart, you need at least one second of data.
- **Leakage:** the FFT assumes the signal window repeats periodically. If a component does not complete an integer number of periods in the window, its energy "leaks" into neighbouring bins. Multiplying the window by a smooth **window function** (e.g. Hann) reduces leakage at the price of slightly wider peaks.

10.3 Bearing fault frequencies

When a rolling element passes a local defect on a raceway, it produces a short impact. These impacts repeat at frequencies that follow from the bearing geometry: with shaft frequency f_r, number of rolling elements n_b, rolling element diameter d, pitch diameter D and contact angle ϕ,

$$\text{BPFO} = \frac{n_b f_r}{2}\left(1 - \frac{d}{D}\cos\phi\right), \qquad \text{BPFI} = \frac{n_b f_r}{2}\left(1 + \frac{d}{D}\cos\phi\right).$$

BPFO and BPFI are the *ball pass frequencies* of the outer and inner race. For the drive-end bearing of the CWRU test rig (SKF 6205), the CWRU documentation gives BPFI $\approx$ 5.415 f_r and BPFO $\approx$ 3.585 f_r – check the bearing specification page before relying on these factors.

Engineering practice: let physics tell you where to look

Knowing the fault frequencies turns a vague search ("is the spectrum different?") into a targeted test ("is there a peak at BPFI and its harmonics?"). Features built from physics – the amplitude at BPFI, the energy in a resonance band – are more robust and easier to explain than features found by blind search.

10.4 Statistical features of a signal window

Definition: Common time-domain features

For a window $x_1, \dots, x_N$:

- **RMS** $= \sqrt{\frac{1}{N}\sum x_i^2}$ – overall vibration energy,
- **peak** $= \max |x_i|$,
- **crest factor** = peak / RMS – impulsiveness,
- **kurtosis** $= m_4/m_2^2 - 3$ (excess kurtosis), with the central moments $m_r = \frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x})^r$ – "spikiness"; 0 for a normal distribution, large for signals with short impacts,
- **skewness** – asymmetry of the amplitude distribution.

Early bearing damage produces short impacts: kurtosis and crest factor rise while the RMS hardly changes. As damage progresses and spreads, the impacts become less distinct, kurtosis may fall again and RMS rises – a reason to use several features together.

Pitfall: splitting windows of the same recording

When a long recording is cut into windows, neighbouring windows are almost identical. If windows of the same recording end up in both training and test set, the model recognises the *recording*, not the fault – the test accuracy is meaningless. Split by recording, machine or time period (e.g. with `GroupKFold`), never randomly by window.

```
# Setup: works locally and in Google Colab
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import signal, stats
from scipy.io import loadmat

sns.set_theme(style="whitegrid")
```

```
# Download helper, as in Chapter 2
from pathlib import Path
import requests

DATA = Path("data")
DATA.mkdir(exist_ok=True)                          # create the folder ./data
 ↪  if it does not exist
```

```
def download(url, fname=None):
    """Download a file once and cache it in ./data."""
    if fname is None:                               # no file name given:
        fname = url.split("/")[-1]                  # use the last part of the
 ↪  URL
    path = DATA / fname                             # "/" joins folder and file
 ↪  name
    if not path.exists():                           # download only if not
     ↪  cached yet
        r = requests.get(url, timeout=120)          # give up after 120 s
 ↪  without response
        r.raise_for_status()                        # stop with an error if the
 ↪  download failed
        path.write_bytes(r.content)                 # open, write and close the
 ↪  file in one step
    return path
```

10.5 Live coding: healthy vs. inner-race fault

The Case Western Reserve University (CWRU) bearing data are a widely used benchmark. Faults of defined size were introduced into the bearings of a 2 hp motor test rig by electro-discharge machining; vibration was measured with accelerometers at the drive end and fan end. We use two files: normal baseline (`97.mat`) and a 0.007-inch inner race fault at the drive end (`105.mat`), both at 0 hp motor load (approx. 1797 rpm).

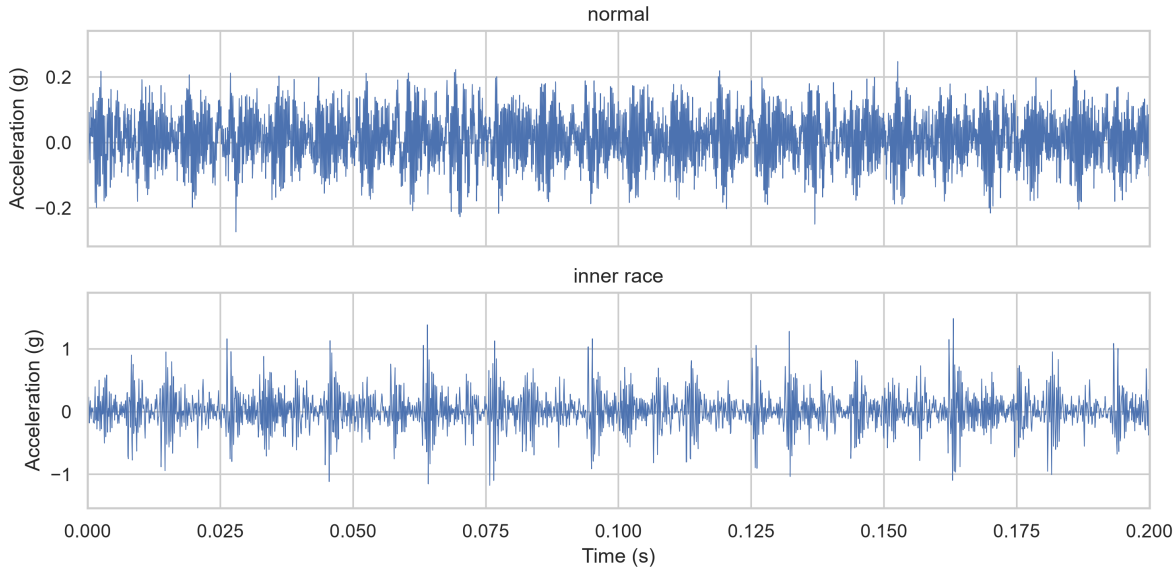
Check the file numbers and the sampling rates (12 kHz or 48 kHz) on the CWRU download pages before using them – the values below are set accordingly and marked.

```
FILES = {"normal": ("97.mat", 48_000), "inner race": ("105.mat", 12_000)}   #
 ↪  file, sampling rate (Hz) - verify!
BASE_URL = "https://engineering.case.edu/sites/default/files/"

signals = {}
for label, (fname, fs) in FILES.items():
    mat = loadmat(download(BASE_URL + fname))
    key = next(k for k in mat if k.endswith("_DE_time"))     # drive-end
 ↪  accelerometer
    signals[label] = (mat[key].ravel(), fs)

fig, axes = plt.subplots(2, 1, figsize=(10, 5), sharex=True)
```

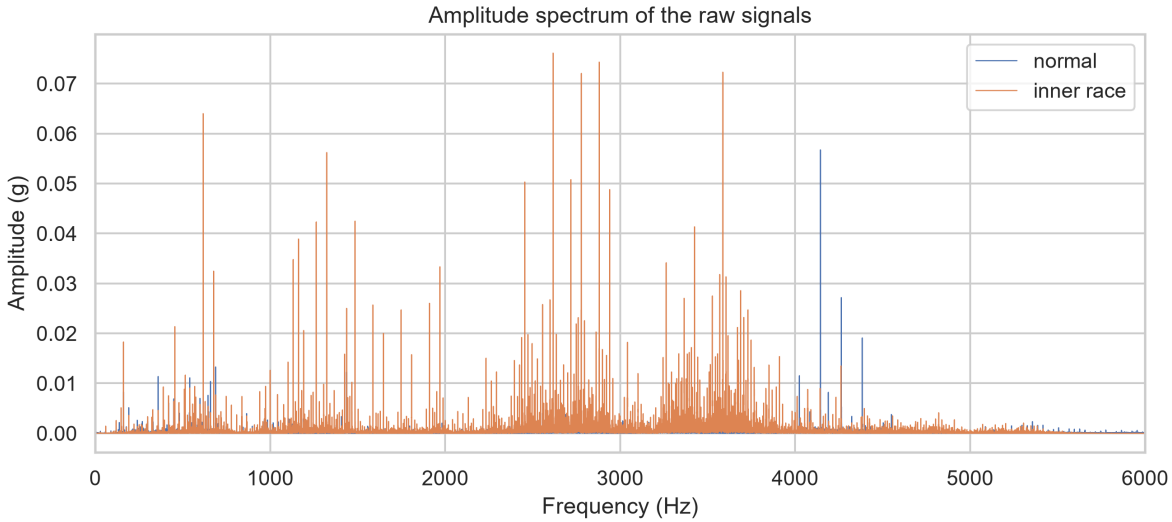
```
for ax, (label, (x, fs)) in zip(axes, signals.items()):
    t = np.arange(len(x)) / fs
    ax.plot(t, x, lw=0.5); ax.set(title=label, ylabel="Acceleration (g)")
axes[1].set(xlim=(0, 0.2), xlabel="Time (s)")
plt.tight_layout(); plt.show()
```



Discussion. Already in the time domain the faulty bearing looks different: short, regularly spaced impacts are visible on top of the background vibration. Count the impacts in 0.1 s – their rate should match the BPFI of about 5.4 × 30 Hz 162 Hz.

10.6 Live coding: amplitude spectrum

```
fig, ax = plt.subplots(figsize=(10, 4))
for label, (x, fs) in signals.items():
    f = np.fft.rfftfreq(len(x), 1 / fs)
    A = 2 * np.abs(np.fft.rfft(x)) / len(x)
    ax.plot(f, A, label=label, lw=0.6)
ax.set(xlim=(0, 6000), xlabel="Frequency (Hz)", ylabel="Amplitude (g)",
       title="Amplitude spectrum of the raw signals")
ax.legend(); plt.show()
```



Discussion. The faulty bearing shows much more energy at high frequencies, in the range of several kilohertz. These are **structural resonances** of the bearing and housing, excited by the impacts – not the fault frequency itself. The impact rate of about 162 Hz modulates these resonances but is hard to see directly in this spectrum. Envelope analysis makes it visible.

10.7 Extension: envelope analysis

Extension – optional material, not required for the exam.

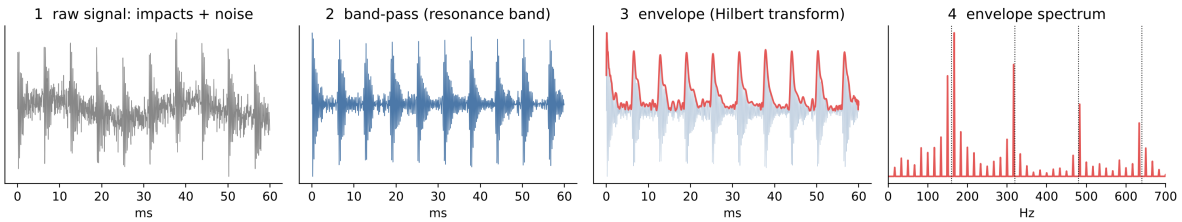


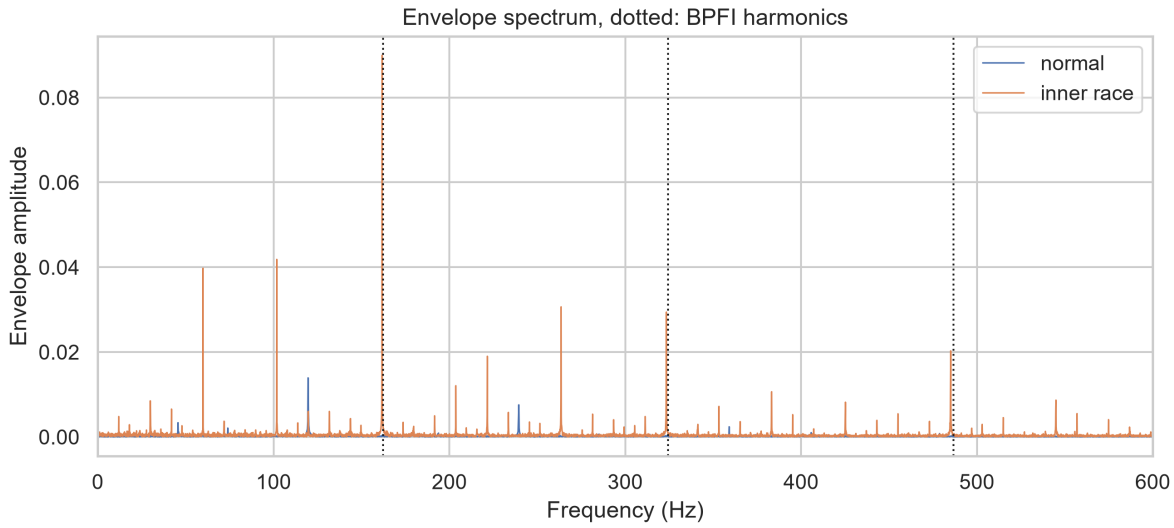
Figure 10.2: Envelope analysis. Periodic impacts excite a high-frequency resonance (1). A band-pass filter isolates the resonance band (2). The envelope – the magnitude of the analytic signal from the Hilbert transform – recovers the impact pattern (3). Its spectrum shows peaks at the impact frequency and its harmonics (4).

The plain spectrum shows *that* the faulty bearing excites high-frequency resonances, but not *why*. The envelope spectrum reveals the repetition rate of the impacts – and thus the damaged component.

```
f_r = 1797 / 60          # shaft frequency (Hz)
bpfi = 5.4152 * f_r

def envelope_spectrum(x, fs, band=(2000, 5000)):
    b, a = signal.butter(4, band, btype="bandpass", fs=fs)
    env = np.abs(signal.hilbert(signal.filtfilt(b, a, x)))
    env -= env.mean()
    f = np.fft.rfftfreq(len(env), 1 / fs)
    return f, np.abs(np.fft.rfft(env)) / len(env)

fig, ax = plt.subplots(figsize=(10, 4))
for label, (x, fs) in signals.items():
    f, A = envelope_spectrum(x, fs)
    ax.plot(f, A, label=label, lw=0.8)
for k in (1, 2, 3):
    ax.axvline(k * bpfi, c="k", ls=":", lw=1)
ax.set(xlim=(0, 600), xlabel="Frequency (Hz)", ylabel="Envelope amplitude",
       title="Envelope spectrum, dotted: BPFI harmonics")
ax.legend(); plt.show()
```



For the inner race fault, peaks appear at BPFI and its harmonics; often there are also sidebands spaced at the shaft frequency, because the defect on the rotating inner ring moves in and out of the load zone once per revolution. The choice of the band (here 2–5 kHz) matters: it should contain the resonance excited by the impacts. Methods such as the *kurtogram* select it automatically.

10.8 Live coding: statistical features

```
def features(x):
    rms = np.sqrt(np.mean(x**2))
    return {"rms": rms, "peak": np.max(np.abs(x)), "crest": np.max(np.abs(x))
     ↪  / rms,
            "kurtosis": stats.kurtosis(x), "skewness": stats.skew(x)}

# features on 1-second windows
rows = []
for label, (x, fs) in signals.items():
    for start in range(0, len(x) - fs, fs):
        rows.append({"condition": label, **features(x[start:start + fs])})
feat = pd.DataFrame(rows)
feat.groupby("condition").mean().round(3)
```

condition	rms	peak	crest	kurtosis	skewness
inner race	0.291	1.601	5.494	2.397	0.164
normal	0.074	0.279	3.787	-0.235	-0.035

Discussion. The faulty bearing has a clearly higher kurtosis and crest factor – the signature of impacts. Such features can feed a classifier (Exercise 3). They are cheap to compute, which matters for monitoring systems running on embedded hardware, and they have an intuitive meaning that maintenance staff understand.

10.9 Back to the decision

A single-feature threshold works if the feature values of healthy and faulty windows do not overlap. We check kurtosis and crest factor and, if they separate, place the threshold in the middle of the gap.

```
for f_name in ["kurtosis", "crest"]:
    healthy_max = feat.loc[feat.condition == "normal", f_name].max()
    faulty_min = feat.loc[feat.condition == "inner race", f_name].min()
    if faulty_min > healthy_max:
        print(f"{f_name:9s}: separates (healthy max {healthy_max:.2f} <
         ↪  faulty min {faulty_min:.2f}) "
```

```
            f"-> threshold {(healthy_max + faulty_min) / 2:.2f}")
    else:
        print(f"{f_name:9s}: overlap (healthy max {healthy_max:.2f} >= faulty
         ↪  min {faulty_min:.2f})")
```

```
kurtosis : separates (healthy max -0.20 < faulty min 2.05) -> threshold 0.92
crest    : separates (healthy max 4.27 < faulty min 5.15) -> threshold 4.71
```

If a feature separates the two recordings, the decision looks easy – but it rests on **one** healthy and **one** faulty recording at one load. Before ordering sensors for dozens of pumps, the threshold must be checked on other loads, other fault types and sizes, and above all on healthy pumps in the field, whose kurtosis can be raised by cavitation or electrical interference. The recommendation is therefore: a pilot on a few pumps with the chosen feature, logging the values for some months before alarms are switched on.

! Key takeaways

- Sample at more than twice the highest relevant frequency and filter before sampling – aliasing cannot be undone.
- The FFT reveals periodic components; frequency resolution is $1/T$, leakage is reduced by window functions.
- Bearing geometry determines where fault frequencies appear; physics-based features are robust and explainable.
- RMS, crest factor and kurtosis summarise signal windows; split windows by recording, not randomly.

10.10 Exercises

1. A vibration signal contains a component at 7 kHz. It is sampled at 10 kHz without anti-aliasing filter. At which frequency does the component appear in the spectrum? What sampling rate would be needed?
2. You record 0.5 s of vibration data at 12 kHz. What is the frequency resolution? Can you separate the BPFI (162.3 Hz) from its sideband at BPFI + f_r?
3. Build a classifier for normal / inner race / ball / outer race faults on the statistical window features. Split each recording in time (first 70 % of the windows for training, last 30 % for testing). Why is a random split of the windows wrong?
4. Hydraulic system: compute features per cycle (mean, std, max) for the 100 Hz pressure sensors and predict the valve condition with a random forest. Use stratified cross-validation.

5. (Extension) Compare the plain amplitude spectrum with the envelope spectrum around BPFI. Then download an outer race fault file and check whether a peak appears at BPFO.

10.10.1 Solutions

Solution to Exercise 1

The Nyquist frequency is 5 kHz. The 7 kHz component is folded back to $|7-10| = 3$ kHz – it appears as a false peak at 3 kHz. To capture 7 kHz correctly, the sampling rate must exceed 14 kHz; in practice one chooses about 2.5 times the highest frequency of interest (17.5 kHz) and uses an anti-aliasing filter with its cut-off below the Nyquist frequency.

Solution to Exercise 2

$\Delta f = 1/T = 1/0.5\,\mathrm{s} = 2$ Hz. The sideband lies $f_r \approx 30$ Hz from the BPFI peak, i.e. 15 bins away – easily separable. Peaks closer than about 2–3 bins (here 4–6 Hz) could not be separated, especially with a window function that widens the peaks. Note that the sampling rate does not affect the resolution; only the duration does.

Solution to Exercise 3

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

FILES4 = {"normal": ("97.mat", 48_000), "inner race": ("105.mat",
 ↪  12_000),
          "ball": ("118.mat", 12_000), "outer race": ("130.mat", 12_000)}
           ↪  # verify on the CWRU pages!
rows = []
for label, (fname, fs) in FILES4.items():
    mat = loadmat(download(BASE_URL + fname))
    x = mat[next(k for k in mat if k.endswith("_DE_time"))].ravel()
    if fs == 48_000:                                    # bring all
     ↪  signals to 12 kHz
        x, fs = signal.decimate(x, 4), 12_000
    win = fs // 4                                       # 0.25 s windows
    starts = range(0, len(x) - win, win)
    n_train = int(0.7 * len(starts))
    for i, s in enumerate(starts):
        rows.append({"condition": label, "train": i < n_train,
 ↪  **features(x[s:s + win])})
fw = pd.DataFrame(rows)
feat_cols = ["rms", "peak", "crest", "kurtosis", "skewness"]
clf = RandomForestClassifier(300, random_state=0).fit(fw.loc[fw.train,
 ↪  feat_cols], fw.loc[fw.train, "condition"])
print(classification_report(fw.loc[~fw.train, "condition"],
 ↪  clf.predict(fw.loc[~fw.train, feat_cols])))
```

```
              precision    recall  f1-score   support

        ball       1.00      1.00      1.00        12
  inner race       1.00      1.00      1.00        12
      normal       1.00      1.00      1.00         6
  outer race       1.00      1.00      1.00        12

    accuracy                           1.00        42
   macro avg       1.00      1.00      1.00        42
weighted avg       1.00      1.00      1.00        42
```

All signals are first brought to the same sampling rate, because features such as peak and RMS depend on the bandwidth. A random split would put neighbouring, nearly identical

windows into training and test, so the test accuracy would measure memorisation. The time split is better, but still optimistic: there is only one recording per condition, so the model may learn properties of the individual test run (sensor mounting, load) rather than the fault. A convincing evaluation needs several recordings per class – e.g. train on 0 and 1 hp, test on 2 and 3 hp.

Solution to Exercise 4

```
import zipfile
from sklearn.model_selection import cross_val_score, StratifiedKFold

hz =
 ↪  download("https://archive.ics.uci.edu/static/public/447/condition+monitoring+of+hydraul
              "hydraulic.zip")

def read_sensor(z, name):
    member = next(n for n in z.namelist() if n.endswith(name))
    return pd.read_csv(z.open(member), sep="\t", header=None).values

with zipfile.ZipFile(hz) as z:
    ps = {s: read_sensor(z, f"{s}.txt") for s in ["PS1", "PS2", "PS3",
 ↪  "PS4", "PS5", "PS6"]}
    profile = pd.DataFrame(read_sensor(z, "profile.txt"),
                           columns=["cooler", "valve", "pump_leak",
 ↪  "accumulator", "stable"])
feat_ps = pd.DataFrame({f"{s}_{stat}": getattr(np, stat)(v, axis=1)
                        for s, v in ps.items() for stat in ["mean",
                         ↪  "std", "max"]})
acc = cross_val_score(RandomForestClassifier(300, random_state=0),
 ↪  feat_ps, profile["valve"].astype(int),
                      cv=StratifiedKFold(5, shuffle=True,
 ↪  random_state=0))
print(f"Valve condition accuracy: {acc.mean():.3f} ± {acc.std():.3f}")
```

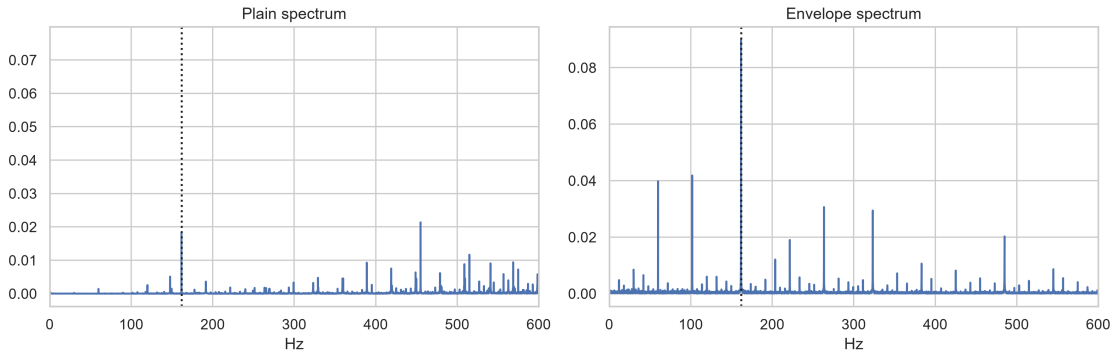
```
Valve condition accuracy: 0.987 ± 0.008
```

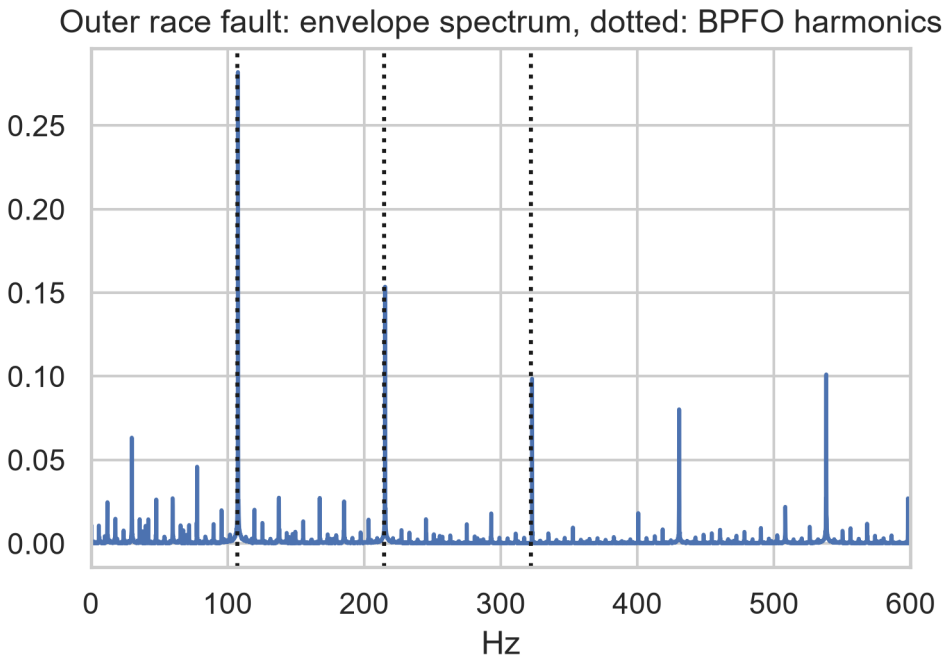
Simple pressure statistics per cycle usually predict the valve condition well, because a degraded valve changes the switching behaviour and thus the pressure curves. Caution: the conditions were changed in blocks over the duration of the experiment, so neighbouring cycles share conditions *and* drifting factors such as oil temperature. A time-based split would give a more conservative estimate than the shuffled cross-validation used here.

Solution to Exercise 5

```
x_ir, fs_ir = signals["inner race"]
f_plain = np.fft.rfftfreq(len(x_ir), 1 / fs_ir)
A_plain = 2 * np.abs(np.fft.rfft(x_ir)) / len(x_ir)
f_env, A_env = envelope_spectrum(x_ir, fs_ir)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(f_plain, A_plain); axes[0].set(xlim=(0, 600), title="Plain
 ↪  spectrum", xlabel="Hz")
axes[1].plot(f_env, A_env); axes[1].set(xlim=(0, 600), title="Envelope
 ↪  spectrum", xlabel="Hz")
for ax in axes:
    ax.axvline(bpfi, c="k", ls=":")
plt.tight_layout(); plt.show()

mat = loadmat(download(BASE_URL + "130.mat"))                  # outer
 ↪  race fault (verify!)
x_or = mat[next(k for k in mat if k.endswith("_DE_time"))].ravel()
f_o, A_o = envelope_spectrum(x_or, 12_000)
bpfo = 3.5848 * f_r
plt.plot(f_o, A_o); plt.xlim(0, 600)
for k in (1, 2, 3):
    plt.axvline(k * bpfo, c="k", ls=":")
plt.title("Outer race fault: envelope spectrum, dotted: BPFO harmonics");
 ↪  plt.xlabel("Hz")
plt.show()
```





In the plain spectrum, the low-frequency range is dominated by shaft harmonics and noise, and the impact rate hardly shows, because the impacts carry their energy at the resonance frequencies. The envelope spectrum demodulates the resonance and reveals BPFI clearly. For the outer race fault, the peaks appear at BPFO; since the outer ring is stationary, the defect stays in the same position relative to the load zone, and shaft-frequency sidebands are typically weaker than for inner race faults.

Part IV

Part D – Time Series, Deep Learning and Review

11 Time Series Forecasting

Grid operators must know tomorrow's electricity demand to schedule power plants; a factory plans shifts based on forecast orders; an energy manager wants to know the peak load of next week. All of these are **forecasting** problems on time series – sequences of measurements ordered in time.

Time series differ from the tabular data of the previous chapters in one fundamental way: the observations are not independent. Today's load is strongly related to yesterday's and to the load one week ago. This dependence is what makes forecasting possible – and it changes the rules for model validation. The single most common mistake in practical forecasting is to evaluate a model with information it would not have had at forecast time.

Scenario: a better day-ahead forecast?

The planning department of a transmission system operator currently forecasts tomorrow's daily load with the seasonal naive method: "same as the same weekday last week". Forecast errors must be balanced with expensive reserve power. A data scientist proposes a gradient boosting model with lags and calendar features. Changing the operational forecasting process costs effort, so the head of planning sets a condition: the new model must reduce the mean absolute error by at least 10 % over a full year.

Your decision: Replace the seasonal naive forecast with the model?

Learning goals. After this chapter you can

- describe a time series by trend, seasonality and noise, and analyse it with the autocorrelation function,
- define the forecasting task precisely (horizon, available information) and build naive and seasonal-naive baselines,
- turn forecasting into a regression problem with lag, calendar and exogenous features,
- validate forecasts correctly in time and compare them with MAE, RMSE and MAPE,
- (extension) fit seasonal ARIMA (SARIMA) models.

Datasets in this chapter

- Open Power System Data – Time Series
- DWD Climate Data Center

11.1 Components and autocorrelation

A time series $y_1, y_2, \ldots, y_T$ is often thought of as a combination of

- a **trend** (slow change of the level, e.g. growing electrification),
- one or several **seasonal** patterns (daily, weekly, yearly cycles),
- **exogenous** influences (temperature, holidays, production plans),
- and an irregular **noise** component.

Definition: Autocorrelation function (ACF)

The autocorrelation at lag h is the correlation of the series with itself shifted by h time steps:

$$\rho(h) = \frac{\operatorname{Cov}(y_t, y_{t-h})}{\operatorname{Var}(y_t)}.$$

Peaks of the ACF at lags 7, 14, 21 (for daily data) indicate a weekly cycle; a slow decay indicates a trend or strong persistence.

Many classical models assume **stationarity**: mean, variance and autocorrelation do not change over time. Trends and seasonal patterns violate this; **differencing** ($y_t - y_{t-1}$, or $y_t - y_{t-7}$ for a weekly season) removes them.

11.2 Defining the forecasting task

Before choosing a model, answer three questions:

1. **Horizon:** how far ahead must we forecast – one hour, one day, one week?
2. **Information set:** which data are available *at the time the forecast is made*? Yesterday's load is known for a day-ahead forecast; today's load is not.
3. **Granularity and metric:** hourly or daily values? Which error matters – average error, peaks, under- or overestimation?

Definition: Forecast error metrics

For forecasts $\hat{y}_t$ and actual values y_t over a test period:

$$\mathrm{MAE} = \frac{1}{n}\sum_t |y_t - \hat{y}_t|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_t (y_t - \hat{y}_t)^2}, \qquad \mathrm{MAPE} = \frac{1}{n}\sum_t \left|\frac{y_t - \hat{y}_t}{y_t}\right|,$$

where the sums run over the n time steps of the test period. RMSE penalises large errors more strongly; MAPE is scale-free but undefined for $y_t = 0$.

Engineering practice: always compare with naive forecasts

The **naive** forecast predicts the last observed value; the **seasonal naive** forecast predicts the value one season ago (e.g. the same weekday last week). They are free, robust and surprisingly hard to beat. A sophisticated model that does not clearly outperform the seasonal naive forecast is not worth its complexity.

11.3 Forecasting as regression

The most flexible approach in practice: build a table in which each row is one time step and the columns contain information available at forecast time – **lagged values** (y_{t-1}, y_{t-7}, …), **rolling statistics** (mean of the last 7 days), **calendar features** (weekday, month, holiday) and **exogenous variables** (temperature forecast). Any regression method from the previous chapters can then be used; gradient boosting is a strong default.

Pitfall: information from the future

- **Random splits** put future observations into the training set and past observations into the test set. The model then "interpolates" instead of forecasting, and the error is unrealistically small.
- **Lags shorter than the horizon:** for a forecast 24 hours ahead, the lag 1 hour is not available at forecast time.
- **Measured instead of forecast exogenous data:** training with the *measured* temperature of tomorrow overstates the accuracy – in operation only a weather *forecast* is available.

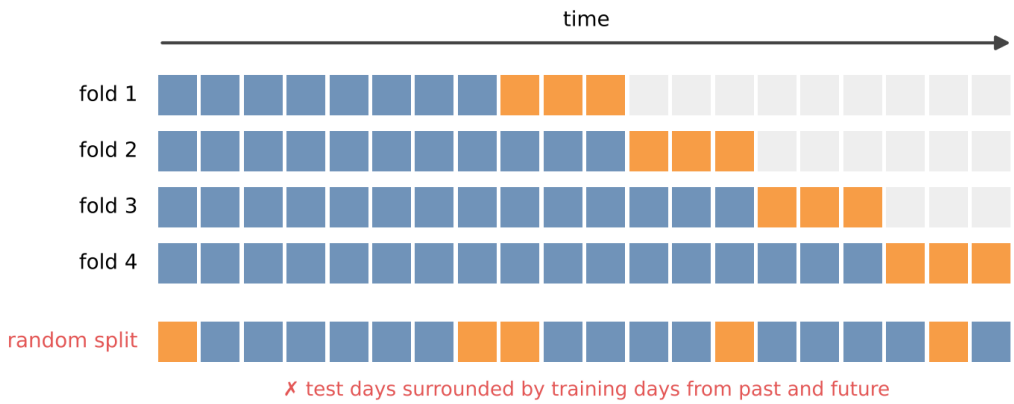


Figure 11.1: Time series cross-validation with an expanding window. The model is always trained on the past (blue) and tested on the period that immediately follows (orange). In a random split (bottom), test observations are surrounded by training observations from before and after.

```
# Setup: works locally and in Google Colab
import importlib.util, subprocess, sys
for pkg in ["statsmodels", "holidays"]:
    if importlib.util.find_spec(pkg) is None:
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg],
 ↪  check=True)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
```

```
# Download helper, as in Chapter 2
from pathlib import Path
import requests

DATA = Path("data")
DATA.mkdir(exist_ok=True)                          # create the folder ./data
 ↪  if it does not exist

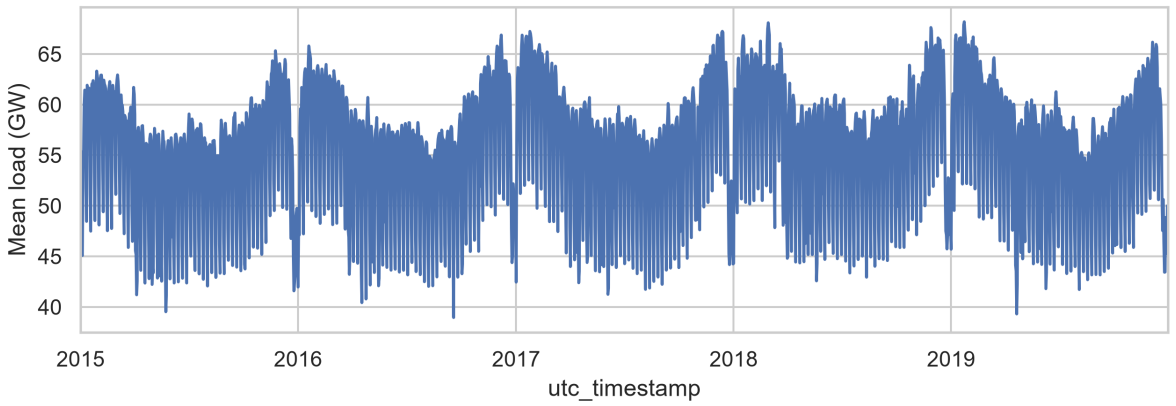
def download(url, fname=None):
    """Download a file once and cache it in ./data."""
```

```
    if fname is None:                                # no file name given:
        fname = url.split("/")[-1]                   # use the last part of the
↪   URL
    path = DATA / fname                              # "/" joins folder and file
↪   name
    if not path.exists():                            # download only if not
     ↪  cached yet
        r = requests.get(url, timeout=300)           # give up after 300 s
↪   without response
        r.raise_for_status()                         # stop with an error if the
↪   download failed
        path.write_bytes(r.content)                  # open, write and close the
↪   file in one step
    return path
```

11.4 Live coding: German electricity load

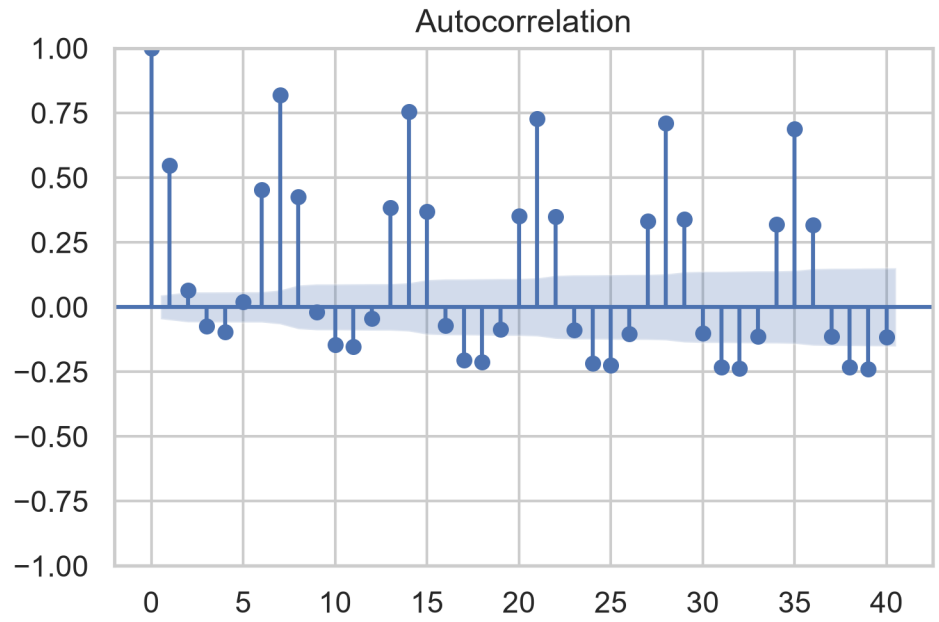
The Open Power System Data (OPSD) time series package collects load, wind and solar generation and prices for 32 European countries from the ENTSO-E Transparency Platform and other sources. We use version 2020-10-06 (2015 to mid-2020) and only the column with the actual German load. The full CSV file is about 124 MB; we read only the two columns we need.

```
url = ("https://data.open-power-system-data.org/time_series/2020-10-06/"
       "time_series_60min_singleindex.csv")
load = pd.read_csv(download(url), usecols=["utc_timestamp",
 ↪  "DE_load_actual_entsoe_transparency"],
                   parse_dates=["utc_timestamp"], index_col="utc_timestamp")
load = load.rename(columns={"DE_load_actual_entsoe_transparency":
 ↪  "load_MW"})["load_MW"]
daily = load.resample("D").mean().loc["2015-01-01":"2019-12-31"] / 1000   #
 ↪  GW
ax = daily.plot(figsize=(10, 3), ylabel="Mean load (GW)")
plt.show()
```



Discussion. The daily mean load shows a yearly cycle (higher in winter: lighting, heating), a strong weekly cycle (lower on weekends) and sharp drops around Christmas and public holidays. We restrict ourselves to 2015–2019 to avoid the exceptional Covid-19 period of 2020 – a reminder that structural breaks can make historical data unrepresentative.

```
from statsmodels.graphics.tsaplots import plot_acf
plot_acf(daily, lags=40); plt.show()
```



The ACF has pronounced peaks at lags 7, 14, 21, … – the weekly cycle – and decays slowly overall, reflecting the yearly cycle and persistence.

11.4.1 Baselines

We forecast each day of the test period one day ahead (horizon 1 day) and use the last year as test period.

```
test_start = "2019-01-01"
y_test = daily[test_start:]
naive = daily.shift(1)[test_start:]                 # yesterday
seasonal_naive = daily.shift(7)[test_start:]        # same weekday last week

def report(name, pred):
    err = y_test - pred
    print(f"{name:22s} MAE = {err.abs().mean():.2f} GW,  MAPE = {(err.abs() /
     ↪  y_test).mean():.1%}")

report("naive (yesterday)", naive)
report("seasonal naive (t-7)", seasonal_naive)
```

```
naive (yesterday)      MAE = 4.31 GW,  MAPE = 8.0%
seasonal naive (t-7)   MAE = 2.30 GW,  MAPE = 4.3%
```

The naive forecast fails at every weekday/weekend transition; the seasonal naive forecast handles these well and fails mainly on holidays and around them.

11.4.2 Feature-based forecasting with gradient boosting

```
import holidays
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.model_selection import TimeSeriesSplit, cross_val_score

df = daily.to_frame("load")
for lag in (1, 7, 14):
    df[f"lag_{lag}"] = df["load"].shift(lag)
df["dow"] = df.index.dayofweek
df["doy"] = df.index.dayofyear
de_holidays = holidays.Germany(years=range(2015, 2021))
df["holiday"] = [d.date() in de_holidays for d in df.index]
df = df.dropna()

X, y = df.drop(columns="load"), df["load"]
```

```
gbr = HistGradientBoostingRegressor(random_state=0)
scores = cross_val_score(gbr, X, y, cv=TimeSeriesSplit(5),
 ↪  scoring="neg_mean_absolute_percentage_error")
print("MAPE per fold (time series CV):", np.round(-scores, 3))

gbr.fit(X[:test_start].iloc[:-1], y[:test_start].iloc[:-1])
report("gradient boosting", pd.Series(gbr.predict(X[test_start:]),
 ↪  index=X[test_start:].index))
```

```
MAPE per fold (time series CV): [0.032 0.029 0.018 0.016 0.016]
gradient boosting      MAE = 0.88 GW,  MAPE = 1.6%
```

Discussion. All features are available at forecast time: lags 1, 7 and 14 days, the calendar and the holiday calendar, which is known in advance. `TimeSeriesSplit` trains on an expanding window and always tests on the following block; the error per fold shows how stable the model is over time. The comparison with the baselines on the same test year shows how much the model really adds.

Weather as an input. Temperature drives heating and cooling demand. Historical hourly temperatures are available from the DWD (see Chapter 2 and Exercise 2). For a real day-ahead forecast, the model must be trained and operated with temperature *forecasts* – using measured temperatures overstates the accuracy.

11.5 Back to the decision

We compare both methods on the test year 2019, which was not used for training. The model must reduce the MAE by at least 10 %.

```
pred_gb = pd.Series(gbr.predict(X[test_start:]), index=X[test_start:].index)
common = pred_gb.index.intersection(seasonal_naive.dropna().index)
mae_naive = (y_test[common] - seasonal_naive[common]).abs().mean()
mae_model = (y_test[common] - pred_gb[common]).abs().mean()
gain = 1 - mae_model / mae_naive
print(f"MAE seasonal naive: {mae_naive:.2f} GW,  MAE gradient boosting:
 ↪  {mae_model:.2f} GW,  improvement: {gain:.0%}")
print("Decision:", "replace the seasonal naive forecast." if gain >= 0.10
 ↪  else "keep the seasonal naive forecast.")
```

```
MAE seasonal naive: 2.30 GW,  MAE gradient boosting: 0.88 GW,  improvement: 62%
Decision: replace the seasonal naive forecast.
```

A one-year test is a good basis, but the recommendation should add: the improvement comes mainly from holidays and transitions – check the error on those days separately; the model should be monitored after go-live, because structural changes (such as the drop in demand in 2020) can make it worse than the simple method; and adding temperature forecasts (Exercise 2) is the obvious next step.

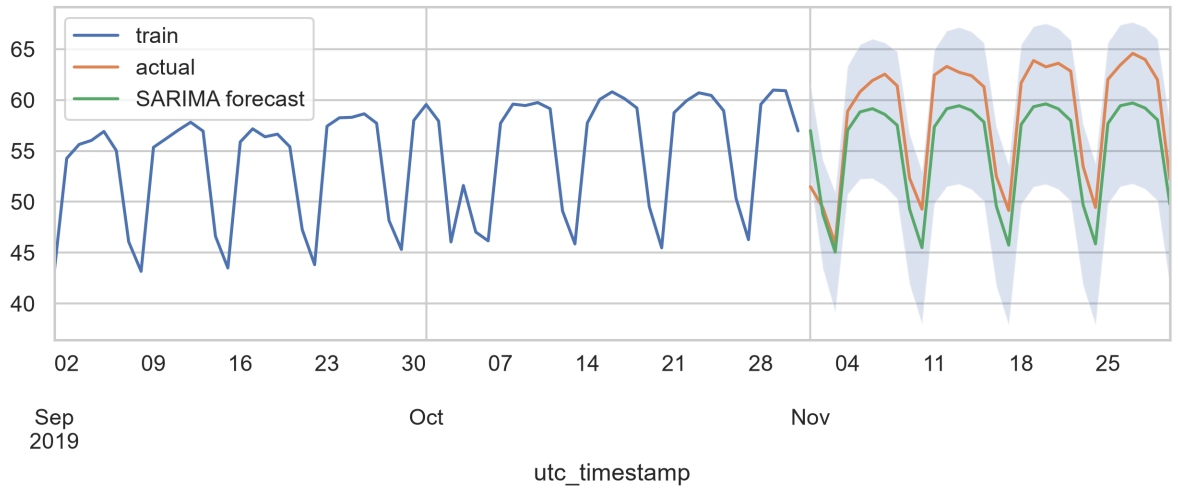
11.6 Extension: seasonal ARIMA

Extension – optional material, not required for the exam.

ARIMA models describe a series by its own past values (**AR**, autoregressive part), past forecast errors (**MA**, moving average part) after differencing d times (**I**, integrated). The seasonal extension SARIMA$(p,d,q)(P,D,Q)_s$ adds the same terms at the seasonal lag s (here $s = 7$ days). ARIMA models are the classical statistical approach: compact, with well-founded prediction intervals, but harder to extend with many exogenous variables than the regression approach.

```
from statsmodels.tsa.statespace.sarimax import SARIMAX

train, test = daily[:"2019-10-31"], daily["2019-11-01":"2019-11-30"]
sarima = SARIMAX(train, order=(1, 0, 1), seasonal_order=(1, 1, 1,
 ↪  7)).fit(disp=False)
fc = sarima.get_forecast(len(test))
ax = train["2019-09-01":].plot(label="train", figsize=(10, 3))
test.plot(ax=ax, label="actual")
fc.predicted_mean.plot(ax=ax, label="SARIMA forecast")
ci = fc.conf_int()
ax.fill_between(ci.index, ci.iloc[:, 0], ci.iloc[:, 1], alpha=0.2)
ax.legend(); plt.show()
mape = (abs(fc.predicted_mean - test) / test).mean()
print(f"MAPE November 2019 (forecast up to 30 days ahead): {mape:.1%}")
```




```
MAPE November 2019 (forecast up to 30 days ahead): 6.1%
```

Note that this is a *multi-step* forecast: all 30 days are forecast at once from the end of October, so the horizon grows from 1 to 30 days, and the prediction interval widens accordingly. It is therefore not directly comparable with the one-day-ahead forecasts above.

! Key takeaways

- Define horizon and information set first; they determine which features are legal.
- Always compare with naive and seasonal naive baselines.
- Lag, calendar and exogenous features turn forecasting into regression; gradient boosting is a strong default.
- Validate in time order (`TimeSeriesSplit`); random splits and future information produce misleadingly small errors.

11.7 Exercises

1. Use a random 5-fold `KFold` with shuffling instead of `TimeSeriesSplit` for the gradient boosting model. Why does the error look better – and why is that wrong?
2. Add the daily mean temperature of the DWD station Trier-Petrisberg (historical hourly data) as a feature. How much does the error improve? Why is this improvement optimistic for real operation?
3. Remove the `holiday` feature. On which days does the error increase most?
4. Build a forecast for 7 days ahead (e.g. for weekly planning). Which lag features may you still use? How does the error compare with the 1-day-ahead forecast?

5. You want to forecast *hourly* load one day ahead. Which seasonal periods and which lags are relevant now?

11.7.1 Solutions

Solution to Exercise 1

```
from sklearn.model_selection import KFold

random_scores = cross_val_score(gbr, X, y, cv=KFold(5, shuffle=True,
 ↪  random_state=0),

 ↪  scoring="neg_mean_absolute_percentage_error")
print("MAPE random KFold:      ", np.round(-random_scores, 3))
print("MAPE TimeSeriesSplit:   ", np.round(-scores, 3))
```

```
MAPE random KFold:       [0.016 0.016 0.015 0.014 0.016]
MAPE TimeSeriesSplit:    [0.032 0.029 0.018 0.016 0.016]
```

With random folds, every test day is surrounded by training days from just before and just after. The model effectively interpolates within known periods – it has, for instance, seen the Christmas holidays of the same year. In operation, the future is never available for training. The time-ordered split measures what we need: the error on a period that lies entirely after the training data.

Solution to Exercise 2

```
import re, zipfile

hist_dir =
 ↪  ("https://opendata.dwd.de/climate_environment/CDC/observations_germany/"
            "climate/hourly/air_temperature/historical/")
listing = download(hist_dir, "dwd_hist_index.html").read_text()
fname = re.search(r"stundenwerte_TU_05100_\d+_\d+_hist\.zip",
 ↪  listing).group(0)
with zipfile.ZipFile(download(hist_dir + fname)) as z:
    name = next(n for n in z.namelist() if n.startswith("produkt"))
    dwd = pd.read_csv(z.open(name), sep=";", na_values=[-999],
 ↪  skipinitialspace=True)
dwd.columns = dwd.columns.str.strip()
dwd["time"] = pd.to_datetime(dwd["MESS_DATUM"].astype(str),
 ↪  format="%Y%m%d%H")
temp = dwd.set_index("time")["TT_TU"].resample("D").mean().rename("temp")
temp.index = temp.index.tz_localize("UTC")          # DWD time stamps are
 ↪  UTC, like the OPSD index

df_t = df.join(temp, how="inner").dropna()
X_t, y_t = df_t.drop(columns="load"), df_t["load"]
s_t = cross_val_score(gbr, X_t, y_t, cv=TimeSeriesSplit(5),
 ↪  scoring="neg_mean_absolute_percentage_error")
print(f"MAPE without temperature: {-scores.mean():.3f},  with
 ↪  temperature: {-s_t.mean():.3f}")
```

```
MAPE without temperature: 0.022,  with temperature: 0.022
```

The temperature helps mostly in the heating season. The improvement is optimistic for two reasons: we use the *measured* temperature of the forecast day, which is not known one day in advance (in operation one would use a weather forecast with its own error), and one station is only a rough proxy for the temperature relevant to German demand – a population-weighted average of many stations would be better.

Solution to Exercise 3

```
X_nh = X.drop(columns="holiday")
m_h =
 ↪  HistGradientBoostingRegressor(random_state=0).fit(X[:test_start].iloc[:-1],
 ↪  y[:test_start].iloc[:-1])
m_nh =
 ↪  HistGradientBoostingRegressor(random_state=0).fit(X_nh[:test_start].iloc[:-1],
 ↪  y[:test_start].iloc[:-1])
err_h = (y[test_start:] - m_h.predict(X[test_start:])).abs()
err_nh = (y[test_start:] - m_nh.predict(X_nh[test_start:])).abs()
(err_nh - err_h).sort_values(ascending=False).head(10).round(2)
```

```
utc_timestamp
2019-05-01 00:00:00+00:00    12.21
2019-01-01 00:00:00+00:00    11.70
2019-10-03 00:00:00+00:00    10.18
2019-04-19 00:00:00+00:00     9.58
2019-06-10 00:00:00+00:00     9.51
2019-04-23 00:00:00+00:00     5.61
2019-04-22 00:00:00+00:00     4.81
2019-12-25 00:00:00+00:00     4.00
2019-05-02 00:00:00+00:00     3.24
2019-05-27 00:00:00+00:00     2.95
Name: load, dtype: float64
```

The largest increases occur on public holidays (Easter, Whit Monday, Christmas, New Year) and often on the following days, because the lag-1 and lag-7 features then contain the unusual holiday load. A “bridge day” feature or a feature “holiday 7 days ago” can reduce these after-effects.

Solution to Exercise 4

```
h = 7
df7 = daily.to_frame("load")
for lag in (h, h + 7, 364):
    df7[f"lag_{lag}"] = df7["load"].shift(lag)
df7["dow"], df7["doy"] = df7.index.dayofweek, df7.index.dayofyear
df7["holiday"] = [d.date() in de_holidays for d in df7.index]
df7 = df7.dropna()
s7 = cross_val_score(HistGradientBoostingRegressor(random_state=0),
 ↪  df7.drop(columns="load"), df7["load"],
                      cv=TimeSeriesSplit(5),
 ↪  scoring="neg_mean_absolute_percentage_error")
print(f"MAPE 1 day ahead: {-scores.mean():.3f},  7 days ahead:
 ↪  {-s7.mean():.3f}")
```

```
MAPE 1 day ahead: 0.022,  7 days ahead: 0.033
```

For a 7-day horizon, the most recent load we can use is from 7 days before the target day, so lags 1 to 6 are forbidden. Lags 7, 14 and the same day one year ago (lag 364, which keeps the weekday) remain. The error typically increases because recent changes – a cold spell, a holiday period – are not yet visible in the data.

Solution to Exercise 5

Hourly load has a **daily** cycle (period 24 h), a **weekly** cycle (168 h) and a yearly cycle (8760 h), and the daily profile itself differs between weekdays and weekends and between summer and winter (interaction of calendar features). For a day-ahead forecast made, say, at noon for all 24 hours of the next day, the most recent usable lags are around 24–36 hours, plus 168 hours (same hour last week). Useful features: hour of day, weekday, holiday, lags of 24, 48 and 168 hours, and temperature forecasts. Also note daylight saving time: in local time, one day per year has 23 and one 25 hours – working in UTC and adding local-time calendar features avoids errors.

12 Deep Learning in Engineering

Neural networks are behind most of the recent headlines about artificial intelligence: image recognition, speech assistants, large language models. In engineering, they are used for visual quality inspection, for virtual sensors, for surrogate models of simulations and for predicting the remaining useful life of machines. Their strength is that they can learn suitable features from raw data – pixels, signal samples, sequences – by themselves, where the methods of the previous chapters needed hand-crafted features.

This chapter gives a compact, application-oriented introduction. We build a neural network from its smallest unit, the neuron, train it with gradient descent and apply it to predict the remaining useful life of jet engines. An extension section shows how pretrained networks detect defects on images. Neural networks in depth are the subject of a separate course; here the goal is to understand the principles and to judge when deep learning is worth its cost.

Scenario: from fixed intervals to condition-based maintenance

In Chapter 3, the airline set a fixed overhaul interval for its engines. The fleet manager now wants to go further: every engine streams its sensor data, and a model predicts its remaining useful life (RUL). An engine should be scheduled for overhaul when its predicted RUL falls below the planning horizon of 20 cycles. Safety demands that at most 5 % of the engines are scheduled too late. The model, of course, is not perfect.

Your decision: Which safety margin must be added to the predicted RUL so that the safety requirement holds?

Learning goals. After this chapter you can

- explain neurons, layers, activation functions, loss functions and backpropagation,
- train a neural network in PyTorch with mini-batch gradient descent and monitor training with a validation set,
- prepare sensor time series as sliding windows for remaining useful life (RUL) prediction,
- judge when deep learning is worth it compared with classical methods,
- (extension) use a pretrained CNN as a feature extractor for visual anomaly detection.

Datasets in this chapter

- NASA C-MAPSS turbofan degradation data
- MVTec Anomaly Detection (MVTec AD)

12.1 From linear regression to neural networks

A linear regression computes a weighted sum of its inputs. A **neuron** does the same and then applies a nonlinear **activation function**:

> **Definition: Neuron, layer, multilayer perceptron**
>
> A **neuron** computes $a = \sigma(w^\top x + b)$ with weights w, bias b and activation function σ. A **layer** applies many neurons to the same input: $a = \sigma(Wx + b)$. A **multilayer perceptron (MLP)** stacks layers:
>
> $$\hat{y} = W_3\,\sigma(W_2\,\sigma(W_1 x + b_1) + b_2) + b_3.$$
>
> Common activations are ReLU, $\sigma(z) = \max(0, z)$, and the sigmoid $1/(1 + e^{-z})$.

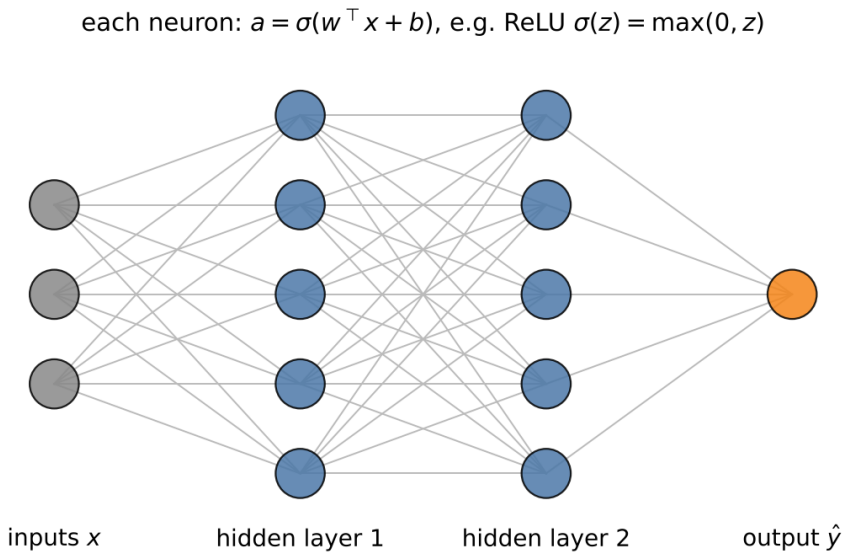


Figure 12.1: A multilayer perceptron with two hidden layers. Each arrow carries a weight; each neuron computes a weighted sum of its inputs and applies an activation function. Without the nonlinear activations, the whole network would collapse into a single linear model.

! Universal approximation

An MLP with one hidden layer and enough neurons can approximate any continuous function on a bounded region arbitrarily well. The theorem guarantees that a suitable network *exists* – not that training will find it, nor how much data is needed. In practice, deeper networks with fewer neurons per layer often learn more efficiently.

12.2 Training: loss, gradients and backpropagation

The network's weights and biases, collected in a parameter vector θ, are found by minimising a **loss function** $\mathcal{L}(\theta)$ on the training data, e.g. the mean squared error for regression or the cross-entropy for classification. Since the loss is a nonconvex function of possibly millions of weights, we use **gradient descent**:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta),$$

with the **learning rate** η. The gradient is computed by **backpropagation** – the chain rule of calculus applied layer by layer from the output back to the input. Frameworks like PyTorch do this automatically (*automatic differentiation*).

Practical ingredients:

- **Mini-batches:** each step uses a small random subset of the data (e.g. 256 examples) – *stochastic* gradient descent. One pass through all data is an **epoch**.
- **Optimisers** like Adam adapt the step size per weight and usually work well with default settings.
- **Scaling:** inputs should be standardised; otherwise training is slow or unstable.
- **Validation and early stopping:** monitor the loss on a validation set during training and stop when it no longer improves – the most important protection against overfitting.

Engineering practice: when is deep learning worth it?

- **Raw, high-dimensional data** (images, spectrograms, long signals, text): yes – neural networks learn the features.
- **Tabular data with meaningful engineered features:** usually no – gradient boosting is typically as good or better, faster to train and easier to explain.
- **Little data:** usually no, unless a **pretrained** network can be reused (transfer learning).
- Always compare with a classical baseline and consider the costs: training time, hardware, expertise, explainability, certification.

12.3 Remaining useful life from sensor sequences

The **remaining useful life (RUL)** of a machine is the time it can still operate before failure. Predicting it is the core of predictive maintenance: replace parts shortly before they fail, not too early (wasted life) and not too late (unplanned failure).

Two modelling choices are typical for RUL prediction:

- **Sliding windows:** the input for a prediction at cycle t is the sensor history of the last L cycles, an $L \times p$ matrix (p sensors). Each trajectory provides many overlapping windows for training.
- **Piecewise linear target:** early in life, degradation is not observable in the sensors – a new engine with 300 remaining cycles looks like one with 200. The RUL target is therefore capped at a constant (commonly 125 cycles) and decreases linearly afterwards.



Figure 12.2: Sliding windows and capped RUL target. From each engine's sensor history, windows of L cycles are cut; the target of a window is the RUL at its last cycle. The RUL is capped at 125 cycles because early degradation is invisible in the data.

Windows of the same engine are highly correlated – exactly like signal windows in Chapter 10. Training and validation data must be split **by engine**.

```
# Setup: works locally and in Google Colab
import importlib.util, os, subprocess, sys
for pkg in ["torch", "torchvision"]:
```

```
    if importlib.util.find_spec(pkg) is None:
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg],
 ↪  check=True)

# Windows + Anaconda: the pip build of PyTorch brings its own copy of the
 ↪  OpenMP runtime
# (libiomp5md.dll), which clashes with the one used by Anaconda's NumPy/MKL
 ↪  and kills the
# kernel ("OMP: Error #15"). This switch lets both copies coexist; it has no
 ↪  effect elsewhere.
os.environ.setdefault("KMP_DUPLICATE_LIB_OK", "TRUE")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import torch
from torch import nn

sns.set_theme(style="whitegrid")
```

12.4 Live coding: remaining useful life

The data are downloaded automatically as in Chapter 3. Subset FD001 has one operating condition and one fault mode (degradation of the high-pressure compressor). The test file contains trajectories that end some time *before* failure; `RUL_FD001.txt` gives the true RUL at the end of each test trajectory.

```
from pathlib import Path
import zipfile, io, requests

def cmapss_dir(root=Path("data")):
    """Download and extract the NASA C-MAPSS data once; return the folder
     ↪  with train_FD001.txt."""
    root.mkdir(exist_ok=True)
    target = root / "CMAPSSData"
    if not any(target.rglob("train_FD001.txt")):
        zip_path = root / "CMAPSSData.zip"
        if not zip_path.exists():
            r =
 ↪  requests.get("https://data.nasa.gov/docs/legacy/CMAPSSData.zip",
 ↪  timeout=300)
```

```
            r.raise_for_status()
            zip_path.write_bytes(r.content)
        with zipfile.ZipFile(zip_path) as z:
            z.extractall(target)
        for inner in list(target.rglob("*.zip")):          # some versions
         ↪  contain a nested zip
            zipfile.ZipFile(inner).extractall(inner.parent)
    return next(target.rglob("train_FD001.txt")).parent

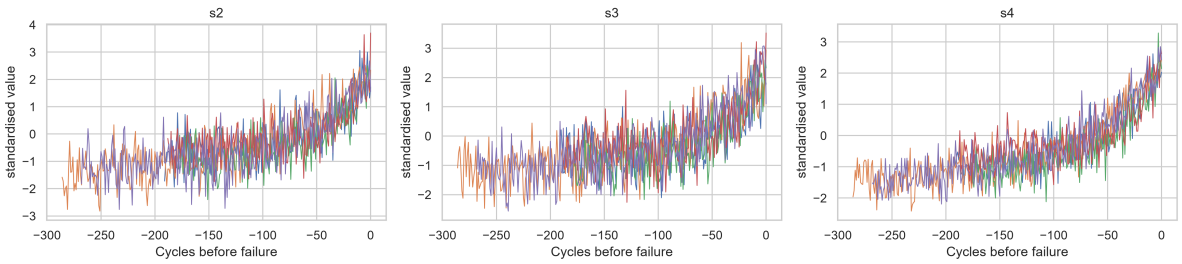
DATA = cmapss_dir()
cols = ["unit", "cycle", "op1", "op2", "op3"] + [f"s{i}" for i in range(1,
 ↪  22)]
read = lambda f: pd.read_csv(DATA / f, sep=r"\s+", header=None, names=cols)

train, test = read("train_FD001.txt"), read("test_FD001.txt")
rul_test = pd.read_csv(DATA / "RUL_FD001.txt", header=None).iloc[:, 0].values

train["rul"] = (train.groupby("unit")["cycle"].transform("max") -
 ↪  train["cycle"]).clip(upper=125)
sensors = [c for c in cols[5:] if train[c].std() > 1e-3]       # drop
 ↪  constant sensors
mu, sd = train[sensors].mean(), train[sensors].std()
train[sensors] = (train[sensors] - mu) / sd
test[sensors] = (test[sensors] - mu) / sd
print("Sensors used:", sensors)
```

```
Sensors used: ['s2', 's3', 's4', 's6', 's7', 's8', 's9', 's11', 's12', 's13', 's14', 's15',
```

```
fig, axes = plt.subplots(1, 3, figsize=(15, 3.5))
for ax, s in zip(axes, sensors[:3]):
    for u in [1, 2, 3, 4, 5]:
        g = train[train.unit == u]
        ax.plot(g["cycle"] - g["cycle"].max(), g[s], lw=0.8)
    ax.set(title=s, xlabel="Cycles before failure", ylabel="standardised
 ↪  value")
plt.tight_layout(); plt.show()
```



Plotted against the cycles *before failure*, several sensors show a clear trend that accelerates towards the end of life – the degradation signature the network must learn. Some sensors are constant in FD001 and were removed.

```
L = 30   # window length (cycles)

def windows(df, with_target=True):
    X, y = [], []
    for _, g in df.groupby("unit"):
        v = g[sensors].values
        if len(v) < L:                                  # pad short test
         ↪  series
            v = np.vstack([np.repeat(v[:1], L - len(v), axis=0), v])
        if with_target:
            r = g["rul"].values
            for end in range(L, len(v) + 1):
                X.append(v[end - L:end]); y.append(r[end - 1])
        else:
            X.append(v[-L:])
    return np.array(X, dtype=np.float32), np.array(y, dtype=np.float32)

X_tr, y_tr = windows(train)
X_te, _ = windows(test, with_target=False)
X_tr.shape, X_te.shape
```

```
((17731, 30, 15), (100, 30, 15))
```

```
torch.manual_seed(0)
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(L * len(sensors), 64), nn.ReLU(),
    nn.Linear(64, 32), nn.ReLU(),
    nn.Linear(32, 1),
)
```

```
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.MSELoss()

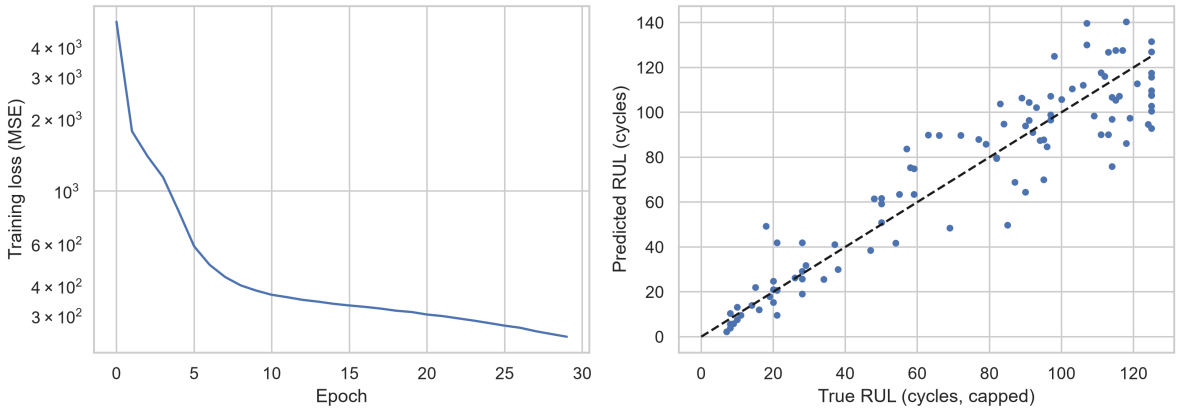
Xt, yt = torch.tensor(X_tr), torch.tensor(y_tr).unsqueeze(1)
loader = torch.utils.data.DataLoader(torch.utils.data.TensorDataset(Xt, yt),
                                     batch_size=256, shuffle=True)
history = []
for epoch in range(30):
    model.train()
    epoch_loss = 0.0
    for xb, yb in loader:
        opt.zero_grad()
        loss = loss_fn(model(xb), yb)
        loss.backward()
        opt.step()
        epoch_loss += loss.item() * len(xb)
    history.append(epoch_loss / len(Xt))

model.eval()
with torch.no_grad():
    pred = model(torch.tensor(X_te)).squeeze().numpy()
rmse = np.sqrt(np.mean((pred - np.minimum(rul_test, 125))**2))
print(f"Test RMSE: {rmse:.1f} cycles")
```

```
Test RMSE: 15.2 cycles
```

The training loop is the same in every PyTorch project: forward pass, loss, `backward()` to compute gradients, `step()` to update the weights. The network has a few tens of thousands of weights – small by deep learning standards.

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].plot(history); axes[0].set(xlabel="Epoch", ylabel="Training loss
 ↪  (MSE)", yscale="log")
axes[1].scatter(np.minimum(rul_test, 125), pred, s=12)
axes[1].plot([0, 125], [0, 125], "k--")
axes[1].set(xlabel="True RUL (cycles, capped)", ylabel="Predicted RUL
 ↪  (cycles)")
plt.tight_layout(); plt.show()
```



Discussion. The predictions follow the diagonal, with larger scatter at high RUL, where degradation is barely visible, and smaller scatter near the end of life – fortunately the region where accurate predictions matter most. The training loss alone says nothing about overfitting; we did not use a validation set here (Exercise 2 fixes this). Also note that the test RMSE is computed on only 100 engines and therefore has considerable uncertainty itself. Whether the network beats a gradient boosting model on window statistics is an open question – Exercise 1.

12.5 Back to the decision

An engine is scheduled too late if the model **overestimates** its RUL. If we schedule an overhaul when *predicted RUL* − *margin* falls below the planning horizon, the margin must cover the overestimates of 95 % of the engines – i.e. it must equal the 95 % quantile of the prediction error $\hat{y} - y$ on the test engines.

```
true_rul = np.minimum(rul_test, 125)
err = pred - true_rul                                    # > 0: overestimate
 ↪  (dangerous)
margin = max(0.0, float(np.quantile(err, 0.95)))
print(f"Overestimated engines: {(err > 0).mean():.0%}, 95 % quantile of the
 ↪  error: {np.quantile(err, 0.95):.1f} cycles")
print(f"Decision: schedule an overhaul when the predicted RUL falls below 20
 ↪  + {margin:.0f} = {20 + margin:.0f} cycles.")
```

```
Overestimated engines: 50%, 95 % quantile of the error: 23.9 cycles
Decision: schedule an overhaul when the predicted RUL falls below 20 + 24 = 44 cycles.
```

The margin is the price of the model's uncertainty: the more accurate the model, the smaller the margin and the more useful life is used. Note that the quantile is estimated from only 100 test engines – the fleet manager should add a reserve and refine the margin with field experience. And as in Chapter 3: C-MAPSS is a simulation, so the procedure must be validated on real engine data before it replaces the fixed interval.

12.6 Extension: CNNs and transfer learning for visual inspection

Extension – optional material, not required for the exam.

Convolutional neural networks (CNNs) process images with small learnable filters (e.g. 3 × 3 pixels) that slide across the image. Early layers detect edges and simple textures, deeper layers combine them into complex patterns. Because the same filter is applied everywhere, CNNs need far fewer weights than an MLP on raw pixels.

Training a CNN from scratch requires many thousands of labelled images. **Transfer learning** avoids this: a network pretrained on millions of everyday images (ImageNet) already provides general visual features that transfer surprisingly well to industrial images. For anomaly detection, a simple but strong approach is: embed all good training images with the pretrained CNN and score a test image by its distance to the nearest good image in this feature space. No defective images are needed for training.

12.6.1 Live coding: anomaly detection on one MVTec category

Data access. MVTec AD is free for non-commercial use (CC BY-NC-SA 4.0) and is downloaded from the MVTec website after accepting the license. Download one category (e.g. `bottle`, a few hundred MB) and extract it to `data/mvtec/bottle/`. A GPU (Colab: *Runtime → Change runtime type*) speeds things up but is not required. Because of the manual download, the two MVTec cells are **not executed** when the script is built – run them yourself after downloading the data.

```
from pathlib import Path
from torchvision import models
from PIL import Image

ROOT = Path("data/mvtec/bottle")
device = "cuda" if torch.cuda.is_available() else "cpu"

weights = models.ResNet18_Weights.DEFAULT
backbone = models.resnet18(weights=weights)
backbone.fc = torch.nn.Identity()           # use the 512-d feature vector
```

```
backbone.eval().to(device)
tf = weights.transforms()

@torch.no_grad()
def embed(paths):
    feats = []
    for p in paths:
        x = tf(Image.open(p).convert("RGB")).unsqueeze(0).to(device)
        feats.append(backbone(x).cpu().numpy()[0])
    return np.array(feats)

train_paths = sorted((ROOT / "train" / "good").glob("*.png"))
test_paths = sorted((ROOT / "test").glob("*/*.png"))
labels = np.array([p.parent.name != "good" for p in test_paths])
F_train, F_test = embed(train_paths), embed(test_paths)
```

```
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import roc_auc_score

nn_model = NearestNeighbors(n_neighbors=1).fit(F_train)
score = nn_model.kneighbors(F_test)[0][:, 0]
print(f"Image-level ROC AUC: {roc_auc_score(labels, score):.3f}")

order = np.argsort(score)[::-1][:5]
fig, axes = plt.subplots(1, 5, figsize=(15, 3))
for ax, i in zip(axes, order):
    ax.imshow(Image.open(test_paths[i])); ax.axis("off")
    ax.set_title(f"{test_paths[i].parent.name}\nscore {score[i]:.1f}")
plt.show()
```

The five most anomalous test images should mostly be defective bottles. State-of-the-art methods (e.g. PatchCore) refine this idea by comparing local image patches instead of whole images, which also localises the defect.

12.7 Outlook

- **Physics-informed machine learning:** physical laws enter the model as constraints or as terms of the loss function – useful when data are scarce and extrapolation matters.
- **Surrogate models:** neural networks or Gaussian processes replace expensive FEM or CFD simulations in optimisation loops.

- **Foundation models:** large pretrained models for time series, images and text, adapted to specific tasks with little data; large language models as assistants for coding and documentation.
- **Deployment and MLOps:** models in production must be monitored for *data drift* (the input data change, e.g. a new sensor or a new product), retrained, versioned and – in safety-relevant applications – validated and certified.

! Key takeaways

- A neural network is a stack of linear maps and nonlinear activations, trained by gradient descent with backpropagation.
- Monitor a validation set and stop early; split by unit (engine, machine), not by window.
- For RUL prediction, sliding windows and a capped target are standard; errors near end of life matter most.
- Deep learning shines on raw images, signals and text; for tabular features, compare with gradient boosting first.

12.8 Exercises

1. Build a baseline: gradient boosting on features of each window (last value, mean and slope of each sensor). Is the neural network better?
2. Hold out 20 training engines as a validation set (split by engine, not by window!) and implement early stopping. Plot training and validation loss.
3. Why is an *underestimate* of the RUL less critical than an overestimate? Design an asymmetric loss function and discuss its effect.
4. (Extension) Replace the MLP by a 1D convolutional network (`nn.Conv1d`). Mind the tensor shape (batch, channels, time).
5. (Extension) MVTec: repeat the anomaly detection for a texture category (e.g. `carpet`) and an object category (e.g. `screw`). Where does the method work better? Choose a threshold so that no defective part passes – what fraction of good parts is rejected?

12.8.1 Solutions

Solution to Exercise 1

```
from sklearn.ensemble import HistGradientBoostingRegressor

def window_features(Xw):
    t = np.arange(L) - (L - 1) / 2
    slope = (Xw * t[None, :, None]).sum(axis=1) / (t**2).sum()
    return np.hstack([Xw[:, -1, :], Xw.mean(axis=1), slope])

gb =
 ↪  HistGradientBoostingRegressor(random_state=0).fit(window_features(X_tr),
 ↪  y_tr)
pred_gb = gb.predict(window_features(X_te))
rmse_gb = np.sqrt(np.mean((pred_gb - np.minimum(rul_test, 125))**2))
print(f"Test RMSE: MLP {rmse:.1f} cycles, gradient boosting {rmse_gb:.1f}
 ↪  cycles")
```

```
Test RMSE: MLP 15.2 cycles, gradient boosting 12.5 cycles
```

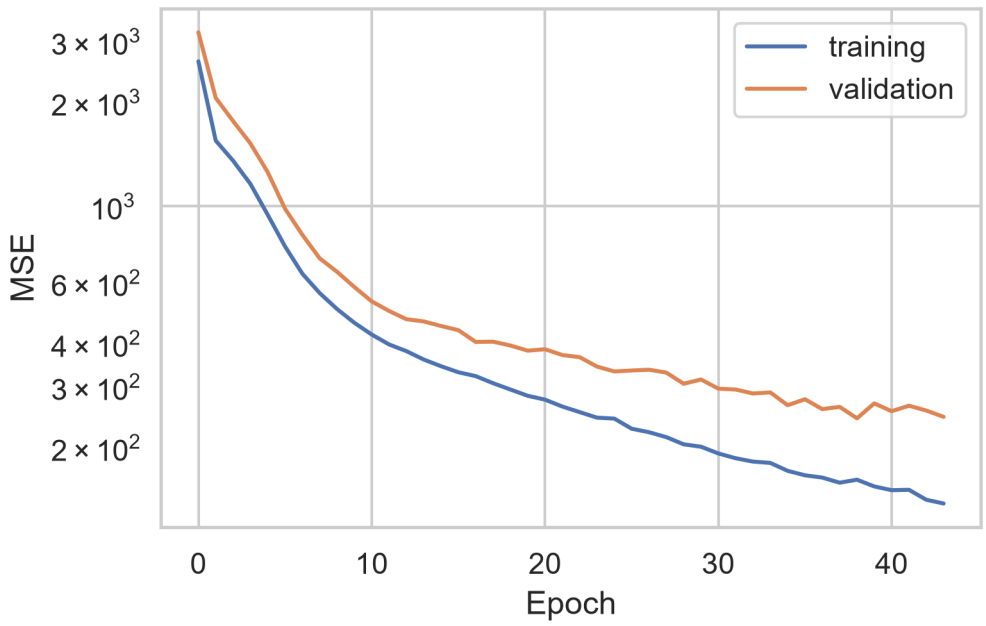
With well-chosen window features (level, mean and trend per sensor), gradient boosting is often on par with a small MLP on FD001 – a reminder that deep learning is not automatically better on sensor data with few channels. Neural networks gain an advantage on the more complex subsets (FD002/FD004 with six operating conditions) and with architectures that exploit the sequence structure, such as 1D CNNs or LSTMs.

Solution to Exercise 2

```
rng = np.random.default_rng(0)
val_units = rng.choice(train["unit"].unique(), size=20, replace=False)
Xa, ya = windows(train[~train.unit.isin(val_units)])
Xv, yv = windows(train[train.unit.isin(val_units)])
Xa_t, ya_t = torch.tensor(Xa), torch.tensor(ya).unsqueeze(1)
Xv_t, yv_t = torch.tensor(Xv), torch.tensor(yv).unsqueeze(1)

def make_mlp():
    return nn.Sequential(nn.Flatten(), nn.Linear(L * len(sensors), 64),
     ↪  nn.ReLU(),
                         nn.Linear(64, 32), nn.ReLU(), nn.Linear(32, 1))

torch.manual_seed(0)
net, best, best_state, patience, wait = make_mlp(), np.inf, None, 5, 0
opt2 = torch.optim.Adam(net.parameters(), lr=1e-3)
dl = torch.utils.data.DataLoader(torch.utils.data.TensorDataset(Xa_t,
 ↪  ya_t), batch_size=256, shuffle=True)
tr_hist, va_hist = [], []
for epoch in range(100):
    net.train()
    for xb, yb in dl:
        opt2.zero_grad(); l = loss_fn(net(xb), yb); l.backward();
 ↪  opt2.step()
    net.eval()
    with torch.no_grad():
        tr_hist.append(loss_fn(net(Xa_t), ya_t).item());
 ↪  va_hist.append(loss_fn(net(Xv_t), yv_t).item())
    if va_hist[-1] < best:
        best, best_state, wait = va_hist[-1], {k: v.clone() for k, v in
 ↪  net.state_dict().items()}, 0
    else:
        wait += 1
        if wait >= patience:
            break
net.load_state_dict(best_state)
plt.plot(tr_hist, label="training"); plt.plot(va_hist,
 ↪  label="validation")
plt.yscale("log"); plt.xlabel("Epoch"); plt.ylabel("MSE"); plt.legend();
 ↪  plt.show()
print(f"stopped after {len(va_hist)} epochs, best validation RMSE
 ↪  {np.sqrt(best):.1f} cycles")
```




```
stopped after 44 epochs, best validation RMSE 15.6 cycles
```

The validation loss typically decreases at first and then flattens or rises while the training loss keeps falling – overfitting. Early stopping keeps the weights of the best validation epoch. Splitting by engine is essential: with a random window split, validation windows would be near-copies of training windows from the same engine, and the validation loss would keep falling.

Solution to Exercise 3

An underestimate leads to maintenance too early: some useful life is wasted, a cost that can be planned. An overestimate means the machine may fail before maintenance: unplanned downtime, secondary damage, possibly a safety risk. An asymmetric loss penalises late predictions (predicted RUL > true RUL) more strongly, e.g. the score of the PHM 2008 challenge with $d = \hat{y} - y$:

$$s(d) = \begin{cases} e^{-d/13} - 1, & d < 0 \\ e^{d/10} - 1, & d \geq 0. \end{cases}$$

A simpler differentiable variant for training is a weighted squared error:

```
def asymmetric_mse(pred, target, w_late=4.0):
    d = pred - target
    return torch.mean(torch.where(d > 0, w_late * d**2, d**2))
```

Training with such a loss shifts the predictions towards lower RUL values – the model becomes deliberately conservative. The weight should reflect the real cost ratio of late and early maintenance.

Solution to Exercise 4

```
class CNN1D(nn.Module):
    def __init__(self, n_sensors):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv1d(n_sensors, 32, kernel_size=5, padding=2),
 ↪  nn.ReLU(),
            nn.Conv1d(32, 32, kernel_size=5, padding=2), nn.ReLU(),
            nn.AdaptiveAvgPool1d(1), nn.Flatten(), nn.Linear(32, 1))

    def forward(self, x):                    # x: (batch, time, sensors)
        return self.net(x.transpose(1, 2))  # Conv1d expects (batch,
         ↪  channels, time)

torch.manual_seed(0)
cnn = CNN1D(len(sensors))
opt3 = torch.optim.Adam(cnn.parameters(), lr=1e-3)
for epoch in range(30):
    cnn.train()
    for xb, yb in loader:
        opt3.zero_grad(); l = loss_fn(cnn(xb), yb); l.backward();
 ↪  opt3.step()
cnn.eval()
with torch.no_grad():
    pred_cnn = cnn(torch.tensor(X_te)).squeeze().numpy()
print(f"Test RMSE 1D CNN: {np.sqrt(np.mean((pred_cnn -
 ↪  np.minimum(rul_test, 125))**2)):.1f} cycles")
```

```
Test RMSE 1D CNN: 18.2 cycles
```

The convolution filters slide along the time axis and share their weights over time, so the network detects local patterns (e.g. a steepening trend) wherever they occur in the

window. The sensors are the input *channels* – hence the transposition from (batch, time, sensors) to (batch, sensors, time).

Solution to Exercise 5

Change `ROOT` to the new category and rerun the two cells. Image-level anomaly detection with global features typically works very well for objects with consistent appearance (bottle) and for many textures, but less well for categories with small, local defects on objects with varying pose (screw), because a global feature vector averages over the whole image. For a threshold that lets no defective part pass:

```
thr = score[labels].min()                          # lowest score among
 ↪  defective images
false_reject = (score[~labels] >= thr).mean()
print(f"threshold {thr:.2f}: {false_reject:.0%} of good parts rejected")
```

The resulting false reject rate shows the price of zero slip. Note that this threshold was chosen on the test set itself and is therefore optimistic; in practice it must be set on separate validation images.

13 Review and Exam Preparation

This chapter collects the methods of the course in one map, lists the most important pitfalls and offers practice questions in the style of the exam. The exam is written, without a computer: it tests understanding, interpretation and method choice – not programming routine. Sections marked *Extension* in the chapters are not part of the exam; in the tables below they are marked (ext.).

13.1 Which method when?

Engineering question	Data	Method	Chapter
How reliable is a component?	lifetimes	Weibull fit, B10 life, bootstrap	3
How much can we trust an alarm?	detection and false alarm rates, base rate	Bayes' theorem	3, 7
Few tests, but prior knowledge	counts, earlier products	Bayesian updating (beta–binomial) (ext.)	3
Did a change improve the process?	measurements of two or more groups	t-test, ANOVA, effect size	4
How many samples do I need?	assumed effect and noise	power analysis	4
How does an output depend on inputs?	continuous, few inputs	linear regression, design of experiments (ext.)	5
Fit a physical or empirical law	measured curve	nonlinear least squares (`curve_fit`)	5
Predict a continuous quantity	many inputs	regularised regression, gradient boosting	6, 8
Expensive experiments or simulations	few runs	Gaussian process, Bayesian optimisation (ext.)	6

Engineering question	Data	Method	Chapter
OK / not OK, fault type	labelled data	logistic regression, trees, ensembles	7, 8
Why does the model decide this way?	trained model	permutation importance, partial dependence, SHAP (ext.)	8
Detect unknown deviations	mostly normal data	PCA (T^2, SPE), Isolation Forest	9
Find operating modes or groups	unlabelled data	k-means, DBSCAN	9
Diagnose rotating machinery	vibration signals	FFT, statistical features, envelope spectrum (ext.)	10
Forecast demand or load	time series	baselines, lag features + boosting, SARIMA (ext.)	11
Remaining useful life	multivariate sequences	neural networks, boosting baseline	12
Inspect surfaces	images	pretrained CNN, anomaly detection (ext.)	12

13.2 Core concepts at a glance

Concept	In one sentence	Chapter
CRISP-DM	Process model: understanding → data → preparation → modelling → evaluation → deployment, iterated.	1
Statistics vs. ML	Statistics focuses on understanding and uncertainty, ML on prediction for new data; they overlap in statistical learning.	1

Concept	In one sentence	Chapter
Robust statistics	Median, IQR and MAD are barely affected by outliers, mean and standard deviation are.	2
MCAR / MAR / MNAR	Whether missingness is random, depends on observed data, or on the missing value itself.	2
Central limit theorem	Means of many independent observations are approximately normal, with standard error $\sigma/\sqrt{n}$.	3
Weibull shape β	< 1 early failures, $= 1$ random failures, > 1 wear-out.	3
Confidence interval	A procedure that captures the true parameter in 95 % of repetitions.	3
p-value	Probability of data at least as extreme as observed, assuming H_0.	4
Effect size	Size of a difference in physical units or standard deviations; decides relevance.	4
Prediction interval	Interval for a single new observation; wider than the confidence interval of the mean.	5
Bias–variance trade-off	Too simple models underfit, too flexible models overfit; the test error is U-shaped.	6
Cross-validation	Estimate of the test error by rotating held-out folds.	6
Precision / recall	Share of alarms that are real / share of failures that are detected.	7
Bagging / boosting	Averaging decorrelated deep trees / adding shallow trees that correct errors.	8

Concept	In one sentence	Chapter
T^2 / SPE	Unusual within the PCA model plane / distance from the plane.	9
Nyquist frequency	Half the sampling rate; higher frequencies alias.	10
Time series validation	Train on the past, test on the following period.	11
Backpropagation	Chain rule to compute the gradient of the loss with respect to all weights.	12

13.3 Checklist: typical pitfalls

⚠ Before you trust a result, check …

- **Data leakage:** features that contain the answer, preprocessing fitted on all data, random splits of time series, windows or parts of the same unit in training and test.
- **Wrong metric:** accuracy or ROC on imbalanced data; MAPE near zero values.
- **Significance without relevance:** tiny effects with huge n; dependent observations treated as independent.
- **Extrapolation:** predictions outside the range (or the combinations) of the training data.
- **Missing baseline:** no comparison with the mean, the seasonal naive forecast or a simple linear model.
- **Correlation vs. causation:** importance and correlation do not prove that a factor causes the effect.

13.4 Typical exam question types

1. **Method choice:** given a scenario, choose and justify a method and an evaluation metric.
2. **Interpretation:** read a regression table, a confusion matrix, a scree plot, a probability plot or a learning curve.
3. **Calculation by hand:** confidence interval, test statistic, Bayes' theorem, precision/recall from a confusion matrix, aliasing frequency.
4. **Code reading:** find the error in a short Python snippet.

13.5 Practice questions

1. A confusion matrix shows TN = 950, FP = 30, FN = 5, TP = 15. Compute accuracy, precision and recall. Is the model useful for failure detection?
2. A sensor detects 99 % of all leaks and raises a false alarm in 2 % of the leak-free tests; 0.5 % of all parts leak. What is the probability that a part with an alarm really leaks?
3. Find the error: `X = StandardScaler().fit_transform(X); X_train, X_test, y_train, y_test = train_test_split(X, y)`.
4. A t-test on 50,000 parts shows $p < 0.001$ for a 0.2 µm difference in mean diameter, tolerance ±20 µm. What do you tell the production manager?
5. Explain to a colleague, in five sentences, why a random train/test split is wrong for load forecasting.
6. Ten measurements of a shaft diameter give $\bar{x} = 25.012$ mm and $s = 0.004$ mm. Compute a 95 % confidence interval for the mean diameter ($t_{9,\,0.975} = 2.262$).
7. A Weibull analysis of pump seals gives $\beta = 0.8$. The maintenance manager wants to replace all seals preventively after 2,000 hours. What do you advise?
8. A regression of tool wear on cutting speed gives $R^2 = 0.95$ on the training data. The CV error is three times the training error. What do you conclude, and what do you do?
9. A vibration signal is sampled at 2 kHz. A gear mesh frequency of 1.6 kHz is present. Where does it appear in the spectrum?
10. In a PCA monitoring chart, a new production run exceeds the SPE limit but not the T^2 limit. How do you interpret this?

13.5.1 Solutions

Solution to Question 1

Accuracy = 965/1000 = 96.5 %, precision = 15/45 = 33 %, recall = 15/20 = 75 %. The trivial model "always OK" would reach 98 % accuracy, so accuracy is misleading here. The model detects three out of four failures with two false alarms per real alarm. Whether it is useful depends on the cost ratio of missed failures and false alarms – often yes.

Solution to Question 2

$$P(\text{leak} \mid \text{alarm}) = \frac{0.99 \cdot 0.005}{0.99 \cdot 0.005 + 0.02 \cdot 0.995} = \frac{0.00495}{0.00495 + 0.0199} \approx 0.20.$$

Only about one in five alarms is a real leak – the base rate effect. Alarmed parts should be re-tested before scrapping.

Solution to Question 3

The scaler is fitted on all data before the split, so means and standard deviations of the test data leak into the preprocessing. Correct: split first, then fit the scaler on the training data only – best by putting scaler and model into a `Pipeline`. (For time series, additionally `shuffle=False` or a time-based split.)

Solution to Question 4

The difference is statistically significant but practically irrelevant: 0.2 µm is 1 % of the tolerance half-width. With 50,000 parts, the test detects even tiny differences. No action is needed because of this difference; what matters is whether the process stays within tolerance (process capability).

Solution to Question 5

Neighbouring hours and days are strongly correlated. With a random split, the test days are surrounded by training days from before *and after*, so the model only interpolates. In operation, a forecast must be made without knowing the future. The random split therefore underestimates the real forecast error. A time-based split – train on the past, test on the following period – reproduces the real situation.

Solution to Question 6

$$25.012 \pm 2.262 \cdot \frac{0.004}{\sqrt{10}} = 25.012 \pm 0.0029 \text{ mm} \;\Rightarrow\; [25.0091,\ 25.0149] \text{ mm}.$$

The interval describes the uncertainty of the *mean* diameter, not the spread of individual shafts (about $\pm 2s = \pm 0.008$ mm).

Solution to Question 7

$\beta < 1$ means a decreasing failure rate – early failures dominate (e.g. installation errors, defective batches). A new seal is then *more* likely to fail soon than an old one, so preventive replacement at 2,000 hours would increase the number of failures and costs. Better: investigate the cause of the early failures (installation, quality of supply) and possibly run-in tests.

Solution to Question 8

The large gap between training and cross-validation error indicates overfitting: the model is too flexible for the amount of data, and the high training R^2 is misleading. Actions: simpler model (fewer terms, lower polynomial degree), regularisation, more data; select the complexity by cross-validation and report the CV or test error, not the training R^2.

Solution to Question 9

The Nyquist frequency is 1 kHz. The 1.6 kHz component aliases to $|1.6 - 2.0| = 0.4$ kHz $= 400$ Hz. The spectrum shows a false peak at 400 Hz. A sampling rate above 3.2 kHz (in practice 4 kHz) with an anti-aliasing filter is needed.

Solution to Question 10

A high SPE with normal T^2 means that the run shows a pattern of variation that the PCA model of normal operation does not contain – a new kind of deviation, e.g. a sensor fault, a broken correlation between variables or a new fault type. The contribution plot of the SPE shows which variables cause the deviation and is the starting point for the root-cause analysis.

A Python Quick Reference

This appendix collects the Python functions used in this script, ordered by **task** – the question you have in mind ("How do I count the missing values?") rather than the library. Each entry says what the function does, gives a short example and names where it is used first.

Notation: `df` is a DataFrame (a table), `s` a single column (a Series), `m` a mask of truth values. `np`, `pd`, `plt` and `sns` are the usual short names for NumPy, pandas, Matplotlib and seaborn (Chapter 1).

A.1 Getting an overview

Function	What it does	Example	Used in
`df.info()`	number of rows, column names, data types, non-missing values	`df.info()`	Ch. 1
`df.describe()`	count, mean, standard deviation, minimum, quartiles, maximum of every numeric column	`df.describe().round(2)`	Ch. 1
`df.head(n)`, `df.tail(n)`	first or last `n` rows (default: 5)	`df.head(10)`	Ch. 2
`df.shape`	number of rows and columns – an attribute, therefore without brackets	`n_rows, n_cols = df.shape`	Ch. 2
`s.value_counts()`	frequency of each value – the description of a categorical variable	`label.value_counts()`	Ch. 2
`df.nunique()`	number of different values per column	`(df.nunique() <= 1).sum()` counts constant columns	Ch. 2

Function	What it does	Example	Used in
`df.corr()`	Pearson correlation of every pair of columns	`sns.heatmap(df.corr(), annot=True)`	Ch. 1
`df.corrwith(s)`	correlation of every column of `df` with one series `s`	`df.corrwith(df["PE"])`	Ch. 2

A.2 Missing values

Function	What it does	Example	Used in
`df.isna()`	`True` wherever a value is missing (`NaN`, `None`, `NaT`)	`df.isna().sum()` counts missing values per column	Ch. 2
`df.notna()`	the opposite: `True` wherever a value is present	`t.notna().all()` – are all values present?	Ch. 2
`df.dropna()`	removes rows with missing values	`s.diff().dropna()`	Ch. 2
`df.fillna(value)`	replaces missing values	`X.fillna(X.median())` – median imputation	Ch. 2
`na_values=[...]` in `pd.read_csv`	treats codes such as `-999` as missing while reading	`pd.read_csv(f, na_values=[-999])`	Ch. 2

⚠ Pitfall: `is None`, `isna()` and `==`

- `x is None` checks whether *nothing* was given, e.g. an optional argument of a function: `if fname is None:`. Use `is` only for `None` – for numbers and text always use `==`.
- `s.isna()` checks whether *measured values* are missing. Never test with `== np.nan`: `NaN` is by definition not equal to anything, not even to itself – `np.nan == np.nan` is `False`.
- `s == None` on a column does not help either: it gives a mask in which even the missing values are `False`, and `if s == None:` raises an error.

A.3 Selecting rows and columns

Function	What it does	Example	Used in
`df["a"]`	one column, as a Series	`df["PE"]`	Ch. 1
`df[["a", "b"]]`	several columns, as a DataFrame (note the double brackets)	`df[["T1", "T_out"]]`	Ch. 2
`df[m]`	the rows where the mask `m` is `True`	`df[df["AT"] > 30]`	Ch. 2
`df.loc[rows, columns]`	rows and columns by label or mask	`X.loc[:, X.nunique() > 1]` – all rows, non-constant columns	Ch. 2
`df.iloc[rows, columns]`	rows and columns by position, counting from 0	`y.iloc[:, 0]` – first column; `s.iloc[0]` – first value	Ch. 5
`df.select_dtypes("number")`	only the numeric columns	`df.select_dtypes("number")`	Ch. 2
`df.drop(columns=[...])`	removes columns	`df.drop(columns=["rv1", "rv2"])`	Ch. 2
`df.sample(n, random_state=0)`	`n` random rows, reproducibly	`df.sample(1000, random_state=0)`	Ch. 1
`df.sort_values("a")`	sorts the rows by a column	`df.sort_values("RMSE", ascending=False)`	Ch. 1

Pitfall: `and`, `or`, `not` do not work on columns

Python's `and`, `or` and `not` handle a *single* truth value. For masks – NumPy arrays and pandas columns – use `&`, `|` and `~`, which work element by element, and put every condition in brackets:

	single value	mask (array, column)
and	`and`	`&`
or	`or`	`|`
not	`not`	`~`

```
df[(df["machine_id"] == "M4") & (df["shift"] == "night")]   # correct
df[df["machine_id"] == "M4" & df["shift"] == "night"]       # error: &
 ↪  binds before ==
```

`not m` on a mask raises an error ("the truth value of an array is ambiguous"), and `~True` on a single value gives `-2`, because `~` is originally a bitwise operator on integers.

A.4 Transforming values

Function	What it does	Example	Used in
`np.where(m, a, b)`	element-wise "if – then – else": `a` where `m` is `True`, otherwise `b`	`np.where(m, estimate, temp)`	Ch. 2
`s.astype(type)`	converts the data type	`s.astype(str)`	Ch. 2
`s.map({...})`	replaces values according to a dictionary, e.g. codes by names	`y.map({-1: "pass", 1: "fail"})`	Ch. 2
`pd.to_datetime(s, format=...)`	turns text into time stamps; always give the format explicitly	`pd.to_datetime(s, format="%Y%m%d%H")`	Ch. 2
`df.set_index("a")`, `df.sort_index()`	uses a column as row index, e.g. the time; sorts by the index	`df.set_index("time").sort_index()`	Ch. 2
`df.rename(columns={...})`	gives columns new names	`df.rename(columns={"T": "temp_C"})`	Ch. 2
`s.str.strip()`	removes blanks at the beginning and end of text	`df.columns.str.strip()`	Ch. 2
`s.tz_localize(...)`, `s.tz_convert(...)`	declares the time zone of time stamps, converts to another one	`s.tz_localize("UTC").tz_convert("Europe/Berlin")`	Ch. 2
`np.log(s)`, `s.abs()`	logarithm (for skewed data), absolute value	`np.log(energy["Appliances"])`	Ch. 2

`np.where` with one argument

With three arguments, `np.where(m, a, b)` selects values. With only one argument, `np.where(m)` does something different: it returns the *positions* where `m` is `True`. For positions, `np.flatnonzero(m)` is clearer.

`.str` and `.dt`: functions for every entry of a column

Text functions such as `strip()` work on a *single* text. To apply them to every entry of a column, pandas offers the accessor `.str`: `s.str.strip()`, `s.str.lower()`, `s.str.startswith("produkt")`. The column must contain text – hence `s.astype(str).str.strip()`. In the same way, `.dt` gives access to the parts of time stamps: `s.dt.hour`, `s.dt.day_name()`. For a time *index*, these are available directly: `df.index.hour`, `df.index.dayofweek`.
Pitfall: some functions exist twice with different meanings.

```
s = pd.Series(["1,5", "2,0"])
s.replace(",", ".")        # replaces entries that are exactly "," -
 ↪ here: no change
s.str.replace(",", ".")    # replaces "," inside every text - "1.5",
 ↪ "2.0"
```

Forgetting `.str` raises no error – the data simply remain unchanged.

Format codes for time stamps

In `pd.to_datetime(s, format=...)`, every code stands for one complete field of the date; its length is part of its definition:

Code	Meaning	Digits	Example
`%Y`	year	4	2025
`%y`	year without century	2	25
`%m`	month	2	04
`%d`	day	2	11
`%H`	hour (00–23)	2	13
`%M`	minute	2	05
`%S`	second	2	00

All other characters (`-`, `.`, `:`, blanks) must appear in the format exactly as in the data. Upper and lower case matter: `%m` is the month, `%M` the minute – a classic error that often raises no error message. Other tools write formats differently, e.g. `YYYYMMDDHH` in Excel, databases or the DWD documentation; such formats have to be translated.
Formats you will meet in practice:

Data look like	Typical source	Reading in pandas
`2025-04-11 13:00:00`, `2025-04-11T13:00:00+02:00`	international standard ISO 8601: databases, web services, modern loggers	`pd.to_datetime(s, format="ISO8601")`
`11.04.2025 13:00`	German software, Excel exports	`format="%d.%m.%Y %H:%M"`
`2025041113`	compact formats of older systems, e.g. the DWD	`format="%Y%m%d%H"` (text, not number)
`1744376400`	seconds since 1 January 1970 (Unix time): sensors, IoT, log files	`pd.to_datetime(s, unit="s")`

A.5 Summarising and grouping

Function	What it does	Example	Used in
`s.mean()`, `s.median()`, `s.std()`, `s.min()`, `s.max()`, `s.sum()`	the usual summary statistics; the mean of a mask is the share of `True` values	`(s == "scrap").mean()` – scrap rate	Ch. 1, 2
`s.quantile([0.25, 0.75])`	quantiles, here the quartiles	`q1, q3 = s.quantile([0.25, 0.75])`	Ch. 2
`s.idxmax()`	the label (e.g. the time) of the largest value	`cycle.idxmax()`	Ch. 2
`df.groupby("g")["x"].mean()`	summary per group	`df.groupby("machine_id")["diameter_mm"].mean()`	Ch. 2
`.agg([...])`	several summaries at once	`.agg(["min", "mean", "max"])`	Ch. 2
`s.resample("D")`	groups a time series into intervals (`"h"` hour, `"D"` day, …)	`s.resample("D").mean()`	Ch. 2

Function	What it does	Example	Used in
`s.rolling("1D")`	moving window, e.g. a 24 h rolling mean	`s.rolling("1D").mean()`	Ch. 2
`s.diff()`, `s.shift()`, `s.cumsum()`	difference to the previous value, shifted series, running sum	`s.index.to_series().diff()` – time steps	Ch. 2

A.6 Reshaping and combining tables

Function	What it does	Example	Used in
`pd.concat([a, b], axis=1)`	puts tables side by side (`axis=0`: one below the other)	`pd.concat([features, targets], axis=1)`	Ch. 1
`df.merge(other, on="key")`	combines two tables via a key column – like SQL `JOIN`	`parts.merge(orders, on="order_id")`	SQL app.
`df.pivot(index=..., columns=..., values=...)`	long table → wide table	`rates.pivot(index="machine_id", columns="shift", values="scrap_rate")`	SQL app.
`.unstack()`	moves an index level into the columns, e.g. after a `groupby` with two keys	`df.groupby([a, b])["x"].mean().unstack()`	Ch. 2
`df.melt(...)`	wide table → long (tidy) table	`df.melt(id_vars="time")`	Ch. 1 (text)

A.7 Reading data

Function	What it does	Example	Used in
`fetch_ucirepo(id=...)`	loads a dataset from the UCI repository (package `ucimlrepo`)	`fetch_ucirepo(id=294)`	Ch. 1

Function	What it does	Example	Used in
`pd.read_csv(f, sep=..., decimal=..., na_values=...)`	reads a text file with separated values; `decimal=","` for numbers with a decimal comma	`pd.read_csv(f, sep=";", na_values=[-999])`	Ch. 2
`zipfile.ZipFile(path)`	opens a zip archive; `.namelist()` lists its files	see Chapter 2	Ch. 2
`pd.read_sql(query, con)`	runs an SQL query and returns the result as a DataFrame	`pd.read_sql("SELECT * FROM machines", con)`	SQL app.

A.8 Random numbers and simple models

Function	What it does	Example	Used in
`rng = np.random.default_rng(0)`	random number generator with a fixed start value: the same "random" numbers in every run	`rng = np.random.default_rng(0)`	Ch. 2
`rng.random(n)`, `rng.uniform(a, b, n)`, `rng.normal(mu, sigma, n)`	`n` random numbers: uniform in [0, 1), uniform in [a, b), normally distributed	`rng.normal(0, 15, 10_000)`	Ch. 2
`rng.integers(a, b)`, `rng.choice(values, p=...)`	random integer, random choice with given probabilities	`rng.choice(["C45", "42CrMo4"], p=[0.7, 0.3])`	SQL app.
`np.polyfit(x, y, deg=1)`	least-squares polynomial; with `deg=1` a straight line → slope and intercept	`slope, intercept = np.polyfit(x, y, deg=1)`	Ch. 1

A.9 Plots

Function	What it does	Used in
`sns.set_theme(style="whitegrid")`	grid on style for all plots	Ch. 1
`sns.histplot(s)`	histogram – the shape of a distribution	Ch. 2
`sns.boxplot(x=s)`	box plot – compact, good for comparing groups	Ch. 2
`sns.pairplot(df)`	scatter plots of all pairs of columns	Ch. 1
`sns.heatmap(table)`	coloured table, e.g. a correlation matrix or a profile hour × weekday	Ch. 1, 2
`s.plot()`, `df.plot()`	quick line plot directly from pandas, e.g. of a time series	Ch. 2
`s.plot.bar()`	bar chart, e.g. of the frequencies from `value_counts()`	Ch. 2
`fig, axes = plt.subplots(rows, cols)`	several plots in one figure	Ch. 2
`plt.show()`	displays the figure	Ch. 1

B Data from Databases

In the chapters of this script, data arrive as files: a table from a repository, a raw text file from a weather service. In a company, engineering data usually live somewhere else – in **databases**:

- **process data** in a *historian*, a database specialised in time series from the control system,
- **quality and order data** in a manufacturing execution system (MES) or an enterprise resource planning system (ERP),
- **test results** in a laboratory database.

If you start a data science project in a company, you will therefore often get access to a database rather than a file. To find, combine and extract the data you need, you have to speak the language of databases: **SQL** (*Structured Query Language*). This appendix teaches the small part of SQL that covers most everyday needs – reading data – and shows how SQL and pandas fit together. No previous knowledge of databases is required.

Guiding question

A production line has six machines that turn shafts. Every part is inspected and either accepted (OK) or scrapped. The production manager asks: *"Which machine has the highest scrap rate – and does it depend on the shift?"* The answer is in the company database, spread over several tables.

Learning goals. After this appendix you can

- explain how a relational database stores data in tables linked by keys,
- read data with the SQL commands `SELECT`, `WHERE`, `GROUP BY` and `JOIN`,
- name the pandas counterpart of each command,
- load the result of a query into a pandas DataFrame,
- avoid typical pitfalls: duplicated rows after a join, missing values (`NULL`), time stamps.

B.1 Tables, keys and relations

A **relational database** stores data in tables. Each table follows the rules of *tidy data* from Chapter 1: one row per observation, one column per variable, one value per cell. The key

idea is to use **one table per kind of observational unit** and to link the tables instead of repeating information.

Our production database has four tables:

Table	One row per …	Columns (selection)
`machines`	machine	`machine_id`, `line`, `year_built`
`orders`	production order	`order_id`, `machine_id`, `start_time`, `shift`, `material`
`inspections`	inspected part	`part_id`, `order_id`, `diameter_mm`, `result`
`maintenance`	maintenance event	`machine_id`, `date`, `kind` (used in the pitfalls below)

Instead of writing the machine's line and year into every one of thousands of inspection rows, each inspection only stores its `order_id`, and each order stores its `machine_id`. If a machine is moved to another line, one row has to be changed – not thousands.

> **Definition: Primary key and foreign key**
>
> A **primary key** identifies each row of a table uniquely, e.g. `machine_id` in `machines`. A **foreign key** is a column that refers to the primary key of another table, e.g. `machine_id` in `orders`. Foreign keys are the links along which tables are combined.

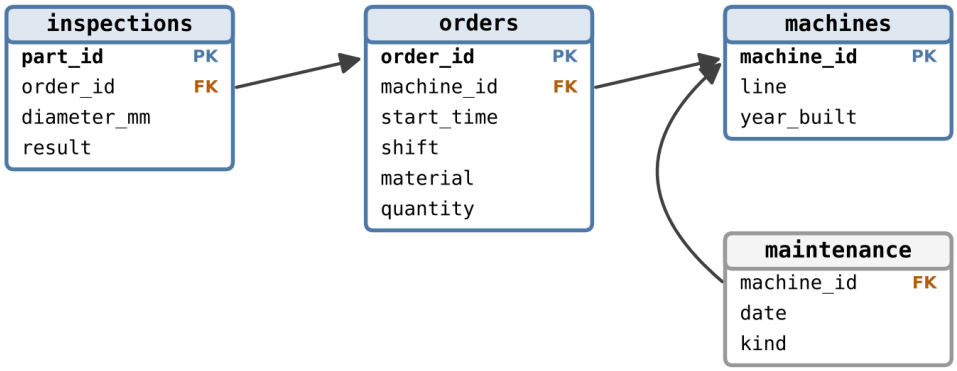


Figure B.1: Tables of the production database and their links. Each arrow points from a foreign key to the primary key it refers to.

We use **SQLite**, a database that consists of a single file, needs no server and is part of every Python installation. It works locally and in Google Colab. The following cell creates the example database; *how* it is created is not important here – in practice, the database already exists and you only read from it.

```
# Create the example database production.db (simulated data; in practice the
 ↪  database exists already)
import sqlite3
from pathlib import Path
import numpy as np
import pandas as pd

def create_production_db(path="data/production.db", seed=1):
    rng = np.random.default_rng(seed)
    Path(path).parent.mkdir(exist_ok=True)
    Path(path).unlink(missing_ok=True)
    machines = pd.DataFrame({
        "machine_id": ["M1", "M2", "M3", "M4", "M5", "M6"],
        "line": ["A", "A", "A", "B", "B", "B"],
        "year_built": [2019, 2019, 2015, 2008, 2021, 2015],
    })
    base_scrap = {"M1": 0.02, "M2": 0.025, "M3": 0.03, "M4": 0.07, "M5":
 ↪  0.015, "M6": 0.03}
    shift_factor = {"early": 1.0, "late": 1.1, "night": 1.5}
    # orders start at 06:00, 14:00 and 22:00 local time (05:00, 13:00, 21:00
     ↪  UTC in winter)
    starts = pd.date_range("2025-01-06 05:00", "2025-03-28 21:00", freq="8h",
 ↪  tz="UTC")
    shift_of = {5: "early", 13: "late", 21: "night"}
    orders, parts = [], []
    order_id, part_id = 1000, 1
    for t in starts:
        shift = shift_of[t.hour]
        for m in machines["machine_id"]:
            if rng.random() < 0.15:                      # machine idle in
             ↪  this shift
                continue
            material = str(rng.choice(["C45", "42CrMo4"], p=[0.7, 0.3]))
            n = int(rng.integers(20, 60))
            orders.append((order_id, m, t.strftime("%Y-%m-%d %H:%M:%S"),
 ↪  shift, material, n))
            p = base_scrap[m] * shift_factor[shift] * (1.3 if material ==
 ↪  "42CrMo4" else 1.0)
```

```
            if rng.random() > 0.03:                    # a few orders were
             ↪  never inspected
                for _ in range(n):
                    scrap = rng.random() < p
                    diam = rng.normal(25.0, 0.04 if scrap else 0.01)
                    if rng.random() < 0.02:
                        diam = None                    # diameter not
↪  recorded
                    parts.append((part_id, order_id, diam, "scrap" if scrap
↪  else "OK"))
                    part_id += 1
            order_id += 1
    maintenance = pd.DataFrame({
        "machine_id": ["M1", "M3", "M4", "M4", "M4", "M6"],
        "date": ["2025-01-20", "2025-02-03", "2025-01-13", "2025-02-10",
         ↪  "2025-03-10", "2025-03-03"],
        "kind": ["inspection", "repair", "repair", "repair", "inspection",
         ↪  "inspection"],
    })
    with sqlite3.connect(path) as con:
        con.execute("CREATE TABLE machines (machine_id TEXT PRIMARY KEY, line
↪  TEXT, year_built INTEGER)")
        con.execute("""CREATE TABLE orders (order_id INTEGER PRIMARY KEY,
↪  machine_id TEXT REFERENCES machines,
                       start_time TEXT, shift TEXT, material TEXT, quantity
↪  INTEGER)""")
        con.execute("""CREATE TABLE inspections (part_id INTEGER PRIMARY KEY,
↪  order_id INTEGER REFERENCES orders,
                       diameter_mm REAL, result TEXT)""")
        con.execute("CREATE TABLE maintenance (machine_id TEXT REFERENCES
↪  machines, date TEXT, kind TEXT)")
        con.executemany("INSERT INTO machines VALUES (?, ?, ?)",
↪  machines.itertuples(index=False))
        con.executemany("INSERT INTO orders VALUES (?, ?, ?, ?, ?, ?)",
↪  orders)
        con.executemany("INSERT INTO inspections VALUES (?, ?, ?, ?)", parts)
        con.executemany("INSERT INTO maintenance VALUES (?, ?, ?)",
↪  maintenance.itertuples(index=False))
    return path

db_path = create_production_db()
```

B.2 Reading data: `SELECT … FROM`

A database query is a short sentence in SQL. `pd.read_sql` sends it to the database and returns the result as a DataFrame. We open a connection once and use it for all queries:

```
con = sqlite3.connect(db_path)          # connection to the database file

pd.read_sql("SELECT * FROM machines", con)   # all columns (*) of the table
 ↪  machines
```

	machine_id	line	year_built
0	M1	A	2019
1	M2	A	2019
2	M3	A	2015
3	M4	B	2008
4	M5	B	2021
5	M6	B	2015

`SELECT` names the columns, `FROM` the table. SQL keywords are written in capitals by convention; SQL itself does not care. Large tables should not be read completely – `LIMIT` shows only the first rows, and `COUNT(*)` counts them:

```
print(pd.read_sql("SELECT COUNT(*) AS n_parts FROM inspections", con))
pd.read_sql("SELECT order_id, machine_id, start_time, shift FROM orders LIMIT
 ↪  5", con)
```

```
   n_parts
0    47583
```

	order_id	machine_id	start_time	shift
0	1000	M1	2025-01-06 05:00:00	early
1	1001	M2	2025-01-06 05:00:00	early
2	1002	M3	2025-01-06 05:00:00	early
3	1003	M4	2025-01-06 05:00:00	early
4	1004	M5	2025-01-06 05:00:00	early

B.3 Selecting rows: `WHERE`

`WHERE` keeps only the rows that satisfy a condition. Conditions can be combined with `AND` and `OR`; text values are written in single quotes.

```
query = """
SELECT order_id, machine_id, start_time, material, quantity
FROM orders
WHERE machine_id = 'M4' AND shift = 'night'
"""
night_m4 = pd.read_sql(query, con)
print(len(night_m4), "night orders on M4")
night_m4.head()
```

```
66 night orders on M4
```

	order_id	machine_id	start_time	material	quantity
0	1031	M4	2025-01-07 21:00:00	C45	20
1	1045	M4	2025-01-08 21:00:00	C45	58
2	1075	M4	2025-01-10 21:00:00	C45	58
3	1091	M4	2025-01-11 21:00:00	42CrMo4	47
4	1104	M4	2025-01-12 21:00:00	C45	54

The pandas counterpart filters a DataFrame that has already been loaded completely:

```
orders = pd.read_sql("SELECT * FROM orders", con)
orders[(orders["machine_id"] == "M4") & (orders["shift"] == "night")]
```

Both give the same result. The difference: with `WHERE`, the database does the filtering and only the selected rows travel to Python. For a table with millions of rows, this is the difference between seconds and minutes.

B.4 Summarising: `GROUP BY`

`GROUP BY` forms groups of rows with the same value and computes one summary per group – like `groupby` in pandas. Typical summaries are `COUNT`, `SUM`, `AVG`, `MIN` and `MAX`.

```
query = """
SELECT result, COUNT(*) AS n_parts
FROM inspections
GROUP BY result
"""
pd.read_sql(query, con)
```

	result	n_parts
0	OK	45607
1	scrap	1976

For the scrap rate we need the share of scrapped parts. A comparison such as `result = 'scrap'` is 1 if true and 0 if false, so its average is exactly the scrap rate. (This short form works in SQLite; other databases need `AVG(CASE WHEN result = 'scrap' THEN 1.0 ELSE 0 END)`.)

```
query = """
SELECT order_id,
       COUNT(*)                    AS n_parts,
       AVG(result = 'scrap')       AS scrap_rate
FROM inspections
GROUP BY order_id
"""
per_order = pd.read_sql(query, con)
per_order.head()
```

	order_id	n_parts	scrap_rate
0	1000	21	0.000000
1	1001	25	0.000000
2	1002	51	0.058824
3	1003	41	0.073171
4	1004	20	0.000000

This gives the scrap rate per *order*. But the question was about *machines* and *shifts* – and these are not in the table `inspections`. We have to combine tables.

B.5 Combining tables: `JOIN`

`JOIN` puts rows of two tables side by side wherever the keys match. We link each inspected part to its order (via `order_id`) and each order to its machine (via `machine_id`):

```
query = """
SELECT m.machine_id, m.year_built,
       COUNT(*)                AS n_parts,
       AVG(i.result = 'scrap') AS scrap_rate
FROM inspections AS i
JOIN orders   AS o ON i.order_id   = o.order_id
JOIN machines AS m ON o.machine_id = m.machine_id
GROUP BY m.machine_id, m.year_built
ORDER BY scrap_rate DESC
"""
per_machine = pd.read_sql(query, con)
per_machine.round(3)
```

	machine_id	year_built	n_parts	scrap_rate
0	M4	2008	8232	0.088
1	M3	2015	7809	0.043
2	M6	2015	7715	0.038
3	M2	2019	8503	0.034
4	M1	2019	7759	0.023
5	M5	2021	7565	0.019

The short names `i`, `o` and `m` (*aliases*) save typing and make clear from which table each column comes. `ORDER BY … DESC` sorts the result in descending order.

Discussion. Machine M4, the oldest one, scraps clearly more parts than all others. Is this a problem of the machine or of the shifts in which it runs? A second grouping variable answers this:

```
query = """
SELECT o.machine_id, o.shift, AVG(i.result = 'scrap') AS scrap_rate
FROM inspections AS i
JOIN orders AS o ON i.order_id = o.order_id
GROUP BY o.machine_id, o.shift
"""
rates = pd.read_sql(query, con)
rates.pivot(index="machine_id", columns="shift",
 ↪  values="scrap_rate").round(3)
```

shift machine_id	early	late	night
M1	0.020	0.022	0.029
M2	0.032	0.031	0.040
M3	0.035	0.042	0.052
M4	0.075	0.081	0.113
M5	0.016	0.018	0.022
M6	0.028	0.041	0.046

Two effects are visible: M4 is worse in every shift, and on every machine the night shift has the highest scrap rate. The answer to the production manager is therefore twofold – a machine problem *and* a shift effect. Note the last line of the cell: the query returns a long table (one row per machine and shift), and `pivot` turns it into a compact overview – the reshaping mentioned in Chapter 1.

The same analysis in pandas uses `merge` for the join and `groupby` for the grouping:

```
inspections = pd.read_sql("SELECT * FROM inspections", con)
orders = pd.read_sql("SELECT * FROM orders", con)
parts = inspections.merge(orders, on="order_id")
parts.groupby(["machine_id", "shift"])["result"].apply(lambda r: (r ==
 ↪  "scrap").mean())
```

Task	SQL	pandas
choose columns	`SELECT a, b`	`df[["a", "b"]]`
choose rows	`WHERE a > 5`	`df[df["a"] > 5]`
summarise per group	`GROUP BY g` with `COUNT`, `AVG`, …	`df.groupby("g").agg(...)`
combine tables	`JOIN … ON key`	`df1.merge(df2, on="key")`
sort	`ORDER BY a DESC`	`df.sort_values("a", ascending=False)`

Engineering practice: filter and aggregate in the database

Let the database do what it is good at – selecting, joining and aggregating large tables – and load only the result into Python. Use pandas for everything that comes afterwards: plots, statistics, models. On a company database, also make sure that you only *read*:

ask for a read-only account, and never run commands such as `DELETE` or `UPDATE` on a production system.

B.6 Pitfalls

B.6.1 A join can multiply rows

A join creates one output row for *every* matching pair. If a key appears several times in both tables, rows are multiplied. The table `maintenance` lists maintenance events – machine M4 has three of them. What happens if we join it to the orders?

```
query = """
SELECT COUNT(*) AS n_rows
FROM orders AS o
JOIN maintenance AS mt ON o.machine_id = mt.machine_id
WHERE o.machine_id = 'M4'
"""
print("orders of M4:            ", pd.read_sql("SELECT COUNT(*) FROM orders
 ↪  WHERE machine_id = 'M4'", con).iloc[0, 0])
print("rows after joining with maintenance:", pd.read_sql(query, con).iloc[0,
 ↪  0])
```

```
orders of M4:             209
rows after joining with maintenance: 627
```

Every order of M4 now appears three times – once per maintenance event. Sums and counts computed after such a join are three times too large, and nothing warns you. Always check the number of rows before and after a join, and be sure what one row of the result means.

B.6.2 Inner and left join

A plain `JOIN` (an *inner join*) keeps only rows that have a partner in both tables. Orders without any inspection therefore disappear silently. A `LEFT JOIN` keeps all rows of the left table and fills the missing columns with `NULL`:

```
query = """
SELECT o.order_id, o.machine_id, o.start_time
FROM orders AS o
LEFT JOIN inspections AS i ON o.order_id = i.order_id
WHERE i.part_id IS NULL
"""
not_inspected = pd.read_sql(query, con)
print(len(not_inspected), "orders without any inspection record")
not_inspected.head()
```

```
34 orders without any inspection record
```

	order_id	machine_id	start_time
0	1039	M4	2025-01-08 13:00:00
1	1048	M3	2025-01-09 05:00:00
2	1116	M1	2025-01-13 21:00:00
3	1124	M4	2025-01-14 05:00:00
4	1129	M4	2025-01-14 13:00:00

Whether these orders were really not inspected or whether the records got lost is a question for the people who run the line – exactly as for missing values in Chapter 2.

B.6.3 `NULL` is a missing value

`NULL` is SQL's missing value; pandas turns it into `NaN` or `None`. Aggregates such as `AVG` and `COUNT(column)` ignore `NULL` values silently, `COUNT(*)` counts all rows. A comparison with `NULL` is never true – use `IS NULL` instead of `= NULL`.

```
query = """
SELECT COUNT(*)            AS all_rows,
       COUNT(diameter_mm)  AS with_diameter,
       AVG(diameter_mm)    AS mean_diameter
FROM inspections
"""
pd.read_sql(query, con).round(4)
```

	all_rows	with_diameter	mean_diameter
0	47583	46617	24.9999

B.6.4 Time stamps are often text

SQLite has no separate type for time stamps; here they are stored as text in UTC. After loading, convert them explicitly, as for the weather data in Chapter 2:

```
orders = pd.read_sql("SELECT * FROM orders", con)
orders["start_time"] = pd.to_datetime(orders["start_time"], format="%Y-%m-%d
 ↪  %H:%M:%S", utc=True)
```

Other databases have proper date types, but the question "UTC or local time?" remains – check the documentation of the database.

B.7 Outlook

The commands of this appendix work with every relational database – SQLite, PostgreSQL, MySQL, Microsoft SQL Server, Oracle – with only small differences in dialect. To connect to a server database, you need a driver and the connection details (server, database, user name, password); the package SQLAlchemy offers a uniform interface for this. Process historians often have their own query interfaces, but usually also offer an SQL-like access or an export to tables.

! Key takeaways

- Relational databases store tidy tables – one per kind of observational unit – linked by primary and foreign keys.
- Four SQL commands cover most reading tasks: `SELECT` (columns), `WHERE` (rows), `GROUP BY` (summaries) and `JOIN` (combining tables). Each has a pandas counterpart.
- Filter and aggregate in the database, analyse in pandas.
- Check the number of rows after every join, know the difference between inner and left join, and treat `NULL` as a missing value.

B.8 Exercises

1. Write a query that returns, for each material, the number of inspected parts and the scrap rate. Is one material more difficult to machine?
2. The production manager suspects that scrap increases towards the end of the week. Compute the scrap rate per weekday. (Hint: load the joined table with `start_time`, convert it as shown above and use the pandas attribute `.dt.day_name()`.)
3. Determine the mean diameter and its standard deviation for OK parts and for scrap parts. What does the result tell you about the cause of the scrap?
4. Compute the scrap rate of M4 in the two weeks before and the two weeks after the repair on 2025-02-10. Can you see an effect of the repair? (Use `WHERE` with a condition on `start_time`; text in the format `YYYY-MM-DD` can be compared like dates.)

B.8.1 Solutions

Solution to Exercise 1

```
query = """
SELECT o.material, COUNT(*) AS n_parts, AVG(i.result = 'scrap') AS
 ↪  scrap_rate
FROM inspections AS i
JOIN orders AS o ON i.order_id = o.order_id
GROUP BY o.material
"""
pd.read_sql(query, con).round(3)
```

	material	n_parts	scrap_rate
0	42CrMo4	14454	0.054
1	C45	33129	0.036

The alloyed steel 42CrMo4 has a higher scrap rate than C45. Before concluding that it is “more difficult”, check whether both materials run on the same machines and in the same shifts – otherwise the comparison mixes several effects.

Solution to Exercise 2

```
query = """
SELECT o.start_time, i.result
FROM inspections AS i
JOIN orders AS o ON i.order_id = o.order_id
"""
parts = pd.read_sql(query, con)
parts["start_time"] = pd.to_datetime(parts["start_time"],
 ↪  format="%Y-%m-%d %H:%M:%S", utc=True)
weekday =
 ↪  parts["start_time"].dt.tz_convert("Europe/Berlin").dt.day_name()
(parts["result"] == "scrap").groupby(weekday).mean().round(3)
```

```
start_time
Friday       0.041
Monday       0.042
Saturday     0.046
Sunday       0.040
Thursday     0.044
Tuesday      0.038
Wednesday    0.040
Name: result, dtype: float64
```

The differences between the weekdays are small (between about 4 and 5 %) and show no trend towards the end of the week – the suspicion is not confirmed. Note the conversion to local time: here, orders start at 06:00, 14:00 and 22:00 local time, so the conversion does not change the weekday. For an order starting at 00:30 local time it would, because in UTC that is still the previous day.

Solution to Exercise 3

```
query = """
SELECT result, COUNT(diameter_mm) AS n, AVG(diameter_mm) AS mean_mm,
       AVG(diameter_mm * diameter_mm) - AVG(diameter_mm) *
 ↪  AVG(diameter_mm) AS var_mm2
FROM inspections
GROUP BY result
"""
stats = pd.read_sql(query, con)
stats["std_mm"] = stats["var_mm2"] ** 0.5      # SQLite has no built-in
 ↪  standard deviation
stats.drop(columns="var_mm2").round(4)
```

	result	n	mean_mm	std_mm
0	OK	44677	24.9999	0.0099
1	scrap	1940	24.9995	0.0397

Both groups have the same mean diameter, but the scrap parts scatter about four times as much. The process is not shifted, it is *unstable* for the scrapped parts – which points to causes such as vibration or tool wear rather than a wrong machine setting. (The variance formula used here is the simple "mean of squares minus square of the mean"; for very large or very precise values, compute the standard deviation in pandas instead.)

Solution to Exercise 4

```
query = """
SELECT CASE WHEN o.start_time < '2025-02-10' THEN 'before' ELSE 'after'
 ↪  END AS period,
       COUNT(*) AS n_parts, AVG(i.result = 'scrap') AS scrap_rate
FROM inspections AS i
JOIN orders AS o ON i.order_id = o.order_id
WHERE o.machine_id = 'M4' AND o.start_time >= '2025-01-27' AND
 ↪  o.start_time < '2025-02-24'
GROUP BY period
"""
pd.read_sql(query, con).round(3)
```

	period	n_parts	scrap_rate
0	after	1233	0.102
1	before	1509	0.080

The scrap rate of M4 is not lower after the repair – in this sample it is even higher. The repair has obviously not solved the problem. Is the increase real? In the simulation, M4 has the same scrap probability throughout, so the difference is pure chance plus a different mix of shifts and materials in the two periods. With real data, such a difference would have to be checked with a statistical test before drawing conclusions (Chapter 4).

C Datasets

All datasets used in this script are publicly available. Please cite the original sources when you use them.

Dataset	Domain	Size	License	Chapters
Combined Cycle Power Plant	power generation	9,568 rows, 4 features	CC BY 4.0	1, 6
Appliances Energy Prediction	building services, sensor network	19,735 rows, 10-min intervals	CC BY 4.0	2
SECOM	semiconductor manufacturing	1,567 rows, 591 features	CC BY 4.0	2, 9
NASA C-MAPSS	turbofan degradation	4 subsets, 21 sensors	NASA Open Data	3, 12
Gas Turbine CO and NOx Emission	gas turbine emissions	36,733 rows, 2011–2015	CC BY 4.0	4
AI4I 2020 Predictive Maintenance	manufacturing (synthetic)	10,000 rows, 5 failure modes	CC BY 4.0	4, 7
Concrete Compressive Strength	materials, civil engineering	1,030 rows, 8 features	CC BY 4.0	5
Airfoil Self-Noise	aerodynamics, acoustics	1,503 rows, 5 features	CC BY 4.0	6
Steel Plates Faults	quality inspection	1,941 rows, 27 features, 7 classes	CC BY 4.0	8
Condition Monitoring of Hydraulic Systems	hydraulic test rig	2,205 cycles, 17 sensors up to 100 Hz	CC BY 4.0	9, 10
CWRU Bearing Data Center	rolling bearings, vibration	normal and faulty bearings, 0–3 hp	freely available	10

Dataset	Domain	Size	License	Chapters
Open Power System Data – Time Series	European power grid	32 countries, 2015–2020, hourly	open (sources: ENTSO-E and others)	11
DWD Climate Data Center	weather in Germany	station data	DWD open data	2, 11
MVTec AD	industrial visual inspection	$>$ 5,000 images, 15 categories	CC BY-NC-SA 4.0	12

Loading. UCI datasets are loaded with the package `ucimlrepo`. MVTec AD has to be downloaded manually after accepting its license (see Chapter 12). All other files are downloaded by the code of the respective chapter and cached in a local `data/` folder.

D Further Reading

- G. James, D. Witten, T. Hastie, R. Tibshirani, J. Taylor: *An Introduction to Statistical Learning with Applications in Python*. Springer, 2023. Free PDF: https://www.statlearning.com – core textbook for Chapters 5–10.
- S. L. Brunton, J. N. Kutz: *Data-Driven Science and Engineering*. 2nd ed., Cambridge University Press, 2022 – SVD/PCA, Fourier analysis, engineering perspective.
- J. VanderPlas: *Python Data Science Handbook*. Free online: https://jakevdp.github.io/PythonDataScienceHandbook/ – NumPy, pandas, Matplotlib (Chapters 1–2).
- NIST/SEMATECH: *e-Handbook of Statistical Methods*. Free online: https://www.itl.nist.gov/div898/handbook/ – distributions, tests, reliability, DoE (Chapters 3–5).
- D. C. Montgomery: *Design and Analysis of Experiments*. Wiley – design of experiments in depth.
- A. Géron: *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3rd ed., O'Reilly, 2022 – practical machine learning.